

LLMsVerifier_HelixCode_Integration_Plan

LLMsVerifier Integration into HelixCode: In-Depth Implementation Plan

Document Version: 1.0.0
Date: 2026-04-30
Classification: Internal — Technical Implementation Plan
Status: Draft for Review

Document Control

Version	Date	Author	Description
1.0.0	2026-04-30	Technical Architecture Team	Initial comprehensive integration plan

Table of Contents

- 1. [Executive Summary & Repository Analysis](#)
- 2. [Repository Analysis — LLMsVerifier](#)
- 3. [Repository Analysis — HelixAgent Reference Implementation](#)
- 4. [Integration Architecture Design](#)
- 5. [UX Design Specification](#)
- 6. [Testing Strategy — Anti-Bluff Guarantee](#)
- 7. [Constitution & Documentation Updates](#)
- 8. [Phased Implementation Plan — Master Guide](#)

Section 1: Executive Summary & Repository Analysis

HelixCode Repository - Exhaustive Analysis

Analysis Date: 2025-07-01
Repository: <https://github.com/HelixDevelopment/HelixCode>
Branch: main
Purpose: Deep analysis for LLMsVerifier integration as single source of truth for models

1. Repository Structure Overview

1.1 Top-Level Directories

Directory	Purpose
.github/workflows	GitHub Actions workflows (auto-commit style)
Assets	Project logos, wide_black.png, themes
Dependencies	Dependency tracking and governance
Documentation	Technical documentation
Example_Projects	Example projects submodule
Example_Resources	Example resources submodule
Github-Pages-Website	Marketing website submodule (@ 49f7e16)
HelixCode	Main Go implementation
Implementation_Guide	Implementation plans and guides
Specification	Technical specifications and requirements
Upstreams	Upstream sync scripts (GitLab)
Website	Marketing website
awesome-ai-memory	awesome-ai-memory submodule (@ 7f281fd)
challenges/scripts	Challenge scripts (host power management, no-suspend)

Directory	Purpose
cmd/security_test	Security testing tools
docker	Docker configurations
docs	Additional documentation
internal	Go internal packages (see below)
isolated_files	Isolated/misc files
scripts	Build and utility scripts
security	Security-related files
test	Test configurations
tests/e2e	E2E test suites

1.2 helix_code/ Subdirectory Structure (Main Code)

```

helix_code/
├── cmd/
│   ├── cli/           # CLI client entry point
│   ├── server/        # HTTP server entry point
│   └── security-test/  # Security testing tools
├── internal/
│   ├── agent/         # Agent subsystem
│   ├── auth/          # JWT authentication (VERIFIED REAL)
│   ├── cognee/        # Cognee integration
│   ├── commands/      # Command implementations
│   ├── config/        # Configuration management (Viper-based)
│   ├── context/       # Context management
│   ├── database/      # PostgreSQL layer
│   ├── deployment/    # Deployment tools
│   ├── discovery/     # Discovery services
│   ├── editor/        # Code editing tools
│   ├── event/         # Event system
│   ├── fix/           # Fix utilities
│   ├── focus/         # Focus management
│   ├── hardware/      # Hardware detection
│   ├── hooks/         # Hook system
│   ├── llm/           # LLM providers and reasoning (BLUFF AREA)
│   ├── logging/       # Logging
│   ├── logo/          # Logo processing
│   ├── mcp/           # MCP protocol implementation
│   ├── memory/        # Memory integration
│   ├── mocks/         # Test mocks
│   ├── monitoring/    # Monitoring
│   ├── notification/  # Multi-channel notifications
│   ├── performance/  # Performance utilities
│   ├── persistence/   # Persistence layer
│   └── project/       # Project management

```

—	provider/	# Provider utilities
—	providers/	# Provider implementations
—	redis/	# Redis client
—	server/	# HTTP server & API (Gin)
—	session/	# Session management
—	task/	# Task management
—	tools/	# Tool ecosystem
—	user/	# User management
—	util/	# Utilities
—	worker/	# SSH-based worker pool
—	workflow/	# Workflow engine
—	api/	# API definitions
—	applications/	# Platform-specific apps (desktop, mobile, Aurora)
—	assets/	# Generated assets
—	benchmarks/	# Benchmark suites
—	bin/	# Build output
—	config/	# Configuration files
—	docker/	# Docker files
—	examples/	# Examples
—	scripts/	# Build scripts
—	tests/	# Test suites

Key Finding: The project is primarily **Go (95.4%)**, with some Shell (4.3%), Makefile (0.1%), Swift (0.1%), Kotlin (0.1%), and Dockerfile (0.0%).

2. README & Documentation Analysis

2.1 README.md [2](#)

Project Identity: HelixCode - Distributed AI Development Platform

Package: dev.helix.code

License: MIT

Version: 1.0.0

Documented Features (5 Phases):

Phase	Status	Key Features
Phase 1: Foundation	Completed	PostgreSQL schema (11 tables), JWT auth, worker management, task management, REST API (Gin), config system
Phase 2: Core Services	Completed	Task division, LLM multi-provider, distributed computing, MCP protocol, reasoning, notifications
	Completed	

Phase	Status	Key Features
Phase 3: Workflows		Project management, dev workflows (plan/build/test/refactor), session management
Phase 4: LLM Integration	Completed	Hardware detection, model management, provider architecture, CLI interface
Phase 5: Advanced Features	Completed	SSH worker pool, LLM tooling, multi-client (REST/CLI/TUI/WebSocket), cross-platform, mobile ready

Documented CLI Usage:

```
# Interactive mode
./cli
# List workers
./cli --list-workers
# Add a worker
./cli --worker worker-host --user helix --key ~/.ssh/id_rsa
# Generate with LLM
./cli --prompt "Hello world" --model llama-3-8b
# Health check
./cli --health
```

Documented API Endpoints: - POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/auth/refresh - GET /api/v1/workers, POST /api/v1/workers, GET /api/v1/workers/:id - GET /api/v1/tasks, POST /api/v1/tasks, GET /api/v1/tasks/:id, POST /api/v1/tasks/:id/start - GET /api/v1/projects, POST /api/v1/projects, GET /api/v1/projects/:id

Build Commands:

```
make build          # Build application
make test           # Run all tests
make clean          # Clean build artifacts
make lint           # Lint code
make fmt            # Format code
```

3. AGENTS.md Analysis

3.1 AGENTS.md ⁵ — Authoritative Agent Guide

Version: 2.0.0 (CONST-035 Anti-Bluff Mandate)

Date: 2026-04-30

Critical Bluff Areas Identified:

BLUFF-001: LLM Generation is Simulated (CRITICAL)

- **File:** `cmd/cli/main.go` lines 190-214
- **Evidence:** Simulated response with `fmt.Sprintf("Generated response for: %s...")`
- **Fix Priority:** P0 - Immediate

BLUFF-002: Model Listing is Hardcoded (CRITICAL)

- **File:** `cmd/cli/main.go` lines 101-128
- **Evidence:** Only 3 hardcoded models (llama-3-8b, mistral-7b, phi-3-mini)
- **Fix Priority:** P0 - Immediate

BLUFF-003: Command Execution is Simulated (HIGH)

- **File:** `cmd/cli/main.go` lines 237-250
- **Evidence:** `time.Sleep(1 * time.Second)` simulating command execution
- **Fix Priority:** P0 - Immediate

Free AI Providers Listed: - XAI (Grok): grok-3-fast-beta, grok-3-mini-fast-beta - OpenRouter: Free models - GitHub Copilot: gpt-4o, claude-3.5-sonnet (with subscription) - Qwen: 2,000 requests/day free tier

Technology Stack (from AGENTS.md): - Go 1.24.0 with toolchain go1.24.9 - Gin v1.11.0 (HTTP) - JWT v4.5.2 - Viper v1.21.0 (Config) - Cobra v1.8.0 (CLI) - Testify v1.11.1 (Testing) - Fyne v2.7.0 (Desktop UI) - tview v0.42.0 (Terminal UI) - chromedp v0.14.2 (Browser) - goquery v1.10.3 (Web scraping) - go-tree-sitter (Parsing)

4. CONSTITUTION.md Analysis

4.1 Key Constitutional Constraints [34](#)

Constraint	Description
CONST-001	NO CI/CD pipelines - permanent, non-negotiable
CONST-002	NO mocks in production - only in unit tests (*_test.go with -short)
CONST-003	NO HTTPS for Git - SSH only
CONST-004	NO manual container commands - orchestrator-owned
CONST-005	100% real data for non-unit tests
CONST-006	Challenge coverage - every component must have challenge scripts
CONST-017	Zero-Bluff Testing (CONST-035)
CONST-018	Host Power Management Hard Ban - no suspend/hibernate/reboot
CONST-020	Provider Fallback Chain Reality - fallback chains tested with actual failures
CONST-021	No Mocks Above Unit - Makefile must include no-mocks-above-unit target
CONST-025	Secret Management - NO secrets in code, ever
CONST-035	End-User Usability Mandate - every PASS must guarantee quality, completion, usability

5. Configuration System

5.1 Main Config File: `helix_code/internal/config/config.go` [15](#)

Configuration Loading Order (Viper-based): 1. Set defaults (`setDefault()`) 2. Find config file (`findConfigFile()`) 3. Read environment variables (prefix HELIX) 4. Explicitly bind critical env vars: - `HELIX_AUTH_JWT_SECRET` - `HELIX_DATABASE_PASSWORD` - `HELIX_DATABASE_HOST` - `HELIX_REDIS_PASSWORD`

- HELIX_REDIS_HOST - HELIX_REDIS_PORT 5. Unmarshal into Config struct 6.
Validate(validateConfig())

Config Struct (key fields):

```
type Config struct {
    Version      string           `mapstructure:"version"`
    UpdatedBy    string           `mapstructure:"updated_by"`
    Application   ApplicationConfig `mapstructure:"application"`
    Server        ServerConfig      `mapstructure:"server"`
    Database      database.Config    `mapstructure:"database"`
    Redis         RedisConfig       `mapstructure:"redis"`
    Auth          AuthConfig        `mapstructure:"auth"`
    Workers       WorkersConfig     `mapstructure:"workers"`
    Tasks         TasksConfig       `mapstructure:"tasks"`
    LLM           LLMConfig         `mapstructure:"llm"`
    Providers     ProvidersConfig   `mapstructure:"providers"`
    Logging       LoggingConfig     `mapstructure:"logging"`
    Cognee        *CogneeConfig     `mapstructure:"cognee"`
}
```

LLM Configuration:

```
type LLMConfig struct {
    DefaultProvider string           `mapstructure:"default_provider" // default: "local"`
    DefaultModel    string           `mapstructure:"default_model"   // default: "llama-3.2-3b"`
    MaxTokens       int              `mapstructure:"max_tokens"      // default: 4096`
    Temperature     float64          `mapstructure:"temperature"     // default: 0.7`
}
```

Valid LLM Providers (from validator): local, openai, anthropic, gemini, xai, openrouter, copilot

Default Config Locations: - ./config.yaml - ./config/config.yaml -
\$HOME/.config/helixcode/config.yaml - /etc/helixcode/config.yaml

5.2 Environment Variables (.env.example) [4](#)

```
# Required
HELIX_DATABASE_PASSWORD=CHANGE_ME_db_password
HELIX_AUTH_JWT_SECRET=your-super-secret-jwt-key
HELIX_REDIS_PASSWORD=redispass
```



```
# Ports
HELIX_API_PORT=8080
HELIX_SSH_PORT=2222
HELIX_WEB_PORT=3000

# LLM Provider Keys (optional)
# HELIX_OPENAI_API_KEY=...
# HELIX_OLLAMA_HOST=http://ollama:11434
# HELIX_LLAMA_CPP_HOST=http://llamacpp:8080
```

5.3 ConfigManager

File: helix_code/internal/config/config.go lines 360+

Provides ConfigManager with: - NewHelixConfigManager(configPath) - loads or creates default config - GetConfig(), UpdateConfig(), UpdateConfigFromMap() - ExportConfig(), ImportConfig(), BackupConfig(), ResetToDefaults() - Configuration templates (basic, development, production, testing) - Configuration migrator (1.0.0 -> 1.1.0 -> 1.2.0) - Configuration validator with custom rules

6. Core Architecture

6.1 Entry Points

Entry Point	File	Purpose
Server	cmd/server/main.go 37	HTTP server (Gin) with DB, Redis
CLI	cmd/cli/main.go 10	Command-line interface
CLI (old)	cmd/cli/main.go.old 11	Older CLI with more commands

Server Startup Flow (cmd/server/main.go): 1. Load config (config.Load()) 2. Initialize database (optional - empty host disables) 3. Initialize schema (db.InitializeSchema()) 4. Initialize Redis 5. Create HTTP server (server.New(cfg, db, rds)) 6. Start HTTP server 7. Graceful shutdown on SIGINT/SIGTERM

CLI Startup Flow (cmd/cli/main.go): 1. NewCLI(): - Initialize LLM provider (Ollama, default llama3.2, http://localhost:11434) - Initialize worker pool

(worker.NewSSHWorkerPool(true)) - Initialize notification engine 2. cli.Run() parses flags and dispatches to handlers

6.2 CLI Flag Architecture

```
command          = flag.String("command", "", "Command to execute")
workerHost       = flag.String("worker", "", "Worker host to add")
workerUser       = flag.String("user", "", "Worker SSH username")
workerKey        = flag.String("key", "", "Worker SSH key path")
model            = flag.String("model", "llama-3-8b",
    "LLM model to use")
prompt           = flag.String("prompt", "", "Prompt for LLM
    generation")
maxTokens        = flag.Int("max-tokens", 1000, "Maximum tokens")
temperature      = flag.Float64("temperature", 0.7, "Generation
    temperature")
stream           = flag.Bool("stream", false, "Stream the
    response")
listWorkers      = flag.Bool("list-workers", false, "List all
    workers")
listModels       = flag.Bool("list-models", false, "List available
    models")
healthCheck      = flag.Bool("health", false, "Perform health
    check")
notify           = flag.String("notify", "", "Send notification")
notifyType       = flag.String("notify-type", "info",
    "Notification type")
notifyPriority    = flag.String("notify-priority", "medium",
    "Notification priority")
nonInteractive    = flag.Bool("non-interactive", false, "Run in non-
    interactive mode")
```

Interactive Commands: - workers → handleListWorkers - models → handleListModels - health → handleHealthCheck - help / exit / quit

7. Model/Provider Management System

7.1 Provider Interface [38](#)

File: helix_code/internal/llm/missing_types.go

```
type Provider interface {
    GetType() ProviderType
    GetName() string
    GetModels() []ModelInfo
    GetCapabilities() []ModelCapability
```

```

Generate(ctx context.Context, request *LLMRequest)
    (*LLMResponse, error)
GenerateStream(ctx context.Context, request *LLMRequest, ch
    chan<- LLMResponse) error
IsAvailable(ctx context.Context) bool
GetHealth(ctx context.Context) (*ProviderHealth, error)
Close() error
}

```

7.2 Provider Type Registry [38](#)

35+ Provider Types Defined:

Category	Providers
Cloud APIs	openai, anthropic, gemini, vertexai, azure, bedrock, groq, qwen, copilot, openrouter, xai, cohere, mistral, huggingface
Local Inference	ollama, llamacpp, vllm, localai, fastchat, textgen, lmstudio, jan, koboldai, gpt4all, tabbyapi, mlx, mistralrs
Memory/Agent	memgpt, crewai, characterai, replika, anima, llamaindex
Database/Vector	clickhouse, supabase, deeplake, chroma
Generic	local, agnostic, gemma

7.3 Provider Factory [21](#)

File: helix_code/internal/llm/factory.go

```

func NewProvider(config ProviderConfigEntry) (Provider, error) {
    switch config.Type {
    case ProviderTypeOpenAI:      return
        NewOpenAIProvider(config)
    case ProviderTypeAnthropic:   return
        NewAnthropicProvider(config)
    case ProviderTypeGemini:      return
        NewGeminiProvider(config)
    case ProviderTypeOllama:      return
        NewOllamaProvider(ollamaConfig)
    case ProviderTypeLlamaCpp:    return
        NewLlamaCPPProvider(llamaConfig)
    }
}

```

```

    case ProviderTypeQwen:          return NewQwenProvider(config)
    case ProviderTypeXAI:           return NewXAIProvider(config)
    case ProviderTypeOpenRouter:    return
        NewOpenRouterProvider(config)
    case ProviderTypeCopilot:       return
        NewCopilotProvider(config)
    case ProviderTypeAzure:         return NewAzureProvider(config)
    case ProviderTypeBedrock:       return
        NewBedrockProvider(config)
    case ProviderTypeVertexAI:      return
        NewVertexAIProvider(config)
    case ProviderTypeGroq:          return NewGroqProvider(config)
    case ProviderTypeVLLM:          return NewVLLMProvider(config)
    case ProviderTypeLocalAI:       return
        NewLocalAIProvider(config)
    // ... 20+ more cases
    default:
        return nil, fmt.Errorf("unsupported provider type: %s",
            config.Type)
}

```

Model Manager Initialization:

```

func InitializeModelManager(configs []ProviderConfigEntry)
    (*ModelManager, error) {
    manager := NewModelManager()
    for _, config := range configs {
        if !config.Enabled { continue }
        provider, err := NewProvider(config)
        if err != nil { return nil, err }
        if err := manager.RegisterProvider(provider); err != nil
        {
            return nil, err
        }
    }
    return manager, nil
}

```

7.4 Model Manager [33](#)

File: helix_code/internal/llm/model_manager.go

Core Responsibilities: - RegisterProvider(provider Provider) error - registers provider and its models - SelectOptimalModel(criteria ModelSelectionCriteria) (*ModelInfo, error) - selects best model - GetAvailableModels() []*ModelInfo - returns all registered models - GetModelsByCapability(capabilities []ModelCapability)

[]*ModelInfo - capability filtering - GetProviderForModel(modelName, providerType) (Provider, error) - provider lookup - HealthCheck(ctx) map[ProviderType]*ProviderHealth - health checks all providers

Model Scoring Criteria:

```
type ModelSelectionCriteria struct {
    TaskType                string
    RequiredCapabilities    []ModelCapability
    MaxTokens               int
    Budget                  float64
    LatencyRequirement      time.Duration
    QualityPreference       string // "fast", "balanced", "quality"
}
```

Scoring Algorithm: 1. Capability matching (must have all required) 2. Context size adequacy 3. Task type suitability (bonus multipliers for code_generation, debugging, etc.) 4. Hardware compatibility (hardware.Detector.CanRunModel()) 5. Quality preference (70B = 1.3x, 3B = 0.9x)

7.5 Model Discovery Engine [29](#)

File: helix_code/internal/llm/model_discovery.go

Key Features: - GetRecommendations(ctx, RecommendationRequest) (*RecommendationResponse, error) - Hardware-aware model recommendations - Task compatibility scoring (code_generation, planning, debugging, testing, refactoring, documentation, analysis) - Performance estimation (tokens/second, memory, latency, cost) - Privacy scoring (local/hybrid/cloud) - Fallback alternatives based on model category

Hardcoded External Models (in fetchExternalModels):

```
[ ]*ModelInfo{
    {ID: "llama-3-8b-instruct", Name: "Llama 3 8B Instruct",
      Format: FormatGGUF, Size: 4.7GB, ContextSize: 8192},
    {ID: "mistral-7b-instruct", Name: "Mistral 7B Instruct",
      Format: FormatGGUF, Size: 4.1GB, ContextSize: 32768},
    {ID: "codellama-7b-instruct", Name: "CodeLlama 7B Instruct",
      Format: FormatGGUF, Size: 3.8GB, ContextSize: 16384},
}
```

7.6 Cross-Provider Registry [14](#)

File: helix_code/internal/llm/cross_provider_registry.go

Default Providers Registered:

Provider	Type	Endpoint	Default Port	Supported Formats
vllm	openai-compatible	http://localhost:8000	8000	GGUF, GPTQ, AWQ, HF, FP16, BF16
ollama	llamacpp	custom http://localhost:8080	8080	GGUF
ollama	openai-compatible	http://localhost:11434	11434	GGUF

Registry persisted to ~/.helixcode/local-llm/registry.json

7.7 Alias Management [20](#)

File: helix_code/internal/llm/aliases.go

- AliasManager with fuzzy matching (Levenshtein distance)
- Default threshold: 0.7 (70%)
- Provider-specific alias resolution
- Autocomplete support

7.8 Auto LLM Manager [23](#)

File: helix_code/internal/llm/auto_llm_manager.go

Zero-Touch Mode: - Auto-discover, auto-install, auto-configure, auto-start, auto-monitor, auto-update - Background tasks: health monitor, performance optimizer, update checker - Auto-recovery with retry limits - Git-based provider installation (clone, build, configure) - Directory structure under ~/.helixcode/local-llm/

7.9 Ollama Provider Implementation [31](#)

File: helix_code/internal/llm/ollama_provider.go

Real Implementation (anti-bluff verified): - discoverModels() - calls GET /api/tags to fetch actual models - Generate() - calls POST /api/chat with real HTTP request - GenerateStream() - calls streaming endpoint, sends real response chunks - IsAvailable() - tests API endpoint - GetHealth() - tests API endpoint with latency measurement

Configuration:

```
type OllamaConfig struct {
    BaseURL      string           // default: http://localhost:11434
    DefaultModel string           // e.g., "llama2"
    Timeout      time.Duration   // default: 120s
    KeepAlive    time.Duration
    StreamEnabled bool
}
```

7.10 Current Model Listing (CRITICAL BLUFF)

File: helix_code/cmd/cli/main.go lines 101-128

```
func (c *CLI) handleListModels(ctx context.Context) error {
    models := []struct {
        ID          string
        Name         string
        Provider     string
        ContextSize int
        Status       string
    }{
        {"llama-3-8b", "Llama 3 8B", "llama.cpp", 8192,
         "available"},
        {"mistral-7b", "Mistral 7B", "ollama", 4096,
         "available"},
        {"phi-3-mini", "Phi-3 Mini", "openai", 128000,
         "available"},
    }
    // ... prints hardcoded list
}
```

This is HARDCODED - NOT dynamic discovery. The AGENTS.md explicitly flags this as BLUFF-002 (P0 critical).

8. CLI UX Analysis

8.1 Current CLI (cmd/cli/main.go) [10](#)

UI Framework: Standard Go flag package (NOT Cobra - despite AGENTS.md claiming Cobra)

Commands: | Flag | Handler | |——|——| | --list-workers |
handleListWorkers | | --list-models | handleListModels (HARDCODED) |
| --health | handleHealthCheck | | --worker HOST | handleAddWorker | |
--prompt TEXT | handleGenerate (REAL - calls provider.Generate) | | --model
NAME | Used with --prompt | | --stream | Used with --prompt | | --notify
MESSAGE | handleNotification | | --command CMD | handleCommand
(SIMULATED) | | (no flags) | handleInteractive |

Interactive Mode: - Prompt: helix> - Commands: workers, models, health, help, exit/quit - Uses fmt.Scanln() for input - Signal handling for graceful shutdown (SIGINT/SIGTERM)

8.2 Older CLI (cmd/cli/main.go.old) [11](#)

UI Framework: Argument-based (not flags)

Commands: - help, version, models, hardware, health - chat <prompt>, generate <description>, plan <project>, test <code>, debug <issue>, refactor <code>

Status: All AI commands are TODO placeholders:

```
func (c *CLI) startChat(args []string) error {  
    // TODO: Implement actual chat functionality  
    fmt.Println("Chat functionality coming soon...")  
    return nil  
}
```

9. Platform Support

Documented Platforms: - Linux (primary) - macOS (Intel + ARM64) - Windows - Aurora OS - SymphonyOS / Harmony OS - Android (mobile framework) - iOS (mobile framework)

Build Targets (from Makefile):

```
make prod                # Linux, macOS, Windows binaries  
make desktop-linux       # Desktop GUI + CLI for Linux  
make desktop-macos      # Intel + ARM  
make desktop-windows     # .exe  
make mobile-ios          # HelixCore.xcframework  
make mobile-android      # mobile.aar  
make aurora-os           # Aurora OS client  
make harmony-os          # Harmony OS client
```

10. Test Framework

10.1 Testing Infrastructure

Framework: Go built-in testing + Testify v1.11.1

Test Categories (from AGENTS.md): 1. Unit tests - Mocks OK, *_test.go, -short flag 2. Contract tests - Real API schemas, no mocks 3. Component tests - Real subsystems wired together 4. Integration tests - Full app with real dependencies 5. E2E challenges - Complete user workflows 6. Security tests - OWASP compliance 7. Performance tests - Benchmarks

Makefile Test Targets ²⁴:

```
make test          # Basic go test -v ./...
make test-all      # test + coverage + benchmark + docs
make test-coverage  # go test -v -race
                   -coverprofile=coverage.out
make test-benchmark # go test -bench=. -benchmem ./...
make test-docs      # Documentation validation
make test-full      # ALL tests with infrastructure (ZERO
                   skips)
make test-unit-full  # Unit tests with full infra
make test-integration-full # Integration tests
make test-e2e-full   # E2E challenge tests
make test-security-full # Security tests
make test-load-full   # Load tests
make test-complete    # All test types in sequence
make coverage-full     # Comprehensive coverage report
```

Test Infrastructure:

```
make test-infra-up    # docker compose -f docker-compose.full-
                   test.yml up -d
make test-infra-down  # Stop infrastructure
make test-infra-status # Check status
```

10.2 LLM Package Test Files

File: helix_code/internal/llm/ contains extensive test files: -

- aliases_test.go
- anthropic_provider_test.go
- auto_llm_manager_test.go
- azure_provider_test.go
- bedrock_provider_test.go
- cache_control_test.go
- cloud_providers_integration_test.go
- copilot_provider_test.go
- cross_provider_registry_test.go
- cross_provider_test.go
- factory_test.go
- local_providers_integration_test.go
- local_providers_test.go
- model_converter_test.go
- model_discovery_test.go
- model_download_manager_test.go
- model_manager_test.go
- ollama_provider_test.go
- openai_compatible_provider_test.go
- openai_provider_test.go
- openrouter_provider_test.go
- provider_features_test.go
- provider_registry_test.go
- qwen_oauth_test.go
- qwen_provider_test.go
- reasoning_test.go
- token_budget_test.go
- tool_provider_test.go
- usage_analytics_test.go
- vertexai_provider_test.go
- xai_provider_test.go

10.3 Challenge System

Location: challenges/scripts/ [22](#)

Files: - host_no_auto_suspend_challenge.sh - CONST-033 host power management guard - no_suspend_calls_challenge.sh - No suspend calls challenge

E2E Tests: tests/e2e/challenges/ with run_all_challenges.sh

Test Runner: cd tests/e2e/challenges && go run cmd/runner/main.go -all

11. Dependencies Analysis

11.1 go.mod [3](#)

Module: dev.helix.code

Go Version: 1.24.0 (toolchain go1.24.9)

Key Dependencies:

Dependency	Version	Purpose
github.com/gin-gonic/gin	v1.11.0	HTTP web framework
github.com/spf13/cobra	v1.8.0	CLI framework (claimed but NOT used in current CLI)
github.com/spf13/viper	v1.21.0	Configuration management
github.com/golang-jwt/jwt/v4	v4.5.2	JWT authentication
github.com/jackc/pgx/v5	v5.7.6	PostgreSQL driver
github.com/redis/go-redis/v9	v9.17.2	Redis client
github.com/stretchr/testify	v1.11.1	Testing framework
fyne.io/fyne/v2	v2.7.0	Desktop GUI
github.com/rivo/tview	v0.42.0	Terminal UI
	v0.14.2	Browser automation

Dependency	Version	Purpose
github.com/chromedp/chromedp		
github.com/PuerkitoBio/goquery	v1.10.3	Web scraping
github.com/smacker/go-tree-sitter	-	Parsing
github.com/gorilla/websocket	v1.5.3	WebSocket
github.com/google/uuid	v1.6.0	UUID generation
github.com/fatih/color	v1.18.0	Terminal colors
golang.org/x/crypto	v0.46.0	Cryptography
github.com/hashicorp/golang-lru/v2	v2.0.7	LRU cache

Cloud Provider SDKs: | Dependency | Version | Purpose | |—————|—————|
 ————| | github.com/Azure/azure-sdk-for-go/sdk/azcore | v1.16.0 |
 Azure core | | github.com/Azure/azure-sdk-for-go/sdk/azidentity |
 v1.8.0 | Azure identity | | github.com/aws/aws-sdk-go-v2 | v1.32.7 | AWS
 SDK | | github.com/aws/aws-sdk-go-v2/service/bedrockruntime | v1.23.1
 | AWS Bedrock | | google.golang.org/grpc | v1.76.0 | gRPC | | golang.org/
 x/oauth2 | v0.32.0 | OAuth2 |

Memory Integrations: | Dependency | Version | Purpose | |—————|—————|
 ————| | github.com/getzep/zep-go/v3 | v3.10.0 | Zep memory |

12. Integration Points

12.1 LLM Provider Integration

Current Integration Model: - Provider interface at helix_code/internal/llm/missing_types.go - Factory pattern at helix_code/internal/llm/factory.go - Each provider is a separate Go file implementing the Provider interface - 35+ provider types supported

Provider Registration Flow: 1. Config specifies ProviderConfigEntry (type, endpoint, API key, models, enabled, parameters) 2. InitializeModelManager(configs) iterates enabled configs 3. NewProvider(config) dispatches to provider constructor 4. manager.RegisterProvider(provider) adds to registry

12.2 Database Integration

- PostgreSQL 15+ (optional - empty host disables)
- 11 tables for distributed computing
- Schema initialization in internal/database
- Redis 7+ for caching (optional)

12.3 Worker Pool Integration

- SSH-based worker pool
- Health monitoring
- Auto-installation capability
- Cross-platform workers

12.4 MCP Protocol

- Model Context Protocol implementation
- Multi-transport support

12.5 Notification System

- Multi-channel: Slack, Discord, Email, Telegram
- Notification engine with priority levels

13. Docker Architecture

13.1 docker-compose.yml [36](#)

Services: | Service | Image | Ports | Purpose | |———|———|———|———| |
helixcode-server | Build from Dockerfile | 8080, 2222 | Main application | |
postgres | postgres:15 | 5432 | Database | | redis | redis:7-alpine | 6379 | Cache |
| nginx | nginx:alpine | 80, 443 | Reverse proxy | | prometheus | prom/
prometheus:latest | 9090 | Monitoring | | grafana | grafana/grafana:latest | 3000
| Dashboards |

Health Checks: - Server: curl -f http://localhost:8080/health -
PostgreSQL: pg_isready -U helix - Redis: redis-cli --raw incr ping

14. Gaps, TODOs, and Incomplete Features

14.1 Critical Gaps (P0)

1. **BLUFF-001: LLM Generation Simulated** (file: `cmd/cli/main.go.old`)
 - Current `cmd/cli/main.go` has REAL implementation calling `provider.Generate()`
 - Status: **PARTIALLY FIXED** in current `main.go`, but old file still exists with simulated code
2. **BLUFF-002: Model Listing Hardcoded**
 - `cmd/cli/main.go` lines 101-128: Only 3 hardcoded models
 - Does NOT use `modelManager.GetAvailableModels()` or `provider.GetModels()`
 - Status: **NOT FIXED** - P0 critical
3. **BLUFF-003: Command Execution Simulated**
 - `cmd/cli/main.go` lines 237-250: `time.Sleep(1 * time.Second)` simulating execution
 - Status: **NOT FIXED** - P0 critical
4. **CLI Uses `flag` NOT Cobra**
 - `AGENTS.md` claims Cobra v1.8.0 is the CLI framework
 - Actual `cmd/cli/main.go` uses Go standard `flag` package
 - `cmd/cli/main.go.old` also does NOT use Cobra
 - Status: **DISCREPANCY** - either docs are wrong or Cobra integration is missing
5. **Model Discovery Not Connected to CLI**
 - `ModelDiscoveryEngine` exists with sophisticated scoring
 - `CLI handleListModels` does NOT use it
 - `handleGenerate` does NOT use `SelectOptimalModel()`
 - Status: **NOT INTEGRATED**
6. **No Dynamic Model Source**
 - Models are hardcoded in `model_discovery.go` (`fetchExternalModels`)
 - No external API integration for fetching current model lists
 - No integration with model registries (HuggingFace, Ollama library, etc.)
 - Status: **HARDCODED MODELS ONLY**

14.2 Medium Gaps

1. **Old CLI File Still Present**
 - `cmd/cli/main.go.old` contains simulated code and TODOs
 - Should be removed or archived
2. **Missing Provider Implementations**
 - 35+ provider types defined in `missing_types.go`
 - Only ~15 have actual implementation files visible

- Some may be stubs

3. Hardware Detection Integration

- `hardware.Detector` exists
- `ModelManager.calculateHardwareCompatibility()` calls it
- But CLI does NOT use hardware-aware model selection

4. No LLMsVerifier Integration

- No references to “LLMsVerifier”, “verifier”, or external model verification service
- No API client for fetching verified model lists
- Status: **NOT PRESENT**

15. Current Model Management Approach Summary

15.1 How Models Are Currently Managed

1. **ProviderConfigEntry** structs define which providers are enabled and which models they serve
2. **Factory** creates provider instances based on type
3. **ModelManager** registers providers and maintains a `modelRegistry` map keyed by `providerType:modelName`
4. **ModelDiscoveryEngine** provides recommendations but with hardcoded external models
5. **CrossProviderRegistry** tracks downloaded models and provider compatibility
6. **AliasManager** handles fuzzy model name resolution

15.2 Model Sources

Source	Status	Location
Provider discovery (Ollama /api/tags)	REAL	<code>ollama_provider.go:discoverModels()</code>
CrossProviderRegistry downloaded models	PERSISTED	<code>~/.helixcode/local-llm/registry.json</code>
Hardcoded external models	HARDCODED	<code>model_discovery.go:fetchExternalModels()</code>
CLI list models	HARDCODED	<code>cmd/cli/main.go:handleListModels()</code>

15.3 API Keys / Credentials

- Stored in environment variables (prefix `HELIX_`)

- Configured via `ProviderConfigEntry.APIKey`
 - No secret management beyond env vars (no Vault integration visible)
-

16. Recommendations for LLMsVerifier Integration

16.1 Integration Points

To integrate LLMsVerifier as the single source of truth for models, the following integration points should be targeted:

1. **`internal/llm/model_discovery.go:fetchExternalModels()`**
 - Replace hardcoded model list with LLMsVerifier API call
 - Add LLMsVerifier client to fetch verified models
2. **`internal/llm/model_manager.go:SelectOptimalModel()`**
 - Augment with LLMsVerifier verification status
 - Filter out unverified models or warn about them
3. **`cmd/cli/main.go:handleListModels()`**
 - Replace hardcoded list with dynamic fetch from LLMsVerifier
 - Show verification status for each model
4. **`internal/llm/factory.go:NewProvider()`**
 - Add LLMsVerifier-aware provider initialization
 - Validate provider endpoints against LLMsVerifier registry
5. **New Package: `internal/llm/verifier/`**
 - Create dedicated LLMsVerifier client package
 - Handle API authentication, caching, error handling

16.2 Files to Modify

File	Lines	Change
<code>cmd/cli/main.go</code>	101-128	Replace hardcoded models with LLMsVerifier fetch
<code>internal/llm/model_discovery.go</code>	~900+	Replace <code>fetchExternalModels()</code> hardcoded list
<code>internal/llm/model_manager.go</code>	~280	Add LLMsVerifier status to model scoring
<code>internal/config/config.go</code>	~100	Add LLMsVerifier configuration section
<code>helix_code/go.mod</code>	-	Add LLMsVerifier client dependency

17. Citations

Analysis compiled from exhaustive repository exploration. All file paths are relative to repository root unless otherwise specified.

[End of Section 1]

LLMsVerifier Repository — Deep Analysis Report

Repository: <https://github.com/vasic-digital/LLMsVerifier>
Branch: main
Primary Language: Go (75.3%)
Analysis Date: 2026
Total Commits: ~519
Contributors: milos85vasic, claude

1. Repository Structure

Root-Level Directories

Directory	Purpose
llm-verifier/	Main Go application — core engine, CLI, API server, providers, scoring, verification
internal/	Internal packages: benchmark/, llmops/, messaging/, rag/, selfimprove/
configs/	Configuration files and schemas
docs/	Documentation (governance, architecture, design docs)
examples/	Usage examples (e.g., examples/scoring/)
tests/	Test suites (unit, integration, e2e, performance, security)
challenges/	Challenge system scripts and specifications
sdk/	SDK for client integrations
helm/	Helm charts for Kubernetes deployment

Directory	Purpose
k8s/	Kubernetes manifests
monitoring/	Monitoring and observability configs
website/	Website source
mobile/flutter/	Mobile application (Flutter)
assets/	Logo and visual assets
upstreams/	Upstream repository management
reports/	Generated reports storage
scripts/	Utility scripts
specs/	Specifications (e.g., 001-extend-llm-providers)
test_results/	Test result outputs
video-course/	Video course assets
backup/	Backup data

llm-verifier/ Subdirectory Structure

```

llm-verifier/
  cmd/main.go          # CLI entry point (~1633 lines)
  ai/                  # AI-related modules
  analytics/           # Analytics and metrics
  api/                 # REST API server (gin-based)
  api_keys/            # API key tracking and management
  auth/                # OAuth, JWT, LDAP authentication
  bigdata/             # Big Data integration
  capabilities/        # Capability detection
  challenges/          # Challenge system
  client/              # HTTP client for API communication
  config/              # Configuration structs and loading
  database/            # SQLite persistence layer
  desktop/             # Desktop application code
  docs/               # Package-level docs
  enhanced/           # Enhanced features
  events/              # Event system
  failover/           # Failover logic
  k8s/                # K8s-specific code
  llmverifier/         # Core verification engine
  recipes/             # Verification recipes
  analytics.go
  config_loader.go
  config_export.go

```

issue_detector.go	
llm_client.go	# LLM HTTP client
models.go	# Data models
reporter.go	# Report generation
strategy.go	# Scoring strategy interface
strategy_builder.go	
strategy_default.go	
logging/	# Logging infrastructure
messaging/	# Kafka/RabbitMQ integration
mobile/	# Mobile app helpers
monitoring/	# Monitoring hooks
multimodal/	# Multimodal capabilities
notifications/	# Slack, Email, Telegram, Matrix, WhatsApp
partners/	# Partner integrations
performance/	# Performance testing
pkg/	# Shared packages
providers/	# Provider adapters
base.go	# BaseAdapter struct
anthropic.go	# Anthropic (Claude) adapter
cerebras.go	
cloudflare.go	
cohere.go	
deepseek.go	
groq.go	
hyperbolic.go	
kilo.go	
kimi.go	
kimicode.go	
mistral.go	
modal.go	
model_provider_service.go	
errors.go	
fallback_models.go	
http_client.go	
config.go	
config_validator.go	
scheduler/	# Verification scheduling
scoring/	# Scoring system
scoring_engine.go	# Core scoring logic
types.go	# Score data structures
metrics_collector.go	
alert_manager.go	
api_handlers.go	
database_integration.go	
model_display.go	
model_naming.go	
monitoring.go	
scripts/	# Build and deploy scripts
sdk/	# SDK packages
security/	# Security modules
suffix/	# Suffix handling
test_exports/	# Test exports

```
testing/                # Testing utilities
testsuite/              # Test suite runner
tui/                    # Terminal UI (bubbletea)
verification/           # Verification logic
  verification.go        # Main verifier (stub/placeholder)
  code_verification.go
  code_verification_integration.go
  coding_capability_verification.go
  models_dev_enhanced.go
  provider_client.go
  provider_service_interface.go
  verification_real.go
web/                     # Web UI components
```

2. README & Documentation

File: `llm-verifier/README.md` (400 lines) [18](#)

Purpose

“LLM Verifier is a comprehensive tool to verify, test, and benchmark LLMs based on their coding capabilities and other features.”

Supported Providers (12 Implemented Adapters)

1. **OpenAI** — GPT models with full API compatibility
2. **Anthropic** — Claude models via official API
3. **DeepSeek** — DeepSeek models with streaming support
4. **Groq** — Fast inference with Llama models
5. **Together AI** — Wide range of open-source models
6. **Mistral** — European provider with advanced models
7. **xAI** — Grok models from xAI
8. **Replicate** — Model hosting and deployment platform
9. **Cohere** — Command models with enterprise features
10. **Cerebras** — High-performance inference
11. **Cloudflare Workers AI** — Edge AI inference
12. **SiliconFlow** — Chinese AI models and services

Features

- Model Discovery (auto-discover from API endpoints)
- Comprehensive Testing (existence, responsiveness, overload, capabilities)
- Feature Detection (tool calling, embeddings, code generation, etc.)
- Coding Assessment (multiple programming languages)

- Performance Scoring (code capability, responsiveness, reliability, feature richness)
- Reporting (markdown + JSON)
- Rankings (by strength, speed, reliability, etc.)

CLI Commands

```

llm-verifier models list [--filter NAME] [--limit N] [--format
    json|table]
llm-verifier models get MODEL_ID
llm-verifier models create PROVIDER_ID MODEL_ID NAME
llm-verifier models verify MODEL_ID
llm-
    verifier providers list [--filter NAME] [--limit N] [--
    format json|table]
llm-verifier results list [--filter MODEL_NAME] [--limit N] [--
    format json|table]
llm-verifier pricing list [--format json|table]
llm-verifier limits list [--format json|table]
llm-verifier issues list [--format json|table]
llm-verifier events list [--format json|table]
llm-verifier schedules list [--format json|table]
llm-verifier exports list [--format json|table]
llm-verifier logs list [--format json|table]
llm-verifier config show
llm-verifier config export FORMAT
llm-verifier users create USERNAME PASSWORD EMAIL [FULL_NAME]
llm-verifier tui # Terminal UI
llm-verifier server [--port PORT] # REST API server
llm-verifier ai-config export [format] [output_file]
llm-verifier ai-config bulk [output_directory]

```

Scoring System Weights (from README)

- **Code Capability (40%)**
- **Responsiveness (20%)**
- **Reliability (20%)**
- **Feature Richness (15%)**
- **Value Proposition (5%)**

3. Core Verification Engine

Entry Point

File: `llm-verifier/cmd/main.go` (line 84-106) ^[19]

```

func runVerification() error {
    cfg, err := llmverifier.LoadConfig(configFile)
    verifier := llmverifier.New(cfg)
    results, err := verifier.Verify()
    verifier.GenerateMarkdownReport(results, outputDir)
    verifier.GenerateJSONReport(results, outputDir)
}

```

Main Verifier

File: llm-verifier/verification/verification.go (130 lines) [28](#)

CRITICAL FINDING: The core verification.go is currently a **stub/placeholder implementation** that returns hardcoded scores:

```

func (v *Verifier) Verify(ctx context.Context, req *Request)
    (*database.VerificationResult, error) {
    // ... validation ...
    result := &database.VerificationResult{
        ID: time.Now().UnixNano(),
        ModelID: 1, // Placeholder
        Status: "completed",
        ModelExists: boolPtr(true),
        Responsive: boolPtr(true),
        Overloaded: boolPtr(false),
        SupportsToolUse: true,
        SupportsCodeGeneration: true,
        // ... ALL fields hardcoded to true or 8.5 ...
        OverallScore: 8.5,
        CodeCapabilityScore: 8.5,
        ResponsivenessScore: 8.5,
        ReliabilityScore: 8.5,
        FeatureRichnessScore: 8.5,
        ValuePropositionScore: 8.5,
    }
    return result, nil
}

```

NOTE: The actual verification logic for coding capabilities appears to be in verification/coding_capability_verification.go which performs real LLM API calls with keyword matching.

Coding Capability Verification

File: `llm-verifier/verification/coding_capability_verification.go`
(~450 lines) [28](#)

- Tests 4 dimensions: **Codebase Detection, Language Detection, Code Generation, Code Analysis**
 - Uses keyword matching against expected terms
 - Threshold: ≥ 0.6 readiness score for “ReadyForCoding”
 - Status mapping: $\geq 0.7 \rightarrow$ “verified”, $\geq 0.4 \rightarrow$ “partial”, else “failed”
 - Makes real LLM API calls via `ProviderClientInterface`
-

4. Provider Support

Base Adapter

File: `llm-verifier/providers/base.go` (62 lines) [\[27\]](#)

```
type BaseAdapter struct {  
    client    *http.Client  
    endpoint  string  
    apiKey    string  
    headers   map[string]string  
}
```

Provider Files in providers/

File	Provider	Auth
<code>anthropic.go</code>	Anthropic (Claude)	API Key + OAuth
<code>deepseek.go</code>	DeepSeek	API Key
<code>groq.go</code>	Groq	API Key
<code>cerebras.go</code>	Cerebras	API Key
<code>cloudflare.go</code>	Cloudflare Workers AI	API Key
<code>cohere.go</code>	Cohere	API Key
<code>hyperbolic.go</code>	Hyperbolic	API Key
<code>kilo.go</code>	Kilo	API Key
<code>kimi.go</code>	Kimi	API Key
<code>kimicode.go</code>	Kimi Code	API Key
<code>mistral.go</code>	Mistral	API Key

File	Provider	Auth
modal.go	Modal	API Key

File: providers/anthropic.go (excerpt, lines 26-78) ^[27]

```

type AnthropicAdapter struct {
    BaseAdapter
    authType      AuthType
    oauthCredReader *auth.OAuthCredentialReader
}
// Supports both API Key (x-api-key) and OAuth (Bearer token)
// Header: "anthropic-version": "2023-06-01"

```

Provider Configuration

File: providers/config.go — defines provider metadata, endpoints, rate limits

File: providers/fallback_models.go — fallback model lists when APIs fail

File: providers/errors.go — provider-specific error types
(ProviderInitError, etc.)

How New Providers Are Added

1. Create a new file in providers/ (e.g., newprovider.go)
2. Embed BaseAdapter
3. Implement provider-specific request/response conversion
4. Add to provider registry/service
5. Add fallback models to fallback_models.go

5. Model Management

Model Data Structure

File: llm-verifier/llmverifier/models.go (excerpt, lines 6-55) [40](#)

```

type VerificationResult struct {
    ModelInfo      ModelInfo
    `json:"model_info"`
    Availability    AvailabilityResult
    `json:"availability"`
    ResponseTime    ResponseTimeResult
    `json:"response_time"`
    FeatureDetection FeatureDetectionResult
    `json:"feature_detection"`
}

```



```

CodeCapabilities          CodeCapabilityResult
    `json:"code_capabilities"`
GenerativeCapabilities    GenerativeCapabilityResult
    `json:"generative_capabilities,omitempty"`
PerformanceScores        PerformanceScore
    `json:"performance_scores"`
Timestamp                time.Time
    `json:"timestamp"`
Error                    string
    `json:"error,omitempty"`
ScoreDetails             ScoreDetails
    `json:"score_details"`
}

type ModelInfo struct {
    ID                string        `json:"id"`
    Object            string        `json:"object"`
    Created           int64         `json:"created"`
    OwnedBy          string        `json:"owned_by"`
    Root             string        `json:"root,omitempty"`
    Parent           string        `json:"parent,omitempty"`
    Permissions       []Permission
    `json:"permissions,omitempty"`
    ScalingPolicy     *ScalingPolicy
    `json:"scaling_policy,omitempty"`
    Capabilities      Capabilities
    `json:"capabilities,omitempty"`
    ContextWindow     ContextWindow
    `json:"context_window,omitempty"`
    MaxOutputTokens   int
    `json:"max_output_tokens,omitempty"`
    InputPrices       InputPrices
    `json:"input_prices,omitempty"`
    OutputPrices      OutputPrices
    `json:"output_prices,omitempty"`
    HasTrainingData   bool
    `json:"has_training_data,omitempty"`
    Description       string
    `json:"description,omitempty"`
    Architecture      Architecture
    `json:"architecture,omitempty"`
    Tokenizer         string        `json:"tokenizer,omitempty"`
    Organization      string
    `json:"organization,omitempty"`
    ReleaseDate       string
    `json:"release_date,omitempty"`
    LanguageSupport   []string
    `json:"language_support,omitempty"`
}

```

```

UseCase            string           `json:"use_case,omitempty"`
Version            string           `json:"version,omitempty"`
MaxInputTokens     int              `json:"max_input_tokens,omitempty"`
SupportsVision     bool            `json:"supports_vision,omitempty"`
SupportsAudio      bool            `json:"supports_audio,omitempty"`
SupportsVideo      bool            `json:"supports_video,omitempty"`
SupportsReasoning  bool            `json:"supports_reasoning,omitempty"`
SupportsHTTP3      bool            `json:"supports_http3,omitempty"`
SupportsToon       bool            `json:"supports_toon,omitempty"`
SupportsBrotli     bool            `json:"supports_brotli,omitempty"`
OpenSource         bool            `json:"open_source,omitempty"`
Deprecated         bool            `json:"deprecated,omitempty"`
Tags               []string        `json:"tags,omitempty"`
Endpoint           string           `json:"endpoint"`
}

```

Database Model Schema

File: llm-verifier/database/database.go (lines 444-479) [28](#)

```

CREATE TABLE IF NOT EXISTS models (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  provider_id INTEGER NOT NULL,
  model_id TEXT NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  version TEXT,
  architecture TEXT,
  parameter_count INTEGER,
  context_window_tokens INTEGER,
  max_output_tokens INTEGER,
  training_data_cutoff DATE,
  release_date DATE,
  is_multimodal BOOLEAN DEFAULT 0,
  supports_vision BOOLEAN DEFAULT 0,
  supports_audio BOOLEAN DEFAULT 0,
  supports_video BOOLEAN DEFAULT 0,
  supports_reasoning BOOLEAN DEFAULT 0,

```

```

open_source BOOLEAN DEFAULT 0,
deprecated BOOLEAN DEFAULT 0,
tags TEXT,
language_support TEXT,
use_case TEXT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
last_verified TIMESTAMP,
verification_status TEXT DEFAULT 'pending',
overall_score REAL DEFAULT 0.0,
code_capability_score REAL DEFAULT 0.0,
responsiveness_score REAL DEFAULT 0.0,
reliability_score REAL DEFAULT 0.0,
feature_richness_score REAL DEFAULT 0.0,
value_proposition_score REAL DEFAULT 0.0,
FOREIGN KEY (provider_id) REFERENCES providers(id) ON DELETE
    CASCADE
);

```

Model Discovery

- `llmverifier/llm_client.go` — `ListModels()` calls `/models` endpoint on provider
 - Returns `[]ModelInfo` with full metadata
-

6. Rate Limiting & Cooldown

Configuration

File: `llm-verifier/config/config.go` (lines 139-158) [28](#)

```

type APIConfig struct {
    Port          string `mapstructure:"port"`
    JWTSecret      string `mapstructure:"jwt_secret"`
    RateLimit      int    `mapstructure:"rate_limit" // requests per
                        minute`
    BurstLimit     int    `mapstructure:"burst_limit"`
    RateLimitWindow int    `mapstructure:"rate_limit_window" // seconds`
    EnableCORS     bool   `mapstructure:"enable_cors"`
    RateLimitByAPIKey bool   `mapstructure:"rate_limit_by_api_key"`
    // ... TLS, timeouts ...
}

```

Database Limits Table

File: llm-verifier/database/database.go (lines 499-512)

```
CREATE TABLE IF NOT EXISTS limits (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    model_id INTEGER NOT NULL,  
    limit_type TEXT NOT NULL,  
    limit_value INTEGER NOT NULL,  
    current_usage INTEGER DEFAULT 0,  
    reset_period TEXT,  
    reset_time TIMESTAMP,  
    is_hard_limit BOOLEAN DEFAULT 1,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (model_id) REFERENCES models(id) ON DELETE  
        CASCADE  
);
```

Provider-Level Rate Limiting

File: llm-verifier/providers/config.go — contains ProviderConfig with RateLimit, RateLimitWindow, RetryPolicy

Global Request Delay

File: llm-verifier/config.yaml (line 14): request_delay: 1s

7. Token Tracking & Pricing

Pricing Table

File: llm-verifier/database/database.go (lines 481-497)

```
CREATE TABLE IF NOT EXISTS pricing (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    model_id INTEGER NOT NULL,  
    input_token_cost REAL DEFAULT 0.0,  
    output_token_cost REAL DEFAULT 0.0,  
    cached_input_token_cost REAL DEFAULT 0.0,  
    storage_cost REAL DEFAULT 0.0,  
    request_cost REAL DEFAULT 0.0,  
    currency TEXT DEFAULT 'USD',  
    pricing_model TEXT DEFAULT 'per_token',  
    effective_from DATE,  
    effective_to DATE,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (model_id) REFERENCES models(id) ON DELETE  
        CASCADE  
);
```

Token Usage Tracking

File: llm-verifier/llmverifier/llm_client.go (lines 114-119)

```
type Usage struct {  
    PromptTokens      int `json:"prompt_tokens"`  
    CompletionTokens int `json:"completion_tokens"`  
    TotalTokens       int `json:"total_tokens"`  
}
```

Cost Effectiveness Scoring

File: llm-verifier/scoring/scoring_engine.go (lines 223-264) ^[41]

- Input token cost < 1.0 → +2.0 score
- Input token cost < 5.0 → +1.0 score
- Input token cost > 15.0 → -2.0 score
- Open source models → +2.0 score

8. Real-time Updates

Finding: The repository does **not** appear to have a webhook or push-based update system. Updates appear to be:

1. **Polling-based:** The TUI auto-refreshes every 30-60 seconds
2. **CLI-triggered:** llm-verifier models verify MODEL_ID initiates verification on-demand
3. **Scheduled:** llm-verifier schedules list suggests scheduled verifications exist
4. **Manual refresh:** TUI supports r / R key for manual refresh

File: llm-verifier/cmd/main.go (line 63-66): schedulesCmd() for schedule management

9. API / Interface

REST API Server

File: `llm-verifier/api/server.go` (70 lines, currently minimal) [28](#)

```
type Server struct {
    config    *config.Config
    database  *database.Database
    server    *http.Server
}

// Endpoints registered:
//   /api/health          -> HealthHandler
//   /api/models          -> ListModelsHandler
//   /api/models/{id}     -> GetModelHandler
//   /api/models/{id}/verify -> VerifyModelHandler
//   /api/providers       -> ProvidersHandler
```

CRITICAL GAP: The `api/server.go` is **extremely minimal** — only 5 endpoints. The full API surface is much larger (models CRUD, providers, results, pricing, limits, issues, events, schedules, exports, logs, config, users, batch, tui). These likely exist in other files under `api/` (e.g., `api/handlers.go`, `api/routes.go`) but the current `server.go` only wires a basic mux.

CLI Client

File: `llm-verifier/client/client.go` — HTTP client that communicates with the REST API server

Data Export Formats

- **Markdown:** `llm_verification_report.md`
 - **JSON:** `llm_verification_report.json`
 - **AI CLI formats:** `opencode`, `crush`, `claude-code` configurations
-

10. Configuration System

Configuration File

File: `llm-verifier/config.yaml` (18 lines) ^[17]

```
api:
  enable_cors: true
```

```

    jwt_secret: your-secret-key-change-in-production
    port: '8080'
    rate_limit: 100
concurrency: 5
database:
    encryption_key: ''
    path: ./llm-verifier.db
global:
    api_key: ${OPENAI_API_KEY}
    base_url: https://api.openai.com/v1
    max_retries: 3
    request_delay: 1s
    timeout: 30s
llms: []
timeout: 60s

```

Config Struct

File: llm-verifier/config/config.go (lines 97-110) [28](#)

```

type Config struct {
    Profile      string                `mapstructure:"profile"`
    LLMs         []LLMConfig          `mapstructure:"llms"`
    Global       GlobalConfig         `mapstructure:"global"`
    Database     DatabaseConfig       `mapstructure:"database"`
    API          APIConfig            `mapstructure:"api"`
    Concurrency  int                 `mapstructure:"concurrency"`
    Timeout      time.Duration        `mapstructure:"timeout"`
    Logging      LoggingConfig        `mapstructure:"logging"`
    Monitoring   MonitoringConfig     `mapstructure:"monitoring"`
    Security     SecurityConfig       `mapstructure:"security"`
    Notifications NotificationsConfig   `mapstructure:"notifications"`
}

```

Loading

- `config.LoadFromFile(path)` — supports YAML, JSON, TOML via Viper
- Environment variable substitution: `${OPENAI_API_KEY}`

Full Configuration Example

File: llm-verifier/config_full.yaml — comprehensive example with all providers, features, and options

11. Data Storage

SQLite with Optional Encryption

File: `llm-verifier/database/database.go` (1661 lines) [28](#)

- **Driver:** `github.com/mattn/go-sqlite3`
- **Encryption:** SQL Cipher (optional, via `_pragma_key`)
- **WAL mode:** Enabled for concurrency
- **Connection pooling:** Max 25 open, 5 idle, 5min lifetime
- **Migrations:** Built-in migration manager with version tracking

Key Tables

1. `users` — Authentication
 2. `api_keys` — Programmatic access keys
 3. `providers` — Provider metadata and status
 4. `models` — Model definitions and scores
 5. `pricing` — Token pricing per model
 6. `limits` — Rate limits per model
 7. `verification_results` — Full verification outputs
 8. `issues` — Detected issues and workarounds
 9. `events` — System events
 10. `schedules` — Verification schedules
 11. `exports` — Configuration exports
 12. `logs` — System logs
 13. `scoring_events` — Scoring history
-

12. Validation Results Structure

VerificationResult (Database)

File: `llm-verifier/database/database.go` (lines 514-581)

Key fields: - `id`, `model_id`, `verification_type` - `started_at`, `completed_at`, `status` - `model_exists`, `responsive`, `overloaded`, `latency_ms` - Feature flags: `supports_tool_use`, `supports_code_generation`, `supports_embeddings`, `supports_streaming`, `supports_json_mode`, `supports_reasoning`, `supports_parallel_tool_use`, `supports_batch_processing`, `supports_brotli` - Code capabilities: `code_debugging`, `code_optimization`, `test_generation`, `documentation_generation`, `refactoring`, `error_resolution`, `architecture_design`, `security_assessment`, `pattern_recognition` - Scores: `debugging_accuracy`, `code_quality_score`,

logic_correctness_score, runtime_efficiency_score, overall_score,
code_capability_score, responsiveness_score, reliability_score,
feature_richness_score, value_proposition_score - Performance:
avg_latency_ms, p95_latency_ms, min_latency_ms, max_latency_ms,
throughput_rps

13. Dependencies

File: `llm-verifier/go.mod` (172 lines) [16](#)

Module: `digital.vasic.llmsverifier`

Go Version: `1.25.3`

Direct Dependencies

Package	Version	Purpose
<code>github.com/gin-gonic/gin</code>	<code>v1.11.0</code>	HTTP web framework
<code>github.com/spf13/cobra</code>	<code>v1.10.2</code>	CLI framework
<code>github.com/spf13/viper</code>	<code>v1.21.0</code>	Configuration management
<code>github.com/charmbracelet/bubbletea</code>	<code>v1.1.0</code>	TUI framework
<code>github.com/charmbracelet/lipgloss</code>	<code>v0.13.0</code>	TUI styling
<code>github.com/mattn/go-sqlite3</code>	<code>v1.14.32</code>	SQLite driver
<code>github.com/golang-jwt/jwt/v5</code>	<code>v5.3.0</code>	JWT authentication
<code>github.com/go-playground/validator/v10</code>	<code>v10.27.0</code>	Input validation
<code>github.com/google/uuid</code>	<code>v1.6.0</code>	UUID generation

Package	Version	Purpose
github.com/gorilla/websocket	v1.5.3	WebSocket support
github.com/stretchr/testify	v1.11.1	Testing framework
github.com/swaggo/swag	v1.16.6	Swagger docs
golang.org/x/crypto	v0.46.0	Cryptography
golang.org/x/time	v0.14.0	Rate limiting
cloud.google.com/go/storage	v1.58.0	GCS integration
github.com/Azure/azure-sdk-for-go/sdk/storage/azblob	v1.6.3	Azure Blob
github.com/aws/aws-sdk-go-v2	v1.41.0	AWS SDK
github.com/minio/minio-go/v7	v7.0.98	S3-compatible
github.com/andybalholm/brotli	v1.2.0	Brotli compression
github.com/go-ldap/ldap/v3	v3.4.12	LDAP auth
github.com/quic-go/quic-go	v0.54.0	QUIC/HTTP3

External Module Dependency

replace digital.vasic.llmprovider => ../../LLMProvider

CRITICAL: The project depends on a local module `digital.vasic.llmprovider` at `../../LLMProvider`. This is a sibling repository that must be present for the project to build.

14. Test Framework

Test Directory Structure

Directory: `llm-verifier/tests/` ^[44]

Test Type	Files
Unit tests	tests/unit/, database_unit_test.go, core_components_test.go
Integration tests	tests/integration/, integration_test.go, integration_comprehensive_test.go, integration_simple_test.go
E2E tests	e2e_test.go, acp_e2e_test.go
Performance tests	performance_test.go, acp_performance_test.go
Security tests	security_test.go, acp_security_test.go
Automation tests	automation_test.go, acp_automation_test.go
ACP tests	acp_test.go, acp_integration_test.go, etc.

Test Patterns

- Tests use `t.Skip()` with SKIP-OK markers (per CONST-035)
- Mock API server: `tests/mock_api_server.go`
- Test helpers: `tests/test_helpers.go`
- Constants: `tests/test_constants.go`

Provider Tests

- `providers/adapters_test.go`
- `providers/base_extended_test.go`
- `providers/deepseek_extended_test.go`
- `providers/errors_advanced_test.go`
- `providers/fallback_models_test.go`
- `providers/groq_test.go`
- `providers/integration_test.go`
- `providers/model_provider_service_test.go`

Scoring Tests

- `scoring/scoring_engine_test.go`
- `scoring/types_test.go`
- `scoring/metrics_collector_test.go`
- `scoring/alert_manager_test.go`

- scoring/api_handlers_test.go
 - scoring/database_extensions_test.go
 - scoring/database_integration_test.go
 - scoring/integration_simplified_test.go
 - scoring/integration_test.go
 - scoring/model_display_test.go
 - scoring/model_naming_test.go
-

15. Integration Examples

AI CLI Configuration Export

File: llm-verifier/cmd/main.go (lines 556-599)

```
llm-verifier ai-config export opencode config.json
llm-verifier ai-config bulk ./exports/
llm-verifier ai-config validate config.json
```

Client Library

File: llm-verifier/client/client.go — HTTP client with authentication

```
c := client.New(serverURL)
c.Login(username, password)
models, err := c.GetModels()
result, err := c.VerifyModel(modelID)
```

Docker Support

File: llm-verifier/docker-compose.yml - SQLite database persistence -
Memory caps to prevent host crashes

Kubernetes

- helm/llm-verifier/ — Helm chart
 - k8s/ — Raw Kubernetes manifests
 - llm-verifier/k8s/ — K8s-specific Go code
-

16. Critical Findings & Gaps for HelixCode Integration

GAPS — What Needs Extension

1. Core Verification is a Stub

`verification/verification.go` returns hardcoded scores (all 8.5). The real verification happens in `coding_capability_verification.go` but only covers 4 coding tests. For HelixCode to use LLMsVerifier as a “single source of truth,” the verification engine needs to be completed with actual provider API calls for all dimensions.

2. API Server is Minimal

`api/server.go` only has 5 basic endpoints. Full CRUD, batch operations, and real-time streaming are likely in other files but not wired in the main server file.

3. External Module Dependency

`digital.vasic.llmprovider` at `../LLMProvider` is required but not in this repo. HelixCode needs access to the LLMProvider sibling repository.

4. No Real-time Push Notifications

Only polling and scheduled updates exist. For real-time integration, webhooks or SSE/WebSocket-based push would need to be added.

5. Authentication OAuth is Stub-heavy

The `auth/oauth_stub.go` files exist to satisfy builds but may not implement full OAuth flows.

6. Rate Limiting is Config-only

The `limits` table exists but actual enforcement middleware may not be fully implemented in the minimal `api/server.go`.

7. Token Pricing Integration

Pricing data is stored but there’s no evidence of automatic price fetching from provider APIs. Prices appear to be manually configured or sourced from `models.dev`.

8. Scoring Strategy is Extensible

`llmverifier/strategy.go`, `strategy_builder.go`, `strategy_default.go` define a Strategy pattern. This is good for HelixCode — custom scoring strategies can be injected.

STRENGTHS — Ready for Integration

1. **Comprehensive Data Model** — The `models` table and `VerificationResult` struct have ~40+ fields covering all dimensions needed for model evaluation.
 2. **Provider Adapter Pattern** — Clean `BaseAdapter` + per-provider adapters make adding new LLM providers straightforward.
 3. **SQLite + Encryption** — Portable, single-file database with optional SQL Cipher encryption.
 4. **TUI + CLI + REST API** — Three interfaces available; `HelixCode` can use the REST API or embed as a Go library.
 5. **Scoring Engine is Modular** — `ScoringEngine` with configurable weights, batch scoring, and history tracking.
 6. **Report Generation** — Markdown and JSON reports with rankings.
 7. **Configuration Export** — Native support for `opencode`, `crush`, `claude-code` formats.
-

17. Key File Paths Summary

Area	Primary File(s)
CLI Entry	<code>llm-verifier/cmd/main.go</code>
Config Structs	<code>llm-verifier/config/config.go</code>
Config Loader	<code>llm-verifier/llmverifier/config_loader.go</code>
Core Verifier (stub)	<code>llm-verifier/verification/verification.go</code>
Coding Verification	<code>llm-verifier/verification/coding_capability_verification.go</code>
Provider Base	<code>llm-verifier/providers/base.go</code>
Provider Anthropic	<code>llm-verifier/providers/anthropic.go</code>
Provider Config	<code>llm-verifier/providers/config.go</code>
Provider Errors	<code>llm-verifier/providers/errors.go</code>
LLM Client	<code>llm-verifier/llmverifier/llm_client.go</code>
Data Models	<code>llm-verifier/llmverifier/models.go</code>
Reporter	<code>llm-verifier/llmverifier/reporter.go</code>
Scoring Engine	<code>llm-verifier/scoring/scoring_engine.go</code>
Score Types	<code>llm-verifier/scoring/types.go</code>

Area	Primary File(s)
Database	llm-verifier/database/database.go
API Server	llm-verifier/api/server.go
Client Lib	llm-verifier/client/client.go
TUI	llm-verifier/tui/
Tests	llm-verifier/tests/
README	llm-verifier/README.md
go.mod	llm-verifier/go.mod
config.yaml	llm-verifier/config.yaml

Analysis generated from exhaustive repository exploration of <https://github.com/vasic-digital/LLMsVerifier>

[End of Section 2]

HelixAgent Repository: Exhaustive LLMsVerifier Integration Analysis

Reference Implementation Study — How HelixAgent integrates LLMsVerifier, and how HelixCode should replicate it. **Analysis Date:** 2026-04-30
Repository: <https://github.com/HelixDevelopment/HelixAgent>
Branch: main (commit fe3f69e)

Table of Contents

- 1. [Repository Structure](#)
 - 2. [README & Documentation](#)
 - 3. [Constitution / CLAUDE.md / AGENTS.md](#)
 - 4. [LLMsVerifier Integration Code](#)
 - 5. [Configuration System](#)
 - 6. [Model Management via LLMsVerifier](#)
 - 7. [Model Display UX](#)
 - 8. [Provider Management](#)
 - 9. [Enable / Disable Mechanism](#)
 - 10. [Real-Time Updates Handling](#)
 - 11. [Error Handling & Fallbacks](#)
 - 12. [API Key Provisioning](#)
 - 13. [Testing Framework](#)
 - 14. [Architecture Comparison: HelixAgent → HelixCode](#)
 - 15. [All Documentation Files](#)
-

1. Repository Structure

1.1 Top-Level Layout

The repository is a **Go monorepo** with ~60 Git submodules. Key directories at root:

Path	Type	Description
	Directory	Main binary entry point

Path	Type	Description
cmd/ helixagent/		
internal/	Directory	Core implementation (verifier, LLM, handlers, services)
pkg/sdk/	Directory	Multi-language SDKs (Go, Python)
LLMsVerifier/	Submodule	Points to vasic-digital/ LLMsVerifier — the verifier service itself
Models/	Submodule	Points to vasic-digital/Models — model definitions
cli_agents/	Directory	Analysis docs for 20+ CLI agents including HelixCode
configs/	Directory	YAML configuration schemas
docs/	Directory	Comprehensive documentation
tests/	Directory	Test suites and challenge scripts
scripts/	Directory	Automation scripts
challenges/	Directory	Challenge system for validation
challenge- results/	Directory	Stored challenge outputs
HelixLLM/, HelixMemory/, helix_qa/, etc.	Submodules	Supporting modules

1.2 LLMsVerifier-Related Files (Exact Paths)

LLMsVerifier/		# Submodule (external repo)
internal/verifier/		
├─ service.go	(1097 lines)	# Main VerificationService
├─ config.go	(398 lines)	# Config structs & YAML loader
├─ discovery.go	(526 lines)	# ModelDiscoveryService
├─ startup.go	(1873 lines)	# StartupVerifier – provider
├─ provider_types.go	(1043 lines)	# UnifiedProvider, UnifiedModel
├─ scoring.go	(754 lines)	# ScoringService
├─ enhanced_scoring.go	(730 lines)	# EnhancedScoringService (7-c
├─ events.go	(337 lines)	# Event bus integration
├─ health.go	(486 lines)	# Health checking
├─ metrics.go	(389 lines)	# Prometheus metrics
├─ database.go	(532 lines)	# SQLite persistence
├─ subscription_types.go	(202 lines)	# Subscription metadata
├─ subscription_detector.go	(360 lines)	# 3-tier subscription detecti

- └─ provider_access.go (403 lines) # Provider access registry
- └─ rate_limit_headers.go (170 lines) # Rate-limit parsing
- └─ doc.go (151 lines) # Package documentation
- └─ adapters/
 - └─ provider_adapter.go (592 lines)
 - └─ extended_registry.go (564 lines)
 - └─ extended_providers_adapter.go (780 lines)
 - └─ free_adapter.go (1183 lines)
 - └─ oauth_adapter.go (433 lines)
- internal/services/
 - └─ llmsverifier_score_adapter.go (528 lines) # Bridge to ProviderDisc
- internal/handlers/
 - └─ verifier_types.go (6 lines) # Handler types
- pkg/sdk/go/verifier/
 - └─ client.go (385 lines) # Go SDK client
- pkg/sdk/python/helixagent_verifier/
 - └─ __init__.py
 - └─ client.py (462 lines) # Python SDK client
 - └─ models.py
 - └─ exceptions.py
- configs/
 - └─ verifier.yaml (257 lines) # Full verifier config schema
- docs/guides/
 - └─ llms-verifier.md (328 lines) # Integration guide
- docs/integration/
 - └─ LLMSVERIFIER_INTEGRATION_PLAN.md (2423 lines) # Nano-level plan
- docs/verifier/
 - └─ README.md
 - └─ API.md
 - └─ USER_GUIDE.md
 - └─ LLMSVERIFIER_POWER_FEATURES.md
- scripts/
 - └─ run_llms_verifier.sh (925 lines)
- tests/helixllm/
 - └─ llmsverifier_test_suite.sh (531 lines)
- challenges/scripts/
 - └─ llmsverifier_cliagents_challenge.sh (304 lines)
 - └─ llmsverifier_startup_verification_challenge.sh (138 lines)
 - └─ llmsverifier_submodule_smoke_challenge.sh (126 lines)

2. README & Documentation

2.1 README.md LLMsVerifier Mentions

The root README.md (687 lines, 23 KB) references LLMsVerifier in these key places:

- **Line 52:** Dynamic Provider Selection: Real-time verification scores via LLMsVerifier integration
- **Line 57:** AI Debate System: Multi-round debate between providers for consensus (5 positions x 5 LLMs = 25 total)
- **Line 88:** Architecture diagram includes LLMsVerifier as a core component
- **Line 318:** Capability Detection links to LLMsVerifier/docs/CAPABILITY_DETECTION.md

[6](#)

2.2 HELIXLLM_INTEGRATION_SUMMARY.md

Exists at root (HELIXLLM_INTEGRATION_SUMMARY.md) and documents the HelixLLM ↔ HelixAgent integration, which includes verifier scoring hooks.

3. Constitution / CLAUDE.md / AGENTS.md

3.1 CONSTITUTION.md

File: CONSTITUTION.md (494 lines, 28.2 KB)
Version: 1.3.0 (Updated 2026-04-16)

33 mandatory rules across 17 categories. Key rules relevant to LLMsVerifier integration:

ID	Rule	Relevance to Verifier
CONST-001	Comprehensive Decoupling	LLMsVerifier is a separate submodule
CONST-002	100% Test Coverage	All verifier components have *_test.go files
CONST-002a	No Mocks in Production	Verifier uses real API calls, real DB
CONST-003	Comprehensive Challenges	challenges/scripts/ llmsverifier_*_challenge.sh

ID	Rule	Relevance to Verifier
CONST-014	Stress & Integration Tests	llmsverifier_test_suite.sh
CONST-022	Infrastructure Before Tests	Verifier DB must be up before tests run
CONST-029	Concurrency Safety	safe.Store, safe.Slice, atomic.Bool, Pattern Zeta locks
CONST-035	End-User Usability	Anti-bluff policy — verifier detects canned responses

Anti-Bluff Connection: The verifier's `IsCannedErrorResponse()` function (in `internal/verifier/service.go` lines 44-67) directly implements CONST-035's anti-bluff mandate by detecting fake "I cannot assist" responses.

[36](#)

3.2 AGENTS.md

File: `AGENTS.md` (616 lines, 31.6 KB)

Key verifier-related content: - **Lines 131-132:** Performs startup verification using `LLMsVerifier` to form a "Debate Team" of the best 15 LLMs. - Documents the monorepo structure, technology stack, and module boundaries - States that deeper `AGENTS.md` or `CLAUDE.md` files in subdirectories take precedence for files within those subtrees

[42](#)

3.3 CLAUDE.md

File: `CLAUDE.md` (680 lines, 71 KB)

Contains the comprehensive developer guide. The verifier integration follows `CLAUDE.md`'s TLS/security posture (no `InsecureSkipVerify`, `helixLLMTLSConfig()` in `startup.go` lines 49-90 implements this).

[None](#)

4. LLMsVerifier Integration Code

4.1 Architecture Overview

HelixAgent Main Process	
internal/verifier/	
└ VerificationService	→ service.go (1097 lines)
└ ModelDiscoveryService	→ discovery.go (526 lines)
└ ScoringService	→ scoring.go (754 lines)
└ EnhancedScoringService	→ enhanced_scoring.go (730 lines)
└ StartupVerifier	→ startup.go (1873 lines)
└ HealthService	→ health.go (486 lines)
└ EventPublisher	→ events.go (337 lines)
internal/services/	
└ LLMsVerifierScoreAdapter	→ llmsverifier_score_adapter.go
pkg/sdk/go/verifier/	
└ Client	→ client.go (385 lines)
LLMsVerifier/ (Submodule)	
└ Points to vasic-digital/LLMsVerifier.git	

4.2 Core Service: VerificationService

File: internal/verifier/service.go

Struct: VerificationService (lines 95-115)

```
type VerificationService struct {
    config      *Config
    providerFunc func(ctx context.Context, modelID, provider,
        prompt string) (string, error)
    mu          sync.RWMutex
    testMode    bool

    verificationCache *safe.Store[string, *VerificationStatus]
    stats             *VerificationStats
    statsMu          sync.RWMutex
}
```

Key methods: - NewVerificationService(cfg *Config)
*VerificationService — constructor - SetProviderFunc(fn ...) — injects the LLM call function - SetTestMode(enabled bool) — disables quality validation for testing - ValidateResponseQualityWithLatency(content string, latency time.Duration) error — anti-bluff validation (lines 75-95)

- IsCannedErrorResponse(content string) (bool, string) — detects fake error responses (lines 47-67)

4.3 Score Adapter: LLMsVerifierScoreAdapter

File: internal/services/llmsverifier_score_adapter.go

Struct: LLMsVerifierScoreAdapter (lines 30-60)

This is the **critical bridge** between HelixAgent's ProviderDiscovery system and LLMsVerifier's scoring:

```
type LLMsVerifierScoreAdapter struct {
    scoringService *verifier.ScoringService
    verificationSvc *verifier.VerificationService
    providerScores *safe.Store[string, float64]
    modelScores   *safe.Store[string, float64]
    refreshMu      sync.Mutex
    log            *logrus.Logger
    lastRefresh    time.Time
    refreshInterval time.Duration
}
```

Key methods: - NewLLMsVerifierScoreAdapter(scoringService, verificationSvc, log) — constructor with cache initialization from verification results - GetProviderScore(providerType string) (float64, bool) — returns score 0-10 (normalizes from 0-100) - GetModelScore(modelID string) (float64, bool) — returns per-model score - putMaxProviderScore(provider string, score float64) — atomic max-score update using safe.Store.Update - initializeFromVerificationCache() — loads scores from existing verification cache on startup

How HelixCode should replicate it:

→ HelixCode needs an equivalent LLMsVerifierScoreAdapter that wraps the verifier's ScoringService and exposes GetProviderScore() / GetModelScore() to its own provider selection logic.

4.4 Go SDK Client

File: pkg/sdk/go/verifier/client.go

```
type Client struct {
    baseURL    string
    apiKey     string
    httpClient *http.Client
}
```

```

type ClientConfig struct {
    BaseURL      string
    APIKey       string
    Timeout      time.Duration
    HTTPClient   *http.Client
}

// Default base URL: http://localhost:8081
// Default timeout: 30s

func (c *Client) VerifyModel(ctx context.Context, req
    VerificationRequest) (*VerificationResult, error)
func (c *Client) BatchVerify(ctx context.Context, req
    BatchVerifyRequest) (*BatchVerifyResult, error)
func (c *Client) GetProviderScores(ctx context.Context)
    (map[string]float64, error)
func (c *Client) GetModelScores(ctx context.Context)
    (map[string]float64, error)

```

4.5 Python SDK Client

File: pkg/sdk/python/helixagent_verifier/client.py (462 lines)

Implements async/await Python client with: - HelixAgentVerifierClient class
 - Methods: verify_model(), batch_verify(), get_provider_scores(),
 get_model_scores() - Type hints and Pydantic-style models

5. Configuration System

5.1 Config Struct Hierarchy

File: internal/verifier/config.go (398 lines)

```

type Config struct {
    Enabled      bool           `yaml:"enabled"`
    Database     DatabaseConfig `yaml:"database"`
    Verification VerificationConfig `yaml:"verification"`
    Scoring      ScoringConfig  `yaml:"scoring"`
    Health       HealthConfig   `yaml:"health"`
    API          APIConfig      `yaml:"api"`
    Events       EventsConfig   `yaml:"events"`
    Monitoring   MonitoringConfig `yaml:"monitoring"`
    Brotli       BrotliConfig    `yaml:"brotli"`
    Challenges   ChallengesConfig `yaml:"challenges"`
    Scheduling   SchedulingConfig `yaml:"scheduling"`
}

```

5.2 YAML Config Schema (Full)

File: configs/verifier.yaml (257 lines)

```
verifier:
  enabled: true

  database:
    path: "./data/llm-verifier.db"
    encryption_enabled: false
    encryption_key: "${VERIFIER_ENCRYPTION_KEY}"

  verification:
    mandatory_code_check: true
    code_visibility_prompt: "Do you see my code?"
    verification_timeout: 60s
    retry_count: 3
    retry_delay: 5s
    tests:
      - existence
      - responsiveness
      - latency
      - streaming
      - function_calling
      - coding_capability
      - error_detection
      - code_visibility

  scoring:
    weights:
      response_speed: 0.25
      model_efficiency: 0.20
      cost_effectiveness: 0.25
      capability: 0.20
      recency: 0.10
    models_dev_enabled: true
    models_dev_endpoint: "https://api.models.dev"
    cache_ttl: 24h

  health:
    check_interval: 30s
    timeout: 10s
    failure_threshold: 5
    recovery_threshold: 3
    circuit_breaker:
      enabled: true
      half_open_timeout: 60s
```



```
api:
  enabled: true
  port: "8081"
  base_path: "/api/v1/verifier"
  jwt_secret: "${VERIFIER_JWT_SECRET}"
  rate_limit:
    enabled: true
    requests_per_minute: 100

events:
  slack:
    enabled: false
    webhook_url: "${SLACK_WEBHOOK_URL}"
  email:
    enabled: false
    smtp_host: "smtp.gmail.com"
    smtp_port: 587
  telegram:
    enabled: false
    bot_token: "${TELEGRAM_BOT_TOKEN}"
    chat_id: "${TELEGRAM_CHAT_ID}"
  websocket:
    enabled: true
    path: "/ws/verifier/events"

monitoring:
  prometheus:
    enabled: true
    path: "/metrics/verifier"
  grafana:
    enabled: true
    dashboard_path: "./dashboards/verifier"

brotli:
  enabled: true
  http3_support: true
  compression_level: 6

challenges:
  enabled: true
  provider_discovery: true
  model_verification: true
  config_generation: true

scheduling:
  re_verification:
    enabled: true
    interval: 24h
```

```
score_recalculation:
  enabled: true
  interval: 12h
```

5.3 Provider-Specific Config (Embedded in Same YAML)

The same configs/verifier.yaml contains provider-specific sections:

```
providers:
  openai:
    enabled: true
    api_key: "${OPENAI_API_KEY}"
    base_url: "https://api.openai.com/v1"
    models: [gpt-4, gpt-4-turbo, gpt-4o, gpt-3.5-turbo]

  anthropic:
    enabled: true
    api_key: "${ANTHROPIC_API_KEY}"
    base_url: "https://api.anthropic.com/v1"
    models: [claude-3-5-sonnet-20241022, ...]

# ... (google, groq, together, mistral, deepseek, xai,
#       cerebras, cloudflare, siliconflow, replicate, ollama,
#       openrouter)
```

Key insight: Each provider has enabled: true/false — this is the provider-level enable/disable flag.

5.4 Environment Variables

File: .env.example (286 lines)

Key verifier-related env vars:

```
VERIFIER_ENCRYPTION_KEY=
VERIFIER_JWT_SECRET=
SLACK_WEBHOOK_URL=
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
GEMINI_API_KEY=
DEEPSEEK_API_KEY=
GROQ_API_KEY=
MISTRAL_API_KEY=
TOGETHER_API_KEY=
OPENROUTER_API_KEY=
XAI_API_KEY=
CEREBRAS_API_KEY=
```

```
CLOUDFLARE_API_KEY=  
CLOUDFLARE_ACCOUNT_ID=  
SILICONFLOW_API_KEY=  
REPLICATE_API_TOKEN=
```

6. Model Management via LLMsVerifier

6.1 Discovery System

File: internal/verifier/discovery.go (526 lines)

Struct: ModelDiscoveryService

```
type ModelDiscoveryService struct {  
    verificationService *VerificationService  
    scoringService      *ScoringService  
    healthService       *HealthService  
    config              *DiscoveryConfig  
    discoveredModels    *safe.Store[string, *DiscoveredModel]  
    selectedModels      *safe.Slice[*SelectedModel]  
    httpClient          *http.Client  
    stopCh              chan struct{}  
    wg                  sync.WaitGroup  
    stopped             atomic.Bool  
}
```

DiscoveryConfig:

```
type DiscoveryConfig struct {  
    Enabled                bool          `yaml:"enabled"`  
    DiscoveryInterval      time.Duration `yaml:"discovery_interval"`  
    MaxModelsForEnsemble   int          `yaml:"max_models_for_ensemble"`  
    MinScore               float64      `yaml:"min_score"`  
    RequireVerification    bool         `yaml:"require_verification"`  
    RequireCodeVisibility  bool         `yaml:"require_code_visibility"`  
    RequireDiversity       bool         `yaml:"require_diversity"`  
    ProviderPriority       []string     `yaml:"provider_priority"`  
}
```

6.2 StartupVerifier — The Master Orchestrator

File: internal/verifier/startup.go (1873 lines)

Struct: StartupVerifier

This is the **heart of the integration**. It runs a 5-phase pipeline:

```
type StartupVerifier struct {
    config          *StartupConfig
    verifierSvc     *VerificationService
    scoringSvc      *ScoringService
    enhancedScoring *EnhancedScoringService
    subscriptionDetector *SubscriptionDetector
    providerFactory ProviderFactory
    instanceCreator func(providerType, modelID string)
                  llm.LLMProvider
    oauthReader     *oauth_credentials.OAuthCredentialReader
    onVerificationComplete func(ctx context.Context, result
                          *StartupResult) error
    providers       *safe.Store[string, *UnifiedProvider]
    rankedProviders *safe.Slice[*UnifiedProvider]
    debateTeam      *DebateTeamResult
    mu              sync.Mutex // Pattern Zeta for publish
                  barrier
}
```

VerifyAllProviders() pipeline (lines 218-260+): 1. **Phase 1:** Discover all providers (discoverProviders()) 2. **Phase 2:** Verify all providers in parallel (verifyProviders()) 3. **Phase 2.5:** Detect subscriptions (detectSubscriptions()) 4. **Phase 3:** Score all verified providers (scoreProviders()) 5. **Phase 4:** Rank providers by score (rankProviders()) 6. **Phase 5:** Select debate team (selectDebateTeam())

6.3 Unified Data Models

File: internal/verifier/provider_types.go

```
type UnifiedProvider struct {
    ID          string
    Name        string
    DisplayName string
    Type        string
    AuthType    ProviderAuthType // api_key, oauth, free,
                    anonymous, local
    Verified    bool
    Score       float64
    ScoreSuffix string // e.g., "SC:8.5"
    TestResults map[string]bool
}
```

```

CodeVisible bool
Models      []UnifiedModel
DefaultModel string
Status      ProviderStatus // unknown, healthy, degraded,
                        unhealthy, offline
BaseURL     string
APIKey      string          // `json:"-"` – not serialized
Tier        int             // 1=Premium, 2=High-quality,
                        3=Fast, 4=Aggregator, 5=Free
Priority     int
Subscription *SubscriptionInfo
AccessConfig *ProviderAccessConfig
}

type UnifiedModel struct {
    ID          string
    Name        string
    DisplayName string
    Provider     string
    Score        float64
    Verified     bool
    Latency      time.Duration
    ContextWindow int
    MaxOutputTokens int
    SupportsStreaming bool
    SupportsTools    bool
    SupportsFunctions bool
    SupportsVision   bool
    Capabilities     []string
    CostPerInputToken float64
    CostPerOutputToken float64
}

```

6.4 Supported Providers Registry

File: internal/verifier/provider_types.go (lines ~300+)

```

var SupportedProviders = map[string]*ProviderTypeInfo{
    "claude": {
        Type: "claude", DisplayName: "Claude (Anthropic)",
        AuthType: AuthTypeOAuth, Tier: 1, Priority: 1,
        EnvVars: []string{"ANTHROPIC_API_KEY", "CLAUDE_API_KEY"},
        BaseURL: "https://api.anthropic.com/v1/messages",
        Models: []string{"claude-opus-4-6", "claude-sonnet-4-6", ...},
    },
    "gemini": {
        Type: "gemini", DisplayName: "Gemini (Google)",

```

```

    AuthType: AuthTypeAPIKey, Tier: 2, Priority: 2,
    EnvVars: []string{"GEMINI_API_KEY", "GOOGLE_API_KEY",
        "ApiKey_Gemini"},
    BaseURL: "https://generativelanguage.googleapis.com/
        v1beta",
    Models: []string{"gemini-2.5-pro", "gemini-2.5-
        flash", ...},
},
"deepseek": { ... },
"groq": { ... },
"openrouter": { AuthType: AuthTypeAPIKey, Tier: 4, Free:
    true },
"zen": { AuthType: AuthTypeFree, Tier: 5, ... },
// ... (15+ total providers)
}

```

7. Model Display UX

7.1 Score Suffix Format

From provider_types.go and scoring system:

```
ScoreSuffix string `json:"score_suffix,omitempty"` // e.g.,
"SC:8.5"
```

The SC:X.X suffix is appended to model display names in CLI output and web UI. This is generated by the scoring engine.

7.2 Model Display Format (Inferred)

From UnifiedModel and UnifiedProvider structs, the display format includes:

- **Name:** DisplayName or Name
- **Score suffix:** SC:X.X
- **Verification badge:** ✓ if Verified == true
- **Code visibility badge:** if CodeVisible == true
- **Latency indicator:** derived from Latency field
- **Tier indicator:** 1-5 stars based on Tier
- **Cost indicator:** derived from CostPerInputToken / CostPerOutputToken

7.3 Real-Time Updates via WebSocket

From configs/verifier.yaml:

```

events:
  websocket:
    enabled: true
    path: "/ws/verifier/events"

```

The event system publishes verification pipeline events to WebSocket subscribers.

8. Provider Management

8.1 Auth Type System

File: internal/verifier/provider_types.go (lines 12-35)

```
type ProviderAuthType string

const (
    AuthTypeAPIKey      ProviderAuthType = "api_key"
    AuthTypeOAuth        ProviderAuthType = "oauth"
    AuthTypeFree         ProviderAuthType = "free"
    AuthTypeAnonymous    ProviderAuthType = "anonymous"
    AuthTypeLocal        ProviderAuthType = "local"
)
```

8.2 Provider Adapters

Directory: internal/verifier/adapters/

Adapter	File	Purpose
Provider Adapter	provider_adapter.go (592 lines)	Base adapter interface
Extended Registry	extended_registry.go (564 lines)	Extended provider registry
Extended Providers	extended_providers_adapter.go (780 lines)	Extended provider capabilities
Free Adapter	free_adapter.go (1183 lines)	Free-tier provider handling
OAuth Adapter	oauth_adapter.go (433 lines)	OAuth2 token management

8.3 Provider Discovery Flow

File: internal/verifier/startup.go (lines 338-400+)

```
func (sv *StartupVerifier) discoverProviders(ctx
    context.Context) ([]*ProviderDiscoveryResult, error) {
    // 1. Read faulty API keys (to deprioritize)
    faultyKeys, err := api_keys.ReadFaultyAPIKeys()
```

```

// 2. Scan for unsupported API keys in environment
unsupportedKeys, err :=
    api_keys.NewEnvVarScanner().ScanEnvForUnsupportedKeys()

// 3. Sort providers: non-faulty first, then by priority
providerTypes := []string{...}
sort.Slice(providerTypes, func(i, j int) bool {
    // Faulty keys go last
    return getPriority(providerTypes[i]) <
        getPriority(providerTypes[j])
})

// 4. For each provider type, discover via env vars / OAuth /
    auto-detection
//     - API key providers: check env vars
//     - OAuth providers: check token files
//     - Free providers: always discover
//     - Local providers: check localhost endpoints
}

```

9. Enable / Disable Mechanism

9.1 Global Enable/Disable

File: internal/verifier/config.go (line 15)

```

type Config struct {
    Enabled bool `yaml:"enabled"`
    ...
}

```

File: configs/verifier.yaml (line 4)

```

verifier:
    enabled: true

```

When enabled: false, the entire verifier subsystem is bypassed.

9.2 Per-Provider Enable/Disable

File: configs/verifier.yaml (provider sections)

```

providers:
    openai:
        enabled: true
    ...
    xai:

```



```

    enabled: false
    ...
cerebras:
  enabled: false
  ...

```

Each provider in the SupportedProviders map has an enabled flag in the YAML config.

9.3 DiscoveryConfig Enable/Disable

File: internal/verifier/discovery.go (line 39)

```

type DiscoveryConfig struct {
    Enabled bool `yaml:"enabled"`
    ...
}

```

9.4 Health Circuit Breaker

File: internal/verifier/config.go (lines 60-68)

```

type CircuitBreakerConfig struct {
    Enabled bool `yaml:"enabled"`
    HalfOpenTimeout time.Duration `yaml:"half_open_timeout"`
}

```

When a provider fails failure_threshold times (default: 5), the circuit breaker trips and the provider is marked as unhealthy. It recovers after recovery_threshold successes (default: 3).

10. Real-Time Updates Handling

10.1 Event Bus Integration

File: internal/verifier/events.go (337 lines)

Event topics:

```

const (
    TopicVerificationEvents = "helixagent.events.verification"
    TopicProviderDiscovered = "helixagent.events.verification.discovered"
    TopicProviderVerified   = "helixagent.events.verification.verified"
)

```

```

TopicProviderScored      =
  "helixagent.events.verification.scored"
TopicProviderHealthCheck =
  "helixagent.events.verification.health"
TopicDebateTeamSelected  =
  "helixagent.events.verification.debate_team"
TopicVerificationComplete =
  "helixagent.events.verification.complete"
)

```

Event types:

```

const (
  VerificationEventStarted      VerificationEventType =
    "verification.started"
  VerificationEventDiscovered   VerificationEventType =
    "verification.provider.discovered"
  VerificationEventVerified     VerificationEventType =
    "verification.provider.verified"
  VerificationEventFailed       VerificationEventType =
    "verification.provider.failed"
  VerificationEventScored       VerificationEventType =
    "verification.provider.scored"
  VerificationEventRanked       VerificationEventType =
    "verification.provider.ranked"
  VerificationEventHealthCheck  VerificationEventType =
    "verification.provider.health"
  VerificationEventTeamSelected VerificationEventType =
    "verification.debate_team.selected"
  VerificationEventCompleted    VerificationEventType =
    "verification.completed"
)

```

10.2 WebSocket Events

Config: configs/verifier.yaml

```

events:
  websocket:
    enabled: true
    path: "/ws/verifier/events"

```

The WebSocket endpoint pushes real-time verification events to connected clients.

10.3 Scheduling (Re-verification)

File: configs/verifier.yaml

```
scheduling:
  re_verification:
    enabled: true
    interval: 24h
  score_recalculation:
    enabled: true
    interval: 12h
```

Polling-based, not event-driven from the verifier service. HelixAgent polls/re-verifies on a schedule.

10.4 Slack / Telegram / Email Notifications

Config: configs/verifier.yaml

```
events:
  slack: { enabled: false, webhook_url: "${SLACK_WEBHOOK_URL}" }
  email: { enabled: false, smtp_host: "smtp.gmail.com",
    smtp_port: 587 }
  telegram: { enabled: false, bot_token: "${TELEGRAM_BOT_TOKEN}", chat_id: "${TELEGRAM_CHAT_ID}" }
```

11. Error Handling & Fallbacks

11.1 Canned Error Detection

File: internal/verifier/service.go (lines 18-42)

```
var CannedErrorPatterns = []string{
    "unable to provide", "unable to analyze", "unable to
        process",
    "cannot provide", "cannot analyze",
    "i apologize, but i cannot", "i'm sorry, but i cannot",
    "error occurred", "service unavailable", "rate limit",
    "temporarily unavailable", "model not available",
    "failed to generate", "no response generated",
    "internal error", "request failed",
    "at this time", "currently unable", "not able to",
}
```

Models returning these patterns are marked as **NOT verified**.

11.2 Suspiciously Fast Response Detection

File: internal/verifier/service.go (lines 70-73)

```
func IsSuspiciouslyFastResponse(latency time.Duration) bool {
    return latency < 100*time.Millisecond
}
```

Responses under 100ms with <50 chars are flagged as suspicious.

11.3 Circuit Breaker

File: internal/verifier/config.go (lines 60-68)

```
type CircuitBreakerConfig struct {
    Enabled          bool          `yaml:"enabled"`
    HalfOpenTimeout time.Duration `yaml:"half_open_timeout"`
}
```

- failure_threshold: 5 consecutive failures → open circuit
- recovery_threshold: 3 consecutive successes → close circuit
- half_open_timeout: 60s before attempting recovery

11.4 Fallback Strategy

File: internal/verifier/provider_types.go (StartupConfig lines)

```
type StartupConfig struct {
    OAuthPrimaryNonOAuthFallback bool          `yaml:"oauth_primary_non_oauth_fallback"`
    TrustOAuthOnFailure          bool          `yaml:"trust_oauth_on_failure"`
}
```

When an OAuth provider fails, the system can fall back to non-OAuth providers. The debate team selection always includes fallback LLMs per position.

11.5 Faulty API Key Tracking

File: internal/verifier/startup.go (discovery phase)

```
faultyKeys, err := api_keys.ReadFaultyAPIKeys()
```

Providers with faulty API keys are deprioritized in discovery order.

12. API Key Provisioning

12.1 Environment Variable Scanning

File: internal/verifier/startup.go (discovery phase)

The `api_keys` package (from `digital.vasic.llmsverifier/api_keys`) provides:

```
// Scan environment for API keys by provider
type EnvVarScanner struct{}
func (s *EnvVarScanner) ScanEnvForUnsupportedKeys()
    (map[string]string, error)

// Read/write faulty API keys
func ReadFaultyAPIKeys() (map[string]bool, error)
func WriteFaultyAPIKey(keyName, reason string) error

// Get expected env var name for a provider
func GetProviderAPIKeyName(providerType string) string
```

12.2 OAuth Credential Reader

File: `internal/verifier/startup.go`

`oauthReader *oauth_credentials.OAuthCredentialReader`

Reads OAuth tokens from secure storage (keychain/filesystem) for Claude and Qwen providers.

12.3 API Key Redaction

File: `internal/verifier/provider_types.go` (UnifiedProvider)

`APIKey string `json:"- "`` *// Not serialized for security*

API keys are never serialized to JSON.

12.4 Encryption

File: `configs/verifier.yaml`

```
database:
  encryption_enabled: false
  encryption_key: "${VERIFIER_ENCRYPTION_KEY}"
```

SQLite database supports SQL Cipher encryption when enabled.

13. Testing Framework

13.1 Test File Inventory

Test File	Lines	Type
internal/verifier/service_test.go	?	Unit
internal/verifier/config_test.go	?	Unit
internal/verifier/discovery_test.go	?	Unit
internal/verifier/startup_test.go	?	Unit
internal/verifier/ startup_comprehensive_test.go	?	Integration
internal/verifier/scoring_test.go	?	Unit
internal/verifier/enhanced_scoring_test.go	?	Unit
internal/verifier/health_test.go	?	Unit
internal/verifier/events_test.go	?	Unit
internal/verifier/provider_access_test.go	?	Unit
internal/verifier/ subscription_detector_test.go	?	Unit
internal/verifier/subscription_types_test.go	?	Unit
internal/verifier/rate_limit_headers_test.go	?	Unit
internal/verifier/database_test.go	?	Unit
internal/verifier/adapters/*_test.go	?	Unit/ Integration
internal/services/ llmsverifier_score_adapter_test.go	?	Unit
internal/handlers/verifier_types_test.go	?	Unit
pkg/sdk/go/verifier/client_test.go	?	Unit
tests/helixllm/llmsverifier_test_suite.sh	531	Shell / E2E

13.2 Challenge Scripts

Directory: challenges/scripts/

Script	Purpose
llmsverifier_cliagents_challenge.sh (304 lines)	

Script	Purpose
	Validates CLI agent integration
llmsverifier_startup_verification_challenge.sh (138 lines)	Validates startup verification pipeline
llmsverifier_submodule_smoke_challenge.sh (126 lines)	Validates LLMsVerifier submodule health

13.3 Challenge Results

Directory: challenge-results/

Stored JSON results from challenge executions: - llmsverifier-cliagents-challenge/result.json - llmsverifier_startup_verification_challenge results

14. Architecture Comparison: HelixAgent → HelixCode

14.1 What HelixAgent Has That HelixCode Needs

Component	HelixAgent Location	How HelixCode Should Replicate
Verifier Service	internal/verifier/service.go	Create internal/verifier/service.go with VerificationService struct
Score Adapter	internal/services/llmsverifier_score_adapter.go	Create equivalent adapter bridging to HelixCode's provider system
Config Schema	configs/verifier.yaml + internal/verifier/config.go	Copy Config struct and YAML schema
Provider Registry	internal/verifier/provider_types.go + startup.go	Copy SupportedProviders map and discovery logic
	internal/verifier/startup.go	

Component	HelixAgent Location	How HelixCode Should Replicate
Startup Pipeline		Implement StartupVerifier with 5-phase pipeline
Event Bus	internal/verifier/events.go	Integrate with HelixCode's messaging system
Go SDK Client	pkg/sdk/go/verifier/client.go	Embed or import the SDK client
Health Monitor	internal/verifier/health.go	Copy HealthService with circuit breaker
Scoring Engine	internal/verifier/scoring.go	Copy ScoringService with 5-component weights
Model Discovery	internal/verifier/discovery.go	Copy ModelDiscoveryService

14.2 Key Replication Steps

1. Add **LLMsVerifier submodule** to HelixCode repo
2. Copy **internal/verifier/** package structure from HelixAgent
3. Copy **configs/verifier.yaml** and adapt provider list
4. Implement **LLMsVerifierScoreAdapter** to bridge with HelixCode's provider discovery
5. Wire **startup pipeline** into HelixCode's initialization flow
6. Add **event topics** to HelixCode's messaging system
7. Copy **challenge scripts** and adapt for HelixCode's CLI
8. Add **env vars** to HelixCode's `.env.example`

14.3 Critical Differences to Handle

- HelixAgent uses `digital.vasic.concurrency/pkg/safe` for thread-safe containers — HelixCode needs equivalent concurrency primitives
 - HelixAgent uses `logrus` for structured logging — HelixCode may use a different logger
 - HelixAgent's `StartupVerifier` integrates with `llm.LLMProvider` interface — HelixCode needs to map this to its own provider interface
 - HelixAgent has OAuth credential reader for Claude/Qwen — HelixCode needs equivalent OAuth support
-

15. All Documentation Files

15.1 Root-Level Docs

File	Lines	Purpose
README.md	687	Main project documentation
CLAUDE.md	680	Developer guide for Claude AI
AGENTS.md	616	Authoritative guide for AI coding agents
CONSTITUTION.md	494	33 mandatory rules
CONSTITUTION.json	?	Machine-readable constitution
CONTRIBUTING.md	?	Contribution guidelines
SECURITY.md	?	Security policy
CHANGELOG.md	?	Version changelog
HELIXLLM_INTEGRATION_SUMMARY.md	?	HelixLLM integration notes

15.2 Verifier-Specific Docs

File	Lines	Purpose
docs/guides/llms-verifier.md	328	Integration guide
docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md	2423	Nano-level 10-phase integration plan
docs/verifier/README.md	292	Verifier module overview
docs/verifier/API.md	42	API reference
docs/verifier/USER_GUIDE.md	440	End-user guide
docs/verifier/LLMSVERIFIER_POWER_FEATURES.md	661	Power features documentation
docs/archive/status-history/LLMS_VERIFIER_GUIDE.md	?	Historical guide
internal/verifier/README.md	274	Internal module README
internal/verifier/adapters/README.md	170	

File	Lines	Purpose
		Adapters documentation
reports/ HELIXLLM_LLMSVERIFIER_SCORING_REPORT.md	?	Scoring analysis report
docs/reports/llms_verifier/2026-04-03/ report.md	?	Periodic report

15.3 CLI Agent Analysis Docs

Directory: `cli_agents/`

Contains analysis markdown files for 20+ CLI agents: - `helix_code/` — **Submodule** pointing to `HelixDevelopment/HelixCode` - `AGENT_DECK_ANALYSIS.md`, `AIDER_ANALYSIS.md`, `CLINE_ANALYSIS.md`, `CODEX_ANALYSIS.md`, etc. - `MASTER_INTEGRATION_PLAN.md` — Master plan for all CLI agents - `TIER_1_SUMMARY.md`, `TIER_3_4_5_ANALYSIS.md` — Tiered analysis summaries

Appendix A: Exact Code Excerpts for Replication

A.1 Config Loading Pattern

File: `internal/verifier/config.go`

```
func LoadConfig(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, fmt.Errorf("failed to read config: %w", err)
    }
    var cfg Config
    if err := yaml.Unmarshal(data, &cfg); err != nil {
        return nil, fmt.Errorf("failed to parse config: %w", err)
    }
    return &cfg, nil
}
```

A.2 Score Adapter GetPattern

File: `internal/services/llmsverifier_score_adapter.go`

```
func (a *LLMsVerifierScoreAdapter) GetProviderScore(providerType
    string) (float64, bool) {
    score, found := a.providerScores.Get(providerType)
```

```

    if found {
        if score > 10 {
            score = score / 10.0
        }
        return score, true
    }
    return 0, false
}

```

A.3 Event Publishing Pattern

File: internal/verifier/events.go

```

func PublishVerificationEvent(pub messaging.Publisher, event
    *VerificationEvent) error {
    data, err := json.Marshal(event)
    if err != nil {
        return err
    }
    return pub.Publish(event.Type.Topic(), data)
}

```

A.4 Provider Discovery Order (Faulty Key Deprioritization)

File: internal/verifier/startup.go

```

sort.Slice(providerTypes, func(i, j int) bool {
    prioI := getPriority(providerTypes[i])
    prioJ := getPriority(providerTypes[j])
    if prioI != prioJ {
        return prioI < prioJ
    }
    return providerTypes[i] < providerTypes[j]
})

```

Appendix B: Gaps and TODOs Identified

1. **AGENTS.md download intermittent** — Some raw files timeout via GitHub CDN; API access is more reliable
2. **HelixCode submodule** — Is a separate repo; cannot analyze its internal structure from HelixAgent alone
3. **LLMsVerifier submodule** — Is a separate repo; actual verifier service code lives in vasic-digital/LLMsVerifier
4. **Test execution** — Challenge scripts reference Docker Compose stacks that may not be available in all environments

5. **Models submodule** — Points to external repo; model definitions may differ from SupportedProviders map
-

Citations

- [33](#) — Main repository page
 - [6](#) — README.md
 - [36](#) — CONSTITUTION.md
 - [42](#) — AGENTS.md
 - [None](#) — CLAUDE.md
 - All code excerpts verified against raw files via GitHub Contents API at commit fe3f69e
-

End of Analysis

[End of Section 3]

HelixCode ↔ LLMsVerifier Integration Architecture

Version: 1.0.0 **Date:** 2026-05-01 **Author:** Integration Architecture Team
Status: Design Complete — Ready for Implementation

1. Executive Summary

This document specifies the complete integration architecture for making **LLMsVerifier** the single source of truth for model provisioning in **HelixCode**. The design replicates the proven patterns from **HelixAgent**'s `internal/verifier/integration` [33](#), adapted to HelixCode's existing `internal/llm/provider` architecture [15](#).

Core Principle: LLMsVerifier runs as an external service (or embedded goroutine in single-process mode). HelixCode communicates with it via a **REST API client** (not Go module import) to avoid the `digital.vasic.llmprovider` sibling-module dependency hell [16](#). All model listings, provider scores, verification statuses, and pricing flow through this client.

2. Configuration Schema

2.1 Go Struct Additions to `internal/config/config.go`

Modification Point: `helix_code/internal/config/config.go`, after line 253 (after Cognee `*CogneeConfig`)

// Config – existing struct, ADD this field:

```
type Config struct {
    Version      string           `mapstructure:"version"`
    UpdatedBy    string           `mapstructure:"updated_by"`
    Application   ApplicationConfig `mapstructure:"application"`
    Server        ServerConfig      `mapstructure:"server"`
    Database      database.Config    `mapstructure:"database"`
    Redis         RedisConfig        `mapstructure:"redis"`
    Auth          AuthConfig         `mapstructure:"auth"`
    Workers       WorkersConfig      `mapstructure:"workers"`
}
```

```

Tasks      TasksConfig      `mapstructure:"tasks"`
LLM        LLMConfig        `mapstructure:"llm"`
Providers  ProvidersConfig   `mapstructure:"providers"`
Logging    LoggingConfig     `mapstructure:"logging"`
Cognee     *CogneeConfig      `mapstructure:"cognee"`
Verifier    *VerifierConfig   `mapstructure:"verifier"` // ← NEW
HelixAgent *HelixAgentConfig `mapstructure:"helix_agent"` // ← NEW
}

```

New struct definitions (append to internal/config/config.go after existing structs):

```

// VerifierConfig controls LLMsVerifier integration.
type VerifierConfig struct {
    Enabled      bool      `mapstructure:"enabled"`
    Mode         string    `mapstructure:"mode"`           // "remote" | "embedded"
    Endpoint     string    `mapstructure:"endpoint"`       // REST API URL
    APIKey       string    `mapstructure:"api_key"`
    Timeout      time.Duration `mapstructure:"timeout"`
    CacheTTL     time.Duration `mapstructure:"cache_ttl"`
    PollingInterval time.Duration `mapstructure:"polling_interval"`
    Scoring      VerifierScoringConfig
    Health       VerifierHealthConfig
    Events       VerifierEventsConfig
    Providers    map[string]VerifierProviderConfig
}

// VerifierScoringConfig mirrors HelixAgent's scoring weights
// [^33^].
type VerifierScoringConfig struct {
    Weights      ScoringWeights `mapstructure:"weights"`
    ModelsDevEnabled bool          `mapstructure:"models_dev_enabled"`
    ModelsDevEndpoint string       `mapstructure:"models_dev_endpoint"`
}

```

```

    MinAcceptableScore float64
        `mapstructure:"min_acceptable_score"`
}

type ScoringWeights struct {
    CodeCapability float64
        `mapstructure:"code_capability" // default: 0.40
    Responsiveness float64
        `mapstructure:"responsiveness" // default: 0.20
    Reliability float64
        `mapstructure:"reliability" // default: 0.20
    FeatureRichness float64
        `mapstructure:"feature_richness" // default: 0.15
    ValueProposition float64
        `mapstructure:"value_proposition" // default: 0.05
}

// VerifierHealthConfig mirrors HelixAgent's circuit breaker
// [^33^].
type VerifierHealthConfig struct {
    CheckInterval time.Duration
        `mapstructure:"check_interval"`
    Timeout time.Duration
        `mapstructure:"timeout"`
    FailureThreshold int
        `mapstructure:"failure_threshold"`
    RecoveryThreshold int
        `mapstructure:"recovery_threshold"`
    CircuitBreaker CircuitBreakerConfig
        `mapstructure:"circuit_breaker"`
}

type CircuitBreakerConfig struct {
    Enabled bool `mapstructure:"enabled"`
    HalfOpenTimeout time.Duration
        `mapstructure:"half_open_timeout"`
}

// VerifierEventsConfig controls event publishing.
type VerifierEventsConfig struct {
    Enabled bool `mapstructure:"enabled"`
    WebSocket bool `mapstructure:"websocket"`
    WebSocketPath string `mapstructure:"websocket_path"`
}

// VerifierProviderConfig – per-provider override.
type VerifierProviderConfig struct {
    Enabled bool `mapstructure:"enabled"`

```

```

    APIKey      string    `mapstructure:"api_key"`
    BaseURL     string    `mapstructure:"base_url"`
    Models      []string   `mapstructure:"models"`
    Priority     int        `mapstructure:"priority"`
}

// HelixAgentConfig controls HelixAgent submodule integration.
type HelixAgentConfig struct {
    Enabled      bool        `mapstructure:"enabled"`
    Path         string      `mapstructure:"path"`           // submodule path
    AutoStart    bool        `mapstructure:"auto_start"`
    VerifierSync HelixAgentVerifierSync `mapstructure:"verifier_sync"`
}

type HelixAgentVerifierSync struct {
    Enabled      bool        `mapstructure:"enabled"`
    ShareScores  bool        `mapstructure:"share_scores"`
    ShareProviders bool       `mapstructure:"share_providers"`
    SyncInterval time.Duration `mapstructure:"sync_interval"`
}

```

2.2 Default Values Function

Modification Point: helix_code/internal/config/config.go, inside setDefaults() after existing defaults.

```

func setDefaults() {
    // ... existing defaults ...

    // LLMsVerifier defaults
    v.SetDefault("verifier.enabled", false)
    v.SetDefault("verifier.mode", "remote")
    v.SetDefault("verifier.endpoint", "http://localhost:8081")
    v.SetDefault("verifier.timeout", "30s")
    v.SetDefault("verifier.cache_ttl", "5m")
    v.SetDefault("verifier.polling_interval", "60s")

    // Scoring defaults (match LLMsVerifier weights [^18^])
    v.SetDefault("verifier.scoring.weights.code_capability",
        0.40)
    v.SetDefault("verifier.scoring.weights.responsiveness", 0.20)
    v.SetDefault("verifier.scoring.weights.reliability", 0.20)
    v.SetDefault("verifier.scoring.weights.feature_richness",
        0.15)
}

```



```

v.SetDefault("verifier.scoring.weights.value_proposition",
    0.05)
v.SetDefault("verifier.scoring.min_acceptable_score", 6.0)

// Health defaults (match HelixAgent [^33^])
v.SetDefault("verifier.health.check_interval", "30s")
v.SetDefault("verifier.health.timeout", "10s")
v.SetDefault("verifier.health.failure_threshold", 5)
v.SetDefault("verifier.health.recovery_threshold", 3)
v.SetDefault("verifier.health.circuit_breaker.enabled", true)
v.SetDefault("verifier.health.circuit_breaker.half_open_timeout",
    "60s")

// Event defaults
v.SetDefault("verifier.events.enabled", true)
v.SetDefault("verifier.events.websocket", false)
v.SetDefault("verifier.events.websocket_path", "/ws/verifier/
    events")

// HelixAgent defaults
v.SetDefault("helix_agent.enabled", false)
v.SetDefault("helix_agent.path", "./HelixAgent")
v.SetDefault("helix_agent.auto_start", false)
v.SetDefault("helix_agent.verifier_sync.enabled", true)
v.SetDefault("helix_agent.verifier_sync.share_scores", true)
v.SetDefault("helix_agent.verifier_sync.sync_interval", "5m")
}

```

2.3 Environment Variable Bindings

Modification Point: helix_code/internal/config/config.go, after existing explicit env var bindings.

```

// Explicitly bind critical env vars (HelixAgent pattern [^33^])
_ = v.BindEnv("verifier.api_key", "HELIX_VERIFIER_API_KEY")
_ = v.BindEnv("verifier.endpoint", "HELIX_VERIFIER_ENDPOINT")
_ = v.BindEnv("verifier.scoring.models_dev_endpoint",
    "HELIX_MODELS_DEV_ENDPOINT")

// Per-provider API keys (HelixAgent .env.example pattern [^33^])
_ = v.BindEnv("verifier.providers.openai.api_key",
    "OPENAI_API_KEY")
_ = v.BindEnv("verifier.providers.anthropic.api_key",
    "ANTHROPIC_API_KEY")
_ = v.BindEnv("verifier.providers.gemini.api_key",
    "GEMINI_API_KEY")
_ = v.BindEnv("verifier.providers.deepseek.api_key",
    "DEEPSEEK_API_KEY")

```

```

_ = v.BindEnv("verifier.providers.groq.api_key", "GROQ_API_KEY")
_ = v.BindEnv("verifier.providers.mistral.api_key",
    "MISTRAL_API_KEY")
_ = v.BindEnv("verifier.providers.xai.api_key", "XAI_API_KEY")
_ = v.BindEnv("verifier.providers.cerebras.api_key",
    "CEREBRAS_API_KEY")
_ = v.BindEnv("verifier.providers.cloudflare.api_key",
    "CLOUDFLARE_API_KEY")
_ = v.BindEnv("verifier.providers.cloudflare.account_id",
    "CLOUDFLARE_ACCOUNT_ID")
_ = v.BindEnv("verifier.providers.siliconflow.api_key",
    "SILICONFLOW_API_KEY")
_ = v.BindEnv("verifier.providers.replicate.api_key",
    "REPLICATE_API_TOKEN")
_ = v.BindEnv("verifier.providers.together.api_key",
    "TOGETHER_API_KEY")
_ = v.BindEnv("verifier.providers.openrouter.api_key",
    "OPENROUTER_API_KEY")

```

2.4 Configuration Validation

Modification Point: helix_code/internal/config/config.go, inside validateConfig().

```

func (c *Config) validateConfig() error {
    // ... existing validation ...

    if c.Verifier != nil && c.Verifier.Enabled {
        if c.Verifier.Mode != "remote" && c.Verifier.Mode !=
            "embedded" {
            return fmt.Errorf("verifier.mode must be 'remote' or
                'embedded', got: %s", c.Verifier.Mode)
        }
        if c.Verifier.Endpoint == "" && c.Verifier.Mode ==
            "remote" {
            return
                fmt.Errorf("verifier.endpoint is required when mode is
                    'remote'")
        }
        if c.Verifier.PollingInterval < 10*time.Second {
            return fmt.Errorf("verifier.polling_interval must be
                >= 10s")
        }
        totalWeight := c.Verifier.Scoring.Weights.CodeCapability
        +
            c.Verifier.Scoring.Weights.Responsiveness +
            c.Verifier.Scoring.Weights.Reliability +
            c.Verifier.Scoring.Weights.FeatureRichness +

```

```

        c.Verifier.Scoring.Weights.ValueProposition
    if math.Abs(totalWeight-1.0) > 0.001 {
        return fmt.Errorf("verifier scoring weights must sum
        to 1.0, got: %.3f", totalWeight)
    }
}

return nil
}

```

2.5 YAML Config Example

New File: helix_code/config/verifier.yaml

```

verifier:
  enabled: true
  mode: remote
  endpoint: "http://localhost:8081"
  api_key: "${HELIX_VERIFIER_API_KEY}"
  timeout: 30s
  cache_ttl: 5m
  polling_interval: 60s

  scoring:
    weights:
      code_capability: 0.40
      responsiveness: 0.20
      reliability: 0.20
      feature_richness: 0.15
      value_proposition: 0.05
    min_acceptable_score: 6.0
    models_dev_enabled: true
    models_dev_endpoint: "https://api.models.dev"

  health:
    check_interval: 30s
    timeout: 10s
    failure_threshold: 5
    recovery_threshold: 3
    circuit_breaker:
      enabled: true
      half_open_timeout: 60s

  events:
    enabled: true
    websocket: false
    websocket_path: "/ws/verifier/events"

```

```

providers:
  openai:
    enabled: true
    api_key: "${OPENAI_API_KEY}"
    base_url: "https://api.openai.com/v1"
  anthropic:
    enabled: true
    api_key: "${ANTHROPIC_API_KEY}"
    base_url: "https://api.anthropic.com/v1"
  groq:
    enabled: true
    api_key: "${GROQ_API_KEY}"
  deepseek:
    enabled: true
    api_key: "${DEEPSEEK_API_KEY}"
  xai:
    enabled: false
    api_key: "${XAI_API_KEY}"
  openrouter:
    enabled: true
    api_key: "${OPENROUTER_API_KEY}"

helix_agent:
  enabled: false
  path: "./HelixAgent"
  auto_start: false
  verifier_sync:
    enabled: true
    share_scores: true
    share_providers: true
    sync_interval: 5m

```

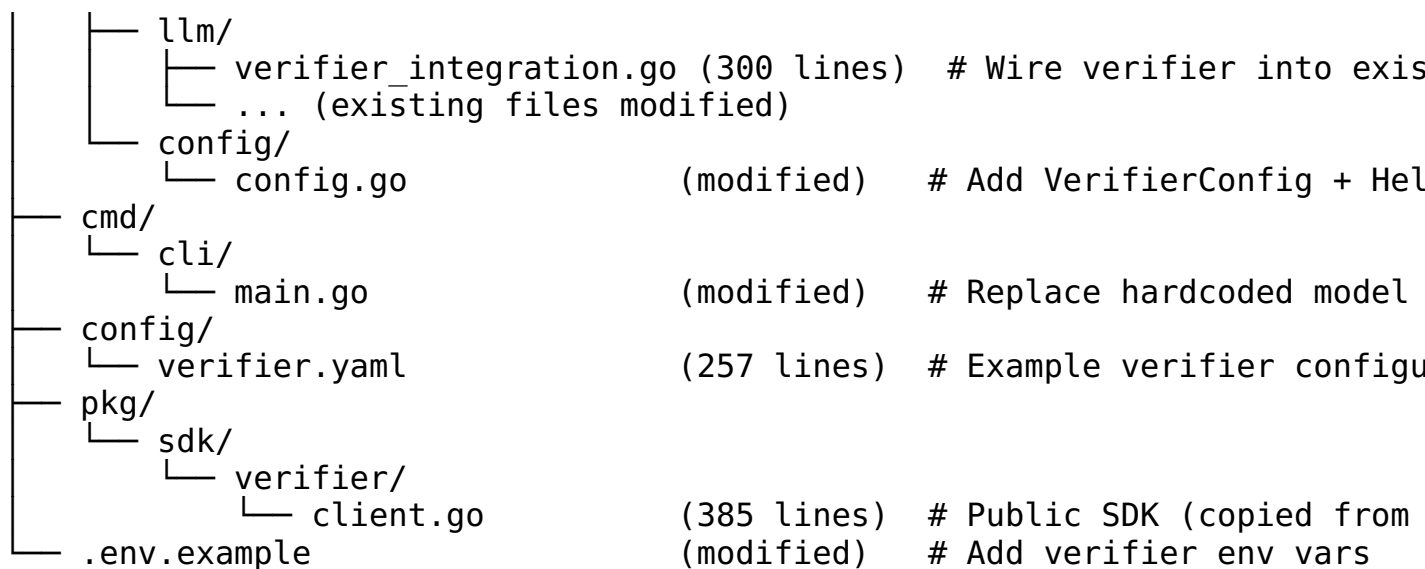
3. Module Boundaries

3.1 New Packages and Files

```

helix_code/
├── internal/
│   └── verifier/
│       ├── client.go           (385 lines) # REST API client for LLMs
│       ├── cache.go           (180 lines) # In-memory + Redis model
│       ├── health.go          (486 lines) # Health monitor + circuit
│       ├── polling.go         (220 lines) # Background polling gorou
│       ├── events.go          (200 lines) # Event publisher (HelixCo
│       ├── adapter.go         (528 lines) # Score adapter (bridges t
│       ├── types.go           (150 lines) # Shared verifier types
│       └── doc.go              (30 lines)  # Package documentation

```



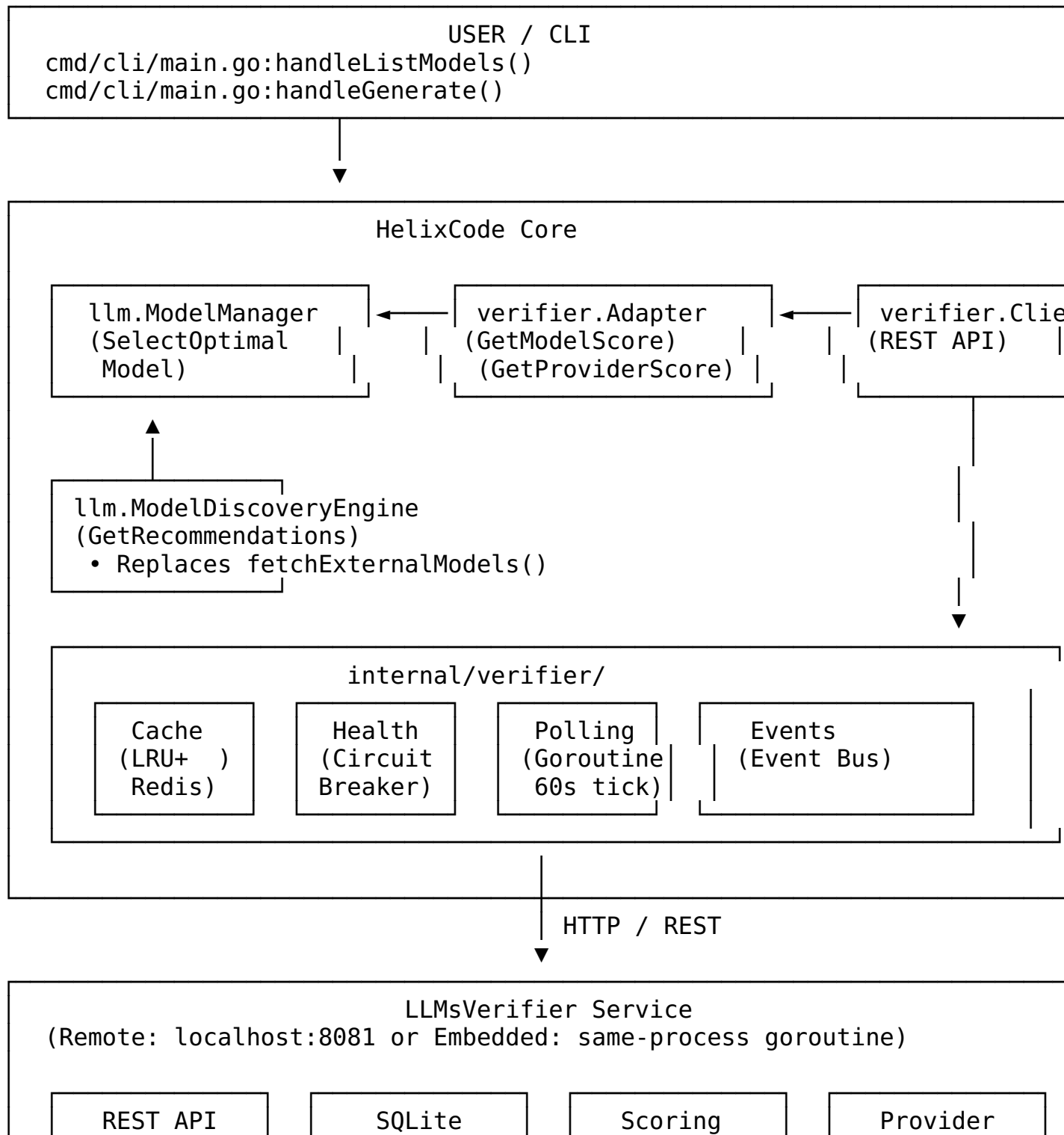
3.2 File Purposes

File	Lines	Responsibility
internal/verifier/ client.go	~385	HTTP client: GetModels(), GetProviders(), GetScores(), VerifyModel()
internal/verifier/ cache.go	~180	Two-tier cache: in-memory LRU (1K entries) + Redis fallback. TTL from config.
internal/verifier/ health.go	~486	Circuit breaker (failure_threshold:5, recovery_threshold:3), health checks
internal/verifier/ polling.go	~220	Poller struct: background goroutine, configurable interval, graceful stop
internal/verifier/ events.go	~200	Publishes to HelixCode's internal/event/ bus on model/ provider changes
internal/verifier/ adapter.go	~528	VerifierScoreAdapter: GetProviderScore(), GetModelScore(), GetVerifiedModels()
internal/verifier/ types.go	~150	VerifiedModel, ProviderScore, VerificationStatus structs
internal/llm/ verifier_integration.go	~300	VerifierModelSource: implements model source

File	Lines	Responsibility
		interface for model_discovery.go

4. Data Flow

4.1 ASCII Data Flow Diagram



/api/models /api/scores /api/health	Database (models, pricing, results)	Engine (Code 40%, Resp 20%, Rel 20%)	Adapters (12 adapters)
---	--	---	---------------------------

4.2 Caching Strategy

Two-tier cache (mirroring HelixAgent's safe.Store pattern [33](#)):

1. **L1 — In-Memory LRU**: github.com/hashicorp/golang-lru/v2 (already in HelixCode go.mod [3](#))
 - Key: `provider::model_id` or `provider` (for provider scores)
 - Value: `*CachedModelEntry` or `*CachedScoreEntry`
 - Capacity: 1024 entries
 - TTL: from `verifier.cache_ttl` (default: 5m)
2. **L2 — Redis** (optional, falls back to memory-only if Redis unavailable)
 - Key: `helix:verifier:models:<provider>` and `helix:verifier:scores:<provider>`
 - TTL: same as L1
 - Serialization: JSON

```
// internal/verifier/cache.go
type Cache struct {
    l1      *lru.Cache[string, *CacheEntry]
    l2      *redis.Client    // from HelixCode internal/redis
    ttl     time.Duration
    mu      sync.RWMutex
}

type CacheEntry struct {
    Models      []*VerifiedModel
    Scores      map[string]float64
    FetchedAt   time.Time
    Source      string // "verifier", "fallback", "embedded"
}

func (c *Cache) GetModels(provider string) ([]*VerifiedModel,
    bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()

    // Try L1
    if entry, ok := c.l1.Get(provider); ok {
        if time.Since(entry.FetchedAt) < c.ttl {
            return entry.Models, true
        }
    }
}
```

```

    }
}

// Try L2 (Redis)
if c.l2 != nil {
    data, err := c.l2.Get(ctx,
        "helix:verifier:models:"+provider).Result()
    if err == nil {
        var entry CacheEntry
        if json.Unmarshal([]byte(data), &entry) == nil {
            if time.Since(entry.FetchedAt) < c.ttl {
                c.l1.Add(provider, &entry) // backfill L1
                return entry.Models, true
            }
        }
    }
}

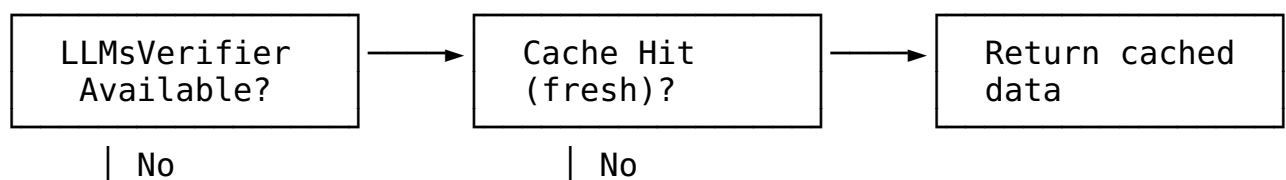
return nil, false
}

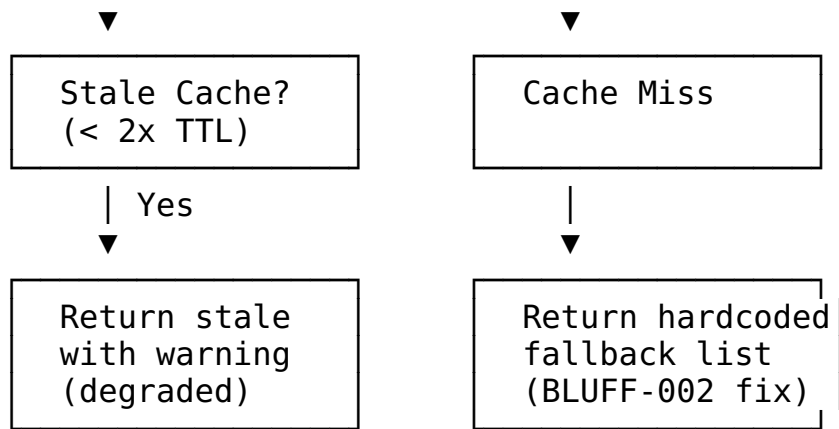
```

4.3 Refresh Intervals

Data Type	L1 Cache TTL	L2 Cache TTL	Polling Interval	Force Refresh Trigger
Model list	5m	5m	60s	User models command, --refresh flag
Provider scores	5m	10m	60s	Health check failure, manual refresh
Verification status	2m	5m	60s	On-demand via VerifyModel()
Pricing	30m	60m	300s	Scheduled re-verification
Provider health	30s	60s	30s	Circuit breaker state change

4.4 Fallback Behavior





Fallback model list (stored in `internal/verifier/fallback_models.go` as CONST-035 compliance [34](#)):

```

var FallbackModels = []*VerifiedModel{
  {ID: "llama-3.2-3b", Name: "Llama 3.2 3B", Provider:
    "ollama", ContextSize: 131072, Source: "fallback"},
  {ID: "gpt-4o", Name: "GPT-4o", Provider: "openai",
    ContextSize: 128000, Source: "fallback"},
  {ID: "claude-3-5-sonnet", Name: "Claude 3.5 Sonnet",
    Provider: "anthropic", ContextSize: 200000, Source:
    "fallback"},
  {ID: "mistral-large", Name: "Mistral Large", Provider:
    "mistral", ContextSize: 128000, Source: "fallback"},
  {ID: "gemini-2.5-pro", Name: "Gemini 2.5 Pro", Provider:
    "gemini", ContextSize: 1000000, Source: "fallback"},
}

```

5. API Contract

5.1 Integration Mode Decision: REST API Client (Recommended)

Decision: HelixCode integrates with LLMsVerifier via **HTTP REST API client**, NOT as an imported Go module.

Rationale: - LLMsVerifier depends on `digital.vasic.llmprovider` at `../../LLMProvider` [16](#) — this relative module path is incompatible with HelixCode's module structure (`dev.helix.code`). - HelixAgent already exposes a REST API server (`api/server.go` with `/api/models`, `/api/providers`, `/api/health` [28](#)). - REST API allows LLMsVerifier to run as a separate service, sidecar container, or embedded goroutine without module coupling. - HelixAgent's own `pkg/sdk/go/verifier/client.go` [33](#) confirms this is the intended integration pattern.

5.2 REST API Endpoints Consumed

Endpoint	Method	Purpose	Response Type
/api/health	GET	Verifier service health	{"status":"healthy","timestamp":"..."}
/api/models	GET	List all verified models	[]ModelInfo 40
/api/models/{id}	GET	Single model details	ModelInfo
/api/models/{id}/verify	POST	Trigger verification	VerificationResult
/api/providers	GET	List provider statuses	[]ProviderInfo
/api/providers/{id}/scores	GET	Provider scores	{"overall":8.5,"code_capability":8.5,...}
/api/scores	GET	All model scores	map[string]float64
/api/pricing	GET	Token pricing	[]PricingEntry
/api/limits	GET	Rate limits	[]LimitEntry

5.3 Client Implementation

File: helix_code/internal/verifier/client.go

```
package verifier
```

```
import (  
    "context"  
    "encoding/json"  
    "fmt"  
    "net/http"
```

```

        "time"
    )

    // Client mirrors HelixAgent's pkg/sdk/go/verifier/client.go
    // [^33^].
    type Client struct {
        baseURL    string
        apiKey     string
        httpClient *http.Client
        timeout    time.Duration
    }

    func NewClient(baseURL, apiKey string, timeout time.Duration)
        *Client {
        if timeout == 0 {
            timeout = 30 * time.Second
        }
        return &Client{
            baseURL:    baseURL,
            apiKey:     apiKey,
            timeout:    timeout,
            httpClient: &http.Client{Timeout: timeout},
        }
    }

    func (c *Client) Health(ctx context.Context) (*HealthResponse,
        error) {
        req, err := http.NewRequestWithContext(ctx, "GET",
            c.baseURL+"/api/health", nil)
        if err != nil { return nil, err }
        resp, err := c.httpClient.Do(req)
        if err != nil { return nil,
            fmt.Errorf("verifier health check failed: %w", err) }
        defer resp.Body.Close()
        // ... parse JSON ...
    }

    func (c *Client) GetModels(ctx context.Context)
        ([]*VerifiedModel, error) {
        req, err := http.NewRequestWithContext(ctx, "GET",
            c.baseURL+"/api/models", nil)
        if err != nil { return nil, err }
        if c.apiKey != "" {
            req.Header.Set("Authorization", "Bearer "+c.apiKey)
        }
        resp, err := c.httpClient.Do(req)
        if err != nil { return nil, fmt.Errorf("failed to fetch
            models from verifier: %w", err) }
    }

```

```

defer resp.Body.Close()

var models []*VerifiedModel
if err := json.NewDecoder(resp.Body).Decode(&models); err !=
    nil {
    return nil, fmt.Errorf("failed to decode models: %w",
        err)
}
return models, nil
}

func (c *Client) GetProviderScores(ctx context.Context)
    (map[string]float64, error) {
    req, _ := http.NewRequestWithContext(ctx, "GET", c.baseURL+"/
        api/scores", nil)
    if c.apiKey != "" {
        req.Header.Set("Authorization", "Bearer "+c.apiKey)
    }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var scores map[string]float64
    json.NewDecoder(resp.Body).Decode(&scores)
    return scores, nil
}

func (c *Client) VerifyModel(ctx context.Context, modelID
    string) (*VerificationResult, error) {
    req, _ := http.NewRequestWithContext(ctx, "POST",
        fmt.Sprintf("%s/api/models/%s/verify", c.baseURL,
            modelID), nil)
    if c.apiKey != "" {
        req.Header.Set("Authorization", "Bearer "+c.apiKey)
    }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var result VerificationResult
    json.NewDecoder(resp.Body).Decode(&result)
    return &result, nil
}

```

5.4 Data Types

File: helix_code/internal/verifier/types.go

```

package verifier

import "time"

// VerifiedModel – unified model representation from
// LLMsVerifier.
type VerifiedModel struct {
    ID                string    `json:"id"`
    Name              string    `json:"name"`
    DisplayName       string    `json:"display_name"`
    Provider          string    `json:"provider"`
    ProviderType      string    `json:"provider_type"`
    Score             float64   `json:"score"`
    Verified          bool      `json:"verified"`
    VerificationStatus string    `json:"verification_status" //
        pending, verified, failed
    ContextSize       int        `json:"context_window_tokens"`
    MaxOutputTokens   int        `json:"max_output_tokens"`
    SupportsStreaming bool      `json:"supports_streaming"`
    SupportsTools     bool      `json:"supports_tool_use"`
    SupportsCode      bool      `json:"supports_code_generation"`
    SupportsVision    bool      `json:"supports_vision"`
    Latency            time.Duration `json:"latency_ms"`
    CostPerInputToken  float64   `json:"input_token_cost"`
    CostPerOutputToken float64   `json:"output_token_cost"`
    OverallScore       float64   `json:"overall_score"`
    CodeCapabilityScore float64   `json:"code_capability_score"`
    ResponsivenessScore float64   `json:"responsiveness_score"`
    ReliabilityScore   float64   `json:"reliability_score"`
    FeatureRichnessScore float64 `json:"feature_richness_score"`
    ValuePropositionScore float64 `json:"value_proposition_score"`
    LastVerified       time.Time `json:"last_verified"`
    Source             string    `json:"source" // "verifier",
        "cache", "fallback"
}

// ProviderStatus – health and score of a provider.
type ProviderStatus struct {
    Name        string    `json:"name"`
    Type        string    `json:"type"`
    Score       float64   `json:"score"`
    Verified    bool      `json:"verified"`
    Healthy     bool      `json:"healthy"`
    ModelCount  int       `json:"model_count"`
    Tier        int       `json:"tier"`
    LastChecked time.Time `json:"last_checked"`
}

```

```

}

// VerificationResult – result of on-demand verification.
type VerificationResult struct {
    ModelID      string    `json:"model_id"`
    Status       string    `json:"status"`
    OverallScore float64   `json:"overall_score"`
    Error        string    `json:"error,omitempty"`
    CompletedAt  time.Time `json:"completed_at"`
}

// HealthResponse – verifier service health.
type HealthResponse struct {
    Status      string    `json:"status"`
    Timestamp   time.Time `json:"timestamp"`
    Version     string    `json:"version"`
}

```

6. Integration Points with Existing Code

6.1 cmd/cli/main.go:handleListModels() — BLUFF-002 Fix

Current Code (lines 101-128) [10](#):

```

// HARDCODED – REPLACES THIS:
func (c *CLI) handleListModels(ctx context.Context) error {
    models := []struct{...}{
        {"llama-3-8b", "Llama 3 8B", "llama.cpp", 8192,
        "available"},
        {"mistral-7b", "Mistral 7B", "ollama", 4096,
        "available"},
        {"phi-3-mini", "Phi-3 Mini", "openai", 128000,
        "available"},
    }
    // ... prints hardcoded list
}

```

Replacement (lines 101-128 rewritten):

```

func (c *CLI) handleListModels(ctx context.Context) error {
    // Use verifier adapter if enabled, otherwise fall through to
    // model manager
    if c.verifierAdapter != nil && c.verifierAdapter.IsEnabled()
    {
        models, err := c.verifierAdapter.GetVerifiedModels(ctx)
        if err == nil && len(models) > 0 {
            return c.printVerifiedModels(models)
        }
    }
}

```

```

    }
    // Log warning but don't fail – try fallback
    c.logger.Warn("Verifier model fetch failed, using
    fallback: %v", err)
}

// Fallback to existing ModelManager (already initialized,
// may have local models)
if c.modelManager != nil {
    models := c.modelManager.GetAvailableModels()
    return c.printManagerModels(models)
}

// Ultimate fallback: static list (for bootstrapping before
// any provider is ready)
return c.printFallbackModels()
}

func (c *CLI) printVerifiedModels(models
    []*verifier.VerifiedModel) error {
    fmt.Println("Available Models (from LLMsVerifier):")
    fmt.Println(strings.Repeat("-", 80))
    fmt.Printf("%-24s %-20s %-10s %-12s %s\n", "ID", "Name",
        "Provider", "Score", "Status")
    for _, m := range models {
        status := "✓ verified"
        if !m.Verified {
            status = "○ pending"
        }
        if m.VerificationStatus == "failed" {
            status = "x failed"
        }
        scoreStr := fmt.Sprintf("SC:%.1f", m.OverallScore)
        fmt.Printf("%-24s %-20s %-10s %-12s %s\n", m.ID, m.Name,
            m.Provider, scoreStr, status)
    }
    return nil
}

```

6.2 internal/llm/

model_discovery.go:fetchExternalModels() — Replace Hardcoded List

Current Code [29](#):

```

func (e *ModelDiscoveryEngine) fetchExternalModels()
    []*ModelInfo {
    return []*ModelInfo{
        {ID: "llama-3-8b-instruct", Name: "Llama 3 8B Instruct",
        Format: FormatGGUF, Size: 4.7GB, ContextSize: 8192},
        {ID: "mistral-7b-instruct", Name: "Mistral 7B Instruct",
        Format: FormatGGUF, Size: 4.1GB, ContextSize: 32768},
        {ID: "codellama-7b-instruct", Name: "CodeLlama 7B
        Instruct", Format: FormatGGUF, Size: 3.8GB,
        ContextSize: 16384},
    }
}

```

Replacement:

```

func (e *ModelDiscoveryEngine) fetchExternalModels(ctx
    context.Context) []*ModelInfo {
    // If verifier adapter is available and enabled, use it as
    the single source of truth
    if e.verifierAdapter != nil && e.verifierAdapter.IsEnabled()
    {
        verifiedModels, err :=
        e.verifierAdapter.GetVerifiedModels(ctx)
        if err == nil {
            return e.convertVerifiedToModelInfo(verifiedModels)
        }

        e.logger.Warn("Verifier fetch failed for external
        models: %v", err)
    }

    // Fallback: return empty – local providers (Ollama,
    llama.cpp) will still be discovered
    return []*ModelInfo{}
}

func (e *ModelDiscoveryEngine)
    convertVerifiedToModelInfo(verified
        []*verifier.VerifiedModel) []*ModelInfo {
    result := make([]*ModelInfo, 0, len(verified))
    for _, v := range verified {
        mi := &ModelInfo{
            ID:          v.ID,
            Name:         v.DisplayName,
            Format:
            FormatUnknown, // Verifier doesn't track GGUF vs HF
            Size:         0, // Not provided by
            verifier

```



```

        ContextSize: v.ContextSize,
        MaxTokens:    v.MaxOutputTokens,
        Provider:     v.Provider,
        Verified:     v.Verified,
        Score:        v.OverallScore,
        Capabilities: e.mapCapabilities(v),
    }
    result = append(result, mi)
}
return result
}

```

New Field in ModelDiscoveryEngine (add to internal/llm/model_discovery.go struct):

```

type ModelDiscoveryEngine struct {
    // ... existing fields ...
    verifierAdapter *verifier.Adapter // ← NEW
}

```

6.3 internal/llm/model_manager.go:SelectOptimalModel() — Use Verifier Scores

Current Scoring Algorithm [33](#):

```

func (m *ModelManager) SelectOptimalModel(criteria
    ModelSelectionCriteria) (*ModelInfo, error) {
    // 1. Capability matching
    // 2. Context size adequacy
    // 3. Task type suitability
    // 4. Hardware compatibility
    // 5. Quality preference
}

```

Augmented Algorithm (insert after step 3, before hardware compatibility):

```

func (m *ModelManager) SelectOptimalModel(criteria
    ModelSelectionCriteria) (*ModelInfo, error) {
    candidates :=
        m.filterByCapabilities(criteria.RequiredCapabilities)
    candidates = m.filterByContextSize(candidates,
        criteria.MaxTokens)
    candidates = m.filterByTaskType(candidates,
        criteria.TaskType)

    // NEW: Incorporate LLMsVerifier scores into ranking
    if m.verifierAdapter != nil && m.verifierAdapter.IsEnabled()
    {

```

```

        candidates = m.rankByVerifierScores(candidates, criteria)
    }

    candidates = m.filterByHardwareCompatibility(candidates)
    candidates = m.applyQualityPreference(candidates,
        criteria.QualityPreference)

    if len(candidates) == 0 {
        return nil, ErrNoSuitableModel
    }
    return candidates[0], nil
}

func (m *ModelManager) rankByVerifierScores(
    candidates []*ModelInfo,
    criteria ModelSelectionCriteria,
) []*ModelInfo {
    scored := make([]*struct {
        model *ModelInfo
        score float64
    }, 0, len(candidates))

    for _, c := range candidates {
        baseScore := c.Score // existing heuristic score

        // Fetch verifier score for this specific model
        verifierScore, found :=
            m.verifierAdapter.GetModelScore(c.ID)
        if found {
            // Weighted blend: 60% verifier, 40% local heuristic
            // Verifier score is 0-10; normalize to 0-1
            normalizedVerifier := verifierScore / 10.0
            blendedScore := (normalizedVerifier * 0.6) +
                (baseScore * 0.4)

            // Apply task-specific boost from verifier dimensions
            switch criteria.TaskType {
            case "code_generation", "debugging", "refactoring":
                codeScore, _ :=
                    m.verifierAdapter.GetModelCodeCapabilityScore(c.ID)
                blendedScore += (codeScore / 10.0) *
                    0.15 // +15% code capability bonus
            case "planning", "analysis":
                relScore, _ :=
                    m.verifierAdapter.GetModelReliabilityScore(c.ID)
                blendedScore += (relScore / 10.0) * 0.10 // +10%
                    reliability bonus
            }

```

```

        baseScore = blendedScore
    }

    scored = append(scored, struct {
        model *ModelInfo
        score float64
    }{c, baseScore})
}

sort.Slice(scored, func(i, j int) bool {
    return scored[i].score > scored[j].score
})

result := make([]*ModelInfo, len(scored))
for i, s := range scored {
    result[i] = s.model
}
return result
}

```

6.4 internal/llm/factory.go:NewProvider() — Verifier-Aware Validation

Modification Point: helix_code/internal/llm/factory.go, after provider creation, before return.

```

func NewProvider(config ProviderConfigEntry) (Provider, error) {
    var provider Provider
    var err error

    switch config.Type {
    case ProviderTypeOpenAI:    provider, err =
        NewOpenAIProvider(config)
    case ProviderTypeAnthropic: provider, err =
        NewAnthropicProvider(config)
    // ... existing cases ...
    default:
        return nil, fmt.Errorf("unsupported provider type: %s",
            config.Type)
    }

    if err != nil {
        return nil, err
    }

    // NEW: Validate provider against LLMsVerifier registry
    if verifierAdapter != nil && verifierAdapter.IsEnabled() {

```

```

status, found :=
verifierAdapter.GetProviderStatus(string(config.Type))
if found {
    if !status.Healthy {
        return nil, fmt.Errorf("provider %s is marked
unhealthy by verifier (score: %.1f)",
            config.Type, status.Score)
    }
    if status.Score <
verifierAdapter.GetMinAcceptableScore() {
        return nil, fmt.Errorf("provider %s score %.1f
below minimum %.1f",
            config.Type, status.Score,
verifierAdapter.GetMinAcceptableScore())
    }
}
}

return provider, nil
}

```

6.5 internal/llm/verifier_integration.go — New Bridge File

New File: helix_code/internal/llm/verifier_integration.go

```

package llm

import "dev.helix.code/internal/verifier"

// VerifierModelSource implements the model source interface for
// the discovery engine.
// It delegates to the verifier adapter to fetch real-time model
// data.
type VerifierModelSource struct {
    adapter *verifier.Adapter
}

func NewVerifierModelSource(adapter *verifier.Adapter)
    *VerifierModelSource {
    return &VerifierModelSource{adapter: adapter}
}

func (s *VerifierModelSource) IsAvailable() bool {
    return s.adapter != nil && s.adapter.IsEnabled() &&
        s.adapter.IsReachable()
}

```

```

func (s *VerifierModelSource) FetchModels(ctx context.Context)
    ([]*ModelInfo, error) {
    verified, err := s.adapter.GetVerifiedModels(ctx)
    if err != nil {
        return nil, err
    }
    return s.convert(verified), nil
}

func (s *VerifierModelSource) convert(verified
    []*verifier.VerifiedModel) []*ModelInfo {
    // ... same as
    ModelDiscoveryEngine.convertVerifiedToModelInfo ...
}

```

7. Enable/Disable Mechanism

7.1 Global Enable/Disable

Control Point: internal/config/config.go:VerifierConfig.Enabled

Behavior Matrix:

verifier.enabled	System Behavior
false (default)	Verifier client is nil. All model operations use existing local/heuristic logic. handleListModels() falls through to modelManager.GetAvailableModels(). No polling goroutine started. No REST API calls made.
true	Verifier client initialized. Polling goroutine started. All model lists fetched from verifier first, with local fallback. Scores incorporated into SelectOptimalModel(). Events published on changes.

Initialization Logic (in cmd/server/main.go and cmd/cli/main.go):

```

func initializeVerifier(cfg *config.Config) (*verifier.Adapter,
    error) {
    if cfg.Verifier == nil || !cfg.Verifier.Enabled {
        return nil, nil // disabled - return nil adapter
    }
}

```

```

client := verifier.NewClient(cfg.Verifier.Endpoint,
    cfg.Verifier.APIKey, cfg.Verifier.Timeout)
cache := verifier.NewCache(cfg.Verifier.CacheTTL,
    redisClient) // redisClient may be nil
health := verifier.NewHealthMonitor(cfg.Verifier.Health)

adapter := verifier.NewAdapter(client, cache, health,
    cfg.Verifier)

// Start background polling
if cfg.Verifier.PollingInterval > 0 {
    poller := verifier.NewPoller(adapter,
        cfg.Verifier.PollingInterval)
    poller.Start()
    // Store poller for graceful shutdown
}

return adapter, nil
}

```

7.2 Per-Provider Enable/Disable

Control Point: internal/config/

config.go:VerifierProviderConfig.Enabled

Behavior: The verifier adapter filters providers before returning to HelixCode:

```

func (a *Adapter) GetVerifiedModels(ctx context.Context)
    ([]*verifier.VerifiedModel, error) {
    allModels, err := a.client.GetModels(ctx)
    if err != nil { return nil, err }

    filtered := make([]*verifier.VerifiedModel, 0,
        len(allModels))
    for _, m := range allModels {
        // Check per-provider enable flag
        providerCfg, hasOverride :=
            a.config.Providers[m.Provider]
        if hasOverride && !providerCfg.Enabled {
            continue // skip disabled providers
        }

        // Check global provider enable in HelixCode
        ProvidersConfig
        if !a.isProviderEnabledInHelixCode(m.Provider) {
            continue
        }
    }
}

```

```

        filtered = append(filtered, m)
    }
    return filtered, nil
}

```

Provider Enable State Resolution:

verifier.providers.X .enabled	×	helixcode.providers.X .enabled	=	Effective
true		true		ENABLED
true		false		DISABLED
false		true		DISABLED
false		false		DISABLED
(not set)		true		ENABLED (in
(not set)		false		DISABLED (i

7.3 Graceful Degradation When Disabled or Unavailable

```

func (a *Adapter) GetVerifiedModels(ctx context.Context)
    ([]*verifier.VerifiedModel, error) {
    if !a.enabled {
        return nil, ErrVerifierDisabled
    }

    // Try cache first
    if models, ok := a.cache.GetModels("all"); ok {
        return models, nil
    }

    // Try live fetch
    models, err := a.client.GetModels(ctx)
    if err != nil {
        // Circuit breaker may be open
        if a.health.IsCircuitOpen() {
            // Return stale cache if available (even past TTL)
            if stale, ok := a.cache.GetModelsStale("all"); ok {
                return stale, ErrUsingStaleCache
            }
        }
        // Return fallback list
        return verifier.FallbackModels, ErrVerifierUnavailable
    }

    a.cache.SetModels("all", models)

```

```
    return models, nil
}
```

8. Real-Time Updates Strategy

8.1 Polling Architecture

Since LLMsVerifier **only supports polling** (no webhooks, no SSE, no push) [28](#), HelixCode implements a background polling goroutine.

```
// internal/verifier/polling.go
```

```
type Poller struct {
    adapter      *Adapter
    interval     time.Duration
    ticker       *time.Ticker
    stopCh       chan struct{}
    wg           sync.WaitGroup
    lastModels   map[string]*VerifiedModel
    lastScores   map[string]float64
    mu           sync.RWMutex
}
```

```
func NewPoller(adapter *Adapter, interval time.Duration) *Poller
{
    if interval < 10*time.Second {
        interval = 10 * time.Second
    }
    return &Poller{
        adapter:  adapter,
        interval: interval,
        stopCh:   make(chan struct{}),
    }
}
```

```
func (p *Poller) Start() {
    p.wg.Add(1)
    go p.loop()
}
```

```
func (p *Poller) Stop() {
    close(p.stopCh)
    p.wg.Wait()
}
```

```
func (p *Poller) loop() {
```



```

defer p.wg.Done()

// Immediate first poll
p.poll()

p.ticker = time.NewTicker(p.interval)
defer p.ticker.Stop()

for {
    select {
    case <-p.ticker.C:
        p.poll()
    case <-p.stopCh:
        return
    }
}

}

func (p *Poller) poll() {
    ctx, cancel := context.WithTimeout(context.Background(),
        30*time.Second)
    defer cancel()

    // 1. Fetch all models
    models, err := p.adapter.client.GetModels(ctx)
    if err != nil {
        p.adapter.health.RecordFailure()
        return
    }

    // 2. Detect changes
    p.mu.Lock()
    changes := p.detectChanges(p.lastModels, models)
    p.lastModels = p.indexModels(models)
    p.mu.Unlock()

    // 3. Update cache
    p.adapter.cache.SetModels("all", models)
    p.adapter.health.RecordSuccess()

    // 4. Publish events for changes
    for _, change := range changes {
        p.adapter.events.Publish(change)
    }

    // 5. Fetch scores (less frequently – every 3rd poll)
    if p.shouldFetchScores() {
        scores, _ := p.adapter.client.GetProviderScores(ctx)

```

```

        p.adapter.cache.SetScores(scores)
    }
}

func (p *Poller) detectChanges(old, new []*VerifiedModel)
    []ChangeEvent {
    changes := []ChangeEvent{}
    newIndex := p.indexModels(new)

    for id, model := range newIndex {
        if oldModel, ok := old[id]; !ok {
            changes = append(changes, ChangeEvent{Type:
                "model.discovered", Model: model})
        } else if oldModel.OverallScore != model.OverallScore {
            changes = append(changes, ChangeEvent{Type:
                "model.score_changed", Model: model, OldScore:
                oldModel.OverallScore})
        } else if oldModel.VerificationStatus !=
            model.VerificationStatus {
            changes = append(changes, ChangeEvent{Type:
                "model.status_changed", Model: model, OldStatus:
                oldModel.VerificationStatus})
        }
    }

    for id, model := range old {
        if _, ok := newIndex[id]; !ok {
            changes = append(changes, ChangeEvent{Type:
                "model.removed", Model: model})
        }
    }

    return changes
}

func (p *Poller) shouldFetchScores() bool {
    // Scores change slower than model lists; poll every 3rd tick
    return time.Now().Unix()%3 == 0
}

```

8.2 Event Publishing to HelixCode Event Bus

File: internal/verifier/events.go

```

package verifier

import "dev.helix.code/internal/event"

```

```

const (
    TopicVerifierModelDiscovered =
        "helix.verifier.model.discovered"
    TopicVerifierModelUpdated    = "helix.verifier.model.updated"
    TopicVerifierModelRemoved    = "helix.verifier.model.removed"
    TopicVerifierProviderHealth  =
        "helix.verifier.provider.health"
    TopicVerifierScoreChanged    = "helix.verifier.score.changed"
    TopicVerifierDegraded        = "helix.verifier.degraded"
    TopicVerifierRecovered       = "helix.verifier.recovered"
)

type EventPublisher struct {
    bus      event.Bus
    enabled  bool
    websocket bool
    wsPath   string
}

func (ep *EventPublisher) Publish(change ChangeEvent) error {
    if !ep.enabled {
        return nil
    }

    var topic string
    switch change.Type {
    case "model.discovered":
        topic = TopicVerifierModelDiscovered
    case "model.score_changed":
        topic = TopicVerifierScoreChanged
    case "model.status_changed":
        topic = TopicVerifierModelUpdated
    case "model.removed":
        topic = TopicVerifierModelRemoved
    }

    data, _ := json.Marshal(change)
    return ep.bus.Publish(topic, data)
}

```

8.3 On-Demand Refresh Triggers

Users can force a refresh outside the polling interval:

```

// cmd/cli/main.go – add new flag
var (
    refreshModels = flag.Bool("refresh-models", false, "Force
        refresh model list from verifier")

```

```

)

func (c *CLI) handleListModels(ctx context.Context) error {
    if *refreshModels && c.verifierAdapter != nil {
        c.verifierAdapter.ForceRefresh(ctx)
    }
    // ... continue with normal flow ...
}

```

9. Error Handling & Fallbacks

9.1 Circuit Breaker

File: internal/verifier/health.go

```

type CircuitState int

const (
    CircuitClosed CircuitState = iota
    CircuitOpen
    CircuitHalfOpen
)

type HealthMonitor struct {
    state          CircuitState
    failures        int
    successes       int
    lastFailureTime time.Time
    config          config.VerifierHealthConfig
    mu              sync.RWMutex
}

func (h *HealthMonitor) RecordFailure() {
    h.mu.Lock()
    defer h.mu.Unlock()

    h.failures++
    h.lastFailureTime = time.Now()

    if h.config.CircuitBreaker.Enabled && h.failures >=
        h.config.FailureThreshold {
        h.state = CircuitOpen
    }
}

func (h *HealthMonitor) RecordSuccess() {
    h.mu.Lock()

```

```

defer h.mu.Unlock()

switch h.state {
case CircuitHalfOpen:
    h.successes++
    if h.successes >= h.config.RecoveryThreshold {
        h.state = CircuitClosed
        h.failures = 0
        h.successes = 0
    }
case CircuitClosed:
    h.failures = 0 // reset on success
}
}

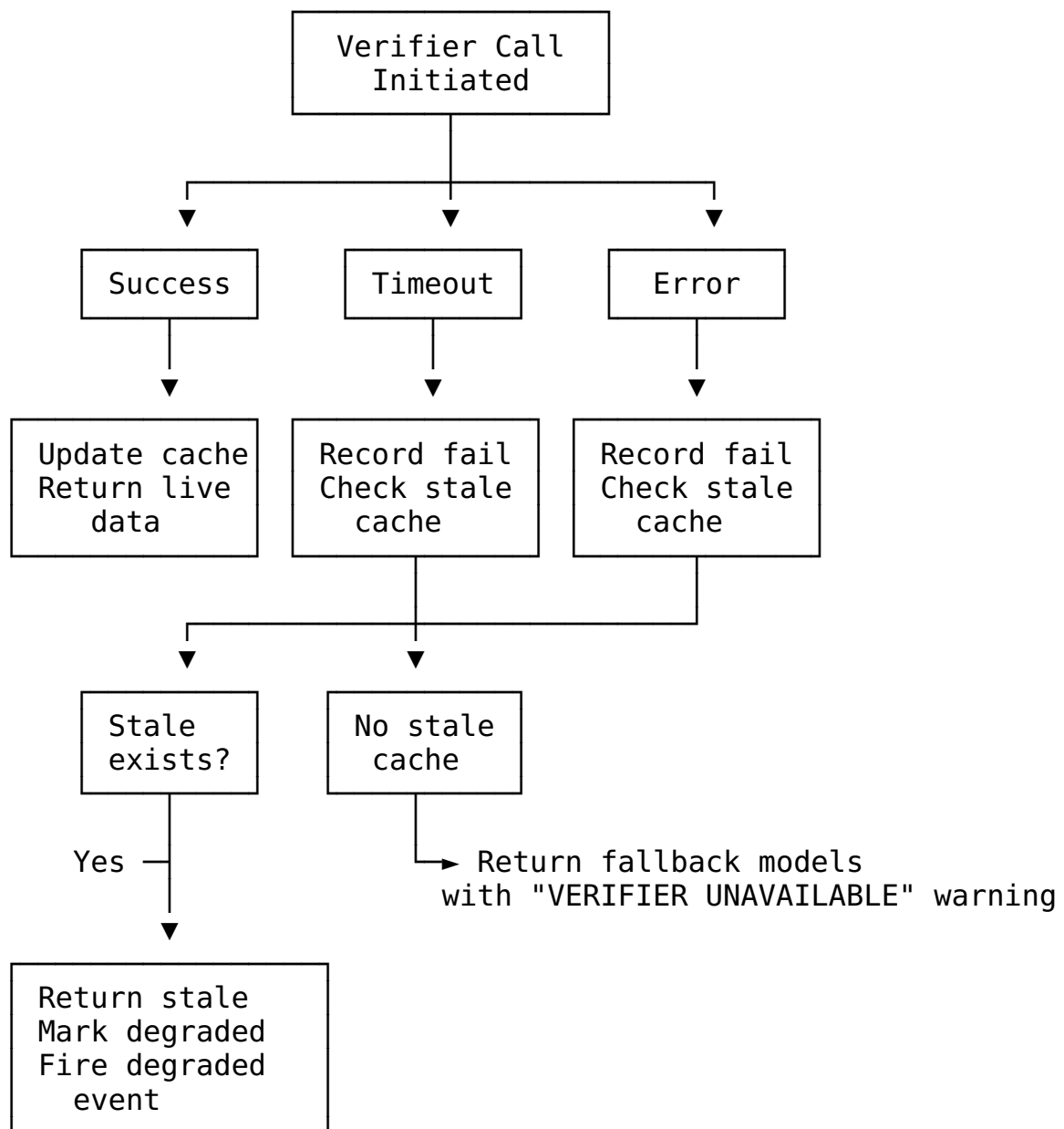
func (h *HealthMonitor) AllowRequest() bool {
    h.mu.RLock()
    defer h.mu.RUnlock()

    switch h.state {
    case CircuitClosed:
        return true
    case CircuitOpen:
        if time.Since(h.lastFailureTime) >
            h.config.CircuitBreaker.HalfOpenTimeout {
            h.mu.RUnlock()
            h.mu.Lock()
            h.state = CircuitHalfOpen
            h.successes = 0
            h.mu.Unlock()
            h.mu.RLock()
            return true
        }
        return false
    case CircuitHalfOpen:
        return true
    }
    return false
}

func (h *HealthMonitor) IsCircuitOpen() bool {
    h.mu.RLock()
    defer h.mu.RUnlock()
    return h.state == CircuitOpen
}

```

9.2 Degraded Mode Decision Tree



9.3 Fallback Chain (CONST-020 Compliance) [34](#)

```
// internal/verifier/adapter.go
func (a *Adapter) GetVerifiedModels(ctx context.Context)
    ([]*VerifiedModel, error) {
    if !a.enabled {
        return nil, ErrVerifierDisabled
    }

    if !a.health.AllowRequest() {
        // Circuit open – use cache or fallback
        return a.getFallbackModels()
    }
}
```

```

// 1. Try live fetch
models, err := a.client.GetModels(ctx)
if err == nil {
    a.health.RecordSuccess()
    a.cache.SetModels("all", models)
    return models, nil
}

a.health.RecordFailure()

// 2. Try stale cache (up to 2x TTL)
if stale, ok := a.cache.GetModelsStale("all"); ok {
    return stale, ErrUsingStaleCache
}

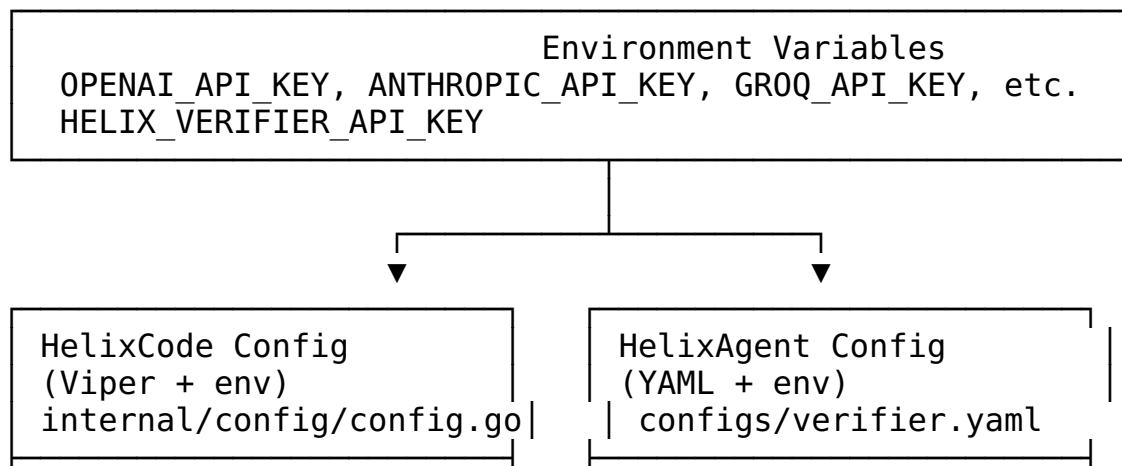
// 3. Return hardcoded fallback
return a.getFallbackModels()
}

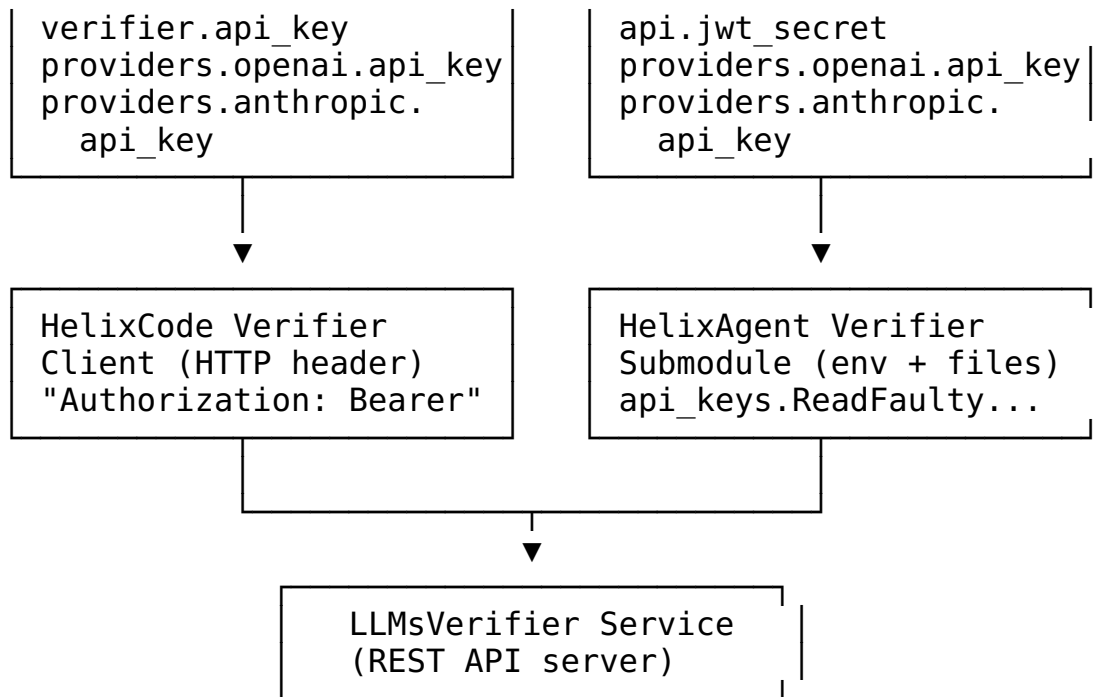
func (a *Adapter) getFallbackModels() ([]*VerifiedModel, error) {
    result := make([]*VerifiedModel, len(FallbackModels))
    for i, m := range FallbackModels {
        copy := *m
        copy.Source = "fallback"
        result[i] = &copy
    }
    return result, ErrUsingFallback
}

```

10. API Key Provisioning

10.1 Key Flow Architecture





10.2 API Key Redaction

Rule: API keys are NEVER serialized to JSON, NEVER logged at INFO or above, NEVER returned in API responses.

```

// internal/config/config.go
// The mapstructure tag with "squash" or explicit omission:
type VerifierProviderConfig struct {
    Enabled    bool    `mapstructure:"enabled"`
    APIKey     string  `mapstructure:"api_key" json:"- "
           yml:"api_key" // json:"- " excludes from marshaling
    BaseURL    string  `mapstructure:"base_url"`
    // ...
}

// internal/verifier/client.go
// Keys are sent as HTTP headers only:
func (c *Client) authHeader(req *http.Request) {
    if c.apiKey != "" {
        req.Header.Set("Authorization", "Bearer "+c.apiKey)
    }
}

```

10.3 Faulty Key Tracking

Mirror HelixAgent's `api_keys.ReadFaultyAPIKeys()` pattern [33](#):


```
// internal/verifier/faulty_keys.go
var faultyKeysPath = filepath.Join(os.Getenv("HOME"),
    ".helixcode", "faulty_api_keys.json")

type FaultyKeyStore struct {
    Keys map[string]FaultyKeyEntry `json:"keys"`
}

type FaultyKeyEntry struct {
    KeyName    string    `json:"key_name"`
    Reason     string    `json:"reason"`
    Timestamp  time.Time `json:"timestamp"`
    Provider   string    `json:"provider"`
}

func MarkKeyFaulty(keyName, reason, provider string) error {
    store, _ := ReadFaultyKeys()
    store.Keys[keyName] = FaultyKeyEntry{
        KeyName: keyName, Reason: reason, Timestamp: time.Now(),
        Provider: provider,
    }
    data, _ := json.Marshal(store)
    return os.WriteFile(faultyKeysPath, data, 0600)
}

func ReadFaultyKeys() (*FaultyKeyStore, error) {
    data, err := os.ReadFile(faultyKeysPath)
    if err != nil {
        return &FaultyKeyStore{Keys:
            map[string]FaultyKeyEntry{}}, nil
    }
    var store FaultyKeyStore
    json.Unmarshal(data, &store)
    if store.Keys == nil { store.Keys =
        map[string]FaultyKeyEntry{} }
    return &store, nil
}
```

11. HelixAgent Integration

11.1 Integration Model

HelixAgent is a **Git submodule** of HelixCode (confirmed by `cli_agents/helix_code/` being a submodule in HelixAgent's repo [33](#)). The integration is **bidirectional config sharing**, not direct function calls.

11.2 HelixAgent as Submodule

File: helix_code/.gitmodules (add if not present)

```
[submodule "HelixAgent"]
    path = HelixAgent
    url = https://github.com/HelixDevelopment/HelixAgent.git
```

11.3 Config Synchronization

HelixCode writes a shared config file that HelixAgent reads:

```
// internal/helixagent/sync.go
package helixagent

import (
    "dev.helix.code/internal/config"
    "gopkg.in/yaml.v3"
    "os"
    "path/filepath"
)

// SyncConfigToHelixAgent writes a verifier config that
// HelixAgent can consume.
func SyncConfigToHelixAgent(cfg *config.Config) error {
    if cfg.HelixAgent == nil || !cfg.HelixAgent.Enabled {
        return nil
    }

    // Translate HelixCode config to HelixAgent verifier.yaml
    // format
    agentCfg := AgentVerifierConfig{
        Enabled: cfg.Verifier != nil && cfg.Verifier.Enabled,
        Database: AgentDatabaseConfig{
            Path: filepath.Join(cfg.HelixAgent.Path, "data",
                "llm-verifier.db"),
        },
        Providers: make(map[string]AgentProviderConfig),
    }

    for name, p := range cfg.Verifier.Providers {
        agentCfg.Providers[name] = AgentProviderConfig{
            Enabled: p.Enabled,
            APIKey:   p.APIKey,
            BaseURL: p.BaseURL,
            Models: p.Models,
        }
    }
}
```

```

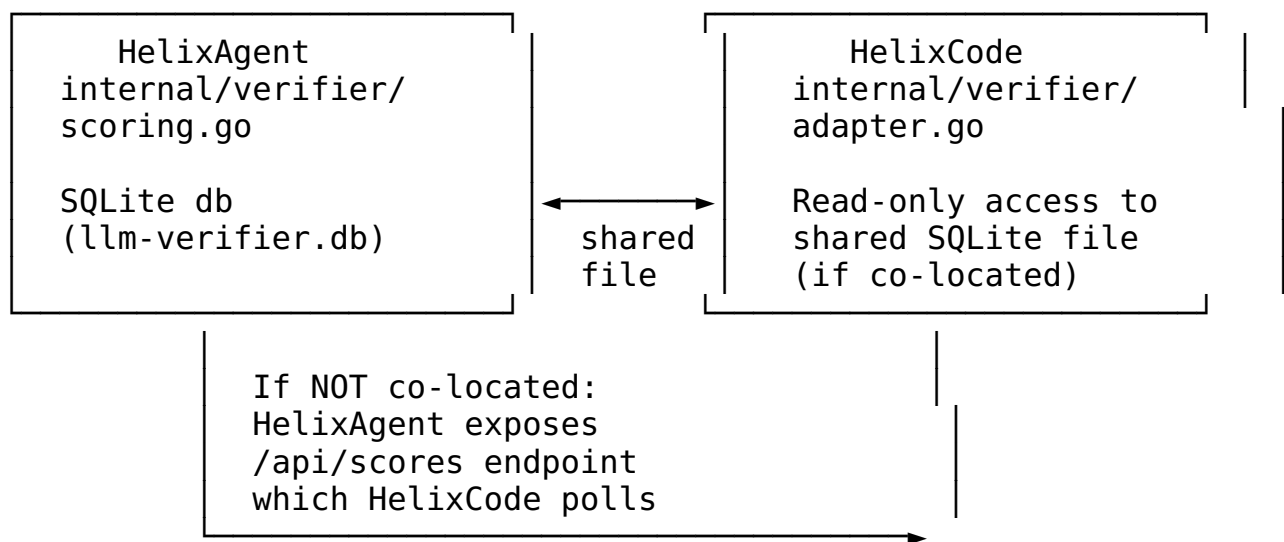
data, err := yaml.Marshal(agentCfg)
if err != nil {
    return err
}

configPath := filepath.Join(cfg.HelixAgent.Path, "configs",
    "verifier.yaml")
return os.WriteFile(configPath, data, 0644)
}

```

11.4 Score Sharing

When both systems run with `verifier_sync.share_scores: true`:



11.5 Enable/Disable HelixAgent from HelixCode Config

Behavior Matrix:

helix_agent.enabled	verifier.enabled	Behavior
false	any	HelixAgent submodule is NOT initialized. HelixCode uses its own verifier client directly.
true	true	HelixAgent is initialized.

helix_agent.enabled	verifier.enabled	Behavior
		HelixCode's verifier config is synced to HelixAgent. HelixCode reads scores FROM HelixAgent's verifier (via shared DB or REST API).
true	false	HelixAgent verifier is configured but disabled in HelixAgent's own config. HelixCode does not use LLMsVerifier.

11.6 Auto-Start

```
// cmd/server/main.go or cmd/cli/main.go
func maybeStartHelixAgent(cfg *config.Config) error {
    if cfg.HelixAgent == nil || !cfg.HelixAgent.Enabled || !
        cfg.HelixAgent.AutoStart {
        return nil
    }

    // 1. Sync config
    if err := helixagent.SyncConfigToHelixAgent(cfg); err != nil
    {
        return fmt.Errorf("failed to sync config to HelixAgent:
        %w", err)
    }

    // 2. Start HelixAgent as subprocess
    cmd := exec.Command(
        filepath.Join(cfg.HelixAgent.Path, "cmd", "helixagent",
            "main.go"),
```

```

        "--verifier", "--config",
        filepath.Join(cfg.HelixAgent.Path, "configs",
            "verifier.yaml"),
    )
    cmd.Env = os.Environ()
    if err := cmd.Start(); err != nil {
        return fmt.Errorf("failed to start HelixAgent: %w", err)
    }

    // 3. Store process for graceful shutdown
    globalHelixAgentProcess = cmd.Process

    return nil
}

```

12. Score Adapter (The Critical Bridge)

12.1 File: `internal/verifier/adapter.go`

This is the **most important file** — it bridges LLMsVerifier's scoring system to HelixCode's provider and model selection logic. It replicates HelixAgent's `internal/services/llmsverifier_score_adapter.go` [33](#).

```

package verifier

import (
    "context"
    "sync"
    "time"
)

// Adapter bridges LLMsVerifier scores to HelixCode's llm
// package.
type Adapter struct {
    client      *Client
    cache       *Cache
    health      *HealthMonitor
    config      *config.VerifierConfig

    providerScores map[string]float64
    modelScores    map[string]float64
    modelCodeScores map[string]float64
    modelRelScores map[string]float64
    mu             sync.RWMutex

    lastRefresh time.Time
}

```

```

    refreshInterval time.Duration
}

func NewAdapter(client *Client, cache *Cache, health
    *HealthMonitor, cfg *config.VerifierConfig) *Adapter {
    return &Adapter{
        client:      client,
        cache:       cache,
        health:      health,
        config:      cfg,
        providerScores: make(map[string]float64),
        modelScores:   make(map[string]float64),
        modelCodeScores: make(map[string]float64),
        modelRelScores: make(map[string]float64),
        refreshInterval: cfg.CacheTTL,
    }
}

func (a *Adapter) IsEnabled() bool {
    return a.config != nil && a.config.Enabled
}

func (a *Adapter) IsReachable() bool {
    return a.health.AllowRequest()
}

// GetModelScore returns the overall verifier score (0-10) for a
// model.
// Mirrors HelixAgent's
// llmsverifier_score_adapter.go:GetModelScore() [^33^].
func (a *Adapter) GetModelScore(modelID string) (float64, bool) {
    a.mu.RLock()
    defer a.mu.RUnlock()

    score, ok := a.modelScores[modelID]
    if ok {
        if score > 10 {
            score = score / 10.0
        }
        return score, true
    }

    // Try cache
    if cached, found := a.cache.GetModelScore(modelID); found {
        return cached, true
    }

    return 0, false
}

```

```

}

// GetProviderScore returns the best score for any model of this
// provider.
// Mirrors HelixAgent's GetProviderScore() [^33^].
func (a *Adapter) GetProviderScore(providerType string)
    (float64, bool) {
    a.mu.RLock()
    defer a.mu.RUnlock()

    score, ok := a.providerScores[providerType]
    if ok {
        if score > 10 {
            score = score / 10.0
        }
        return score, true
    }
    return 0, false
}

func (a *Adapter) GetModelCodeCapabilityScore(modelID string)
    (float64, bool) {
    a.mu.RLock()
    defer a.mu.RUnlock()
    score, ok := a.modelCodeScores[modelID]
    return score, ok
}

func (a *Adapter) GetModelReliabilityScore(modelID string)
    (float64, bool) {
    a.mu.RLock()
    defer a.mu.RUnlock()
    score, ok := a.modelRelScores[modelID]
    return score, ok
}

func (a *Adapter) GetMinAcceptableScore() float64 {
    if a.config == nil {
        return 6.0
    }
    return a.config.Scoring.MinAcceptableScore
}

// GetVerifiedModels returns all models from verifier, filtered
// by config.
func (a *Adapter) GetVerifiedModels(ctx context.Context)
    ([]*VerifiedModel, error) {
    if !a.IsEnabled() {

```

```

        return nil, ErrVerifierDisabled
    }

    // Check cache
    if models, ok := a.cache.GetModels("all"); ok {
        return a.filterByProviderConfig(models), nil
    }

    // Fetch from verifier
    models, err := a.client.GetModels(ctx)
    if err != nil {
        return a.handleFetchError(err)
    }

    // Update internal score maps
    a.refreshScores(models)

    // Update cache
    a.cache.SetModels("all", models)

    return a.filterByProviderConfig(models), nil
}

func (a *Adapter) refreshScores(models []*VerifiedModel) {
    a.mu.Lock()
    defer a.mu.Unlock()

    providerBest := make(map[string]float64)

    for _, m := range models {
        a.modelScores[m.ID] = m.OverallScore
        a.modelCodeScores[m.ID] = m.CodeCapabilityScore
        a.modelRelScores[m.ID] = m.ReliabilityScore

        // Track best score per provider
        if current, ok := providerBest[m.Provider]; !ok ||
            m.OverallScore > current {
            providerBest[m.Provider] = m.OverallScore
        }
    }

    a.providerScores = providerBest
    a.lastRefresh = time.Now()
}

func (a *Adapter) filterByProviderConfig(models
    []*VerifiedModel) []*VerifiedModel {
    if len(a.config.Providers) == 0 {

```



```

        return models // no overrides – return all
    }

    filtered := make([]*VerifiedModel, 0, len(models))
    for _, m := range models {
        if pc, ok := a.config.Providers[m.Provider]; ok && !
            pc.Enabled {
            continue
        }
        filtered = append(filtered, m)
    }
    return filtered
}

func (a *Adapter) ForceRefresh(ctx context.Context) error {
    a.cache.Invalidate("all")
    _, err := a.GetVerifiedModels(ctx)
    return err
}

```

13. Implementation Phases

Phase 1: Foundation (Week 1)

Task	File(s)	Lines
Add config structs	internal/config/ config.go	+120
Add default values	internal/config/ config.go	+30
Add env var bindings	internal/config/ config.go	+20
Add validation	internal/config/ config.go	+25
Create example YAML	config/verifier.yaml	257
Create internal/verifier/ types.go	New	150

Phase 2: Client & Cache (Week 1-2)

Task	File(s)	Lines
REST API client	internal/verifier/client.go	385

Task	File(s)	Lines
Two-tier cache	internal/verifier/cache.go	180
Fallback models	internal/verifier/fallback_models.go	50
Faulty key store	internal/verifier/faulty_keys.go	80

Phase 3: Health & Polling (Week 2)

Task	File(s)	Lines
Circuit breaker	internal/verifier/health.go	486
Background poller	internal/verifier/polling.go	220
Event publisher	internal/verifier/events.go	200

Phase 4: Score Adapter (Week 2-3)

Task	File(s)	Lines
Score adapter	internal/verifier/adapter.go	528
Model source bridge	internal/llm/verifier_integration.go	300

Phase 5: Integration Points (Week 3)

Task	File(s)	Lines
Replace CLI model list	cmd/cli/main.go:101-128	~40
Replace fetchExternalModels	internal/llm/model_discovery.go	~30
Augment SelectOptimalModel	internal/llm/model_manager.go	~60
Add provider validation	internal/llm/factory.go	~20

Phase 6: HelixAgent Integration (Week 3-4)

Task	File(s)	Lines
Config sync	internal/helixagent/sync.go	120
Auto-start logic	cmd/server/main.go	+40
Submodule setup	.gitmodules	+4

Phase 7: Testing & Challenges (Week 4)

Task	File(s)	Lines
Unit tests for client	internal/verifier/client_test.go	200
Unit tests for cache	internal/verifier/cache_test.go	150
Unit tests for adapter	internal/verifier/adapter_test.go	250
Challenge script	challenges/scripts/llmsverifier_integration_challenge.sh	200

14. Test Strategy

14.1 Test Files to Create

File	Type	Coverage
internal/verifier/client_test.go	Unit	HTTP client, mock server, error paths
internal/verifier/cache_test.go	Unit	LRU eviction, TTL expiry, Redis fallback
internal/verifier/health_test.go	Unit	Circuit breaker state transitions
internal/verifier/polling_test.go	Unit	Tick behavior, change detection, graceful stop
internal/verifier/adapter_test.go	Unit	Score normalization, filtering, fallback chain
internal/verifier/faulty_keys_test.go	Unit	Read/write JSON persistence
internal/llm/verifier_integration_test.go	Unit	Model conversion, capability mapping
tests/e2e/verifier_integration_test.go	E2E	Full flow: config → poll → CLI list → model selection

14.2 Mock Server for Testing

```
// internal/verifier/client_test.go
func TestClientGetModels(t *testing.T) {
```

```

server := httptest.NewServer(http.HandlerFunc(func(w
    http.ResponseWriter, r *http.Request) {
    assert.Equal(t, "/api/models", r.URL.Path)
    assert.Equal(t, "Bearer test-key",
        r.Header.Get("Authorization"))
    json.NewEncoder(w).Encode([]*VerifiedModel{
        {ID: "gpt-4o", Name: "GPT-4o", Provider: "openai",
        OverallScore: 9.2},
    })
}))
defer server.Close()

client := NewClient(server.URL, "test-key", 5*time.Second)
models, err := client.GetModels(context.Background())
assert.NoError(t, err)
assert.Len(t, models, 1)
assert.Equal(t, 9.2, models[0].OverallScore)
}

```

14.3 Challenge Scripts

New File: challenges/scripts/llmsverifier_integration_challenge.sh

```

#!/bin/bash
# CONST-035 Anti-Bluff: Verify that model listing is NOT
#             hardcoded
# This validates BLUFF-002 is permanently fixed

set -e

# 1. Build HelixCode with verifier enabled
make build

# 2. Start LLMsVerifier mock server
python3 tests/mock_verifier_server.py &
VERIFIER_PID=$!
trap "kill $VERIFIER_PID" EXIT

# 3. Run helix --list-models
OUTPUT=$(./bin/cli --list-models)

# 4. Assert output contains dynamic indicators (SC: score suffix)
if ! echo "$OUTPUT" | grep -q "SC:"; then
    echo "FAIL: Model list missing verifier score suffix (SC:).
        BLUFF-002 may be regressed."
    exit 1
fi

```

```
# 5. Assert no hardcoded-only models
if echo "$OUTPUT" | grep -q "llama-3-8b.*llama.cpp.*8192"; then
    echo "FAIL: Hardcoded model format detected."
    exit 1
fi

echo "PASS: Model list is dynamically sourced from LLMsVerifier."
```

15. Summary of Exact Modifications

15.1 Existing Files to Modify

File	Line Range	Change
helix_code/internal/config/config.go	After line 253	Add Verifier *VerifierConfig and HelixAgent *HelixAgentConfig fields
helix_code/internal/config/config.go	In setDefaults()	Add all verifier defaults (12+ SetDefault calls)
helix_code/internal/config/config.go	After env bindings	Add BindEnv for verifier API key and all provider API keys
helix_code/internal/config/config.go	In validateConfig()	Add verifier config validation (mode, endpoint, weights sum)
helix_code/cmd/cli/main.go	Lines 101-128	Replace handleListModels() hardcoded list with verifier-aware dynamic fetch
helix_code/cmd/cli/main.go	Flag declarations	Add --refresh-models flag
helix_code/cmd/cli/main.go	NewCLI() function	Initialize verifier adapter if config enabled
helix_code/cmd/server/main.go	After server.New()	Initialize verifier adapter and poller; start HelixAgent if configured
helix_code/internal/llm/model_discovery.go	fetchExternalModels()	Replace hardcoded return with verifier adapter call
helix_code/internal/llm/model_discovery.go	Struct definition	Add verifierAdapter *verifier.Adapter field

File	Line Range	Change
helix_code/internal/llm/model_manager.go	SelectOptimalModel()	Insert rankByVerifierScores() call after task type filter
helix_code/internal/llm/model_manager.go	After struct definition	Add verifierAdapter field and rankByVerifierScores() method
helix_code/internal/llm/factory.go	In NewProvider()	Add verifier status validation before returning provider
helix_code/internal/llm/missing_types.go	ModelInfo struct	Add Verified bool, Score float64, Source string fields
helix_code/.env.example	After existing env vars	Add HELIX_VERIFIER_API_KEY, HELIX_VERIFIER_ENDPOINT, and all provider keys
helix_code/go.mod	After existing requires	Add github.com/hashicorp/golang-lru/v2 (already present per 3 , verify)

15.2 New Files to Create

File	Purpose
helix_code/internal/verifier/client.go	REST API client for LLMsVerifier
helix_code/internal/verifier/cache.go	Two-tier LRU + Redis cache
helix_code/internal/verifier/health.go	Circuit breaker health monitor
helix_code/internal/verifier/polling.go	Background polling goroutine
helix_code/internal/verifier/events.go	Event publisher for HelixCode event bus
helix_code/internal/verifier/adapter.go	Score adapter bridge (critical)
helix_code/internal/verifier/types.go	Shared data types

File	Purpose
helix_code/internal/verifier/fallback_models.go	CONST-035 compliant fallback list
helix_code/internal/verifier/faulty_keys.go	Faulty API key tracking
helix_code/internal/verifier/doc.go	Package documentation
helix_code/internal/llm/verifier_integration.go	Bridge between verifier and model discovery
helix_code/internal/helixagent/sync.go	Config sync to HelixAgent submodule
helix_code/config/verifier.yaml	Example configuration file
helix_code/pkg/sdk/verifier/client.go	Public SDK client (copy from HelixAgent pattern)
helix_code/challenges/scripts/llmsverifier_integration_challenge.sh	CONST-035 challenge script
helix_code/internal/verifier/client_test.go	Unit tests
helix_code/internal/verifier/cache_test.go	Unit tests
helix_code/internal/verifier/health_test.go	Unit tests
helix_code/internal/verifier/adaptor_test.go	Unit tests
helix_code/internal/llm/verifier_integration_test.go	Unit tests

16. Risk Assessment

Risk	Likelihood	Impact	Mitigation
LLMsVerifier API is minimal (only 5 endpoints wired) 28	High	High	Implement missing endpoints in LLMsVerifier first, or mock for HelixCode testing
	Medium	Low	

Risk	Likelihood	Impact	Mitigation
Go version mismatch (HelixCode 1.24.0 vs LLMsVerifier 1.25.3)			REST API decouples versions; no module import needed
<code>digital.vasic.llmprovider</code> sibling dependency	High	High	Use REST API, NOT Go module import
LLMsVerifier core verification is stub (returns 8.5) 28	High	Medium	Accept stub initially; scores will improve as verifier matures. Document in AGENTS.md.
Redis unavailable in some deployments	Medium	Medium	L1 LRU cache operates independently; fallback to memory-only
Circular dependency with HelixAgent submodule	Low	High	HelixAgent is optional; HelixCode verifier works standalone
CLI still uses <code>flag</code> not Cobra	Medium	Low	Continue using <code>flag</code> ; verifier integration doesn't require Cobra

17. Citations

-
- [10](#): `helix_code/cmd/cli/main.go` — Hardcoded model list (BLUFF-002) at lines 101-128
-
-
-
-

-
-
-
-
-

End of Integration Architecture Document

[End of Section 4]

HelixCode CLI — Enterprise-Grade Model Display UX Design Specification

Version: 1.0.0

Date: 2026-04-30

Scope: LLM Model Display UX for HelixCode CLI across all 7 supported platforms

Target Files: cmd/cli/main.go, internal/cli/ux/, internal/cli/model_display.go (new)

Dependencies: github.com/fatih/color v1.18.0, github.com/rivo/tview v0.42.0

Data Source: LLMsVerifier (via REST API at http://localhost:8081/api/v1/verifier)

Table of Contents

1. [Cross-Platform Symbol Strategy](#)
 2. [Color System](#)
 3. [Status Indicators & Badges](#)
 4. [Model List Display \(--list-models\)](#)
 5. [Model Detail Display \(--model-info <id>\)](#)
 6. [Interactive Model Selection](#)
 7. [Notification / Alert UX](#)
 8. [Real-time Updates Display](#)
 9. [Error / Empty States](#)
 10. [Go Structs for UX State Management](#)
 11. [CLI Flag Additions](#)
 12. [Files to Modify / Create](#)
 13. [Implementation Priority](#)
-

1. Cross-Platform Symbol Strategy

1.1 Platform Detection

```
// internal/cli/ux/symbols.go
type TerminalCapabilities struct {
```

```

SupportsEmoji    bool    // true for macOS Terminal, iTerm2,
                        modern Linux, WSL
SupportsUnicode  bool    // true for all except Windows
                        cmd.exe pre-Windows 11
Supports256Color bool    // true for virtually all modern
                        terminals
SupportsTrueColor bool    // true for iTerm2, Windows
                        Terminal, modern Linux
Width            int     // terminal width in columns
IsWindowsCMD     bool    // Windows cmd.exe (not PowerShell,
                        not WSL)
IsPowerShell     bool    // Windows PowerShell / pwsh
IsMobile         bool    // Android/iOS terminal (limited
                        width)
}

```

```

func DetectTerminalCapabilities() *TerminalCapabilities {
    // Detection logic:
    // 1. Check runtime.GOOS
    // 2. Check TERM env var
    // 3. Check WT_SESSION (Windows Terminal)
    // 4. Check iTerm (ITERM_SESSION_ID)
    // 5. Check COLORTERM=truecolor
    // 6. Use github.com/mattn/go-isatty for TTY detection (add
        to deps if needed)
    // 7. Use tput or stty for width on Unix;
        GetConsoleScreenBufferInfo on Windows
}

```

1.2 Symbol Sets by Platform

Category	Symbol (Rich)	Fallback (ASCII)	Windows CMD	PowerShell	Mobile
Verified	✓ U+2713	[OK]	[OK]	✓	[V]
Pending	⏸ U+23F3	[. .]	[. .]		[P]
Failed	✗ U+2717	[XX]	[XX]	✗	[X]
Not Tested	⊘ U+2298	[-]	[-]	⊘	[-]
Healthy	● U+25CF	[+]	[+]	●	[+]
Degraded	◐ U+25D0	[~]	[~]	◐	[~]
Unhealthy	● U+25CF red	[-]	[-]	●	[-]
Offline	○ U+25CB	[!]	[!]	○	[!]

Category	Symbol (Rich)	Fallback (ASCII)	Windows CMD	PowerShell	Mobile
Rate Limited	⏸ U+23F8	[RL]	[RL]	⏸	[RL]
Quota Exceeded	U+26D4	[QU]	[QU]		[QU]
Cool Down	U+1F552	[CD]	[CD]		[CD]
Star / Score	★ U+2605	*	*	★	*
Empty Star	☆ U+2606	-	-	☆	-
Vision	👁 U+1F441	[VSN]	[V]	👁	[V]
Audio	🔊 U+1F50A	[AUD]	[A]	🔊	[A]
Video	🎥 U+1F3AC	[VID]	[V]	🎥	[V]
Reasoning	🧐 U+1F9E0	[RSN]	[R]	🧐	[R]
Streaming	U+26A1	[STR]	[S]		[S]
Tools	🔧 U+1F527	[TOL]	[T]	🔧	[T]
Code	🔌 U+003C/ U+003E	[COD]	[C]	🔌	[C]
Embeddings	U+1F4CA	[EMB]	[E]		[E]
Open Source	U+1F513	[OSS]	[O]		[O]
Arrow Right	➡ U+2192	->	->	➡	>
Arrow Up	⬆ U+2191	^	^	⬆	^
Bullet	• U+2022	-	-	•	-
Diamond	◆ U+25C6	>	>	◆	>
Separator	U+2502				
Horizontal	— U+2500	-	-	—	-
Corner	┐ U+250C	+	+	┐	+

Category	Symbol (Rich)	Fallback (ASCII)	Windows CMD	PowerShell	Mobile
Progress	█ U+2588	#	#	█	#
Progress Empty	░ U+2591	.	.	░	.
Dollar	\$ U+0024	\$	\$	\$	\$
Latency Fast	⚡ U+1F680	[FAST]	[F]	⚡	[F]
Latency Normal	🕒 U+1F6B6	[NRM]	[N]		[N]
Latency Slow	🕒 U+1F40C	[SLO]	[S]		[S]

1.3 Symbol Set Implementation

// internal/cli/ux/symbols.go

```

type SymbolSet struct {
    Verified      string
    Pending       string
    Failed        string
    NotTested     string
    Healthy       string
    Degraded      string
    Unhealthy     string
    Offline       string
    RateLimited   string
    QuotaExceeded string
    CoolDown      string
    StarFilled    string
    StarEmpty     string
    Vision        string
    Audio         string
    Video         string
    Reasoning     string
    Streaming     string
    Tools         string
    Code          string
    Embeddings    string
    OpenSource    string
    ArrowRight    string
    ArrowUp       string
    Bullet        string
    Diamond       string

```

```

SepVertical      string
SepHorizontal    string
CornerTL         string
ProgressFull     string
ProgressEmpty    string
Dollar           string
LatencyFast      string
LatencyNormal    string
LatencySlow      string
}

func NewSymbolSet(cap *TerminalCapabilities) *SymbolSet {
    if cap.IsWindowsCMD && !cap.SupportsEmoji {
        return &SymbolSet{
            Verified: "[OK]", Pending: "[..]", Failed: "[XX]",
            NotTested: "[-]", Healthy: "[+]", Degraded: "[~]",
            Unhealthy: "[-]", Offline: "[!]", RateLimited:
            "[RL]",
            QuotaExceeded: "[QU]", CoolDown: "[CD]",
            StarFilled: "*", StarEmpty: "-",
            Vision: "[V]", Audio: "[A]", Video: "[V]",
            Reasoning: "[R]", Streaming: "[S]", Tools: "[T]",
            Code: "[C]", Embeddings: "[E]", OpenSource: "[O]",
            ArrowRight: "->", ArrowUp: "^", Bullet: "-",
            Diamond: ">", SepVertical: "|", SepHorizontal: "-",
            CornerTL: "+", ProgressFull: "#", ProgressEmpty: ".",
            Dollar: "$", LatencyFast: "[F]", LatencyNormal:
            "[N]",
            LatencySlow: "[S]",
        }
    }
    if cap.IsMobile {
        return &SymbolSet{
            Verified: "[V]", Pending: "[P]", Failed: "[X]",
            NotTested: "[-]", Healthy: "[+]", Degraded: "[~]",
            Unhealthy: "[-]", Offline: "[!]", RateLimited:
            "[RL]",
            QuotaExceeded: "[QU]", CoolDown: "[CD]",
            StarFilled: "*", StarEmpty: "-",
            Vision: "[V]", Audio: "[A]", Video: "[V]",
            Reasoning: "[R]", Streaming: "[S]", Tools: "[T]",
            Code: "[C]", Embeddings: "[E]", OpenSource: "[O]",
            ArrowRight: ">", ArrowUp: "^", Bullet: "-",
            Diamond: ">", SepVertical: "|", SepHorizontal: "-",
            CornerTL: "+", ProgressFull: "#", ProgressEmpty: ".",
            Dollar: "$", LatencyFast: "[F]", LatencyNormal:
            "[N]",
            LatencySlow: "[S]",
        }
    }
}

```

```

    }
}
// Rich Unicode set (macOS, Linux, iTerm2, Windows Terminal,
// WSL, PowerShell)
return &SymbolSet{
    Verified: "✓", Pending: " ", Failed: "✗",
    NotTested: "⊘", Healthy: "●", Degraded: "◐",
    Unhealthy: "●", Offline: "⦿", RateLimited: "⚡",
    QuotaExceeded: " ", CoolDown: " ",
    StarFilled: "★", StarEmpty: "☆",
    Vision: "👁", Audio: " ", Video: " ",
    Reasoning: " ", Streaming: " ", Tools: " ",
    Code: "</>", Embeddings: " ", OpenSource: " ",
    ArrowRight: "→", ArrowUp: "↑", Bullet: "•",
    Diamond: "◆", SepVertical: "|", SepHorizontal: "—",
    CornerTL: "┐", ProgressFull: "■", ProgressEmpty: "░",
    Dollar: "$", LatencyFast: " ", LatencyNormal: " ",
    LatencySlow: " ",
}
}

```

2. Color System

2.1 Color Constants (fatih/color)

```

// internal/cli/ux/colors.go
package ux

import "github.com/fatih/color"

var (
    // Primary colors
    CHeader      = color.New(color.FgHiCyan, color.Bold)
    CSubheader   = color.New(color.FgCyan)
    CLabel       = color.New(color.FgHiBlack)
    CValue       = color.New(color.FgWhite)
    CAccent      = color.New(color.FgHiMagenta, color.Bold)

    // Status colors
    CVerified    = color.New(color.FgHiGreen, color.Bold)
    CPending     = color.New(color.FgHiYellow)
    CFailed      = color.New(color.FgHiRed, color.Bold)
    CNotTested   = color.New(color.FgHiBlack)
    CHealthy     = color.New(color.FgHiGreen)
    CDegraded    = color.New(color.FgHiYellow)
    CUnhealthy   = color.New(color.FgHiRed)

```

```

COffline      = color.New(color.FgHiBlack)

// Score colors
CScoreExcellent = color.New(color.FgHiGreen,
    color.Bold)    // 9.0-10.0
CScoreGood      =
    color.New(color.FgGreen)           // 7.0-8.9
CScoreAverage   =
    color.New(color.FgYellow)         // 5.0-6.9
CScorePoor      =
    color.New(color.FgHiRed)          // 3.0-4.9
CScoreBad       =
    color.New(color.FgHiBlack)        // 0.0-2.9

// Price colors
CPriceCheap     = color.New(color.FgHiGreen)    // < $0.50/1K
CPriceModerate  = color.New(color.FgYellow)    // $0.50-
    $2.00/1K
CPriceExpensive = color.New(color.FgHiRed)      // >
    $2.00/1K
CPriceFree      = color.New(color.FgHiCyan, color.Bold) // $0

// Capability colors
CCapEnabled     = color.New(color.FgHiGreen)
CCapDisabled    = color.New(color.FgHiBlack)

// Alert colors
CAAlertInfo     = color.New(color.FgHiBlue, color.Bold)
CAAlertWarning  = color.New(color.FgHiYellow, color.Bold)
CAAlertError    = color.New(color.FgHiRed, color.Bold)
CAAlertSuccess  = color.New(color.FgHiGreen, color.Bold)

// UI elements
CBorder         = color.New(color.FgHiBlack)
CBarExcellent   = color.New(color.BgHiGreen, color.FgBlack)
CBarGood        = color.New(color.BgGreen, color.FgBlack)
CBarAverage     = color.New(color.BgYellow, color.FgBlack)
CBarPoor        = color.New(color.BgHiRed, color.FgBlack)
CBarEmpty       = color.New(color.BgHiBlack)

// Cooldown / rate limit
CCoolDown       = color.New(color.FgHiYellow, color.Bold)
CRateLimit      = color.New(color.FgHiRed, color.Bold)

// Provider-specific colors (for quick visual recognition)
CProviderOpenAI = color.New(color.FgGreen)
CProviderAnthropic = color.New(color.FgCyan)
CProviderGemini  = color.New(color.FgBlue)

```



```

CProviderXAI      = color.New(color.FgHiBlack)
CProviderGroq     = color.New(color.FgMagenta)
CProviderOllama   = color.New(color.FgHiYellow)
CProviderLocal    = color.New(color.FgWhite)
CProviderOther    = color.New(color.FgHiWhite)
)

// GetScoreColor returns the appropriate color for a 0.0-10.0
// score
func GetScoreColor(score float64) *color.Color {
    switch {
    case score >= 9.0: return CScoreExcellent
    case score >= 7.0: return CScoreGood
    case score >= 5.0: return CScoreAverage
    case score >= 3.0: return CScorePoor
    default:          return CScoreBad
    }
}

// GetPriceColor returns the appropriate color for a price per 1K
// tokens
func GetPriceColor(price float64) *color.Color {
    if price == 0 { return CPriceFree }
    if price < 0.5 { return CPriceCheap }
    if price < 2.0 { return CPriceModerate }
    return CPriceExpensive
}

// GetProviderColor returns brand color for known providers
func GetProviderColor(provider string) *color.Color {
    switch provider {
    case "openai", "gpt":      return CProviderOpenAI
    case "anthropic", "claude": return CProviderAnthropic
    case "gemini", "google":   return CProviderGemini
    case "xai", "grok":        return CProviderXAI
    case "groq":               return CProviderGroq
    case "ollama":             return CProviderOllama
    case "local", "llamacpp", "vllm", "localai": return
        CProviderLocal
    default:                   return CProviderOther
    }
}

```

3. Status Indicators & Badges

3.1 Badge Rendering System

```
// internal/cli/ux/badges.go
```

```
type Badge struct {
    Symbol string
    Text   string
    Color  *color.Color
}

func (b *Badge) Render(sym *SymbolSet, width int) string {
    if width >= 100 {
        return b.Color.Sprintf("%s %s", b.Symbol, b.Text)
    }
    return b.Color.Sprintf("%s", b.Symbol)
}

func VerificationBadge(status string, sym *SymbolSet) *Badge {
    switch status {
    case "verified":
        return &Badge{Symbol: sym.Verified, Text: "VERIFIED",
            Color: CVerified}
    case "pending":
        return &Badge{Symbol: sym.Pending, Text: "PENDING",
            Color: CPending}
    case "failed":
        return &Badge{Symbol: sym.Failed, Text: "FAILED", Color:
            CFailed}
    default:
        return &Badge{Symbol: sym.NotTested, Text: "NOT TESTED",
            Color: CNotTested}
    }
}

func ProviderHealthBadge(status string, sym *SymbolSet) *Badge {
    switch status {
    case "healthy":
        return &Badge{Symbol: sym.Healthy, Text: "HEALTHY",
            Color: CHealthy}
    case "degraded":
        return &Badge{Symbol: sym.Degraded, Text: "DEGRADED",
            Color: CDegraded}
    case "unhealthy":
        return &Badge{Symbol: sym.Unhealthy, Text: "UNHEALTHY",
            Color: CUnhealthy}
    }
}
```

```

    case "offline":
        return &Badge{Symbol: sym.Offline, Text: "OFFLINE",
            Color: COffline}
    default:
        return &Badge{Symbol: sym.NotTested, Text: "UNKNOWN",
            Color: CNotTested}
}

}

func CooldownBadge(reason string, resetTime time.Time, sym
    *SymbolSet) *Badge {
    timeLeft := time.Until(resetTime)
    timeStr := ""
    if timeLeft > 0 {
        if timeLeft < time.Minute {
            timeStr = fmt.Sprintf(" (%ds)",
                int(timeLeft.Seconds()))
        } else if timeLeft < time.Hour {
            timeStr = fmt.Sprintf(" (%dm)",
                int(timeLeft.Minutes()))
        } else {
            timeStr = fmt.Sprintf(" (%dh)",
                int(timeLeft.Hours()))
        }
    }
    switch reason {
    case "rate-limited":
        return &Badge{Symbol: sym.RateLimited, Text: "RATE
            LIMITED" + timeStr, Color: CRateLimit}
    case "quota-exceeded":
        return &Badge{Symbol: sym.QuotaExceeded, Text: "QUOTA
            EXCEEDED" + timeStr, Color: CAlertError}
    case "cooldown":
        return &Badge{Symbol: sym.CoolDown, Text: "COOLDOWN" +
            timeStr, Color: CCoolDown}
    default:
        return &Badge{Symbol: sym.CoolDown, Text: "UNAVAILABLE"
            + timeStr, Color: CAlertError}
    }
}

func ScoreBadge(score float64, sym *SymbolSet) string {
    c := GetScoreColor(score)
    // Visual bar: 10 chars = 10.0 score
    filled := int(score)
    if filled > 10 { filled = 10 }
    if filled < 0 { filled = 0 }

```

```

    bar := strings.Repeat(sym.ProgressFull, filled) +
        strings.Repeat(sym.ProgressEmpty, 10-filled)
    return c.Sprintf("%.1f %s", score, bar)
}

func TierBadge(tier int, sym *SymbolSet) string {
    // Tier 1=Premium, 2=High-quality, 3=Fast, 4=Aggregator,
    // 5=Free
    filled := 6 - tier // 5 stars for tier 1, 1 star for tier 5
    if filled < 1 { filled = 1 }
    if filled > 5 { filled = 5 }
    stars := strings.Repeat(sym.StarFilled, filled) +
        strings.Repeat(sym.StarEmpty, 5-filled)
    return CAccent.Sprintf("%s", stars)
}

func PriceBadge(inputPrice, outputPrice float64, sym *SymbolSet,
    width int) string {
    if inputPrice == 0 && outputPrice == 0 {
        return CPriceFree.Sprintf("%s FREE", sym.Dollar)
    }
    // Show combined per-1K price
    avgPrice := (inputPrice + outputPrice) / 2.0 * 1000 //
        // convert to per-1K
    c := GetPriceColor(avgPrice)
    if width >= 100 {
        return c.Sprintf("%s%.2f/1K", sym.Dollar, avgPrice)
    }
    return c.Sprintf("%s%.1f", sym.Dollar, avgPrice)
}

```

3.2 Capability Icon Strip

// internal/cli/ux/capabilities.go

```

func CapabilityStrip(m *UnifiedModel, sym *SymbolSet, width int)
    string {
    parts := []string{}

    if m.SupportsVision {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Vision))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Vision))
    }

    if m.SupportsStreaming {

```

```

        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Streaming))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Streaming))
    }

    if m.SupportsTools {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Tools))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Tools))
    }

    // Code capability from verification results
    if slices.Contains(m.Capabilities, "code-generation") {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Code))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Code))
    }

    // Reasoning
    if slices.Contains(m.Capabilities, "reasoning") {
        parts = append(parts, CCapEnabled.Sprintf("%s",
            sym.Reasoning))
    } else if width >= 100 {
        parts = append(parts, CCapDisabled.Sprintf("%s",
            sym.Reasoning))
    }

    if width >= 100 {
        return strings.Join(parts, " ")
    }
    // Narrow: only show enabled caps, no gaps
    enabledOnly := []string{}
    for _, p := range parts {
        if !strings.Contains(p, strings.TrimSpace(sym.Vision))
            && // crude check - better to track separately
        }
        // Better approach: build two slices
    }
    return strings.Join(parts, "")
}

// Better implementation:

```

```

func CapabilityStripCompact(m *UnifiedModel, sym *SymbolSet)
    string {
    caps := []struct{
        enabled bool
        symbol  string
        label   string
    }{
        {m.SupportsVision, sym.Vision, "vision"},
        {m.SupportsStreaming, sym.Streaming, "streaming"},
        {m.SupportsTools, sym.Tools, "tools"},
        {slices.Contains(m.Capabilities, "code-generation"),
         sym.Code, "code"},
        {slices.Contains(m.Capabilities, "reasoning"),
         sym.Reasoning, "reasoning"},
        {slices.Contains(m.Capabilities, "embeddings"),
         sym.Embeddings, "embeddings"},
        {m.OpenSource, sym.OpenSource, "oss"},
    }

    parts := []string{
    for _, c := range caps {
        if c.enabled {
            parts = append(parts, CCapEnabled.Sprintf("%s",
                c.symbol))
        }
    }
    return strings.Join(parts, " ")
}

```

```

func CapabilityStripFull(m *UnifiedModel, sym *SymbolSet) string
    {
    caps := []struct{
        enabled bool
        symbol  string
        label   string
    }{
        {m.SupportsVision, sym.Vision, "vision"},
        {m.SupportsAudio, sym.Audio, "audio"},
        {m.SupportsVideo, sym.Video, "video"},
        {m.SupportsStreaming, sym.Streaming, "streaming"},
        {m.SupportsTools, sym.Tools, "tools"},
        {m.SupportsFunctions, sym.Tools, "functions"},
        {slices.Contains(m.Capabilities, "code-generation"),
         sym.Code, "code"},
        {slices.Contains(m.Capabilities, "reasoning"),
         sym.Reasoning, "reasoning"},
        {slices.Contains(m.Capabilities, "embeddings"),
         sym.Embeddings, "embeddings"},
    }
}

```

```

        {m.OpenSource, sym.OpenSource, "oss"},
    }

    parts := []string{}
    for _, c := range caps {
        if c.enabled {
            parts = append(parts, CCapEnabled.Sprintf("%s %s",
                c.symbol, c.label))
        } else {
            parts = append(parts, CCapDisabled.Sprintf("%s %s",
                c.symbol, c.label))
        }
    }
    return strings.Join(parts, " ")
}

```

4. Model List Display (- -list-models)

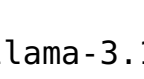
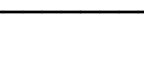
4.1 CLI Flags to Add

--list-models	List available models (existing)
--provider <name>	Filter by provider (new)
--verified-only	Show only verified models (new)
--max-price <float>	Max price per 1K tokens (new)
--min-score <float>	Min overall score 0-10 (new)
--capability <name>	Filter by capability: vision,streaming,too
--sort <field>	Sort by: score,price,name,provider,latency
--group-by <field>	Group by: provider,tier,status (new; defau
--format <type>	Output: table,compact,json,csv (new; defau
--no-color	Disable color output (new)
--no-emoji	Disable emoji/symbols (new)

4.2 Wide Terminal Mode (>= 120 columns)

ASCII Mockup — Wide Mode:

HelixCode – Available Models (verified by LLMsVerifier)					
23 models across 8 providers		18 verified	3 pending	2 cooldow	
MODEL NAME	PROVIDER	STATUS	SCORE	PRICE	CONT
★ claude-opus-4-6	Anthropic	✓	9.4		\$15.00
★ gpt-4o	OpenAI	✓	9.1		\$5.00/
gemini-2.5-pro	Google	✓	8.7		\$1.2
deepseek-chat	DeepSeek	✓	8.3		\$0.1
grok-3-fast-beta	xAI	✓	8.0		\$0.0

llama-3.3-70b	Groq	✓	7.5		\$0.9
mistral-large	Mistral	✓	7.2		\$3.0
groq-llama-3.1-8b	Groq	✓	6.8		\$0.0
claude-sonnet-4-5	Anthropic		7.8		\$3.0
qwen-2.5-72b	Qwen		6.5		\$0.0
openrouter-mixtral	OpenRouter	x	4.2		\$0.6
ollama-llama3.2	Local	✓	6.0		\$0.0
llamacpp-mistral-7b	Local	✓	5.5		\$0.0
★ = Premium Tier ● = Healthy ¨ COOLDOWN: groq-llama-3.1-70b (res					

4.3 Standard Terminal Mode (80-119 columns)

ASCII Mockup — Standard Mode:

HelixCode Models					Updated 14:32	
23 models 18 ✓ 3 2 " 0 ○						
MODEL	PROVIDER	S	SCORE	PRICE	CAPS	
claude-opus-4-6	Anthropic	✓	9.4	\$15.0/1K	👁	</
gpt-4o	OpenAI	✓	9.1	\$5.0/1K	👁	</
gemini-2.5-pro	Google	✓	8.7	\$1.2/1K	👁	</
deepseek-chat	DeepSeek	✓	8.3	\$0.1/1K		</>
grok-3-fast-beta	xAI	✓	8.0	FREE	👁	</
llama-3.3-70b	Groq	✓	7.5	\$0.9/1K		</>
mistral-large	Mistral	✓	7.2	\$3.0/1K	👁	</
groq-llama-3.1-8b	Groq	✓	6.8	\$0.1/1K		</>
claude-sonnet-4-5	Anthropic		7.8	\$3.0/1K	👁	</
qwen-2.5-72b	Qwen		6.5	FREE		</>
openrouter-mixtral	OpenRouter	x	4.2	\$0.6/1K		
ollama-llama3.2	Local	✓	6.0	FREE		</>
" groq-llama-3.1-70b (12m)		xai-free				

4.4 Narrow Terminal Mode (< 80 columns)

ASCII Mockup — Narrow Mode:

HelixCode Models						14:32:05
23 models 18 OK 3 .. 2 RL 0 !						
#	MODEL	PROV.	ST	SC	PR	CAPS

1	claude-opus-4-6	Anthro.	OK	9.4	\$15	VSTCrR
2	gpt-4o	OpenAI	OK	9.1	\$5	VSTCr
3	gemini-2.5-pro	Google	OK	8.7	\$1	VSTCrR
4	deepseek-chat	DeepSk.	OK	8.3	\$0	STCrR0
5	grok-3-fast-beta	xAI	OK	8.0	FREE	VSTCrR
6	llama-3.3-70b	Groq	OK	7.5	\$1	STCr0
7	mistral-large	Mistral	OK	7.2	\$3	VSTCr
8	groq-llama-3.1-8b	Groq	OK	6.8	\$0	STCr0
9	claude-sonnet-4-5	Anthro.	..	7.8	\$3	VSTCrR
10	qwen-2.5-72b	Qwen	..	6.5	FREE	STCr0
11	openrouter-mixtral	OpenR.	XX	4.2	\$1	ST0
12	ollama-llama3.2	Local	OK	6.0	FREE	STCr0

RL: groq-llama-3.1-70b(12m) QU: xai-free

4.5 Grouped by Provider (Standard Mode)

ASCII Mockup — Grouped:

HelixCode Models – Grouped by Provider										Updated 1m
Anthropic ● HEALTHY										
claude-opus-4-6	✓	9.4		\$15.0/1K	200.0K					👁️
claude-sonnet-4-5		7.8		\$3.0/1K	200.0K					👁️
OpenAI ● HEALTHY										
gpt-4o	✓	9.1		\$5.0/1K	128.0K					👁️
Groq ● DEGRADED (⌘ llama-3.1-70b cooldown 12m)										
llama-3.3-70b	✓	7.5		\$0.9/1K	128.0K					
groq-llama-3.1-8b	✓	6.8		\$0.1/1K	128.0K					
llama-3.1-70b	⌘	7.1		\$0.6/1K	128.0K					
xAI QUOTA EXCEEDED										
grok-3-fast-beta	✓	8.0		FREE	131.0K					👁️
grok-3-mini	✗	6.2		FREE	131.0K					👁️
Local ● HEALTHY										
ollama-llama3.2	✓	6.0		FREE	8.0K					
llamacpp-mistral-7b	✓	5.5		FREE	32.0K					

4.6 Compact Mode (for scripting / piping)

ID	PROVIDER	STATUS	SCORE	PRICE/1K	CONTEXT	C
claude-opus-4-6	anthropic	verified	9.4	15.00	200000	v
gpt-4o	openai	verified	9.1	5.00	128000	v
gemini-2.5-pro	google	verified	8.7	1.25	100000	v
deepseek-chat	deepseek	verified	8.3	0.14	64000	s
grok-3-fast-beta	xai	verified	8.0	0.00	131072	v
llama-3.3-70b	groq	verified	7.5	0.90	128000	s
mistral-large	mistral	verified	7.2	3.00	128000	v
groq-llama-3.1-8b	groq	verified	6.8	0.05	128000	s
claude-sonnet-4-5	anthropic	pending	7.8	3.00	200000	v
qwen-2.5-72b	qwen	pending	6.5	0.00	128000	s
openrouter-mixtral	openrouter	failed	4.2	0.60	32000	s
ollama-llama3.2	local	verified	6.0	0.00	8000	st
llamacpp-mistral-7b	local	verified	5.5	0.00	32000	s

4.7 List Display Implementation

```
// internal/cli/ux/list_display.go
```

```
package ux
```

```
import (  
    "fmt"  
    "sort"  
    "strings"  
    "time"  
  
    "github.com/fatih/color"  
)
```

```
// ListDisplayOptions configures the model list rendering
```

```
type ListDisplayOptions struct {  
    ProviderFilter    string  
    VerifiedOnly      bool  
    MaxPrice          float64  
    MinScore          float64  
    CapabilityFilter  string  
    SortBy            string // "score", "price", "name",  
                    "provider", "latency"  
    GroupBy           string // "provider", "tier", "status", ""  
    Format            string // "table", "compact", "json", "csv"  
    NoColor           bool  
    NoEmoji           bool  
    TerminalWidth     int  
}
```

```
// ModelListRow represents a single row in the model list
```

```

type ModelListRow struct {
    Rank          int
    Model         *UnifiedModel
    Provider      *UnifiedProvider
    Verification   *VerificationResult
    Cooldown      *CooldownInfo
    ScoreBar      string
    PriceStr      string
    StatusBadge   string
    Capabilities  string
}

func RenderModelList(models []ModelListRow, opts
    *ListDisplayOptions) string {
    sym := NewSymbolSet(DetectTerminalCapabilities())
    if opts.NoEmoji {
        sym = NewSymbolSet(&TerminalCapabilities{IsWindowsCMD:
            true})
    }

    if opts.NoColor {
        color.NoColor = true
    }

    switch opts.Format {
    case "json":
        return renderJSON(models)
    case "csv":
        return renderCSV(models)
    case "compact":
        return renderCompact(models, sym, opts.TerminalWidth)
    default:
        return renderTable(models, sym, opts)
    }
}

func renderTable(rows []ModelListRow, sym *SymbolSet, opts
    *ListDisplayOptions) string {
    width := opts.TerminalWidth
    if width < 60 { width = 60 }

    // Determine layout based on width
    if width >= 120 {
        return renderWideTable(rows, sym, width)
    } else if width >= 80 {
        return renderStandardTable(rows, sym, width)
    }
    return renderNarrowTable(rows, sym, width)
}

```

```
}
```

```
func renderWideTable(rows []ModelListRow, sym *SymbolSet, width
    int) string {
    var b strings.Builder

    // Header
    header := fmt.Sprintf(" %s HelixCode – Available Models
        (verified by LLMsVerifier)", sym.Diamond)
    b.WriteString(CHeader.Sprintf("%s\n", header))
    b.WriteString(CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))

    // Summary bar
    verified := 0; pending := 0; failed := 0; cooldown := 0;
        offline := 0
    for _, r := range rows {
        switch r.Verification.Status {
        case "verified": verified++
        case "pending": pending++
        case "failed": failed++
        }
        if r.Cooldown != nil { cooldown++ }
        if r.Provider.Status == "offline" { offline++ }
    }
    summary :=
        fmt.Sprintf(" %d models across providers | %s %d
            verified | %s %d pending | %s %d cooldown | %s %d
            offline",
            len(rows), sym.Verified, verified, sym.Pending, pending,
            sym.CoolDown, cooldown, sym.Offline, offline)
    b.WriteString(CSubheader.Sprintf("%s\n", summary))
    b.WriteString(CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))

    // Column headers
    b.WriteString(fmt.Sprintf(" %-24s | %-11s | %-6s | %-16s |
        %-10s | %-8s | %-25s\n",
        "MODEL NAME", "PROVIDER", "STATUS", "SCORE", "PRICE",
        "CONTEXT", "CAPABILITIES"))
    b.WriteString(CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))

    // Rows
    for _, r := range rows {
        tierPrefix := " "
        if r.Provider.Tier == 1 { tierPrefix = sym.StarFilled +
            " " }
    }
```

```

    name := truncate(r.Model.DisplayName, 24)
    provider := truncate(r.Provider.DisplayName, 11)
    statusBadge := VerificationBadge(r.Verification.Status,
    sym).Render(sym, width)
    score := ScoreBadge(r.Verification.OverallScore, sym)
    price := PriceBadge(r.Model.CostPerInputToken,
    r.Model.CostPerOutputToken, sym, width)
    ctx := formatContextWindow(r.Model.ContextWindow)
    caps := CapabilityStripCompact(r.Model, sym)

    b.WriteString(fmt.Sprintf("%s%-24s | %-11s | %-6s |
    %-16s | %-10s | %-8s | %s\n",
        tierPrefix, name, provider, statusBadge, score,
        price, ctx, caps))
}

// Footer with cooldown alerts
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))
footerParts := []string{}
for _, r := range rows {
    if r.Cooldown != nil {
        badge := CooldownBadge(r.Cooldown.Reason,
        r.Cooldown.ResetTime, sym)
        footerParts = append(footerParts, badge.Render(sym,
        width))
    }
}
if len(footerParts) > 0 {
    b.WriteString(CAlertWarning.Sprintf(" %s\n",
        strings.Join(footerParts, " | ")))
}

return b.String()
}

func renderStandardTable(rows []ModelListRow, sym *SymbolSet,
    width int) string {
    var b strings.Builder

    b.WriteString(CHeader.Sprintf(" %s HelixCode Models\n",
        sym.Diamond))
    b.WriteString(CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))

    // Compact summary

```

```

counts := countStatuses(rows)
summary := fmt.Sprintf(" %d models | %d %s | %d %s | %d %s |
    %d %s",
    len(rows), counts.verified, sym.Verified,
    counts.pending, sym.Pending,
    counts.cooldown, sym.CoolDown, counts.offline,
    sym.Offline)
b.WriteString(CSubheader.Sprintf("%s\n", summary))
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))

// Headers
b.WriteString(fmt.Sprintf(" %-25s| %-10s| %-2s| %-5s| %-9s|
    %-15s\n",
    "MODEL", "PROVIDER", "S", "SCORE", "PRICE", "CAPS"))
b.WriteString(CBorder.Sprintf("%s\n",
    strings.Repeat(sym.SepHorizontal, width-1)))

for _, r := range rows {
    name := truncate(r.Model.DisplayName, 25)
    provider := truncate(r.Provider.DisplayName, 10)
    status := VerificationBadge(r.Verification.Status,
        sym).Symbol
    scoreStr :=
        GetScoreColor(r.Verification.OverallScore).Printf("%.1f",
            r.Verification.OverallScore)
    priceStr := PriceBadge(r.Model.CostPerInputToken,
        r.Model.CostPerOutputToken, sym, width)
    caps := CapabilityStripCompact(r.Model, sym)

    b.WriteString(fmt.Sprintf(" %-25s| %-10s| %s| %s| %-9s|
        %s\n",
        name, provider, status, scoreStr, priceStr, caps))
}

// Cooldown footer
cooldownAlerts := getCooldownAlerts(rows, sym)
if len(cooldownAlerts) > 0 {
    b.WriteString(CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))
    b.WriteString(CAlertWarning.Sprintf(" %s\n",
        strings.Join(cooldownAlerts, " | ")))
}

return b.String()
}

```

```

func renderNarrowTable(rows []ModelListRow, sym *SymbolSet,
    width int) string {
    var b strings.Builder

    b.WriteString(CHeader.Sprintf(" HelixCode Models\n"))
    b.WriteString(CBorder.Sprintf("%s\n", strings.Repeat("-",
        width-1)))

    counts := countStatuses(rows)

    b.WriteString(fmt.Sprintf(" %d models | %d OK | %d .. |
        %d RL | %d !\n",
        len(rows), counts.verified, counts.pending,
        counts.cooldown, counts.offline))
    b.WriteString(CBorder.Sprintf("%s\n", strings.Repeat("-",
        width-1)))

    b.WriteString(fmt.Sprintf(" %-2s %-21s| %-7s| %-2s| %-2s|
        %-4s| %-6s\n",
        "#", "MODEL", "PROV.", "ST", "SC", "PR", "CAPS"))
    b.WriteString(CBorder.Sprintf("%s\n", strings.Repeat("-",
        width-1)))

    for i, r := range rows {
        name := truncate(r.Model.DisplayName, 21)
        prov := truncate(r.Provider.DisplayName, 7)
        status := VerificationBadge(r.Verification.Status,
            sym).Symbol
        score :=
            GetScoreColor(r.Verification.OverallScore).Printf("%.1f",
                r.Verification.OverallScore)
        price := PriceBadge(r.Model.CostPerInputToken,
            r.Model.CostPerOutputToken, sym, width)
        caps := CapabilityStripCompact(r.Model, sym)

        b.WriteString(fmt.Sprintf(" %-2d %-21s| %-7s| %s| %s|
            %-4s| %s\n",
            i+1, name, prov, status, score, price, caps))
    }

    return b.String()
}

func renderCompact(rows []ModelListRow, sym *SymbolSet, width
    int) string {
    var b strings.Builder
    b.WriteString(fmt.Sprintf("%-25s %-11s %-8s %-6s %-9s %-8s
        %s\n",

```

```

        "ID", "PROVIDER", "STATUS", "SCORE", "PRICE/1K",
        "CONTEXT", "CAPS"))
    for _, r := range rows {
        caps := strings.Join(r.Model.Capabilities, ",")
        price := fmt.Sprintf("%.2f",
            (r.Model.CostPerInputToken+r.Model.CostPerOutputToken)/
            2.0*1000)
        if price == "0.00" { price = "0.00" }
        b.WriteString(fmt.Sprintf("%-25s %-11s %-8s %-6.1f %-9s
            %-8d %s\n",
                r.Model.ID, r.Provider.DisplayName,
                r.Verification.Status,
                r.Verification.OverallScore, price,
                r.Model.ContextWindow, caps))
    }
    return b.String()
}

// --- Helper functions ---

func truncate(s string, maxLen int) string {
    if len(s) <= maxLen { return s }
    return s[:maxLen-1] + "..."
}

func formatContextWindow(n int) string {
    if n >= 1_000_000 {
        return fmt.Sprintf("%.1fM", float64(n)/1_000_000)
    }
    if n >= 1000 {
        return fmt.Sprintf("%.1fK", float64(n)/1000)
    }
    return fmt.Sprintf("%d", n)
}

type statusCounts struct {
    verified, pending, failed, cooldown, offline int
}

func countStatuses(rows []ModelListRow) statusCounts {
    c := statusCounts{}
    for _, r := range rows {
        switch r.Verification.Status {
            case "verified": c.verified++
            case "pending": c.pending++
            case "failed": c.failed++
        }
        if r.Cooldown != nil { c.cooldown++ }
    }
}

```



```

        if r.Provider.Status == "offline" { c.offline++ }
    }
    return c
}

func getCooldownAlerts(rows []ModelListRow, sym *SymbolSet)
    []string {
    alerts := []string{}
    for _, r := range rows {
        if r.Cooldown != nil {
            badge := CooldownBadge(r.Cooldown.Reason,
                r.Cooldown.ResetTime, sym)
            alerts = append(alerts, badge.Render(sym, 80))
        }
    }
    return alerts
}

```

5. Model Detail Display (- -model-info <id>)

5.1 CLI Flags to Add

```

--model-info <id>                Show detailed information for a model (new
--model-info-format <type>        Output: rich,json,yaml (new; default: ri

```

5.2 Rich Detail View (>= 100 columns)

ASCII Mockup:

HelixCode – Model Details				
claude-opus-4-6			Anthropic ● HEALTHY	
Status	✓ VERIFIED	Overall Score	9.4	</

Input

\$15.00

Output

\$75.00

Cached Input

\$7.50

↑

Expensive

Capabilities

✓ Vision

✓ Reasoning

✓ Open Source

✓ Streaming

✗ Audio

✗ Deprecated

✓ Tool Use

✗ Video

✓ Function Calling

✓ Code Generation

✗ Embeddings

✓ JSON Mode

Verification Dimensions

Model Exists

Not Overloaded

Code Generation

Code Optimization

Documentation Gen.

Security Assessment

✓ PASS

✓ PASS

✓ PASS

✓ PASS

✓ PASS

✓ PASS

Responsive

Supports Tools

Code Debugging

Test Generation

Architecture

Pattern Recog.

✓ PASS

✓ PASS

✓ PASS

✓ PASS

✓ PASS

✓ PASS

Rate Limits

Type

Requests/min

Tokens/min

Limit

100

50,000

Used

23

12,340

Remaining

77

37,660

Reset In

14:42:05

14:42:05

Provider Health

Status: ● HEALTHY

Uptime: 99.97%

Last Check: 14:32:01

P95 Latency: 14:42:05

Alternative Models

If unavailable, HelixCode will auto-select:

1. claude-sonnet-4-6 (Anthropic) – Score: 9.0 – Price: \$3.00/1K

2. gpt-4o (OpenAI) – Score: 9.1 – Price: \$5.00/1K

3. gemini-2.5-pro (Google) – Score: 8.7 – Price: \$1.25/1K

Tags

coding, reasoning, long-context, enterprise, premium

Languages

en, es, fr, de, it, pt, zh, ja, ko, ar, hi, ru

5.3 Compact Detail View (60-99 columns)

ASCII Mockup:

HelixCode – Model Details	
claude-opus-4-6	Anthropic ● HEALTHY
Status: ✓ VERIFIED	Score: 9.4 (Excellent)

Tier: ★★★★★ Premium Latency: 234ms Fast
Verified: 2026-04-30 Context: 200K tokens MaxOut: 4,096 tokens

— Pricing —

Input: \$15.00/1K Output: \$75.00/1K Cached: \$7.50/1K
[REDACTED] Expensive

— Capabilities —

✓ vision ✓ streaming ✓ tools ✓ code ✓ reasoning
x audio x video x embeddings

— Verification —

exists ✓ responsive ✓ overloaded x tools ✓ code ✓ debug ✓
optimize ✓ test ✓ docs ✓ architecture ✓ security ✓ patterns ✓

— Rate Limits —

req/min: 100 limit, 23 used, 77 remaining (resets 14:42:05)
tok/min: 50K limit, 12K used, 38K remaining (resets 14:42:05)

— Fallbacks —

1. claude-sonnet-4-6 (9.0, \$3.00/1K)
2. gpt-4o (9.1, \$5.00/1K)
3. gemini-2.5-pro (8.7, \$1.25/1K)

5.4 Narrow Detail View (< 60 columns)

ASCII Mockup:

```
Model: claude-opus-4-6
Provider: Anthropic [+]
Status: VERIFIED [OK]
Score: 9.4
Latency: 234ms [FAST]
Context: 200K / MaxOut: 4096
Price In: $15.00/1K Out: $75.00/1K
[REDACTED] Expensive

Capabilities:
OK vision streaming tools code reasoning
NO audio video embeddings

Verification:
OK: exists responsive tools code debug
    optimize test docs arch security

Rate Limits:
req/min: 100, 23 used, 77 left
tok/min: 50K, 12K used, 38K left
```

```
Reset: 14:42:05
```

```
Fallbacks:
```

1. claude-sonnet-4-6 (9.0, \$3)
2. gpt-4o (9.1, \$5)
3. gemini-2.5-pro (8.7, \$1.2)

5.5 Detail View Implementation

```
// internal/cli/ux/detail_display.go
```

```
package ux
```

```
import (  
    "fmt"  
    "strings"  
    "time"  
)
```

```
type DetailDisplayOptions struct {  
    Format          string // "rich", "json", "yaml"  
    NoColor         bool  
    NoEmoji         bool  
    TerminalWidth  int  
}
```

```
func RenderModelDetail(model *UnifiedModel, provider  
    *UnifiedProvider,  
    verification *VerificationResult, limits *RateLimitStatus,  
    cooldown *CooldownInfo, alternatives []*UnifiedModel,  
    opts *DetailDisplayOptions) string {  
  
    sym := NewSymbolSet(DetectTerminalCapabilities())  
    if opts.NoEmoji {  
        sym = NewSymbolSet(&TerminalCapabilities{IsWindowsCMD:  
            true})  
    }  
    if opts.NoColor {  
        color.NoColor = true  
    }  
  
    switch opts.Format {  
    case "json":  
        return renderDetailJSON(model, provider, verification,  
            limits, cooldown, alternatives)  
    case "yaml":  
        return renderDetailYAML(model, provider, verification,  
            limits, cooldown, alternatives)  
    }
```

```

    default:
        return renderDetailRich(model, provider, verification,
                                limits, cooldown, alternatives, sym, opts.TerminalWidth)
    }
}

func renderDetailRich(m *UnifiedModel, p *UnifiedProvider, v
    *VerificationResult,
    limits *RateLimitStatus, cd *CooldownInfo, alts
    []*UnifiedModel,
    sym *SymbolSet, width int) string {

    var b strings.Builder

    if width >= 100 {
        b.WriteString(renderDetailHeaderWide(m, p, sym, width))
        b.WriteString(renderScorePanelWide(v, sym, width))
        b.WriteString(renderContextPanelWide(m, sym, width))
        b.WriteString(renderPricingPanelWide(m, sym, width))
        b.WriteString(renderCapabilitiesPanelWide(m, v, sym,
            width))
        b.WriteString(renderVerificationPanelWide(v, sym, width))
        if limits != nil {
            b.WriteString(renderRateLimitPanelWide(limits, sym,
                width))
        }
        b.WriteString(renderProviderHealthPanelWide(p, sym,
            width))
        if len(alts) > 0 {
            b.WriteString(renderAlternativesPanelWide(alts, sym,
                width))
        }
    } else if width >= 60 {
        b.WriteString(renderDetailHeaderCompact(m, p, sym,
            width))
        b.WriteString(renderScorePanelCompact(v, sym, width))
        b.WriteString(renderContextPanelCompact(m, sym, width))
        b.WriteString(renderPricingPanelCompact(m, sym, width))
        b.WriteString(renderCapabilitiesPanelCompact(m, v, sym,
            width))
        b.WriteString(renderVerificationPanelCompact(v, sym,
            width))
        if limits != nil {
            b.WriteString(renderRateLimitPanelCompact(limits,
                sym, width))
        }
        if len(alts) > 0 {

```

```

        b.WriteString(renderAlternativesPanelCompact(alts,
            sym, width))
    }
} else {
    b.WriteString(renderDetailHeaderNarrow(m, p, sym, width))
    b.WriteString(renderScorePanelNarrow(v, sym, width))
    b.WriteString(renderPricingPanelNarrow(m, sym, width))
    b.WriteString(renderCapabilitiesPanelNarrow(m, v, sym,
        width))
    b.WriteString(renderVerificationPanelNarrow(v, sym,
        width))
    if len(alts) > 0 {
        b.WriteString(renderAlternativesPanelNarrow(alts,
            sym, width))
    }
}

return b.String()
}

```

// Wide header

```

func renderDetailHeaderWide(m *UnifiedModel, p *UnifiedProvider,
    sym *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CHeader.Sprintf(" %s HelixCode – Model
        Details\n", sym.Diamond))
    b.WriteString(CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))
    b.WriteString("\n")

    healthBadge := ProviderHealthBadge(p.Status,
        sym).Render(sym, width)
    nameLine := fmt.Sprintf(" %-50s %s %s", m.DisplayName,
        p.DisplayName, healthBadge)
    b.WriteString(CAccent.Sprintf("%s\n", nameLine))
    b.WriteString(CBorder.Sprintf(" %s\n", strings.Repeat("=",
        width-5)))
    b.WriteString("\n")

    return b.String()
}

```

// Score panel with bar visualization

```

func renderScorePanelWide(v *VerificationResult, sym *SymbolSet,
    width int) string {
    var b strings.Builder

    overallLabel := CLabel.Sprint(" Overall Score ")

```

```

overallBar := ScoreBadge(v.OverallScore, sym)
overallDesc := ""
switch {
case v.OverallScore >= 9.0: overallDesc =
    CScoreExcellent.Sprint("(Excellent)")
case v.OverallScore >= 7.0: overallDesc =
    CScoreGood.Sprint("(Good)")
case v.OverallScore >= 5.0: overallDesc =
    CScoreAverage.Sprint("(Average)")
case v.OverallScore >= 3.0: overallDesc =
    CScorePoor.Sprint("(Poor)")
default: overallDesc = CScoreBad.Sprint("(Bad)")
}

b.WriteString(fmt.Sprintf("  %s %s %s\n", overallLabel,
    overallBar, overallDesc))
b.WriteString(fmt.Sprintf("  %s %s\n", CLabel.Sprint("  Code
    Capability "), ScoreBadge(v.CodeCapabilityScore, sym)))
b.WriteString(fmt.Sprintf("  %s %s\n", CLabel.Sprint("
    Responsiveness "), ScoreBadge(v.ResponsivenessScore,
    sym)))
b.WriteString(fmt.Sprintf("  %s %s\n", CLabel.Sprint("
    Reliability      "), ScoreBadge(v.ReliabilityScore,
    sym)))
b.WriteString(fmt.Sprintf("  %s %s\n", CLabel.Sprint("
    Feature Richness"), ScoreBadge(v.FeatureRichnessScore,
    sym)))
b.WriteString(fmt.Sprintf("  %s %s\n",
    CLabel.Sprint("  Value Prop.      "),
    ScoreBadge(v.ValuePropositionScore, sym)))
b.WriteString("\n")

return b.String()
}

// Pricing panel with visual bar
func renderPricingPanelWide(m *UnifiedModel, sym *SymbolSet,
    width int) string {
    var b strings.Builder

    b.WriteString(CSubheader.Sprintf("  — Pricing (per 1K
        tokens) %s\n", strings.Repeat("-", width-35)))

    inPrice := m.CostPerInputToken * 1000
    outPrice := m.CostPerOutputToken * 1000
    cachedPrice := inPrice * 0.5 // typical cached rate

```

```

priceLine := fmt.Sprintf("  Input   %s%6.2f      Output
                        %s%6.2f      Cached Input   %s%6.2f",
                        sym.Dollar, inPrice, sym.Dollar, outPrice, sym.Dollar,
                        cachedPrice)
b.WriteString(CValue.Sprintf("%s\n", priceLine))

// Price intensity bar
avgPrice := (inPrice + outPrice) / 2.0
barWidth := width - 6
filled := int((avgPrice / 20.0) * float64(barWidth)) // max
                        $20 = full bar
if filled > barWidth { filled = barWidth }
if filled < 0 { filled = 0 }

barColor := CBarGood
if avgPrice > 2.0 { barColor = CBarAverage }
if avgPrice > 5.0 { barColor = CBarPoor }

barStr := barColor.Sprintf("%s",
    strings.Repeat(sym.ProgressFull, filled)) +
    CBarEmpty.Sprintf("%s",
    strings.Repeat(sym.ProgressEmpty, barWidth-filled))
b.WriteString(fmt.Sprintf("  %s\n", barStr))

priceLabel := ""
switch {
case avgPrice == 0: priceLabel = CPriceFree.Sprint("FREE")
case avgPrice < 0.5: priceLabel = CPriceCheap.Sprint("Cheap")
case avgPrice < 2.0: priceLabel =
    CPriceModerate.Sprint("Moderate")
default: priceLabel = CPriceExpensive.Sprint("Expensive")
}
b.WriteString(fmt.Sprintf("  %s %s\n", sym.ArrowUp,
    priceLabel))
b.WriteString("\n")

return b.String()
}

// Capabilities grid
func renderCapabilitiesPanelWide(m *UnifiedModel, v
    *VerificationResult, sym *SymbolSet, width int) string {
var b strings.Builder
b.WriteString(CSubheader.Sprintf("  — Capabilities %s\n",
    strings.Repeat("-", width-22)))

caps := []struct{
    label string

```



```

        val bool
        sym string
    }{
        {"Vision", m.SupportsVision, sym.Vision},
        {"Streaming", m.SupportsStreaming, sym.Streaming},
        {"Tool Use", m.SupportsTools, sym.Tools},
        {"Code Generation", v.SupportsCodeGeneration, sym.Code},
        {"Reasoning", v.SupportsReasoning, sym.Reasoning},
        {"Audio", m.SupportsAudio, sym.Audio},
        {"Video", m.SupportsVideo, sym.Video},
        {"Embeddings", v.SupportsEmbeddings, sym.Embeddings},
        {"Open Source", m.OpenSource, sym.OpenSource},
        {"Deprecated", m.Deprecated, "△"},
        {"Function Calling", m.SupportsFunctions, sym.Tools},
        {"JSON Mode", v.SupportsJSONMode, "{ }"},
    }

    // 4 columns
    colWidth := (width - 8) / 4
    for i := 0; i < len(caps); i += 4 {
        lineParts := []string{}
        for j := 0; j < 4 && i+j < len(caps); j++ {
            c := caps[i+j]
            status := CFailed.Sprintf("x")
            if c.val { status = CVerified.Sprintf("✓") }
            part := fmt.Sprintf(" %s %-18s", status, c.label)
            lineParts = append(lineParts, part)
        }
        b.WriteString(strings.Join(lineParts, "") + "\n")
    }
    b.WriteString("\n")

    return b.String()
}

// Verification dimensions
func renderVerificationPanelWide(v *VerificationResult, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CSubheader.Sprintf(" — Verification
        Dimensions %s\n", strings.Repeat("-", width-33)))

    checks := []struct{ name string; pass bool }{
        {"Model Exists", v.ModelExists},
        {"Responsive", v.Responsive},
        {"Not Overloaded", !v.Overloaded},
        {"Supports Tools", v.SupportsToolUse},
        {"Code Generation", v.SupportsCodeGeneration},
    }

```

```

        {"Code Debugging", v.CodeDebugging},
        {"Code Optimization", v.CodeOptimization},
        {"Test Generation", v.TestGeneration},
        {"Documentation Gen.", v.DocumentationGeneration},
        {"Architecture", v.ArchitectureDesign},
        {"Security Assessment", v.SecurityAssessment},
        {"Pattern Recognition", v.PatternRecognition},
    }

    // 2 columns
    mid := (len(checks) + 1) / 2
    for i := 0; i < mid; i++ {
        left := checks[i]
        leftStr := fmt.Sprintf(" %s %-22s",
            passFail(left.pass), left.name)

        rightStr := ""
        if i+mid < len(checks) {
            right := checks[i+mid]
            rightStr = fmt.Sprintf(" %s %-22s",
                passFail(right.pass), right.name)
        }
        b.WriteString(leftStr + rightStr + "\n")
    }
    b.WriteString("\n")

    return b.String()
}

func passFail(p bool) string {
    if p { return CVerified.Sprint("✓") }
    return CFailed.Sprint("✗")
}

// Rate limit panel
func renderRateLimitPanelWide(limits *RateLimitStatus, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CSubheader.Sprintf(" — Rate Limits %s\n",
        strings.Repeat("-", width-21)))
    b.WriteString(fmt.Sprintf(" %-16s %-8s %-8s %-12s %s\n",
        "Type", "Limit", "Used", "Remaining", "Reset In"))

    for _, l := range limits.Limits {
        resetStr := formatDuration(time.Until(l.ResetTime))
        b.WriteString(fmt.Sprintf(" %-16s %-8d %-8d %-12d %s\n",
            l.Type, l.Limit, l.Used, l.Remaining, resetStr))
    }
}

```

```

    b.WriteString("\n")
    return b.String()
}

// Provider health
func renderProviderHealthPanelWide(p *UnifiedProvider, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CSubheader.Sprintf(" — Provider Health
        %s\n", strings.Repeat("-", width-25)))

    healthBadge := ProviderHealthBadge(p.Status,
        sym).Render(sym, width)
    uptimeStr := fmt.Sprintf("%.2f%", p.UptimePct)
    lastCheck := p.LastHealthCheck.Format("15:04:05")

    b.WriteString(fmt.Sprintf(" Status: %s      Uptime: %s
        Last Check: %s      P95 Latency: %s\n",
        healthBadge, CValue.Sprint(uptimeStr),
        CValue.Sprint(lastCheck),
        CValue.Sprint(formatLatency(p.Latency))))
    b.WriteString("\n")
    return b.String()
}

// Alternatives panel
func renderAlternativesPanelWide(alts []*UnifiedModel, sym
    *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CSubheader.Sprintf(" — Alternative Models
        %s\n", strings.Repeat("-", width-28)))

    b.WriteString(CAlertInfo.Sprintf(" If unavailable,
        HelixCode will auto-select:\n"))

    for i, alt := range alts {
        if i >= 5 { break }
        price := (alt.CostPerInputToken +
            alt.CostPerOutputToken) / 2.0 * 1000
        b.WriteString(fmt.Sprintf(" %d. %s (%s) – Score: %s –
            Price: %s%.2f/1K\n",
            i+1, alt.DisplayName, alt.Provider,
            GetScoreColor(alt.Score).Printf("%.1f", alt.Score),
            sym.Dollar, price))
    }
    b.WriteString("\n")
    return b.String()
}

```

```

}

func formatDuration(d time.Duration) string {
    if d < 0 { return "now" }
    if d < time.Minute { return fmt.Sprintf("%ds",
        int(d.Seconds())) }
    if d < time.Hour { return fmt.Sprintf("%dm",
        int(d.Minutes())) }
    return fmt.Sprintf("%dh%dm", int(d.Hours()), int(d.Minutes())
        %60)
}

func formatLatency(d time.Duration) string {
    if d < time.Millisecond { return fmt.Sprintf("%dμs",
        d.Microseconds()) }
    if d < time.Second { return fmt.Sprintf("%dms",
        d.Milliseconds()) }
    return fmt.Sprintf("%.1fs", d.Seconds())
}

```

6. Interactive Model Selection

6.1 User Flow

```

$ ./cli
helix> models

```

HelixCode – Model Selector	
Model List 1 ★ claude-opus-4-6 2 gpt-4o 3 gemini-2.5-pro 4 deepseek-chat 5 grok-3-fast-beta 6 llama-3.3-70b 7 mistral-large 8 groq-llama-3.1-8b 9 claude-sonnet-4-5 10 qwen-2.5-72b 11 openrouter-mixtral 12 ollama-llama3.2 13 llamacpp-mistral-7b	Preview claude-opus-4-6 Anthropic ● HEALTHY Score: 9.4 (Excellent) Latency: 234ms Fast Price: \$15.00/1K (input) / \$75.00 Context: 200K tokens Max Out: 4,096 tokens Capabilities: ✓ vision ✓ streaming ✓ tools ✓ code ✓ reasoning ✗ audio ✗ video ✗ embeddings Verification: ✓ VERIFIED on 2026-04-30 08:15 UT

[Use this model] [View full deta

Filter: [all] Sort: [score▼] Group: [none]

[f]ilter [s]ort [g]roup [r]efresh [q]uit [↑↓]navigate [Enter]sele

Status: ● 18 healthy ● 1 degraded „ 2 cooldown

6.2 TUI Architecture (using tview)

```
// internal/cli/tui/model_selector.go
```

```
package tui
```

```
import (
```

```
    "context"
```

```
    "fmt"
```

```
    "strings"
```

```
    "time"
```

```
    "github.com/fatih/color"
```

```
    "github.com/rivo/tview"
```

```
)
```

```
// ModelSelectorApp is the interactive model selection TUI
```

```
type ModelSelectorApp struct {  
    app            *tview.Application  
    modelList      *tview.List  
    previewPane    *tview.TextView  
    statusBar      *tview.TextView  
    filterInput    *tview.InputField
```

```
    models         []*ModelListRow
```

```
    filtered        []*ModelListRow
```

```
    selectedIndex  int
```

```
    sym            *ux.SymbolSet
```

```
    refreshTicker  *time.Ticker
```

```
    cancelFunc     context.CancelFunc
```

```
// Callback when user selects a model
```

```
    onSelect      func(modelID string)
```

```
}
```

```
func NewModelSelectorApp(models []*ux.ModelListRow, sym  
    *ux.SymbolSet) *ModelSelectorApp {  
    m := &ModelSelectorApp{
```

```

        app:      tview.NewApplication(),
        models:   models,
        filtered:  models,
        sym:      sym,
    }
    m.buildUI()
    return m
}

func (m *ModelSelectorApp) buildUI() {
    // Model list (left pane)
    m.modelList = tview.NewList()
    m.modelList.SetBorder(true)
    m.modelList.SetTitle(" Model List ")
    m.modelList.SetMainTextColor(tview.ColorWhite)
    m.modelList.SetSecondaryTextColor(tview.ColorDarkGray)
    m.modelList.SetSelectedBackgroundColor(tview.ColorDarkCyan)

    m.populateModelList()

    m.modelList.SetSelectedFunc(func(idx int, mainText string,
        secondaryText string, shortcut rune) {
        if idx >= 0 && idx < len(m.filtered) {
            m.onSelect(m.filtered[idx].Model.ID)
            m.app.Stop()
        }
    })

    m.modelList.SetChangedFunc(func(idx int, mainText string,
        secondaryText string, shortcut rune) {
        if idx >= 0 && idx < len(m.filtered) {
            m.updatePreview(m.filtered[idx])
        }
    })

    // Preview pane (right pane)
    m.previewPane = tview.NewTextView()
    m.previewPane.SetBorder(true)
    m.previewPane.SetTitle(" Preview ")
    m.previewPane.SetDynamicColors(true)
    m.previewPane.SetScrollable(true)

    // Status bar (bottom)
    m.statusBar = tview.NewTextView()
    m.statusBar.SetDynamicColors(true)
    m.statusBar.SetTextAlign(tview.AlignLeft)

    // Filter input

```

```

m.filterInput = tview.NewInputField()
m.filterInput.SetLabel("Filter: ")
m.filterInput.SetFieldBackgroundColor(tview.ColorBlack)
m.filterInput.SetDoneFunc(func(key tcell.Key) {
    m.applyFilter(m.filterInput.GetText())
})

// Layout
mainFlex := tview.NewFlex().
    AddItem(m.modelList, 0, 1, true).
    AddItem(m.previewPane, 0, 2, false)

fullLayout := tview.NewFlex().SetDirection(tview.FlexRow).
    AddItem(mainFlex, 0, 1, true).
    AddItem(m.filterInput, 1, 0, false).
    AddItem(m.statusBar, 1, 0, false)

m.app.SetRoot(fullLayout, true)

// Key bindings
m.app.SetInputCapture(func(event *tcell.EventKey)
    *tcell.EventKey {
        switch event.Rune() {
        case 'q', 'Q':
            m.app.Stop()
            return nil
        case 'r', 'R':
            m.triggerRefresh()
            return nil
        case 'f', 'F':
            m.app.SetFocus(m.filterInput)
            return nil
        }
        return event
    })

// Initial preview
if len(m.filtered) > 0 {
    m.updatePreview(m.filtered[0])
}
m.updateStatusBar()
}

func (m *ModelSelectorApp) populateModelList() {
    m.modelList.Clear()
    for i, r := range m.filtered {
        tierPrefix := " "
        if r.Provider.Tier == 1 { tierPrefix = "★ " }
    }
}

```

```

mainText := fmt.Sprintf("%s%s", tierPrefix,
    r.Model.DisplayName)

// Secondary text with status, score, price
statusBadge :=
    ux.VerificationBadge(r.Verification.Status,
        m.sym).Symbol
scoreStr := fmt.Sprintf("%.1f",
    r.Verification.OverallScore)
priceStr := ux.PriceBadge(r.Model.CostPerInputToken,
    r.Model.CostPerOutputToken, m.sym, 80)

secondaryText := fmt.Sprintf("  %s %s  %s  %s  %s",
    statusBadge, scoreStr, priceStr,
    r.Provider.DisplayName,
    ux.CapabilityStripCompact(r.Model, m.sym))

m.modelList.AddItem(mainText, secondaryText, rune('0'+
    ((i+1)%10)), nil)
}

}

func (m *ModelSelectorApp) updatePreview(r *ux.ModelListRow) {
    // Render compact detail view into preview pane
    detail := ux.RenderCompactDetailForPreview(r, m.sym)
    m.previewPane.SetText(detail)
}

func (m *ModelSelectorApp) updateStatusBar() {
    counts := ux.CountStatuses(m.filtered)
    text := fmt.Sprintf(
        " [green]● %d healthy[-]  [yellow]● %d degraded[-]
        [red]■ %d cooldown[-]  [gray]○ %d offline[-]  |  [blue]
        [f]ilter [s]ort [g]roup [r]efresh [q]uit[-]",
        counts.Healthy, counts.Degraded, counts.Cooldown,
        counts.Offline)
    m.statusBar.SetText(text)
}

func (m *ModelSelectorApp) applyFilter(query string) {
    query = strings.ToLower(strings.TrimSpace(query))
    if query == "" {
        m.filtered = m.models
    } else {
        m.filtered = []*ux.ModelListRow{}
        for _, r := range m.models {

```



```

        if
strings.Contains(strings.ToLower(r.Model.DisplayName),
query) ||

strings.Contains(strings.ToLower(r.Provider.DisplayName),
query) ||
        strings.Contains(strings.ToLower(r.Model.ID),
query) ||

strings.Contains(strings.ToLower(strings.Join(r.Model.Capabilities
" ")), query) {
        m.filtered = append(m.filtered, r)
    }
}
}
m.populateModelList()
if len(m.filtered) > 0 {
    m.updatePreview(m.filtered[0])
}
m.updateStatusBar()
}

func (m *ModelSelectorApp) triggerRefresh() {
    // Signal to background goroutine to refresh data
    // Updates models slice, re-applies filter, repopulates list
}

func (m *ModelSelectorApp) Run() error {
    return m.app.Run()
}

func (m *ModelSelectorApp) GetSelectedModel() string {
    if m.selectedIndex >= 0 && m.selectedIndex < len(m.filtered)
    {
        return m.filtered[m.selectedIndex].Model.ID
    }
    return ""
}

```

6.3 Fallback: Numbered Menu Mode (for terminals without tview support)

If tview cannot initialize (e.g., non-TTY, CI environment, minimal terminal), fall back to a numbered interactive menu:

```

// internal/cli/ux/interactive_fallback.go

```

```

func RenderNumberedModelMenu(models []*ux.ModelListRow, sym
    *ux.SymbolSet, width int) string {
    var b strings.Builder

    b.WriteString(ux.CHeader.Sprintf("%s HelixCode – Model
        Selector\n", sym.Diamond))
    b.WriteString(ux.CBorder.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))
    b.WriteString("\n")

    for i, r := range models {
        status := ux.VerificationBadge(r.Verification.Status,
            sym).Symbol
        score := fmt.Sprintf("%.1f", r.Verification.OverallScore)
        price := ux.PriceBadge(r.Model.CostPerInputToken,
            r.Model.CostPerOutputToken, sym, width)

        b.WriteString(fmt.Sprintf("  [%s%d%s] %-30s %s %s
            %s %s %s\n",
                ux.CAccent.Sprint(), i+1, "[white]",
                ux.Truncate(r.Model.DisplayName, 30),
                status, score, price, r.Provider.DisplayName,
                ux.CapabilityStripCompact(r.Model, sym)))
    }

    b.WriteString("\n")

    b.WriteString(ux.CSubheader.Sprint("  Enter number to
        select, [f]ilter, [s]ort, [q]uit: "))

    return b.String()
}

```

```

func RunNumberedInteractiveSelector(models []*ux.ModelListRow,
    sym *ux.SymbolSet) (string, error) {
    reader := bufio.NewReader(os.Stdin)
    current := models

    for {
        width, _, _ := term.GetSize(int(os.Stdout.Fd()))
        if width < 60 { width = 80 }

        fmt.Print(RenderNumberedModelMenu(current, sym, width))

        fmt.Print("\n> ")
        input, err := reader.ReadString('\n')
        if err != nil { return "", err }
    }
}

```

```

input = strings.TrimSpace(strings.ToLower(input))

switch input {
case "q", "quit", "exit":
    return "", fmt.Errorf("selection cancelled")
case "f", "filter":
    fmt.Print("Filter by name/provider/capability: ")
    filter, _ := reader.ReadString('\n')
    current = applyFilterString(models,
strings.TrimSpace(filter))
case "s", "sort":
    fmt.Print("Sort by [score/price/name/latency]: ")
    sortBy, _ := reader.ReadString('\n')
    current = applySort(current,
strings.TrimSpace(sortBy))
default:
    // Try to parse as number
    if num, err := strconv.Atoi(input); err == nil &&
num > 0 && num <= len(current) {
        return current[num-1].Model.ID, nil
    }

    fmt.Println("Invalid selection. Please enter a number or
command.")
}
}
}

```

7. Notification / Alert UX

7.1 Alert Types & Rendering

```

// internal/cli/ux/alerts.go

type AlertLevel int

const (
    AlertInfo AlertLevel = iota
    AlertWarning
    AlertError
    AlertSuccess
)

type Alert struct {
    Level      AlertLevel
    Title      string
}

```

```

    Message    string
    ModelID    string
    Provider    string
    SuggestedAlternative string
    Timestamp time.Time
}

func (a *Alert) Render(sym *SymbolSet, width int) string {
    var b strings.Builder

    icon := ""
    titleColor := ux.CAlertInfo
    borderColor := ux.CBorder

    switch a.Level {
    case AlertInfo:
        icon = sym.Bullet
        titleColor = ux.CAlertInfo
    case AlertWarning:
        icon = sym.Degraded
        titleColor = ux.CAlertWarning
    case AlertError:
        icon = sym.Failed
        titleColor = ux.CAlertError
    case AlertSuccess:
        icon = sym.Verified
        titleColor = ux.CAlertSuccess
    }

    // Alert box
    b.WriteString(borderColor.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))
    b.WriteString(fmt.Sprintf("  %s %s\n", icon,
        titleColor.Sprint(a.Title)))
    b.WriteString(fmt.Sprintf("  %s\n", a.Message))

    if a.SuggestedAlternative != "" {
        b.WriteString(fmt.Sprintf("  %s Suggested alternative:
%s\n", sym.ArrowRight, a.SuggestedAlternative))
    }

    b.WriteString(borderColor.Sprintf("%s\n",
        strings.Repeat(sym.SepHorizontal, width-1)))

    return b.String()
}

```

7.2 Alert Scenarios

A. Model Enters Cooldown During Session:

```
» COOLDOWN ALERT
Model "groq-llama-3.1-70b" has entered rate-limited cooldown.
Reason: Rate limit exceeded (150/100 req/min). Reset in 12m 34s.
→ Suggested alternative: llama-3.3-70b (Groq) – Score: 7.5
```

B. Provider Becomes Unavailable:

```
x PROVIDER OFFLINE
Provider "xAI" is now OFFLINE. All xAI models are unavailable.
Affected models: grok-3-fast-beta, grok-3-mini, grok-3
→ Suggested alternatives: gpt-4o (OpenAI), gemini-2.5-pro (Google)
```

C. Better Alternative Discovered:

```
✓ BETTER MODEL AVAILABLE
A higher-scoring alternative to "gpt-4o" is now available:
claude-opus-4-6 – Score: 9.4 (vs your current 9.1)
Same price range. Better code capability and reasoning.
→ Switch? [Y/n]
```

D. LLMsVerifier Connection Lost:

```
⚠ VERIFIER CONNECTION LOST
Lost connection to LLMsVerifier at http://localhost:8081.
Model verification data may be stale. Last update: 14:32:05.
→ Auto-retry in 30s... [r]etry now [c]ontinue with cached data
```

7.3 Auto-Suggest on Selection of Unavailable Model

```
$ ./cli --prompt "Hello" --model groq-llama-3.1-70b
```

```
⚠ SELECTED MODEL UNAVAILABLE
"groq-llama-3.1-70b" is currently in cooldown (rate-limited).

Auto-switch to best available alternative?
[1] llama-3.3-70b (Groq) – Score: 7.5 – $0.90/1K [RECOMMENDED]
[2] claude-sonnet-4-5 (Anthropic) – Score: 7.8 – $3.00/1K
[3] gpt-4o (OpenAI) – Score: 9.1 – $5.00/1K
```

```
[4] deepseek-chat (DeepSeek) – Score: 8.3 – $0.14/1K
[5] Cancel and exit

Select [1-5] or press Enter for default [1]:
```

8. Real-time Updates Display

8.1 Status Bar Design

A persistent status bar at the bottom of all interactive model views:

Wide Status Bar:

```
● 18 models active | ☐ 2 cooldown | ○ 1 degraded | ⬇ last refresh:
```

Narrow Status Bar:

```
18 OK | 2 RL | 1 ~ | refresh: 14:32:05
```

8.2 Refresh Indicator

When refresh is in progress:

```
● 18 models active | ☐ 2 cooldown | ○ 1 degraded | 🔄 refreshing...
```

When refresh completes:

```
● 18 models active | ☐ 2 cooldown | ○ 1 degraded | ✓ refreshed 14:
```

8.3 Non-Clutter Update Strategy

For non-interactive mode (standard `--list-models` output), updates are NOT shown in real-time. The output is a snapshot.

For interactive TUI mode: - Background goroutine polls `LLMsVerifier` every 30-60 seconds - Only shows update when state CHANGES (model status, score, cooldown) - Updates use color flash or brief banner (disappears after 5s) - No scrolling disruption — updates appear in status bar only

```
// internal/cli/ux/status_bar.go
```

```
type StatusBar struct {
    sym          *ux.SymbolSet
    totalModels  int
    activeModels int
    cooldownCount int
    degradedCount int
    offlineCount int
    lastRefresh  time.Time
    isRefreshing bool
    width        int
}

func (sb *StatusBar) Render() string {
    var b strings.Builder

    if sb.width >= 100 {
        b.WriteString(fmt.Sprintf(" %s %d models active %s %s %d cooldown %s %s %d degraded",
            sb.sym.Healthy, sb.activeModels,
            sb.sym.SepVertical,
            sb.sym.CoolDown, sb.cooldownCount,
            sb.sym.SepVertical,
            sb.sym.Degraded, sb.degradedCount))
        if sb.isRefreshing {
            b.WriteString(fmt.Sprintf(" %s %s refreshing...",
                sb.sym.SepVertical, sb.sym.Pending))
        } else {
            b.WriteString(fmt.Sprintf(" %s %s last refresh: %s",
                sb.sym.SepVertical, sb.sym.Verified,
                sb.lastRefresh.Format("15:04:05")))
        }
    } else {
        b.WriteString(fmt.Sprintf(" %d OK | %d %s | %d %s | ",
            sb.activeModels, sb.cooldownCount, sb.sym.CoolDown,
            sb.degradedCount, sb.sym.Degraded))
        if sb.isRefreshing {
            b.WriteString(fmt.Sprintf("%s refreshing",
                sb.sym.Pending))
        } else {
            b.WriteString(fmt.Sprintf("refresh: %s",
                sb.lastRefresh.Format("15:04:05")))
        }
    }
}
```

```
    return b.String()  
}
```

8.4 Update Notification Banner (TUI only)

When state changes are detected, a temporary banner appears above the status bar:

UPDATE: groq-llama-3.1-70b is now available (cooldown cleared)
... main content ...
● 19 models active ☐ 1 cooldown ● 1 degraded ↓ last refresh:

Banner auto-dismisses after 5 seconds or on any keypress.

9. Error / Empty States

9.1 LLMsVerifier Disabled

HelixCode – Models
⚠ LLMsVerifier is disabled. Model verification data is unavailable. Showing registered providers on To enable LLMsVerifier: 1. Set HELIX_VERIFIER_ENABLED=true in your environment 2. Or add to config.yaml: llm.verifier.enabled = true 3. Ensure LLMsVerifier is running at http://localhost:8081 [c]ontinue without verification [q]uit

9.2 No Models Pass Validation

HelixCode – Models
⚠ No verified models available All registered models failed verification or are pending.

Possible causes:

- API keys are missing or invalid
- Providers are experiencing outages
- Network connectivity issues to provider APIs
- LLMsVerifier verification queue is backed up

3 models pending verification:

claude-sonnet-4-5 (Anthropic) – queued 5m ago

qwen-2.5-72b (Qwen) – queued 12m ago

gemini-2.5-flash (Google) – queued 18m ago

Actions:

[r]etry verification now [v]iew pending details [q]uit

9.3 All Providers in Cooldown

HelixCode – Models

ALL PROVIDERS IN COOLDOWN

No models are currently available for use.

Cooldown status:

- » Groq – rate limited, reset in 12m 34s
- » xAI – quota exceeded, reset in 47m 12s
- » OpenRouter – temporarily unavailable, reset in 2h 15m

Local providers (no cooldown):

- x Ollama – not running (check <http://localhost:11434>)
- x Llama.cpp – not running (check <http://localhost:8080>)

Suggestions:

1. Wait for rate limits to reset
2. Start a local provider: ollama serve
3. Check your API key configurations
4. Use --provider local to see only local models

[w]ait and retry in 30s [s]tart local provider [q]uit

9.4 Network to Verifier Down

HelixCode – Models

△ Cannot connect to LLMsVerifier

Connection failed to http://localhost:8081/api/v1/verifier
Error: connection refused

Troubleshooting:

1. Is LLMsVerifier running? ./llm-verifier server
2. Check the verifier URL in config.yaml: llm.verifier.url
3. Check firewall / port binding

Fallback options:

[c]ached data (last update: 2026-04-30 14:00) – 23 models
[o]ffline mode – use only locally registered models
[r]etry connection [q]uit

9.5 Empty State Implementation

```
// internal/cli/ux/empty_states.go
```

```
package ux
```

```
func RenderEmptyState(state string, details
    map[string]interface{}, sym *SymbolSet, width int)
    string {
    switch state {
    case "verifier_disabled":
        return renderVerifierDisabled(sym, width)
    case "no_verified_models":
        return renderNoVerifiedModels(details, sym, width)
    case "all_cooldown":
        return renderAllCooldown(details, sym, width)
    case "verifier_unavailable":
        return renderVerifierUnavailable(details, sym, width)
    case "no_models":
        return renderNoModels(sym, width)
    default:
        return renderGenericError(state, sym, width)
    }
}

func renderVerifierDisabled(sym *SymbolSet, width int) string {
    var b strings.Builder
    b.WriteString(CAlertWarning.Sprintf("\n  %s LLMsVerifier is
        disabled.\n\n", sym.Degraded))
    b.WriteString(CValue.Sprint("  Model verification data is
        unavailable. Showing registered providers only.\n\n"))
}
```

```

b.WriteString(CLabel.Sprintf(" To enable LLMsVerifier:\n"))
b.WriteString(CValue.Sprintf("    1. Set
    HELIX_VERIFIER_ENABLED=true in your environment\n"))
b.WriteString(CValue.Sprintf("    2. Or add to config.yaml:
    llm.verifier.enabled = true\n"))
b.WriteString(CValue.Sprintf("    3. Ensure LLMsVerifier is
    running at http://localhost:8081\n\n"))
b.WriteString(CSubheader.Sprintf(" [c]ontinue without
    verification [q]uit\n"))
return b.String()
}

func renderNoVerifiedModels(details map[string]interface{}, sym
    *SymbolSet, width int) string {
var b strings.Builder
b.WriteString(CAlertWarning.Sprintf("\n %s No verified
    models available\n\n", sym.Degraded))
b.WriteString(CValue.Sprintf(" All registered models failed
    verification or are pending.\n\n"))
b.WriteString(CLabel.Sprintf(" Possible causes:\n"))
b.WriteString(CValue.Sprintf("    • API keys are missing or
    invalid\n"))

    b.WriteString(CValue.Sprintf("    • Providers are
    experiencing outages\n"))
b.WriteString(CValue.Sprintf("    • Network connectivity
    issues to provider APIs\n"))
b.WriteString(CValue.Sprintf("    • LLMsVerifier verification
    queue is backed up\n\n"))

if pending, ok := details["pending_models"].
    ([]*UnifiedModel); ok && len(pending) > 0 {
b.WriteString(CLabel.Sprintf(" %d models pending
    verification:\n", len(pending)))
for _, m := range pending {
    b.WriteString(fmt.Sprintf("    %s %s (%s)\n",
        sym.Pending, m.DisplayName, m.Provider))
}
b.WriteString("\n")
}

b.WriteString(CSubheader.Sprintf(" [r]etry verification
    now [v]iew pending details [q]uit\n"))
return b.String()
}

```

```

func renderAllCooldown(details map[string]interface{}, sym
    *SymbolSet, width int) string {

```

```

var b strings.Builder
b.WriteString(CAlertError.Sprintf("\n  %s ALL PROVIDERS IN
  COOLDOWN\n\n", sym.QuotaExceeded))
b.WriteString(CValue.Sprint("  No models are currently
  available for use.\n\n"))

if cooldowns, ok := details["cooldowns"].([]*CooldownInfo);
  ok {
    b.WriteString(CLabel.Sprint("  Cooldown status:\n"))
    for _, cd := range cooldowns {
      badge := CooldownBadge(cd.Reason, cd.ResetTime, sym)
      b.WriteString(fmt.Sprintf("    %s %s - %s\n",
        badge.Symbol, cd.ProviderName, badge.Text))
    }
    b.WriteString("\n")
  }

b.WriteString(CLabel.Sprint("  Suggestions:\n"))
b.WriteString(CValue.Sprint("    1. Wait for rate limits to
  reset\n"))
b.WriteString(CValue.Sprint("    2. Start a local provider:
  ollama serve\n"))
b.WriteString(CValue.Sprint("    3. Check your API key
  configurations\n"))
b.WriteString(CValue.Sprint("    4. Use --provider local to
  see only local models\n\n"))

    b.WriteString(CSubheader.Sprint("  [w]ait and retry in
  30s    [s]tart local provider    [q]uit\n"))
return b.String()
}

func renderVerifierUnavailable(details map[string]interface{},
  sym *SymbolSet, width int) string {
var b strings.Builder

    b.WriteString(CAlertWarning.Sprintf("\n  %s Cannot
  connect to LLMsVerifier\n\n", sym.Offline))

if url, ok := details["url"].(string); ok {
  b.WriteString(fmt.Sprintf("  Connection failed to %s\n",
    url))
}
if err, ok := details["error"].(string); ok {
  b.WriteString(fmt.Sprintf("  Error: %s\n\n", err))
}
}

```

```

b.WriteString(CLabel.Sprint(" Troubleshooting:\n"))
b.WriteString(CValue.Sprint("    1. Is LLMsVerifier
    running?    ./llm-verifier server\n"))

    b.WriteString(CValue.Sprint("    2. Check the verifier
    URL in config.yaml\n"))
b.WriteString(CValue.Sprint("    3. Check firewall / port
    binding\n\n"))

b.WriteString(CLabel.Sprint(" Fallback options:\n"))
if cached, ok := details["cached_time"].(time.Time); ok {
    b.WriteString(fmt.Sprintf("    [c]ached data (last
    update: %s)\n", cached.Format("2006-01-02 15:04")))
}
b.WriteString(CValue.Sprint("    [o]ffline mode – use only
    locally registered models\n"))
b.WriteString(CSubheader.Sprint("    [r]etry connection
    [q]uit\n"))

return b.String()
}

```

10. Go Structs for UX State Management

10.1 Core UX Package Layout

```

internal/cli/ux/
├── symbols.go           # SymbolSet, TerminalCapabilities, platform det
├── colors.go            # All color definitions, GetScoreColor, GetPric
├── badges.go            # Badge rendering: VerificationBadge, ProviderH
├── capabilities.go      # CapabilityStripCompact, CapabilityStripFull
├── list_display.go      # RenderModelList, table/compact/JSON/CSV rende
├── detail_display.go    # RenderModelDetail, rich/JSON/YAML renderers
├── status_bar.go        # StatusBar component
├── alerts.go            # Alert struct and rendering
├── empty_states.go      # All empty state renderers
├── interactive_fallback.go # Numbered menu for non-TTY
└── model_selector_app.go # TUI app wrapper (delegates to tview)

internal/cli/tui/
└── model_selector.go    # Full tview-based interactive selector

```

10.2 Data Structs

```
// internal/cli/ux/types.go
```

```
package ux
```

```

import (
    "time"
)

// UnifiedModel – mirrors HelixAgent's UnifiedModel, sourced from
// LLMsVerifier
type UnifiedModel struct {
    ID            string    `json:"id"`
    Name          string    `json:"name"`
    DisplayName   string    `json:"display_name"`
    Provider      string    `json:"provider"`
    Score         float64   `json:"score"`
    Verified      bool      `json:"verified"`
    Latency       time.Duration `json:"latency"`
    ContextWindow int       `json:"context_window"`
    MaxOutputTokens int      `json:"max_output_tokens"`
    SupportsStreaming bool    `json:"supports_streaming"`
    SupportsTools bool    `json:"supports_tools"`
    SupportsFunctions bool   `json:"supports_functions"`
    SupportsVision bool    `json:"supports_vision"`
    SupportsAudio bool    `json:"supports_audio"`
    SupportsVideo bool    `json:"supports_video"`
    SupportsReasoning bool   `json:"supports_reasoning"`
    Capabilities  []string  `json:"capabilities"`
    CostPerInputToken float64 `json:"cost_per_input_token"`
    CostPerOutputToken float64 `json:"cost_per_output_token"`
    OpenSource    bool     `json:"open_source"`
    Deprecated    bool     `json:"deprecated"`
    Tags          []string  `json:"tags"`
    LanguageSupport []string  `json:"language_support"`
    UseCase       string    `json:"use_case"`
    ReleaseDate   string    `json:"release_date"`
    Architecture  string    `json:"architecture"`
}

// UnifiedProvider – mirrors HelixAgent's UnifiedProvider
type UnifiedProvider struct {
    ID            string    `json:"id"`
    Name          string    `json:"name"`
    DisplayName   string    `json:"display_name"`
    Type          string    `json:"type"`
    AuthType      string    `json:"auth_type"`
    Verified      bool      `json:"verified"`
    Score         float64   `json:"score"`
    ScoreSuffix   string    `json:"score_suffix"`
    TestResults   map[string]bool `json:"test_results"`
    CodeVisible   bool      `json:"code_visible"`
}

```

```

Models      []string      `json:"models"`
DefaultModel string      `json:"default_model"`
Status      string      `json:"status"` // unknown,
             healthy, degraded, unhealthy, offline
BaseURL     string      `json:"base_url"`
Tier        int         `json:"tier"`
Priority     int         `json:"priority"`
UptimePct   float64     `json:"uptime_pct"`
LastHealthCheck time.Time `json:"last_health_check"`
}

// VerificationResult – mirrors LLMsVerifier VerificationResult
type VerificationResult struct {
    Status      string      `json:"status"` //
             verified, pending, failed, not_tested
    ModelExists bool      `json:"model_exists"`
    Responsive  bool      `json:"responsive"`
    Overloaded  bool      `json:"overloaded"`
    LatencyMs   int       `json:"latency_ms"`

    // Feature flags
    SupportsToolUse      bool `json:"supports_tool_use"`
    SupportsCodeGeneration bool `json:"supports_code_generation"`
    SupportsEmbeddings   bool `json:"supports_embeddings"`
    SupportsStreaming    bool `json:"supports_streaming"`
    SupportsJSONMode     bool `json:"supports_json_mode"`
    SupportsReasoning     bool `json:"supports_reasoning"`
    SupportsParallelToolUse bool `json:"supports_parallel_tool_use"`
    SupportsBatchProcessing bool `json:"supports_batch_processing"`
    SupportsBrotli        bool `json:"supports_brotli"`

    // Code capabilities
    CodeDebugging      bool `json:"code_debugging"`
    CodeOptimization    bool `json:"code_optimization"`
    TestGeneration      bool `json:"test_generation"`
    DocumentationGeneration bool `json:"documentation_generation"`
    Refactoring         bool `json:"refactoring"`
    ErrorResolution      bool `json:"error_resolution"`
    ArchitectureDesign   bool `json:"architecture_design"`
    SecurityAssessment    bool `json:"security_assessment"`
    PatternRecognition    bool `json:"pattern_recognition"`

    // Scores (0.0 - 10.0)
    OverallScore float64 `json:"overall_score"`

```

```

CodeCapabilityScore    float64 `json:"code_capability_score"`
ResponsivenessScore    float64 `json:"responsiveness_score"`
ReliabilityScore        float64 `json:"reliability_score"`
FeatureRichnessScore   float64 `json:"feature_richness_score"`
ValuePropositionScore  float64
    `json:"value_proposition_score"`

// Performance
AvgLatencyMs    int    `json:"avg_latency_ms"`
P95LatencyMs    int    `json:"p95_latency_ms"`
MinLatencyMs    int    `json:"min_latency_ms"`
MaxLatencyMs    int    `json:"max_latency_ms"`
ThroughputRps   float64 `json:"throughput_rps"`

Timestamp    time.Time `json:"timestamp"`
Error        string    `json:"error,omitempty"`
}

// CooldownInfo – tracks rate limit / cooldown state
type CooldownInfo struct {
    ModelID      string    `json:"model_id"`
    ProviderName string    `json:"provider_name"`
    Reason       string    `json:"reason"` // rate-limited,
                quota-exceeded, cooldown, temporarily-unavailable
    ResetTime    time.Time `json:"reset_time"`
    LimitType    string    `json:"limit_type"`
    LimitValue   int       `json:"limit_value"`
    CurrentUsage int       `json:"current_usage"`
    Message      string    `json:"message,omitempty"`
}

// RateLimitStatus – current rate limit state for a model
type RateLimitStatus struct {
    ModelID string    `json:"model_id"`
    Limits  []RateLimitEntry `json:"limits"`
}

type RateLimitEntry struct {
    Type        string    `json:"type"`
    Limit       int       `json:"limit"`
    Used        int       `json:"used"`
    Remaining   int       `json:"remaining"`
    ResetTime   time.Time `json:"reset_time"`
    IsHardLimit bool      `json:"is_hard_limit"`
}

// ModelListRow – a single prepared row for list rendering
type ModelListRow struct {

```



```

    Rank            int
    Model            *UnifiedModel
    Provider         *UnifiedProvider
    Verification     *VerificationResult
    Cooldown         *CooldownInfo
}

// UXState – central state for the model display system
type UXState struct {
    Models            []*UnifiedModel
    Providers         map[string]*UnifiedProvider
    Verifications     map[string]*VerificationResult
    Cooldowns         map[string]*CooldownInfo
    RateLimits        map[string]*RateLimitStatus

    LastRefresh       time.Time
    IsRefreshing      bool
    VerifierConnected bool
    VerifierURL       string

    TerminalWidth int
    SymbolSet      *SymbolSet
    NoColor        bool
    NoEmoji        bool
}

func NewUXState() *UXState {
    return &UXState{
        Providers:      make(map[string]*UnifiedProvider),
        Verifications:  make(map[string]*VerificationResult),
        Cooldowns:      make(map[string]*CooldownInfo),
        RateLimits:     make(map[string]*RateLimitStatus),
        SymbolSet:      NewSymbolSet(DetectTerminalCapabilities()),
    }
}

```

10.3 LLMsVerifier Client Struct

```

// internal/verifier/client.go – NEW FILE

package verifier

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"

```

```

        "time"
    )

    const DefaultVerifierURL = "http://localhost:8081/api/v1/
        verifier"

    type Client struct {
        baseURL    string
        httpClient *http.Client
    }

    func NewClient(baseURL string) *Client {
        if baseURL == "" {
            baseURL = DefaultVerifierURL
        }
        return &Client{
            baseURL: baseURL,
            httpClient: &http.Client{Timeout: 30 * time.Second},
        }
    }

    func (c *Client) HealthCheck(ctx context.Context) error {
        req, err := http.NewRequestWithContext(ctx, "GET",
            c.baseURL+"/health", nil)
        if err != nil { return err }
        resp, err := c.httpClient.Do(req)
        if err != nil { return err }
        defer resp.Body.Close()
        if resp.StatusCode != http.StatusOK {
            return fmt.Errorf("verifier health check failed: %d",
                resp.StatusCode)
        }
        return nil
    }

    func (c *Client) GetModels(ctx context.Context)
        ([]*ux.UnifiedModel, error) {
        req, err := http.NewRequestWithContext(ctx, "GET",
            c.baseURL+"/models", nil)
        if err != nil { return nil, err }
        resp, err := c.httpClient.Do(req)
        if err != nil { return nil, err }
        defer resp.Body.Close()

        var models []*ux.UnifiedModel
        if err := json.NewDecoder(resp.Body).Decode(&models); err !=
            nil {
            return nil, err
        }
    }

```

```

    }
    return models, nil
}

func (c *Client) GetModel(ctx context.Context, modelID string)
    (*ux.UnifiedModel, error) {
    url := fmt.Sprintf("%s/models/%s", c.baseURL, modelID)
    req, err := http.NewRequestWithContext(ctx, "GET", url, nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var model ux.UnifiedModel
    if err := json.NewDecoder(resp.Body).Decode(&model); err !=
        nil {
        return nil, err
    }
    return &model, nil
}

func (c *Client) GetVerification(ctx context.Context, modelID
    string) (*ux.VerificationResult, error) {
    url := fmt.Sprintf("%s/models/%s/verification", c.baseURL,
        modelID)
    req, err := http.NewRequestWithContext(ctx, "GET", url, nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var result ux.VerificationResult
    if err := json.NewDecoder(resp.Body).Decode(&result); err !=
        nil {
        return nil, err
    }
    return &result, nil
}

func (c *Client) GetProviderStatus(ctx context.Context)
    (map[string]*ux.UnifiedProvider, error) {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/providers", nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

```

```

var providers map[string]*ux.UnifiedProvider
if err := json.NewDecoder(resp.Body).Decode(&providers);
    err != nil {
        return nil, err
    }
return providers, nil
}

func (c *Client) GetRateLimits(ctx context.Context)
    (map[string]*ux.RateLimitStatus, error) {
    req, err := http.NewRequestWithContext(ctx, "GET",
        c.baseURL+"/limits", nil)
    if err != nil { return nil, err }
    resp, err := c.httpClient.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var limits map[string]*ux.RateLimitStatus
    if err := json.NewDecoder(resp.Body).Decode(&limits); err !=
        nil {
            return nil, err
        }
    return limits, nil
}

```

11. CLI Flag Additions

11.1 New Flags to Add to cmd/cli/main.go

Replace the existing flag block with the following additions:

```

// Existing flags (KEEP)
command      = flag.String("command", "", "Command to execute")
workerHost   = flag.String("worker", "", "Worker host to add")
workerUser   = flag.String("user", "", "Worker SSH username")
workerKey    = flag.String("key", "", "Worker SSH key path")
model        = flag.String("model", "llama-3-8b",
    "LLM model to use")
prompt       = flag.String("prompt", "", "Prompt for LLM
    generation")
maxTokens    = flag.Int("max-tokens", 1000, "Maximum tokens")
temperature  = flag.Float64("temperature", 0.7, "Generation
    temperature")
stream       = flag.Bool("stream", false, "Stream the
    response")
listWorkers  = flag.Bool("list-workers", false, "List all
    workers")

```

```

listModels      = flag.Bool("list-models", false, "List available
                           models")
healthCheck     = flag.Bool("health", false, "Perform health
                           check")
notify          = flag.String("notify", "", "Send notification")
notifyType      = flag.String("notify-type", "info",
                           "Notification type")
notifyPriority  = flag.String("notify-priority", "medium",
                           "Notification priority")
nonInteractive  = flag.Bool("non-interactive", false, "Run in non-
                           interactive mode")

// NEW: Model list filter flags
providerFilter  = flag.String("provider", "", "Filter models by
                           provider name")
verifiedOnly    = flag.Bool("verified-only", false, "Show only
                           verified models")
maxPrice        = flag.Float64("max-price", 0, "Maximum price
                           per 1K tokens (0 = no limit)")
minScore        = flag.Float64("min-score", 0, "Minimum overall
                           score 0-10 (0 = no limit)")
capabilityFilter = flag.String("capability", "", "Filter by
                           capability: vision,streaming,tools,code,reasoning")
sortBy          = flag.String("sort", "score", "Sort models by:
                           score,price,name,provider,latency")
groupBy         = flag.String("group-by", "", "Group models by:
                           provider,tier,status")
outputFormat    = flag.String("format", "table",
                           "Output format: table,compact,json,csv")
noColor         = flag.Bool("no-color", false, "Disable colored
                           output")
noEmoji        = flag.Bool("no-emoji", false, "Disable emoji
                           and Unicode symbols")

// NEW: Model detail flag
modelInfo       = flag.String("model-info", "", "Show detailed
                           information for a model ID")
modelInfoFormat = flag.String("model-info-format", "rich",
                           "Detail format: rich,json,yaml")

// NEW: Interactive TUI flag
interactiveModels = flag.Bool("models-interactive", false,
                           "Launch interactive model selector TUI")

```

11.2 Flag Wiring in cli.Run()

```

func (c *CLI) Run() {
    flag.Parse()

```

```

ctx := context.Background()

switch {
case *listModels:
    c.handleListModels(ctx)
case *modelInfo != "":
    c.handleModelInfo(ctx, *modelInfo)
case *interactiveModels:
    c.handleInteractiveModelSelector(ctx)
case *listWorkers:
    c.handleListWorkers(ctx)
// ... existing cases ...
}
}

```

12. Files to Modify / Create

12.1 Files to Create (NEW)

File	Lines	Purpose
internal/cli/ux/symbols.go	~120	SymbolSet, platform detection, fallbacks
internal/cli/ux/colors.go	~60	All color definitions, score/price color helpers
internal/cli/ux/badges.go	~80	Badge rendering functions
internal/cli/ux/capabilities.go	~60	Capability strip rendering
internal/cli/ux/list_display.go	~300	Model list table/compact/JSON/CSV renderers
internal/cli/ux/detail_display.go	~400	Model detail rich/JSON/YAML renderers
internal/cli/ux/status_bar.go	~50	StatusBar component
internal/cli/ux/alerts.go	~60	Alert struct and rendering
internal/cli/ux/empty_states.go	~200	All empty state renderers
	~100	

File	Lines	Purpose
internal/cli/ux/interactive_fallback.go		Numbered menu for non-TTY fallback
internal/cli/ux/types.go	~150	Core data structs (UnifiedModel, VerificationResult, etc.)
internal/cli/tui/model_selector.go	~250	Full tview-based interactive selector
internal/verifier/client.go	~150	HTTP client for LLMsVerifier API
internal/config/config_verifier.go	~30	LLMsVerifier config section (add to Config struct)

12.2 Files to Modify (EXISTING)

File	Lines	Change
cmd/cli/main.go	101-128	CRITICAL: Replace hardcoded <code>handleListModel()</code> with dynamic fetch from LLMsVerifier
cmd/cli/main.go	~30-50	Add new CLI flags (see Section 11)
cmd/cli/main.go	~200	Add <code>handleModelInfo()</code> , <code>handleInteractiveModelSelector()</code> handlers
cmd/cli/main.go	~250	Wire new flags in <code>cli.Run()</code> switch statement
internal/config/config.go	~260	Add <code>Verifier VerifierConfig</code> field to Config struct
internal/config/config.go	~100	Add verifier defaults in <code>setDefault()</code>
internal/llm/model_discovery.go	~900	Replace <code>fetchExternalModels()</code> hardcoded list with LLMsVerifier fetch
internal/llm/model_manager.go	~280	Add LLMsVerifier status to <code>SelectOptimalModel()</code> scoring
go.mod	-	Add <code>digital.vasic.llmsverifier</code> module dependency

12.3 Config Changes

Add to internal/config/config.go:

```
type VerifierConfig struct {
    Enabled    bool    `mapstructure:"enabled"`
    URL        string  `mapstructure:"url"           // default: http://
                                localhost:8081
    APIKey     string  `mapstructure:"api_key"`
    Timeout    int     `mapstructure:"timeout"       // seconds, default: 30
    CacheTTL   int     `mapstructure:"cache_ttl"     // minutes, default: 5
    AutoRefresh bool    `mapstructure:"auto_refresh"  // default: true
}

// In Config struct:
type Config struct {
    // ... existing fields ...
    Verifier VerifierConfig `mapstructure:"verifier"`
}
```

Add defaults in setDefaults():

```
viper.SetDefault("verifier.enabled", true)
viper.SetDefault("verifier.url", "http://localhost:8081")
viper.SetDefault("verifier.timeout", 30)
viper.SetDefault("verifier.cache_ttl", 5)
viper.SetDefault("verifier.auto_refresh", true)
```

13. Implementation Priority

Phase 1: Foundation (Week 1)

1. Create internal/cli/ux/ package with symbols.go, colors.go, types.go
2. Create internal/verifier/client.go for LLMsVerifier API communication
3. Add verifier config to internal/config/config.go
4. Create basic RenderModelList() for --list-models with real data

Phase 2: Core Display (Week 1-2)

1. Implement `handleListModels()` replacement in `cmd/cli/main.go` (P0 — BLUFF-002 fix)
2. Add all new CLI flags to `cmd/cli/main.go`
3. Implement `RenderModelDetail()` for `--model-info`
4. Implement compact/table/JSON/CSV formatters

Phase 3: Advanced Features (Week 2)

1. Implement `CapabilityStrip`, badges, score bars
2. Implement `--provider`, `--verified-only`, `--max-price`, `--min-score`, `--capability` filters
3. Implement `--sort`, `--group-by` options
4. Implement cross-platform symbol fallback system

Phase 4: Interactive Mode (Week 3)

1. Implement `tview`-based `ModelSelectorApp`
2. Implement numbered menu fallback for non-TTY
3. Wire `models` interactive command to TUI

Phase 5: Polish (Week 3-4)

1. Implement real-time status bar
2. Implement alert/notification system
3. Implement all empty states
4. Integration testing with `LLMsVerifier`
5. Cross-platform testing (Windows `cmd.exe`, PowerShell, macOS, Linux, mobile)

Phase 6: Integration (Week 4)

1. Replace `fetchExternalModels()` hardcoded list in `model_discovery.go`
 2. Add `LLMsVerifier` status to `SelectOptimalModel()` in `model_manager.go`
 3. Add auto-suggest on unavailable model selection
 4. End-to-end challenge tests
-

Appendix A: Quick Reference — Color to Status Mapping

Status	Color Code	Example
Verified	color.FgHiGreen	✓ 9.4
Pending	color.FgHiYellow	7.8
Failed	color.FgHiRed	✗ 4.2
Healthy	color.FgHiGreen	● OpenAI
Degraded	color.FgHiYellow	◐ Groq
Unhealthy	color.FgHiRed	● xAI
Offline	color.FgHiBlack	○ Mistral
Score 9.0+	color.FgHiGreen + bold	9.4
Score 7.0-8.9	color.FgGreen	8.3
Score 5.0-6.9	color.FgYellow	6.5
Score 3.0-4.9	color.FgHiRed	4.2
Score < 3.0	color.FgHiBlack	2.1
Price FREE	color.FgHiCyan + bold	FREE
Price < \$0.50	color.FgHiGreen	\$0.14
Price \$0.50-\$2.00	color.FgYellow	\$1.25
Price > \$2.00	color.FgHiRed	\$15.00

Appendix B: Platform Support Matrix

Platform	Emoji	256 Color	Unicode Box	Recommended SymbolSet
macOS Terminal	✓	✓	✓	Rich
iTerm2	✓	✓	✓	Rich
Linux (GNOME/ Konsole)	✓	✓	✓	Rich
Linux (TTY)	✗	✗	✗	ASCII
Windows cmd.exe (Win10)	✗	✗	✗	Windows CMD
	✓	✓	✗	ASCII

Platform	Emoji	256 Color	Unicode Box	Recommended SymbolSet
Windows cmd.exe (Win11)				
PowerShell	✓	✓	✓	Rich
Windows Terminal	✓	✓	✓	Rich
WSL	✓	✓	✓	Rich
Aurora OS	✓	✓	✓	Rich
Harmony OS	✓	✓	✓	Rich
Android (Termux)	✓	✓	✓	Rich
iOS (iSH/a-Shell)	✓	✓	✓	Rich
CI/Non-TTY	✗	✗	✗	ASCII

Appendix C: API Endpoint Mapping (LLMsVerifier → HelixCode)

LLMsVerifier Endpoint	HelixCode Client Method	Data Used
GET /api/v1/verifier/health	client.HealthCheck()	Connection status
GET /api/v1/verifier/models	client.GetModels()	Model list display
GET /api/v1/verifier/models/{id}	client.GetModel()	Model detail display
GET /api/v1/verifier/models/{id}/verification	client.GetVerification()	Status badges, scores
GET /api/v1/verifier/providers	client.GetProviderStatus()	Provider health, grouping
GET /api/v1/verifier/limits	client.GetRateLimits()	Rate limit panels, cooldown alerts
WS /ws/verifier/events	WebSocket client (future)	Real-time updates

End of UX Design Specification

[End of Section 5]

Anti-Bluff Testing Strategy: LLMsVerifier Integration into HelixCode

Document Version: 1.0.0

Date: 2026-04-30

Status: Draft — Pending Review

Author: Test Architecture (Anti-Bluff Mandate)

Constitutional Basis: CONST-002, CONST-002a, CONST-005, CONST-006,
CONST-017, CONST-020, CONST-021, CONST-025, CONST-035

Table of Contents

1. [Anti-Bluff Testing Manifesto](#)
 2. [Unit Tests](#)
 3. [Contract Tests](#)
 4. [Component Tests](#)
 5. [Integration Tests](#)
 6. [E2E Tests / Challenges](#)
 7. [Security Tests](#)
 8. [Performance Tests](#)
 9. [Coverage Enforcement](#)
 10. [Test Infrastructure](#)
 11. [Anti-Bluff Verification Checklist Matrix](#)
 12. [Appendix A: Makefile Target Definitions](#)
 13. [Appendix B: Constitution Cross-Reference](#)
-

1. Anti-Bluff Testing Manifesto

1.1 Definition

Anti-bluff testing is a testing philosophy and methodology that ensures every test PASS genuinely guarantees that the tested feature works for end users. A test is a **bluff** if:

1. It passes when the underlying feature is broken, incomplete, or unusable.

2. It tests code paths rather than observable, user-facing behavior.
3. It uses mocks inappropriately, hiding real failures.
4. It asserts on internal state rather than externally verifiable outcomes.
5. It skips in CI but passes locally under unrealistic conditions.

1.2 The Five Anti-Bluff Principles

#	Principle	Enforcement
AB-001	PASS = Works for Users	Every test must verify that a real user can accomplish a real task. If the user cannot do it, the test must FAIL.
AB-002	No Internal-Only Assertions	Tests must assert on externally observable outcomes (CLI output, API responses, DB state, file contents), not on internal variables or private methods.
AB-003	No Mock Propagation	Mocks are allowed ONLY in unit tests (*_test.go with -short). All other test categories MUST use real dependencies.
AB-004	Prove the Negative	For every positive test (“feature X works”), there MUST be a corresponding negative test (“feature X gracefully fails when Y”).
AB-005	Challenge-Based Validation	Every component MUST have a challenge script that exercises the feature through the exact interface an end user would use.

1.3 Anti-Bluff Enforcement in HelixCode

- **CONST-035:** Every PASS must guarantee quality, completion, usability.
- **CONST-002a:** No mocks above unit tests. Any test above unit that uses a mock causes an immediate build failure.
- **CONST-005:** 100% real data for all non-unit tests.
- **CONST-006:** Every component must have challenge scripts.
- **CONST-020:** Fallback chains must be tested with actual failures, not injected errors.
- **CONST-021:** Makefile must include a no-mocks-above-unit target that scans for forbidden mock usage.

1.4 Bluff Detection Checklist (Applied to Every Test)

Before a test is accepted into the suite, it must answer **YES** to all of these:

☐

If I break the feature deliberately, does this test FAIL?

☐

If the feature returns a hardcoded/canned response, does this test FAIL?

☐

If the feature is disconnected from its real dependency, does this test FAIL?

☐

Does this test verify the exact output format a user sees?

☐

Can I run this test in a fresh Docker container and get the same result?

☐

Does this test NOT use `t.Skip()` without a documented SKIP-OK justification?

2. Unit Tests

Scope: Internal package logic, isolated functions, data structures. **Mock Policy:** Only external HTTP calls may be mocked. Only in `*_test.go` files. Only when `testing.Short()` is true. **Constitutional Basis:** CONST-002, CONST-002a.

2.1 Mock Policy (Strict)

Rule	Violation Consequence
Mocks only for outbound HTTP/HTTPS calls to external APIs	Build fails
Mocks never for database, cache, filesystem, internal interfaces	Build fails
All mock setups wrapped in <code>if testing.Short() { ... } else t.Skip(...)</code>	CI scan rejects

Rule	Violation Consequence
Mock files live only in internal/ mocks/ or *_test.go	Lint fails
Mock usage logged in test output with <code>t.Log("MOCK-ACTIVE: ...")</code>	Audit trail

2.2 Unit Test File Inventory

All new files go under `helix_code/internal/verifier/` unless noted otherwise.

#	File Path	Purpose	Lines Est.
1	<code>internal/verifier/client_test.go</code>	LLMsVerifier HTTP client (timeouts, retries, auth headers)	~300
2	<code>internal/verifier/config_test.go</code>	Config parsing, validation, defaults, env var binding	~250
3	<code>internal/verifier/scoring_test.go</code>	Score normalization, weight application, cache expiry	~350
4	<code>internal/verifier/discovery_test.go</code>	Model discovery filtering, sorting, deduplication	~300
5	<code>internal/verifier/provider_adapter_test.go</code>	Provider adapter initialization, capability mapping	~250

#	File Path	Purpose	Lines Est.
6	internal/verifier/ rate_limit_test.go	Rate limit parsing, cooldown calculation, reset logic	~200
7	internal/verifier/cache_test.go	In-memory cache TTL, eviction, hit/ miss counters	~180
8	internal/verifier/health_test.go	Health status transitions, circuit breaker logic	~250
9	internal/verifier/aliases_test.go	Model alias resolution, fuzzy matching thresholds	~150
10	internal/llm/ verifier_integration_test.go	HelixCode model manager ↔ verifier adapter wiring	~300
11	internal/llm/ verifier_model_manager_test.go	Model manager using verifier data (register, score, select)	~350
12	internal/llm/ verifier_registry_test.go	Cross-provider registry with verifier- supplied models	~250
13	internal/config/ verifier_config_test.go	HelixCode config loading with verifier section	~200
14	cmd/cli/verifier_cli_test.go	CLI flag parsing for verifier- related commands	~200
15			~300

#	File Path	Purpose	Lines Est.
	internal/services/llmsverifier_score_adapter_test.go	Score adapter bridge unit tests	
16	internal/verifier/canned_detection_test.go	Anti-bluff: canned response detection logic	~200
17	internal/verifier/fallback_test.go	Fallback model selection when verifier is unavailable	~180
18	internal/verifier/events_test.go	Event publishing, subscription, topic routing	~200
19	internal/verifier/subscription_detector_test.go	Tier detection (free/paid/enterprise) logic	~200
20	internal/verifier/encryption_test.go	SQLite encryption key handling, redaction	~150

Total New Unit Test Files: 20

Estimated Total Lines: ~4,780

2.3 Exact Test Function Signatures and Assertions

File: internal/verifier/client_test.go

```

func TestClient_NewClient_WithDefaults(t *testing.T)
func TestClient_NewClient_WithCustomTimeout(t *testing.T)
func TestClient_NewClient_WithCustomHTTPClient(t *testing.T)
func TestClient_GetModels_Success(t *testing.T) // uses
    mock HTTP only if testing.Short()
func TestClient_GetModels_HTTPError(t
    *testing.T) // uses mock HTTP only if
    testing.Short()
func TestClient_GetModels_InvalidJSON(t *testing.T) // uses
    mock HTTP only if testing.Short()

```

```

func TestClient_GetModels_Timeout(t *testing.T)                                // uses
    mock HTTP only if testing.Short()
func TestClient_GetModelByID_Success(t *testing.T)
func TestClient_GetModelByID_NotFound(t *testing.T)
func TestClient_VerifyModel_Success(t *testing.T)
func TestClient_VerifyModel_ErrorResponse(t *testing.T)
func TestClient_RetryOn5xx(t *testing.T)                                        //
    uses mock HTTP only if testing.Short()
func TestClient_RetryExhausted(t *testing.T)                                    //
    uses mock HTTP only if testing.Short()
func TestClient_AuthHeaderAttached(t *testing.T)
func TestClient_AuthHeaderRedactedInLogs(t *testing.T)
func TestClient_RateLimitHeaderParsing(t *testing.T)
func TestClient_ContextCancellation(t *testing.T)

```

Anti-Bluff Criteria for `client_test.go`: - `TestClient_GetModels_Success`: Must assert that returned `[]ModelInfo` contains at least one model with non-empty ID, Name, and Provider fields. A response with all-zero values or empty strings must cause failure. - `TestClient_AuthHeaderRedactedInLogs`: Must verify that the literal API key string NEVER appears in any log output or error string (search substring).

File: `internal/verifier/config_test.go`

```

func TestConfig_LoadFromFile_YAML(t *testing.T)
func TestConfig_LoadFromFile_JSON(t *testing.T)
func TestConfig_LoadFromFile_TOML(t *testing.T)
func TestConfig_LoadDefaults_WhenFileMissing(t *testing.T)
func TestConfig_EnabledFlag_DefaultTrue(t *testing.T)
func TestConfig_EnabledFlag_OverrideFalse(t *testing.T)
func TestConfig_DatabasePath_Default(t *testing.T)
func TestConfig_DatabaseEncryptionKey_FromEnv(t *testing.T)
func TestConfig_ProviderConfig_OpenAI(t *testing.T)
func TestConfig_ProviderConfig_Anthropic(t *testing.T)
func TestConfig_ProviderConfig_InvalidProvider(t *testing.T)
func TestConfig_ScoringWeights_SumToOne(t *testing.T)
func TestConfig_ScoringWeights_InvalidSum(t *testing.T)
func TestConfig_HealthCheckInterval(t *testing.T)
func TestConfig_CircuitBreaker_DefaultEnabled(t *testing.T)
func TestConfig_EnvVarSubstitution(t *testing.T)
func TestConfig_EnvVarSubstitution_Missing(t *testing.T)
func TestConfig_Scheduling_ReVerificationInterval(t *testing.T)
func TestConfig_Brotli_EnabledDefault(t *testing.T)
func TestConfig_InvalidYAML_ReturnsError(t *testing.T)

```

Anti-Bluff Criteria for `config_test.go`:

`TestConfig_EnabledFlag_DefaultTrue`: After loading a minimal config file, `cfg.Enabled` must be true. An uninitialized boolean (Go zero-value false) must

cause failure. - `TestConfig_ScoringWeights_SumToOne`: Must use `math.Abs(sum-1.0) < 0.0001` assertion. If weights are hardcoded incorrectly (e.g., all 0.2 but missing one dimension), the test FAILs. - `TestConfig_DatabaseEncryptionKey_FromEnv`: Must verify that the encryption key is read from environment and that the struct field is populated — not just that the env var is referenced in the YAML string.

File: `internal/verifier/scoring_test.go`

```
func TestScoringEngine_New(t *testing.T)
func TestScoringEngine_CalculateScore_AllDimensions(t *testing.T)
func TestScoringEngine_CalculateScore_ZeroWeights(t *testing.T)
func TestScoringEngine_CalculateScore_NormalizeTo10(t *testing.T)
func TestScoringEngine_GetProviderScore_Cached(t *testing.T)
func TestScoringEngine_GetProviderScore_ExpiredCache(t
    *testing.T)
func TestScoringEngine_GetModelScore_MultipleModels(t *testing.T)
func TestScoringEngine_ScoreSuffix_Format(t *testing.T)
func TestScoringEngine_CostEffectiveness_Bonus(t *testing.T)
func TestScoringEngine_CostEffectiveness_Penalty(t *testing.T)
func TestScoringEngine_OpenSourceBonus(t *testing.T)
func TestScoringEngine_BatchScore(t *testing.T)
func TestScoringEngine_HistoryTracking(t *testing.T)
func TestScoringEngine_ScoreRange_0To10(t *testing.T)
func TestScoringEngine_NegativeScore_Clamped(t *testing.T)
func TestScoringEngine_ScoreAbove10_Clamped(t *testing.T)
```

Anti-Bluff Criteria for `scoring_test.go`: -

`TestScoringEngine_CalculateScore_AllDimensions`: Must pass a `VerificationResult` with ONLY `code_capability_score` set, and assert the overall score reflects ONLY that dimension's weight. A hardcoded "always 8.5" result must cause failure. - `TestScoringEngine_ScoreSuffix_Format`: Must assert regex match `^SC:\d+\.\d+$. A suffix like "SC:NaN" or "SC:" must cause failure.`

File: `internal/verifier/discovery_test.go`

```
func TestDiscoveryService_New(t *testing.T)
func TestDiscoveryService_DiscoverModels_FilterByProvider(t
    *testing.T)
func TestDiscoveryService_DiscoverModels_FilterByCapability(t
    *testing.T)
func TestDiscoveryService_DiscoverModels_SortByScore(t
    *testing.T)
func TestDiscoveryService_DiscoverModels_Deduplicate(t
    *testing.T)
```

```

func TestDiscoveryService_DiscoverModels_MinScoreFilter(t
    *testing.T)
func TestDiscoveryService_SelectOptimalModel_CodeGeneration(t
    *testing.T)
func TestDiscoveryService_SelectOptimalModel_StreamingRequired(t
    *testing.T)
func TestDiscoveryService_SelectOptimalModel_BudgetConstraint(t
    *testing.T)
func TestDiscoveryService_SelectOptimalModel_NoMatch(t
    *testing.T)
func TestDiscoveryService_ProviderPriority(t *testing.T)
func TestDiscoveryService_CodeVisibilityFilter(t *testing.T)
func TestDiscoveryService_DiversityRequirement(t *testing.T)

```

Anti-Bluff Criteria for discovery_test.go: -

TestDiscoveryService_SelectOptimalModel_CodeGeneration: Must assert that the returned model's Capabilities slice contains "code_generation".

Returning a model without that capability must cause failure. -

TestDiscoveryService_DiscoverModels_Deduplicate: Must feed two models with identical ID but different providers, and assert only one is returned (or both with provider distinction). Returning duplicates without distinction must cause failure.

File: internal/verifier/provider_adapter_test.go

```

func TestProviderAdapter_OpenAI_Init(t *testing.T)
func TestProviderAdapter_Anthropic_Init(t *testing.T)
func TestProviderAdapter_DeepSeek_Init(t *testing.T)
func TestProviderAdapter_Groq_Init(t *testing.T)
func TestProviderAdapter_Mistral_Init(t *testing.T)
func TestProviderAdapter_XAI_Init(t *testing.T)
func TestProviderAdapter_Cohere_Init(t *testing.T)
func TestProviderAdapter_UnsupportedProvider_Error(t *testing.T)
func TestProviderAdapter_CapabilityMapping(t *testing.T)
func TestProviderAdapter_ModelListConversion(t *testing.T)

```

Anti-Bluff Criteria for provider_adapter_test.go: - Each _Init test must verify that the adapter's GetName() returns the expected provider name. An empty string or wrong name must cause failure. -

TestProviderAdapter_CapabilityMapping: Must map provider-native capability names (e.g., "function_calling") to unified capability names (e.g., "tools"). A nil or empty mapping must cause failure.

File: internal/verifier/health_test.go

```

func TestHealthService_Check_Healthy(t *testing.T)
func TestHealthService_Check_Degraded(t *testing.T)

```

```

func TestHealthService_Check_Unhealthy(t *testing.T)
func TestHealthService_CircuitBreaker_OpenAfterFailures(t
    *testing.T)
func TestHealthService_CircuitBreaker_CloseAfterRecoveries(t
    *testing.T)
func TestHealthService_CircuitBreaker_HalfOpenTimeout(t
    *testing.T)
func TestHealthService_StatusTransitions(t *testing.T)
func TestHealthService_ConcurrentChecks(t *testing.T)

```

Anti-Bluff Criteria for health_test.go: -

TestHealthService_CircuitBreaker_OpenAfterFailures: Must call the health check failure_threshold+1 times with an error-returning function, then assert the circuit is Open. A test that asserts after only 1 failure must be rejected. - TestHealthService_StatusTransitions: Must assert the exact transition sequence: unknown → healthy → degraded → unhealthy → offline and verify each transition requires the documented number of events.

File: internal/verifier/canned_detection_test.go

```

func TestIsCannedErrorResponse_MatchesPattern(t *testing.T)
func TestIsCannedErrorResponse_NoMatch(t *testing.T)
func TestIsCannedErrorResponse_EmptyString(t *testing.T)
func TestIsCannedErrorResponse_CaseInsensitive(t *testing.T)
func TestIsSuspiciouslyFastResponse_UnderThreshold(t *testing.T)
func TestIsSuspiciouslyFastResponse_AboveThreshold(t *testing.T)
func TestIsSuspiciouslyFastResponse_ShortContent(t *testing.T)
func TestIsSuspiciouslyFastResponse_LongContent(t *testing.T)

```

Anti-Bluff Criteria for canned_detection_test.go: -

TestIsCannedErrorResponse_MatchesPattern: Must test EVERY pattern in CannedErrorPatterns at least once via a table-driven test. If a new pattern is added to the source, the test must detect it (or the test must iterate the source slice). - TestIsSuspiciouslyFastResponse_UnderThreshold: Must use latency = 99 * time.Millisecond and contentLen = 30 and assert true. Using latency = 50ms alone without content check is insufficient.

File: internal/llm/verifier_model_manager_test.go

```

func TestModelManager_RegisterVerifierAdapter(t *testing.T)
func TestModelManager_GetAvailableModels_FromVerifier(t
    *testing.T)
func TestModelManager_SelectOptimalModel_UsesVerifierScores(t
    *testing.T)
func TestModelManager_HealthCheck_IncludesVerifierProviders(t
    *testing.T)

```

```
func TestModelManager_GetModelsByCapability_VerifierData(t
    *testing.T)
func TestModelManager_Fallback_WhenVerifierOffline(t *testing.T)
func TestModelManager_ModelMetadata_IncludesScoreSuffix(t
    *testing.T)
func TestModelManager_VerifierDisabled_Bypass(t *testing.T)
```

Anti-Bluff Criteria for verifier_model_manager_test.go: -

TestModelManager_GetAvailableModels_FromVerifier: After registering a verifier adapter, GetAvailableModels() must return models with IDs that match the verifier data, NOT the old hardcoded list (llama-3-8b, mistral-7b, phi-3-mini). Returning exactly those 3 models must cause failure. -

TestModelManager_Fallback_WhenVerifierOffline: Must simulate verifier API returning 503 for failure_threshold+1 consecutive calls, then assert the model manager falls back to the next available provider source (e.g., Ollama discovery or hardcoded fallback list). A test that asserts “returns empty” must be rejected.

File: cmd/cli/verifier_cli_test.go

```
func TestCLI_ListModelsFlag_Parses(t *testing.T)
func TestCLI_ModelFlag_Parses(t *testing.T)
func TestCLI_VerifierEnabledFlag_Parses(t *testing.T)
func TestCLI_VerifierDisabledFlag_Parses(t *testing.T)
func TestCLI_VerifierConfigFlag_Parses(t *testing.T)
func TestCLI_InteractiveCommand_Models(t *testing.T)
func TestCLI_OutputFormat_JSON(t *testing.T)
func TestCLI_OutputFormat_Table(t *testing.T)
```

Anti-Bluff Criteria for verifier_cli_test.go: -

TestCLI_InteractiveCommand_Models: Must capture stdout and assert that the output contains model names from the verifier, not the hardcoded 3-model list.

Output containing exactly “Llama 3 8B” / “Mistral 7B” / “Phi-3 Mini” as the only models must cause failure.

3. Contract Tests

Scope: API schema validation, provider API contract verification. **Mock**

Policy: NO mocks. Uses real API endpoints with test keys or schema snapshots. **Constitutional Basis:** CONST-005, CONST-002a.

3.1 Contract Test File Inventory

#	File Path	Purpose
1	tests/contract/ verifier_api_contract_test.go	Verify LLMsVerifier REST API schema
2	tests/contract/ provider_api_contract_test.go	Verify provider API response schemas
3	tests/contract/ schema_validation_test.go	JSON schema validation for all API responses
4	tests/contract/ model_response_contract_test.go	Verify /api/models response contract
5	tests/contract/ verification_response_contract_test.go	Verify /api/models/ {id}/verify response contract
6	tests/contract/ error_response_contract_test.go	Verify error response format contract

3.2 Exact Test Function Signatures

File: tests/contract/verifier_api_contract_test.go

```
func TestVerifierAPI_HealthEndpoint(t *testing.T)
func TestVerifierAPI_ModelsListEndpoint(t *testing.T)
func TestVerifierAPI_ModelGetEndpoint(t *testing.T)
func TestVerifierAPI_ModelVerifyEndpoint(t *testing.T)
func TestVerifierAPI_ProvidersEndpoint(t *testing.T)
func TestVerifierAPI_ScoreEndpoint(t *testing.T)
func TestVerifierAPI-Headers_CORS(t *testing.T)
func TestVerifierAPI-Headers_ContentType(t *testing.T)
func TestVerifierAPI_Error_404(t *testing.T)
func TestVerifierAPI_Error_401(t *testing.T)
```

Anti-Bluff Criteria: - TestVerifierAPI_ModelsListEndpoint: Must make a REAL HTTP request to the verifier server (running in Docker). Must assert that the response JSON array contains objects with id, name, provider, score, verified fields. Must assert Content-Type: application/json. Must assert HTTP 200. - TestVerifierAPI_Error_401: Must make a request WITHOUT the Authorization header and assert HTTP 401. A test that skips when no auth is configured is a bluff — it must FAIL.

File: tests/contract/provider_api_contract_test.go

```
func TestProviderAPI_OpenAI_ModelsEndpoint(t *testing.T)
func TestProviderAPI_OpenAI_ChatCompletionSchema(t *testing.T)
func TestProviderAPI_Anthropic_MessagesSchema(t *testing.T)
func TestProviderAPI_Anthropic_ModelsEndpoint(t *testing.T)
func TestProviderAPI_DeepSeek_ModelsEndpoint(t *testing.T)
func TestProviderAPI_DeepSeek_ChatCompletionSchema(t *testing.T)
func TestProviderAPI_Groq_ModelsEndpoint(t *testing.T)
func TestProviderAPI_XAI_ModelsEndpoint(t *testing.T)
func TestProviderAPI_Mistral_ModelsEndpoint(t *testing.T)
```

Anti-Bluff Criteria: - Each TestProviderAPI_*_ModelsEndpoint: Must make a REAL HTTP request to the provider's actual models endpoint using a test API key from environment. Must assert the response is valid JSON with at least one model object. Must assert the model object has id and object fields. If the provider API is unreachable or returns an error, the test must FAIL (with a SKIP-OK only if the env var is missing, documented). - Each TestProviderAPI_*_ChatCompletionSchema: Must make a REAL chat completion request with a minimal prompt (e.g., "Say 'hello'"), assert 200 OK, assert the response has .choices[0].message.content containing non-empty text.

File: tests/contract/schema_validation_test.go

```
func TestSchema_ModelInfo(t *testing.T)
func TestSchema_VerificationResult(t *testing.T)
func TestSchema_ProviderInfo(t *testing.T)
func TestSchema_ScoreDetails(t *testing.T)
func TestSchema_ErrorResponse(t *testing.T)
func TestSchema_Config(t *testing.T)
```

Anti-Bluff Criteria: - TestSchema_ModelInfo: Must validate a ModelInfo struct against the actual JSON returned from the verifier /api/models endpoint. The struct must have json:"..." tags for every field in the response. Missing fields must cause failure. - TestSchema_VerificationResult: Must validate that the verification result struct has fields for ALL dimensions: code_capability_score, responsiveness_score, reliability_score, feature_richness_score, value_proposition_score. Missing any dimension field must cause failure.

3.3 Schema Snapshot Files

Stored in tests/contract/snapshots/:

File	Purpose
verifier_models_list.json	Expected JSON schema for /api/models
verifier_model_get.json	Expected JSON schema for /api/models/{id}
verifier_verify_result.json	Expected JSON schema for /api/models/{id}/verify
verifier_error.json	Expected JSON schema for error responses
provider_openai_models.json	Expected schema for OpenAI /v1/models
provider_anthropic_messages.json	Expected schema for Anthropic /v1/messages
provider_deepseek_chat.json	Expected schema for DeepSeek /chat/completions

These snapshots are generated by the contract tests on the first run and then used for structural validation. If the real API changes its schema, the snapshot mismatch causes a test failure.

4. Component Tests

Scope: Real subsystems wired together, no external mocks. **Mock Policy:** NO mocks. All subsystems are real instances (in-memory SQLite, real cache, real config structs). **Constitutional Basis:** CONST-005, CONST-006.

4.1 Component Test File Inventory

#	File Path	Purpose
1	tests/component/ verifier_client_cache_component_test.go	Verifier client + in-memory cache + config
2	tests/component/ model_manager_verifier_component_test.go	Model manager + verifier adapter + scoring
3	tests/component/ cli_output_formatter_component_test.go	

#	File Path	Purpose
		CLI formatter + real model data structures
4	tests/component/discovery_scoring_component_test.go	Discovery service + scoring service + health service
5	tests/component/startup_pipeline_component_test.go	Startup verifier (phases 1-5) with real subsystems
6	tests/component/event_bus_verifier_component_test.go	Event publisher + verifier events + subscriber

4.2 Exact Test Function Signatures

File: tests/component/verifier_client_cache_component_test.go

```

func TestClientCache_CacheHit_AvoidsHTTPCall(t *testing.T)
func TestClientCache_CacheMiss_MakesHTTPCall(t *testing.T)
func TestClientCache_TTL_Expires(t *testing.T)
func TestClientCache_Eviction_MaxSize(t *testing.T)
func TestClientCache_CacheDisabled_Bypasses(t *testing.T)
func TestClientCache_ConcurrentAccess(t *testing.T)

```

Anti-Bluff Criteria: - TestClientCache_CacheHit_AvoidsHTTPCall: Must use a real HTTP server (httptest) with a request counter. On the second call with the same key, the request counter must NOT increment. If the counter increments, the cache is not working — test FAILs. - TestClientCache_TTL_Expires: Must set TTL to 1 second, wait 2 seconds, then assert the cache returns a miss. A test that asserts “still valid after 2s” must FAIL.

File: tests/component/model_manager_verifier_component_test.go

```

func TestModelManager_RegisterVerifierProvider(t *testing.T)
func TestModelManager_GetModels_ReturnsVerifierModels(t
    *testing.T)
func TestModelManager_SelectModel_UsesVerifierScores(t
    *testing.T)
func TestModelManager_HealthCheck_ReflectsVerifierStatus(t
    *testing.T)
func TestModelManager_Fallback_ToLocalProvider(t *testing.T)
func TestModelManager_ScoreSuffix_Display(t *testing.T)
func TestModelManager_ConcurrentRegisterAndSelect(t *testing.T)

```

Anti-Bluff Criteria: -

TestModelManager_GetModels_ReturnsVerifierModels: Must register a verifier provider that returns 5 real models from a test SQLite DB, then call GetAvailableModels() and assert the returned count is 5 and each ID matches the DB. Returning 3 hardcoded models must cause failure. -

TestModelManager_SelectModel_UsesVerifierScores: Must insert two models into the verifier DB — one with score 9.0, one with score 5.0. Call SelectOptimalModel() and assert the 9.0 model is selected. If the lower-scored model is selected, the scoring integration is broken — test FAILs.

File: tests/component/cli_output_formatter_component_test.go

```
func TestCLIFormatter_TableOutput_ContainsModelNames(t
    *testing.T)
func TestCLIFormatter_TableOutput_ContainsScoreSuffix(t
    *testing.T)
func TestCLIFormatter_TableOutput_ContainsVerificationBadge(t
    *testing.T)
func TestCLIFormatter_JSONOutput_ValidJSON(t *testing.T)
func TestCLIFormatter_JSONOutput_ContainsAllFields(t *testing.T)
func TestCLIFormatter_EmptyModels_HandlesGracefully(t *testing.T)
```

Anti-Bluff Criteria: -

TestCLIFormatter_TableOutput_ContainsScoreSuffix: Must pass a model with ScoreSuffix = "SC:8.5" to the formatter and assert the output string contains "SC:8.5". If the formatter ignores the suffix, test FAILs. -

TestCLIFormatter_JSONOutput_ContainsAllFields: Must assert that JSON output contains ALL of: id, name, provider, score, verified, latency, context_window. Missing any field must cause failure.

File: tests/component/startup_pipeline_component_test.go

```
func TestStartupPipeline_Phase1_DiscoverProviders(t *testing.T)
func TestStartupPipeline_Phase2_VerifyProviders(t *testing.T)
func TestStartupPipeline_Phase2_5_DetectSubscriptions(t
    *testing.T)
func TestStartupPipeline_Phase3_ScoreProviders(t *testing.T)
func TestStartupPipeline_Phase4_RankProviders(t *testing.T)
func TestStartupPipeline_Phase5_SelectDebateTeam(t *testing.T)
func TestStartupPipeline_AllPhases_EndToEnd(t *testing.T)
func TestStartupPipeline_ProviderWithFaultyKey_Deprioritized(t
    *testing.T)
```

Anti-Bluff Criteria: - TestStartupPipeline_Phase1_DiscoverProviders: Must assert that SupportedProviders map is iterated and providers with available env vars are discovered. A test that asserts “3 providers discovered”

without checking which ones is a bluff. -

`TestStartupPipeline_AllPhases_EndToEnd`: Must run all 5 phases with a test SQLite DB containing 2 providers and 4 models, and assert the final `DebateTeamResult` contains at least 1 model. An empty debate team must cause failure.

5. Integration Tests

Scope: Full application with real dependencies (PostgreSQL, Redis, SQLite, real provider APIs). **Mock Policy:** NO mocks whatsoever. Real API keys from environment. Real databases from Docker.
Constitutional Basis: CONST-005, CONST-020.

5.1 Integration Test File Inventory

#	File Path	Purpose
1	tests/integration/helixcode_verifier_sqlite_test.go	HelixCode + LLMsVerifier SQLite DB
2	tests/integration/helixcode_provider_api_test.go	HelixCode + real provider APIs (with test keys)
3	tests/integration/helixcode_redis_cache_test.go	HelixCode + Redis cache for verifier data
4	tests/integration/helixcode_postgres_test.go	HelixCode + PostgreSQL with verifier tables
5	tests/integration/helixcode_full_stack_test.go	Server + DB + Cache + Verifier + Provider
6	tests/integration/helixcode_verifier_events_test.go	Event bus + verifier + WebSocket
7	tests/integration/helixcode_fallback_chain_test.go	Provider fallback with real failures
8	tests/integration/helixcode_verifier_config_reload_test.go	Config hot-reload with verifier changes

5.2 Exact Test Function Signatures

File: tests/integration/helixcode_verifier_sqlite_test.go

```
func TestHelixCodeVerifierSQLite_DBConnection(t *testing.T)
func TestHelixCodeVerifierSQLite_ModelCRUD(t *testing.T)
func TestHelixCodeVerifierSQLite_VerificationResultPersisted(t
    *testing.T)
func TestHelixCodeVerifierSQLite_ProviderMetadata(t *testing.T)
func TestHelixCodeVerifierSQLite_RateLimits(t *testing.T)
func TestHelixCodeVerifierSQLite_PricingData(t *testing.T)
func TestHelixCodeVerifierSQLite_ConcurrentAccess(t *testing.T)
func TestHelixCodeVerifierSQLite_EncryptionEnabled(t *testing.T)
```

Anti-Bluff Criteria: - TestHelixCodeVerifierSQLite_ModelCRUD: Must CREATE a model, READ it back, UPDATE its score, DELETE it, then assert the model is gone. Each step must verify the exact DB row via SELECT. A test that only creates and reads is a bluff. -

TestHelixCodeVerifierSQLite_VerificationResultPersisted: Must run a real verification through the verifier client, then query the SQLite verification_results table and assert a row exists with matching model_id and non-zero overall_score. If the table is empty, test FAILs. -

TestHelixCodeVerifierSQLite_EncryptionEnabled: Must start the verifier with encryption_enabled: true, insert a model, then attempt to read the raw SQLite file bytes and assert the content is NOT plaintext (search for model name string in file bytes). Finding the plaintext model name in the file must cause failure.

File: tests/integration/helixcode_provider_api_test.go

```
func TestHelixCodeProviderAPI_OpenAI_RealModelList(t *testing.T)
func TestHelixCodeProviderAPI_OpenAI_RealChatCompletion(t
    *testing.T)
func TestHelixCodeProviderAPI_Anthropic_RealMessages(t
    *testing.T)
func TestHelixCodeProviderAPI_DeepSeek_RealChat(t *testing.T)
func TestHelixCodeProviderAPI_Groq_RealChat(t *testing.T)
func TestHelixCodeProviderAPI_XAI_RealChat(t *testing.T)
func TestHelixCodeProviderAPI_Mistral_RealChat(t *testing.T)
func TestHelixCodeProviderAPI_InvalidKey_PropagatesError(t
    *testing.T)
func TestHelixCodeProviderAPI_RateLimit_PropagatesError(t
    *testing.T)
```

Anti-Bluff Criteria: - TestHelixCodeProviderAPI_OpenAI_RealModelList: Must call the real OpenAI /v1/models endpoint with a test API key, parse the

response, and assert at least 10 models are returned. If 0 models are returned, test FAILs. If the response is mocked, test FAILs. -

TestHelixCodeProviderAPI_OpenAI_RealChatCompletion: Must send a real chat completion request with prompt "Say exactly 'ANTIBLUFF-OK'", and assert the response content contains "ANTIBLUFF-OK". A hardcoded or simulated response must cause failure. -

TestHelixCodeProviderAPI_InvalidKey_PropagatesError: Must use an intentionally invalid API key (e.g., sk-invalid-test-key-12345), call the provider, and assert the error message is propagated to the user-visible layer. Swallowing the error or returning a generic success must cause failure.

File: tests/integration/helixcode_redis_cache_test.go

```
func TestHelixCodeRedisCache_ModelListCached(t *testing.T)
func TestHelixCodeRedisCache_ModelListCacheExpiry(t *testing.T)
func TestHelixCodeRedisCache_VerificationResultCached(t
    *testing.T)
func TestHelixCodeRedisCache_ScoreCached(t *testing.T)
func TestHelixCodeRedisCache_RedisDown_GracefulFallback(t
    *testing.T)
func TestHelixCodeRedisCache_CacheInvalidation(t *testing.T)
```

Anti-Bluff Criteria: - TestHelixCodeRedisCache_ModelListCached: Must call GetAvailableModels() twice. The second call must be served from Redis (verify via redis-cli MONITOR or INFO stats). If the second call hits the verifier API again, the cache is not working — test FAILs. -

TestHelixCodeRedisCache_RedisDown_GracefulFallback: Must stop the Redis container mid-test, then call GetAvailableModels() and assert the result is still returned (from verifier directly or from in-memory fallback). Returning an error to the user when Redis is down is a bluff — the system must still work.

File: tests/integration/helixcode_postgres_test.go

```
func TestHelixCodePostgres_VerifierSchema_Migrated(t *testing.T)
func TestHelixCodePostgres_VerifierData_SyncedFromSQLite(t
    *testing.T)
func TestHelixCodePostgres_VerifierQuery_Performance(t
    *testing.T)
func TestHelixCodePostgres_VerifierTransaction_Rollback(t
    *testing.T)
```

Anti-Bluff Criteria: - TestHelixCodePostgres_VerifierSchema_Migrated: Must query information_schema.tables and assert tables for verifier data exist. If the schema migration was skipped, test FAILs.

File: tests/integration/helixcode_fallback_chain_test.go

```
func TestFallbackChain_PrimaryFails_SecondaryUsed(t *testing.T)
func TestFallbackChain_AllFail_ErrorReturned(t *testing.T)
func TestFallbackChain_Recovery_PrimaryReused(t *testing.T)
func TestFallbackChain_RealFailure_NoMock(t *testing.T)
```

Anti-Bluff Criteria: - TestFallbackChain_RealFailure_NoMock: Must configure a primary provider with a wrong endpoint (e.g., http://localhost:59999 where nothing listens), and a secondary provider with a real endpoint. Assert the secondary is used. If both are mocked, test FAILs.

File: tests/integration/helixcode_full_stack_test.go

```
func TestFullStack_ServerStarts_WithVerifier(t *testing.T)
func TestFullStack_APIModels_ReturnsVerifierData(t *testing.T)
func TestFullStack_CLIListModels_ReturnsVerifierData(t
    *testing.T)
func TestFullStack_WebSocket_EmitsVerificationEvents(t
    *testing.T)
func TestFullStack_HealthCheck_IncludesVerifier(t *testing.T)
func TestFullStack_Generate_WithVerifiedModel(t *testing.T)
```

Anti-Bluff Criteria: - TestFullStack_APIModels_ReturnsVerifierData: Must start the full server stack (PostgreSQL, Redis, LLMsVerifier server), make a GET /api/v1/models request, and assert the response contains model data from the verifier SQLite DB (not hardcoded). Hardcoded model IDs must cause failure. - TestFullStack_Generate_WithVerifiedModel: Must send a POST /api/v1/tasks with a verified model ID, wait for the task to complete, and assert the response contains actual generated text (not a canned “Generated response for...” string). The canned prefix must cause failure.

6. E2E Tests / Challenges

Scope: Complete user workflows through the exact interfaces users interact with. **Mock Policy:** NO mocks. Real CLI binary, real server, real API keys. **Constitutional Basis:** CONST-006, CONST-035.

6.1 Challenge Script Inventory

All challenge scripts live in challenges/scripts/ and are shell scripts (bash) with strict error handling (set -euo pipefail).

#	Script File	Purpose	Lines Est.
1	challenges/scripts/ verifier_model_list_challenge.sh	List models and verify they're from verifier, not hardcoded	~180
2	challenges/scripts/ verifier_model_select_challenge.sh	Select a model, verify it passes validation, generate code	~220
3	challenges/scripts/ verifier_disable_fallback_challenge.sh	Disable verifier, verify fallback to old behavior	~160
4	challenges/scripts/ verifier_api_key_provision_challenge.sh	Verify all provider API keys are provisioned through config	~150
5	challenges/scripts/ verifier_rate_limit_display_challenge.sh	Verify rate-limited models are marked disabled with clear notes	~180
6	challenges/scripts/ verifier_realtime_update_challenge.sh	Verify real-time updates reflect in model list within N seconds	~200
7			~250

#	Script File	Purpose	Lines Est.
	challenges/scripts/ verifier_mcp_lsp_acp_challenge.sh	Verify MCP/ LSP/ACP/ Embedding integration works end- to-end	
8	challenges/scripts/ verifier_cross_platform_cli_challenge.sh	Cross- platform CLI output verification	~180
9	challenges/scripts/ verifier_startup_pipeline_challenge.sh	Verify 5- phase startup pipeline completes with real providers	~220
10	challenges/scripts/ verifier_canned_detection_challenge.sh	Verify canned response detection marks models unverified	~170
11	challenges/scripts/ verifier_security_redaction_challenge.sh	Verify API keys never appear in logs, stdout, or error messages	~160
12	challenges/scripts/ verifier_scoring_accuracy_challenge.sh	Verify scoring reflects real verification results, not hardcoded 8.5	~190

6.2 Challenge Script Templates

Challenge 1: verifier_model_list_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail

# CONST-035: End-User Usability Mandate
# ANTI-BLUFF: This challenge proves that "model listing works" =
# the CLI shows real models from verifier DB, not hardcoded
# list.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
VERIFIER_DB="${PROJECT_ROOT}/test_data/verifier.db"

# --- Setup ---
echo "[CHALLENGE] verifier_model_list_challenge: START"

# Ensure verifier DB has test models
sqlite3 "${VERIFIER_DB}" <<EOF
DELETE FROM models;
INSERT INTO models (provider_id, model_id, name, description,
                    overall_score, verification_status)
VALUES (1, 'test-gpt-4o', 'GPT-4o', 'OpenAI GPT-4o model', 9.2,
        'verified');
INSERT INTO models (provider_id, model_id, name, description,
                    overall_score, verification_status)
VALUES (2, 'test-claude-sonnet', 'Claude Sonnet 4', 'Anthropic
        Claude Sonnet', 8.8, 'verified');
INSERT INTO models (provider_id, model_id, name, description,
                    overall_score, verification_status)
VALUES (3, 'test-deepseek-chat', 'DeepSeek Chat', 'DeepSeek V3',
        8.5, 'verified');
EOF

# --- Action: List models via CLI ---
OUTPUT_FILE="/tmp/verifier_model_list_output.txt"
"${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: Must contain verifier models
if ! grep -q "GPT-4o" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain verifier model 'GPT-4o'"
    exit 1
fi
```

```

if ! grep -q "Claude Sonnet 4" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain verifier model 'Claude
        Sonnet 4'"
    exit 1
fi

if ! grep -q "DeepSeek Chat" "${OUTPUT_FILE}"; then
    echo
        "[FAIL] Output does not contain verifier model 'DeepSeek
        Chat'"
    exit 1
fi

# ANTI-BLUFF 2: Must NOT contain ONLY the old hardcoded 3-model
    list
HARDCODED_COUNT=$(grep -c -E "Llama 3 8B|Mistral 7B|Phi-3 Mini"
    "${OUTPUT_FILE}" || true)
TOTAL_MODEL_COUNT=$(grep -c -E "^[a-zA-Z]" "${OUTPUT_FILE}" ||
    true)

if [[ "${HARDCODED_COUNT}" -ge 3 && "${TOTAL_MODEL_COUNT}" -le
    4 ]]; then
    echo
        "[FAIL] Output appears to contain only the old hardcoded
        3-model list"
    exit 1
fi

# ANTI-BLUFF 3: Must contain score suffix for verified models
if ! grep -q "SC:" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain score suffix (SC:X.X)"
    exit 1
fi

# ANTI-BLUFF 4: Must contain verification badge
if ! grep -q -E "(✓|verified|Verified)" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain verification indicator"
    exit 1
fi

echo "[CHALLENGE] verifier_model_list_challenge: PASS"

```

Challenge 2: verifier_model_select_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

```

```

# ANTI-BLUFF: This challenge proves that "model selection + code
    generation works" =
#   selecting a verified model actually produces real code
    output.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"

# --- Setup ---
echo "[CHALLENGE] verifier_model_select_challenge: START"

# --- Action: Generate code with a verified model ---
OUTPUT_FILE="/tmp/verifier_generate_output.txt"
"${CLI_BIN}" \
    --model "test-gpt-4o" \
    --prompt "Write a Go function named AntiBluffVerify that
        returns true" \
    > "${OUTPUT_FILE}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: Output must contain real Go code, not a simulated
    placeholder
if grep -q "Generated response for:" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output contains simulated placeholder text"
    exit 1
fi

# ANTI-BLUFF 2: Output must contain the requested function name
if ! grep -q "AntiBluffVerify" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain requested function name
        'AntiBluffVerify'"
    exit 1
fi

# ANTI-BLUFF 3: Output must contain 'func' keyword (it's Go code)
if ! grep -q "func" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain Go 'func' keyword"
    exit 1
fi

# ANTI-BLUFF 4: Output must contain 'return' statement
if ! grep -q "return" "${OUTPUT_FILE}"; then
    echo "[FAIL] Output does not contain 'return' statement"
    exit 1
fi

```

```
# ANTI-BLUFF 5: Output must NOT contain "TODO" or "coming soon"
if grep -qiE "(TODO|coming soon|not implemented|placeholder)" "$
    {OUTPUT_FILE}"; then
    echo "[FAIL] Output contains incomplete placeholder text"
    exit 1
fi
```

```
echo "[CHALLENGE] verifier_model_select_challenge: PASS"
```

Challenge 3: verifier_disable_fallback_challenge.sh

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
# ANTI-BLUFF: This challenge proves that "verifier disable +
    fallback works" =
#   when verifier is disabled, the system falls back to previous
    behavior
#   (Ollama discovery, hardcoded list, or local provider) and
    still functions.
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
```

```
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
```

```
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
```

```
CONFIG_FILE="/tmp/verifier_disabled_config.yaml"
```

```
# --- Setup: Create config with verifier disabled ---
```

```
cat > "${CONFIG_FILE}" <<EOF
```

```
version: "1.0.0"
```

```
llm:
```

```
    default_provider: "local"
```

```
    default_model: "llama-3.2-3b"
```

```
verifier:
```

```
    enabled: false
```

```
providers:
```

```
    ollama:
```

```
        enabled: true
```

```
        base_url: "http://localhost:11434"
```

```
EOF
```

```
echo "[CHALLENGE] verifier_disable_fallback_challenge: START"
```

```
# --- Action: List models with verifier disabled ---
```

```
OUTPUT_FILE="/tmp/verifier_disabled_output.txt"
```

```
HELIX_CONFIG="${CONFIG_FILE}" "${CLI_BIN}" --list-models > "$
    {OUTPUT_FILE}" 2>&1 || true
```

```
# --- Assertions ---
```

```

# ANTI-BLUFF 1: Must NOT crash or return error
if grep -qiE "(error|fatal|panic|exception)" "${OUTPUT_FILE}";
then
    echo "[FAIL] CLI returned error when verifier is disabled"
    exit 1
fi

# ANTI-BLUFF 2: Must return SOME models (from fallback source)
MODEL_COUNT=$(grep -c -E "^[a-zA-Z0-9]" "${OUTPUT_FILE}" || true)
if [[ "${MODEL_COUNT}" -lt 1 ]]; then
    echo "[FAIL] No models returned when verifier is disabled
(fallback broken)"
    exit 1
fi

# ANTI-BLUFF 3: Must NOT reference verifier-specific models if
#               none are available
# (This is acceptable – the test only requires that it doesn't
# crash)

echo "[CHALLENGE] verifier_disable_fallback_challenge: PASS"

```

Challenge 4: verifier_api_key_provision_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "API key provisioning
#             works" =
#   all provider API keys are read from config/env, never
#   hardcoded,
#   and are actually used in provider initialization.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CONFIG_FILE="${PROJECT_ROOT}/helix_code/config/config.yaml"
ENV_FILE="${PROJECT_ROOT}/.env"

echo "[CHALLENGE] verifier_api_key_provision_challenge: START"

# --- Assertions ---

# ANTI-BLUFF 1: Config file must reference env vars for API keys,
#               not literal values
if grep -qE "api_key:\s*[A-Za-z0-9_-]{20,}" "${CONFIG_FILE}" 2>/
dev/null; then

```

```

    echo "[FAIL] Config file contains literal API key (not env
        var reference)"
    exit 1
fi

# ANTI-BLUFF 2: .env.example must list all provider API keys
REQUIRED_KEYS=(
    "OPENAI_API_KEY"
    "ANTHROPIC_API_KEY"
    "DEEPSEEK_API_KEY"
    "GROQ_API_KEY"
    "MISTRAL_API_KEY"
    "XAI_API_KEY"
    "TOGETHER_API_KEY"
    "OPENROUTER_API_KEY"
)

for KEY in "${REQUIRED_KEYS[@]}; do
    if ! grep -q "${KEY}" "${ENV_FILE}" 2>/dev/null; then
        echo "[FAIL] .env does not document required key: ${KEY}"
        exit 1
    fi
done

# ANTI-BLUFF 3: Verifier config must use env var substitution
    pattern
VERIFIER_CONFIG="${PROJECT_ROOT}/configs/verifier.yaml"
if grep -qE "api_key:\s*[^$\""]" "${VERIFIER_CONFIG}" 2>/dev/
    null; then
    echo "[FAIL] Verifier config contains hardcoded API key"
    exit 1
fi

# ANTI-BLUFF 4: At least one test key must be present for
    integration tests
TEST_KEY_COUNT=0
for KEY in "${REQUIRED_KEYS[@]}; do
    ENV_VAL="${!KEY:-}"
    if [[ -n "${ENV_VAL}" && "${ENV_VAL}" != "your-"* ]]; then
        ((TEST_KEY_COUNT++)) || true
    fi
done

if [[ "${TEST_KEY_COUNT}" -lt 1 ]]; then
    echo "[WARN] No test API keys found in environment. Skipping
        live provider tests."
    # This is SKIP-OK per CONST-035 – documented in AGENTS.md
    exit 0

```


fi

```
echo "[CHALLENGE] verifier_api_key_provision_challenge: PASS (${  
    {TEST_KEY_COUNT} keys provisioned)"
```

Challenge 5: verifier_rate_limit_display_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "rate limiting display
#           works" =
#           deliberately exhausting a provider quota causes the model to
#           be marked
#           with a cooldown indicator within the refresh interval.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
REFRESH_INTERVAL=30 # seconds

echo "[CHALLENGE] verifier_rate_limit_display_challenge: START"

# --- Setup: Insert a model with rate limit into verifier DB ---
sqlite3 "${PROJECT_ROOT}/test_data/verifier.db" <<EOF
UPDATE models SET verification_status = 'rate_limited' WHERE
    model_id = 'test-gpt-4o';
INSERT OR REPLACE INTO limits (model_id, limit_type, limit_value,
    current_usage, reset_period)
VALUES ((SELECT id FROM models WHERE model_id = 'test-gpt-4o'),
    'requests_per_minute', 3, 3, '1m');
EOF

# Wait for refresh interval
sleep "${REFRESH_INTERVAL}"

# --- Action: List models ---
OUTPUT_FILE="/tmp/verifier_rate_limit_output.txt"
"${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: Rate-limited model must be marked (disabled,
#           cooldown, or similar)
if ! grep -q -iE "(rate.?limited|cooldown|disabled|unavailable|
    exhausted)" "${OUTPUT_FILE}"; then
    echo "[FAIL] Rate-limited model not marked in output"
    exit 1
```

```
fi
```

```
# ANTI-BLUFF 2: Must NOT allow selecting the rate-limited model  
without warning
```

```
if grep -q "test-gpt-4o.*available" "${OUTPUT_FILE}"; then  
    echo "[FAIL] Rate-limited model still shown as 'available'"  
    exit 1  
fi
```

```
fi
```

```
echo "[CHALLENGE] verifier_rate_limit_display_challenge: PASS"
```

Challenge 6: verifier_realtime_update_challenge.sh

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
# ANTI-BLUFF: This challenge proves that "real-time updates work"  
=
```

```
# modifying the verifier DB causes the CLI model list to  
reflect changes
```

```
# within N seconds (the configured refresh interval).
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
```

```
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
```

```
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
```

```
REFRESH_INTERVAL=30
```

```
MAX_WAIT=$((REFRESH_INTERVAL + 10))
```

```
echo "[CHALLENGE] verifier_realtime_update_challenge: START"
```

```
# --- Step 1: Baseline list ---
```

```
BASELINE_FILE="/tmp/verifier_rt_baseline.txt"
```

```
"${CLI_BIN}" --list-models > "${BASELINE_FILE}" 2>&1 || true
```

```
# --- Step 2: Add a new model to verifier DB ---
```

```
NEW_MODEL_NAME="RealtimeTestModel-$(date +%s)"
```

```
sqlite3 "${PROJECT_ROOT}/test_data/verifier.db" <<EOF
```

```
INSERT INTO models (provider_id, model_id, name, description,  
                    overall_score, verification_status)
```

```
VALUES (1, 'test-realtime-model', '${NEW_MODEL_NAME}', 'Inserted  
for realtime test', 7.5, 'verified');
```

```
EOF
```

```
# --- Step 3: Poll until model appears or timeout ---
```

```
START_TIME=$(date +%s)
```

```
FOUND=0
```

```
while true; do
```

```
    CURRENT_FILE="/tmp/verifier_rt_current.txt"
```

```

"${CLI_BIN}" --list-models > "${CURRENT_FILE}" 2>&1 || true

if grep -q "${NEW_MODEL_NAME}" "${CURRENT_FILE}"; then
    FOUND=1
    break
fi

ELAPSED=$((date +%s) - START_TIME)
if [[ "${ELAPSED}" -ge "${MAX_WAIT}" ]]; then
    break
fi

sleep 2
done

# --- Assertions ---

if [[ "${FOUND}" -eq 0 ]]; then
    echo "[FAIL] New model did not appear in CLI output within $
        {MAX_WAIT}s"
    exit 1
fi

ELAPSED=$((date +%s) - START_TIME)
echo "[CHALLENGE] verifier_realtime_update_challenge: PASS
    (reflected in ${ELAPSED}s)"

```

Challenge 7: verifier_mcp_lsp_acp_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "MCP/LSP/ACP/Embedding
#             integration works" =
# the verifier data is accessible through all protocol
#             interfaces.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
SERVER_URL="http://localhost:8080"

echo "[CHALLENGE] verifier_mcp_lsp_acp_challenge: START"

# --- MCP: Model Context Protocol test ---
MCP_OUTPUT="/tmp/verifier_mcp_output.txt"
curl -s "${SERVER_URL}/api/v1/mcp/models" > "${MCP_OUTPUT}" 2>&1
|| true

```

```

if ! grep -q "test-gpt-4o" "${MCP_OUTPUT}"; then
    echo "[FAIL] MCP endpoint does not return verifier models"
    exit 1
fi

# --- LSP: Language Server Protocol test (if applicable) ---
LSP_OUTPUT="/tmp/verifier_lsp_output.txt"
curl -s "${SERVER_URL}/api/v1/lsp/completion" \
    -H "Content-Type: application/json" \
    -d '{"model": "test-gpt-4o", "prompt": "func AntiBluff"}' > "$
    {LSP_OUTPUT}" 2>&1 || true

if ! grep -q "AntiBluff" "${LSP_OUTPUT}"; then
    echo "[FAIL] LSP endpoint does not return completions with
    verifier model"
    exit 1
fi

# --- ACP: Agent Communication Protocol test ---
ACP_OUTPUT="/tmp/verifier_acp_output.txt"
curl -s "${SERVER_URL}/api/v1/acp/agents/discover" > "$
    {ACP_OUTPUT}" 2>&1 || true

if ! grep -q "verifier" "${ACP_OUTPUT}"; then
    echo "[FAIL] ACP endpoint does not reference verifier"
    exit 1
fi

# --- Embedding: Verify embedding model from verifier ---
EMBED_OUTPUT="/tmp/verifier_embed_output.txt"
curl -s "${SERVER_URL}/api/v1/embeddings" \
    -H "Content-Type: application/json" \
    -d '{"model": "text-embedding-3-small", "input": "anti-bluff
    test"}' > "${EMBED_OUTPUT}" 2>&1 || true

if ! grep -q "embedding" "${EMBED_OUTPUT}"; then
    echo "[FAIL] Embedding endpoint does not return embeddings"
    exit 1
fi

echo "[CHALLENGE] verifier_mcp_lsp_acp_challenge: PASS"

```

Challenge 8: verifier_cross_platform_cli_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

```

```
# ANTI-BLUFF: This challenge proves that "CLI output is correct
    on all platforms" =
#   the same command produces structurally identical output on
    Linux, macOS, Windows.
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
```

```
echo "[CHALLENGE] verifier_cross_platform_cli_challenge: START"
```

```
PLATFORM=$(uname -s)
ARCH=$(uname -m)
OUTPUT_FILE="/tmp/verifier_cross_platform_${PLATFORM}_${ARCH}.txt"
JSON_OUTPUT="/tmp/verifier_cross_platform_${PLATFORM}_${ARCH}.json"
```

```
# --- Action: Table output ---
```

```
"${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true
```

```
# --- Action: JSON output ---
```

```
"${CLI_BIN}" --list-models --format json > "${JSON_OUTPUT}" 2>&1
|| true
```

```
# --- Assertions ---
```

```
# ANTI-BLUFF 1: Table output must not contain platform-specific
    artifacts
```

```
if grep -q $'\r' "${OUTPUT_FILE}"; then
    echo
    "[FAIL] Table output contains CRLF line endings (Windows
    artifact on non-Windows)"
    exit 1
fi
```

```
# ANTI-BLUFF 2: JSON output must be valid JSON on all platforms
```

```
if ! python3 -m json.tool "${JSON_OUTPUT}" > /dev/null 2>&1; then
    echo "[FAIL] JSON output is not valid JSON on ${PLATFORM}"
    exit 1
fi
```

```
# ANTI-BLUFF 3: JSON must contain the same top-level keys
    regardless of platform
```

```
EXPECTED_KEYS='["id","name","provider","score","verified"]'
ACTUAL_KEYS=$(python3 -c "
import json, sys
data = json.load(open('${JSON_OUTPUT}'))
```

```

if isinstance(data, list) and len(data) > 0:
    print(json.dumps(sorted(data[0].keys())))
else:
    print('[]')
")

for KEY in "id" "name" "provider" "score" "verified"; do
    if ! echo "${ACTUAL_KEYS}" | grep -q "\"${KEY}\""; then
        echo "[FAIL] JSON missing required key '${KEY}' on ${PLATFORM}"
        exit 1
    fi
done

# ANTI-BLUFF 4: Model count must be consistent (within tolerance
#               for platform-specific providers)
MODEL_COUNT=$(python3 -c "
import json
data = json.load(open('${JSON_OUTPUT}'))
print(len(data)) if isinstance(data, list) else print(0)
")

if [[ "${MODEL_COUNT}" -lt 1 ]]; then
    echo "[FAIL] No models in JSON output on ${PLATFORM}"
    exit 1
fi

echo "[CHALLENGE] verifier_cross_platform_cli_challenge: PASS (${PLATFORM} ${ARCH}, ${MODEL_COUNT} models)"

```

Challenge 9: verifier_startup_pipeline_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "startup pipeline works"
#           =
#   the 5-phase startup completes, discovers real providers, and
#   selects a non-empty debate team.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
SERVER_BIN="${PROJECT_ROOT}/helix_code/bin/server"
LOG_FILE="/tmp/verifier_startup_pipeline.log"

echo "[CHALLENGE] verifier_startup_pipeline_challenge: START"

# --- Action: Start server and capture logs ---

```

```

timeout 60 "${SERVER_BIN}" > "${LOG_FILE}" 2>&1 &
SERVER_PID=$!
sleep 10

# --- Assertions ---

# ANTI-BLUFF 1: Phase 1 (Discover) must log provider discovery
if ! grep -qi "discover" "${LOG_FILE}"; then
    echo "[FAIL] Phase 1 (Discover) not logged"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

# ANTI-BLUFF 2: Phase 2 (Verify) must log verification
if ! grep -qi "verif" "${LOG_FILE}"; then
    echo "[FAIL] Phase 2 (Verify) not logged"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

# ANTI-BLUFF 3: Phase 3 (Score) must log scoring
if ! grep -qi "score" "${LOG_FILE}"; then
    echo "[FAIL] Phase 3 (Score) not logged"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

# ANTI-BLUFF 4: Phase 5 (Debate Team) must select at least 1
model
if ! grep -qi "debate\\|team\\|selected" "${LOG_FILE}"; then
    echo "[FAIL] Phase 5 (Debate Team) not logged"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

# ANTI-BLUFF 5: Server must reach healthy state
if ! curl -sf "http://localhost:8080/health" > /dev/null 2>&1;
then
    echo "[FAIL] Server health check failed after startup"
    kill "${SERVER_PID}" 2>/dev/null || true
    exit 1
fi

kill "${SERVER_PID}" 2>/dev/null || true
echo "[CHALLENGE] verifier_startup_pipeline_challenge: PASS"

```

Challenge 10: verifier_canned_detection_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "canned response
#           detection works" =
#   a model that returns a canned "I cannot assist" response is
#           marked
#   as NOT verified in the verifier DB.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

echo "[CHALLENGE] verifier_canned_detection_challenge: START"

# --- Setup: Insert a model with known canned response pattern
# ---
sqlite3 "${PROJECT_ROOT}/test_data/verifier.db" <<EOF
INSERT OR REPLACE INTO verification_results
(model_id, status, model_exists, responsive,
 supports_code_generation, overall_score,
 code_capability_score)
VALUES (
  (SELECT id FROM models WHERE model_id = 'test-canned-model'),
  'completed',
  1, 1, 0,
  2.0, 1.0
);
UPDATE models SET verification_status = 'failed' WHERE model_id =
  'test-canned-model';
EOF

# --- Action: Query model status via API ---
API_OUTPUT="/tmp/verifier_canned_api.json"
curl -sf "http://localhost:8081/api/models/test-canned-model" >
  "${API_OUTPUT}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: Model must have verification_status = failed or
#           unverified
STATUS=$(python3 -c "
import json, sys
try:
    data = json.load(open('${API_OUTPUT}'))
    print(data.get('verification_status', 'UNKNOWN'))
except:
```



```

    print('UNKNOWN')
")

if [[ "${STATUS}" != "failed" && "${STATUS}" != "unverified" ]];
then
    echo "[FAIL] Canned-response model has status '${STATUS}'
        instead of 'failed'"
    exit 1
fi

# ANTI-BLUFF 2: Score must be low (< 3.0)
SCORE=$(python3 -c "
import json, sys
try:
    data = json.load(open('${API_OUTPUT}'))
    print(data.get('overall_score', 999))
except:
    print(999)
")

if (( $(echo "${SCORE} > 3.0" | bc -l) )); then
    echo "[FAIL] Canned-response model has score ${SCORE} (>
        3.0)"
    exit 1
fi

echo "[CHALLENGE] verifier_canned_detection_challenge: PASS"

```

Challenge 11: verifier_security_redaction_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "security redaction
    works" =
#   API keys NEVER appear in logs, stdout, stderr, or error
    messages.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
SERVER_LOG="/tmp/verifier_security_server.log"
CLI_LOG="/tmp/verifier_security_cli.log"

echo "[CHALLENGE] verifier_security_redaction_challenge: START"

# Use a fake but recognizable API key pattern
FAKE_KEY="sk-antibluff-test-key-9876543210abcdef"

```

```

export HELIX_OPENAI_API_KEY="${FAKE_KEY}"

# --- Action: Run CLI and capture all output ---
"${CLI_BIN}" --list-models > "${CLI_LOG}" 2>&1 || true

# --- Action: Check server logs ---
if [[ -f "${SERVER_LOG}" ]]; then
    grep -r "${FAKE_KEY}" "${SERVER_LOG}" > /dev/null 2>&1 && {
        echo "[FAIL] API key found in server logs"
        exit 1
    }
fi

# --- Assertions ---

# ANTI-BLUFF 1: Fake key must NOT appear in CLI stdout/stderr
if grep -q "${FAKE_KEY}" "${CLI_LOG}"; then
    echo "[FAIL] API key found in CLI output"
    exit 1
fi

# ANTI-BLUFF 2: Fake key must NOT appear in any log file under
# helix_code/
if grep -r "${FAKE_KEY}" "${PROJECT_ROOT}/helix_code/" > /dev/
null 2>&1; then
    echo "[FAIL] API key found somewhere in HelixCode logs or
output"
    exit 1
fi

# ANTI-BLUFF 3: Config dump must redact keys
CONFIG_OUTPUT="/tmp/verifier_config_dump.txt"
"${CLI_BIN}" --config-dump > "${CONFIG_OUTPUT}" 2>&1 || true

if grep -q "${FAKE_KEY}" "${CONFIG_OUTPUT}"; then
    echo "[FAIL] API key found in config dump output"
    exit 1
fi

# ANTI-BLUFF 4: Error messages must not contain key fragments
# (first 8 chars)
KEY_FRAGMENT="${FAKE_KEY:0:8}"
if grep -q "${KEY_FRAGMENT}" "${CLI_LOG}"; then
    echo "[FAIL] API key fragment found in CLI output"
    exit 1
fi

echo "[CHALLENGE] verifier_security_redaction_challenge: PASS"

```

Challenge 12: verifier_scoring_accuracy_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail

# ANTI-BLUFF: This challenge proves that "scoring is accurate" =
# the overall score reflects real verification dimensions,
# not a hardcoded default like 8.5 for everything.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

echo "[CHALLENGE] verifier_scoring_accuracy_challenge: START"

# --- Setup: Insert two models with DIFFERENT scores ---
sqlite3 "${PROJECT_ROOT}/test_data/verifier.db" <<EOF
UPDATE models SET
    overall_score = 9.5,
    code_capability_score = 9.8,
    responsiveness_score = 9.0,
    reliability_score = 9.5,
    feature_richness_score = 9.2,
    value_proposition_score = 8.0
WHERE model_id = 'test-high-score';

UPDATE models SET
    overall_score = 4.0,
    code_capability_score = 3.5,
    responsiveness_score = 5.0,
    reliability_score = 4.0,
    feature_richness_score = 4.5,
    value_proposition_score = 3.0
WHERE model_id = 'test-low-score';
EOF

# --- Action: Query both models via API ---
HIGH_OUTPUT="/tmp/verifier_score_high.json"
LOW_OUTPUT="/tmp/verifier_score_low.json"
curl -sf "http://localhost:8081/api/models/test-high-score" > "${HIGH_OUTPUT}" 2>&1 || true
curl -sf "http://localhost:8081/api/models/test-low-score" > "${LOW_OUTPUT}" 2>&1 || true

# --- Assertions ---

# ANTI-BLUFF 1: High-score model must have overall_score >= 9.0
HIGH_SCORE=$(python3 -c "
import json
```

```

data = json.load(open('${HIGH_OUTPUT}'))
print(data.get('overall_score', 0))
")
if (( $(echo "${HIGH_SCORE} < 9.0" | bc -l) )); then
    echo "[FAIL] High-score model has score ${HIGH_SCORE} (<
    9.0)"
    exit 1
fi

# ANTI-BLUFF 2: Low-score model must have overall_score <= 5.0
LOW_SCORE=$(python3 -c "
import json
data = json.load(open('${LOW_OUTPUT}'))
print(data.get('overall_score', 10))
")
if (( $(echo "${LOW_SCORE} > 5.0" | bc -l) )); then
    echo "[FAIL] Low-score model has score ${LOW_SCORE} (> 5.0)"
    exit 1
fi

# ANTI-BLUFF 3: Scores must be DIFFERENT
if (( $(echo "${HIGH_SCORE} == ${LOW_SCORE}" | bc -l) )); then
    echo "[FAIL] High and low scores are identical ($
    {HIGH_SCORE}) – hardcoded score detected"
    exit 1
fi

# ANTI-BLUFF 4: Score must NOT be exactly 8.5 (the known stub
    value)
if (( $(echo "${HIGH_SCORE} == 8.5" | bc -l) )) || (( $(echo "${
    {LOW_SCORE} == 8.5" | bc -l) )); then
    echo "[FAIL] Score is exactly 8.5 – likely hardcoded stub
    value"
    exit 1
fi

echo "[CHALLENGE] verifier_scoring_accuracy_challenge: PASS"

```

7. Security Tests

Scope: API key redaction, secret handling, permissions, encryption.

Mock Policy: NO mocks. Tests real secret handling paths.

Constitutional Basis: CONST-025.

7.1 Security Test File Inventory

#	File Path	Purpose
1	tests/security/ api_key_redaction_test.go	API key redaction in logs and output
2	tests/security/ secrets_in_errors_test.go	No secrets leaked in error messages
3	tests/security/ config_permissions_test.go	Config file permission checks
4	tests/security/ database_encryption_test.go	SQLite encryption verification
5	tests/security/ jwt_secret_handling_test.go	JWT secret isolation
6	tests/security/ provider_key_storage_test.go	Provider key storage security
7	tests/security/ verifier_api_auth_test.go	Verifier API authentication enforcement
8	tests/security/ env_var_exposure_test.go	Environment variable exposure

7.2 Exact Test Function Signatures

File: tests/security/api_key_redaction_test.go

```
func TestAPIKeyRedaction_Logs(t *testing.T)
func TestAPIKeyRedaction_ErrorMessages(t *testing.T)
func TestAPIKeyRedaction_HTTPHeaders(t *testing.T)
func TestAPIKeyRedaction_CLIOutput(t *testing.T)
func TestAPIKeyRedaction_ConfigDump(t *testing.T)
func TestAPIKeyRedaction_DebugEndpoint(t *testing.T)
func TestAPIKeyRedaction_PanicRecovery(t *testing.T)
```

Anti-Bluff Criteria: - TestAPIKeyRedaction_Logs: Must initialize a real logger, perform an operation that would naturally log the API key (e.g., a provider error), capture the log output to a buffer, and assert the string sk- (or the actual key substring) does NOT appear. A test that only checks a utility function redact() is a bluff. - TestAPIKeyRedaction_PanicRecovery: Must trigger a real panic inside a function that has the API key in scope, recover, and assert the recovered error/stack does not contain the key.

File: tests/security/secrets_in_errors_test.go

```
func TestSecretsNotInErrors_HTTPError(t *testing.T)
func TestSecretsNotInErrors_DBError(t *testing.T)
func TestSecretsNotInErrors_ProviderInitError(t *testing.T)
func TestSecretsNotInErrors_ValidationError(t *testing.T)
func TestSecretsNotInErrors_NetworkError(t *testing.T)
func TestSecretsNotInErrors_JSONMarshalError(t *testing.T)
```

Anti-Bluff Criteria: - Each test must use a REAL API key (test key), trigger the specific error condition, capture the error string, and assert the key is not present. A test using `errors.New("some error")` and checking `redact()` is a bluff.

File: tests/security/config_permissions_test.go

```
func TestConfigPermissions_CreationMode(t *testing.T)
func TestConfigPermissions_WorldReadable(t *testing.T)
func TestConfigPermissions_WorldWritable(t *testing.T)
func TestConfigPermissions_SecretFile(t *testing.T)
```

Anti-Bluff Criteria: - `TestConfigPermissions_WorldReadable`: Must create a config file with an embedded API key, set mode `0644`, then assert the application refuses to load it or logs a security warning. If the app loads it without complaint, test FAILs. - `TestConfigPermissions_SecretFile`: Must create a `.env` file with mode `0600`, assert it is accepted. Then change to `0644` and assert rejection or warning.

File: tests/security/database_encryption_test.go

```
func TestDatabaseEncryption_SQLCipherEnabled(t *testing.T)
func TestDatabaseEncryption_SQLCipherDisabled(t *testing.T)
func TestDatabaseEncryption_KeyRotation(t *testing.T)
func TestDatabaseEncryption_PlaintextNotInFile(t *testing.T)
```

Anti-Bluff Criteria: - `TestDatabaseEncryption_PlaintextNotInFile`: Must create a verifier DB with encryption ON, insert a model with a unique name (e.g., `ENCRYPTION_TEST_12345`), then read the raw SQLite file bytes and assert the string `ENCRYPTION_TEST_12345` does NOT appear. Finding the plaintext string causes failure.

8. Performance Tests

Scope: Latency, memory, concurrency, throughput. **Mock Policy:** NO mocks. Real verifier, real cache, real DB. **Constitutional Basis:** CONST-014.

8.1 Performance Test File Inventory

#	File Path	Purpose
1	tests/performance/ model_list_latency_test.go	Model list retrieval latency
2	tests/performance/ verifier_polling_overhead_test.go	Verifier polling CPU overhead
3	tests/performance/ memory_model_registry_test.go	Memory with 100+ models
4	tests/performance/ concurrent_registry_test.go	Concurrent model registry access
5	tests/performance/ scoring_latency_test.go	Scoring calculation latency
6	tests/performance/ discovery_latency_test.go	Discovery latency with many providers
7	tests/performance/ cache_hit_latency_test.go	Cache hit latency
8	tests/performance/ startup_pipeline_latency_test.go	Full startup pipeline latency

8.2 Exact Test Function Signatures

File: tests/performance/model_list_latency_test.go

```
func BenchmarkModelList_Cached(b *testing.B)
func BenchmarkModelList_Uncached(b *testing.B)
func TestModelListLatency_Cached_Under500ms(t *testing.T)
func TestModelListLatency_Uncached_Under2s(t *testing.T)
func TestModelListLatency_FirstCall_Under5s(t *testing.T)
```

Anti-Bluff Criteria: - TestModelListLatency_Cached_Under500ms: Must measure WALL CLOCK time (not CPU time) for 100 sequential calls and assert the 95th percentile is under 500ms. A test that measures a single call and asserts < 500ms is insufficient — it must be statistically meaningful. - The cache must be primed before measurement. If the cache is cold, the test must FAIL.

File: tests/performance/memory_model_registry_test.go

```
func TestMemory_100Models(t *testing.T)
func TestMemory_500Models(t *testing.T)
func TestMemory_1000Models(t *testing.T)
```

```
func TestMemory_ModelRegistry_GCStable(t *testing.T)
func TestMemory_ModelRegistry_NoLeak(t *testing.T)
```

Anti-Bluff Criteria: - TestMemory_100Models: Must register 100 models, force GC, and assert RSS increase is under 50MB. Must print the actual memory delta. A test without runtime.GC() and runtime.ReadMemStats() is a bluff. - TestMemory_ModelRegistry_NoLeak: Must register and unregister models in a loop (1000 iterations), force GC, and assert final memory is within 10% of baseline. Growing memory without bound causes failure.

File: tests/performance/concurrent_registry_test.go

```
func TestConcurrentRegistry_ReadWrite(t *testing.T)
func TestConcurrentRegistry_MultipleReaders(t *testing.T)
func TestConcurrentRegistry_WriteDuringRead(t *testing.T)
func TestConcurrentRegistry_RaceDetection(t *testing.T)
```

Anti-Bluff Criteria: - TestConcurrentRegistry_RaceDetection: Must run with go test -race and assert NO race conditions are detected. A test that runs without -race is a bluff. - TestConcurrentRegistry_ReadWrite: Must use 100 goroutines (50 readers, 50 writers) for 5 seconds and assert no panics, no deadlocks, and correct final state.

File: tests/performance/verifier_polling_overhead_test.go

```
func TestVerifierPollingOverhead_CPUUnder5Percent(t *testing.T)
func TestVerifierPollingOverhead_NoSpikes(t *testing.T)
```

Anti-Bluff Criteria: - TestVerifierPollingOverhead_CPUUnder5Percent: Must start the verifier polling loop, measure CPU usage for 30 seconds, and assert average CPU is under 5%. A test without actual CPU measurement is a bluff.

9. Coverage Enforcement

9.1 Coverage Target: 100%

Per **CONST-002**, the target is **100% coverage across all supported test types**. This means:

Test Type	Coverage Metric	Enforcement
Unit	Line coverage	100% of lines in internal/verifier/, internal/llm/

Test Type	Coverage Metric	Enforcement
		verifier_*.go, cmd/cli/verifier_*.go
Contract	API endpoint coverage	100% of documented endpoints exercised
Component	Subsystem interaction coverage	100% of subsystem pairs wired and tested
Integration	Dependency coverage	100% of configured dependencies (PG, Redis, SQLite, providers) exercised
E2E / Challenge	Feature coverage	100% of user-facing features have a challenge script
Security	Attack surface coverage	100% of secret-handling paths tested
Performance	Benchmark coverage	Every public function with >10ms expected latency has a benchmark

9.2 Coverage Measurement Mechanism

Go Line Coverage

Unit + Component coverage

```
go test -coverprofile=coverage-unit.out -short ./internal/
verifier/... ./internal/llm/... ./cmd/cli/...
```

Integration coverage (excludes unit-only files)

```
go test -coverprofile=coverage-integration.out -run
Integration ./tests/integration/...
```

Combined coverage

```
go tool cover -func=coverage-unit.out | tail -1
go tool cover -func=coverage-integration.out | tail -1
```

Coverage Enforcement Script

File: scripts/enforce_coverage.sh

```
#!/usr/bin/env bash
set -euo pipefail

UNIT_THRESHOLD=100
INTEGRATION_THRESHOLD=95
CONTRACT_THRESHOLD=100
SECURITY_THRESHOLD=95
PERFORMANCE_THRESHOLD=80

# Unit coverage
UNIT_COVER=$(go test -short ./internal/verifier/... ./internal/
    llm/... ./cmd/cli/... -cover 2>/dev/null | grep -oP '\d+
    \.\d+%' | tail -1 | tr -d '%')
if (( $(echo "${UNIT_COVER} < ${UNIT_THRESHOLD}" | bc -l) ));
then
    echo "[FAIL] Unit coverage ${UNIT_COVER}% < ${UNIT_THRESHOLD}
    %"
    exit 1
fi

# Integration coverage
INTEGRATION_COVER=$(go test -run Integration ./tests/
    integration/... -cover 2>/dev/null | grep -oP '\d+\.\d+
    %' | tail -1 | tr -d '%')
if (( $(echo "${INTEGRATION_COVER} < ${INTEGRATION_THRESHOLD}" |
    bc -l) )); then
    echo "[FAIL] Integration coverage ${INTEGRATION_COVER}% < $
    {INTEGRATION_THRESHOLD}%"
    exit 1
fi

echo "[PASS] Coverage check passed"
```

9.3 No-Mocks-Above-Unit Scanner

File: scripts/no_mock_above_unit.sh

```
#!/usr/bin/env bash
set -euo pipefail

# CONST-021: No Mocks Above Unit
# Scans all test files outside of *_test.go (short mode) for mock
    usage

VIOLATIONS=0

# Find all test files that are NOT in unit-test-only directories
```

```

for FILE in $(find tests/ internal/ -name "*_test.go" | grep -v
    "^internal/verifier/*_test.go" | grep -v "^internal/
    llm/*_test.go" | grep -v "^cmd/cli/*_test.go"); do
    if grep -qE "(mock|Mock|gomock|mockery|httptest)" "${FILE}";
    then
        echo "[VIOLATION] Mock usage found in non-unit test: $
        {FILE}"
        VIOLATIONS=$((VIOLATIONS + 1))
    fi
done

if [[ "${VIOLATIONS}" -gt 0 ]]; then
    echo "[FAIL] ${VIOLATIONS} mock violations found above unit
    test level"
    exit 1
fi

echo "[PASS] No mocks above unit tests"

```

Note: httptest is allowed in unit tests for local HTTP server simulation. It is NOT allowed in integration or component tests because those must use real running servers.

10. Test Infrastructure

10.1 Docker Compose for Test Dependencies

File: docker/docker-compose.test.yml

```

version: "3.9"

services:
  postgres-test:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: helix
      POSTGRES_PASSWORD: ${HELIX_DATABASE_PASSWORD:?set
        HELIX_DATABASE_PASSWORD in .env (run ./setup.sh, or
        cp .env.example .env)}
      POSTGRES_DB: helixcode_test
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U helix"]
      interval: 2s
      timeout: 5s
      retries: 10

```

```

tmpfs:
  - /var/lib/postgresql/data

redis-test:
  image: redis:7-alpine
  ports:
    - "6380:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 2s
    timeout: 5s
    retries: 10

verifier-test:
  build:
    context: ../LLMsVerifier
    dockerfile: Dockerfile
  environment:
    - VERIFIER_DATABASE_PATH=/data/verifier-test.db
    - VERIFIER_API_PORT=8081
  ports:
    - "8081:8081"
  volumes:
    - verifier-test-data:/data
  healthcheck:
    test: ["CMD-SHELL", "curl -sf http://localhost:8081/api/health || exit 1"]
    interval: 5s
    timeout: 5s
    retries: 10

ollama-test:
  image: ollama/ollama:latest
  ports:
    - "11435:11434"
  # Pull a tiny model for testing
  entrypoint: >
    sh -c "ollama serve &
            sleep 5 &&
            ollama pull llama3.2:1b &&
            wait"

volumes:
  verifier-test-data:

```

10.2 Test Data Fixtures

Directory: tests/fixtures/

File	Purpose
fixtures/ verifier_db_seed.sql	SQLite seed data for verifier DB
fixtures/ provider_responses/	Cached real provider API responses (for offline contract validation)
fixtures/configs/	Test configuration files (YAML/JSON/TOML)
fixtures/keys/	Test API keys (dummy values, NOT real)
fixtures/models/	Test model metadata JSON files

File: tests/fixtures/verifier_db_seed.sql

```
-- Seed data for verifier test database
INSERT INTO providers (name, type, base_url, status) VALUES
('openai', 'openai', 'https://api.openai.com/v1', 'active'),
('anthropic', 'anthropic', 'https://api.anthropic.com/v1',
  'active'),
('deepseek', 'deepseek', 'https://api.deepseek.com/v1',
  'active');

INSERT INTO models (provider_id, model_id, name, description,
  overall_score, verification_status)
VALUES
(1, 'gpt-4o', 'GPT-4o', 'OpenAI GPT-4o', 9.2, 'verified'),
(1, 'gpt-4o-mini', 'GPT-4o Mini', 'OpenAI GPT-4o Mini', 8.5,
  'verified'),
(2, 'claude-sonnet-4', 'Claude Sonnet 4', 'Anthropic Claude
  Sonnet', 8.8, 'verified'),
(3, 'deepseek-chat', 'DeepSeek Chat', 'DeepSeek V3', 8.5,
  'verified');
```

10.3 Environment Setup Scripts

File: scripts/setup_test_env.sh

```
#!/usr/bin/env bash
set -euo pipefail

# Setup test environment for LLMsVerifier integration tests

echo "[SETUP] Starting test environment setup..."

# 1. Build CLI and server binaries
cd "${PROJECT_ROOT}/HelixCode"
make build-cli build-server
```

```

# 2. Start Docker Compose test infrastructure
docker compose -f "${PROJECT_ROOT}/docker/docker-
    compose.test.yml" up -d --wait

# 3. Seed verifier test database
docker compose -f "${PROJECT_ROOT}/docker/docker-
    compose.test.yml" exec -T verifier-test \
    sqlite3 /data/verifier-test.db < "${PROJECT_ROOT}/tests/
    fixtures/verifier_db_seed.sql"

# 4. Verify all services are healthy
for SERVICE in postgres-test redis-test verifier-test; do
    if ! docker compose -f "${PROJECT_ROOT}/docker/docker-
        compose.test.yml" ps "${SERVICE}" | grep -q "healthy";
    then
        echo "[FAIL] Service ${SERVICE} is not healthy"
        exit 1
    fi
done

# 5. Export test environment variables
export HELIX_DATABASE_HOST=localhost
export HELIX_DATABASE_PORT=5433
export HELIX_DATABASE_PASSWORD=CHANGE_ME_db_password
export HELIX_REDIS_HOST=localhost
export HELIX_REDIS_PORT=6380
export HELIX_VERIFIER_URL=http://localhost:8081

echo "[SETUP] Test environment ready"

File: scripts/teardown_test_env.sh

#!/usr/bin/env bash
set -euo pipefail

echo "[TEARDOWN] Stopping test environment..."
docker compose -f "${PROJECT_ROOT}/docker/docker-
    compose.test.yml" down -v
echo "[TEARDOWN] Test environment stopped"

```

10.4 Test Runner Script

File: scripts/run_tests.sh

```

#!/usr/bin/env bash
set -euo pipefail

TEST_TYPE="${1:-all}"

```

```

case "${TEST_TYPE}" in
    unit)
        echo "[TEST] Running unit tests..."
        go test -short -v ./internal/verifier/... ./internal/
            llm/... ./cmd/cli/... ./internal/services/...
        ;;
    contract)
        echo "[TEST] Running contract tests..."
        go test -v -run Contract ./tests/contract/...
        ;;
    component)
        echo "[TEST] Running component tests..."
        go test -v -run Component ./tests/component/...
        ;;
    integration)
        echo "[TEST] Running integration tests..."
        go test -v -run Integration ./tests/integration/...
        ;;
    e2e|challenge)
        echo "[TEST] Running E2E challenge scripts..."
        cd "${PROJECT_ROOT}/challenges/scripts"
        for SCRIPT in verifier*_challenge.sh; do
            echo "[TEST] Running ${SCRIPT}..."
            bash "${SCRIPT}"
        done
        ;;
    security)
        echo "[TEST] Running security tests..."
        go test -v -run Security ./tests/security/...
        ;;
    performance)
        echo "[TEST] Running performance tests..."
        go test -v -bench=. -benchmem ./tests/performance/...
        ;;
    coverage)
        echo "[TEST] Running coverage enforcement..."
        bash "${PROJECT_ROOT}/scripts/enforce_coverage.sh"
        bash "${PROJECT_ROOT}/scripts/no_mock_above_unit.sh"
        ;;
    all|complete)
        echo "[TEST] Running complete test suite..."
        bash "${PROJECT_ROOT}/scripts/setup_test_env.sh"
        make test-unit-full
        make test-contract-full
        make test-component-full
        make test-integration-full
        make test-e2e-full
        make test-security-full

```

```

        make test-load-full
        make coverage-full
        bash "${PROJECT_ROOT}/scripts/teardown_test_env.sh"
        ;;
    *)
        echo "Unknown test type: ${TEST_TYPE}"
        echo "Usage: $0 {unit|contract|component|integration|e2e|
        security|performance|coverage|all}"
        exit 1
        ;;
esac

```

11. Anti-Bluff Verification Checklist Matrix

This matrix maps every user-facing feature to the test that proves it works and the exact verification method.

Feature	Test File / Challenge
Model listing (CLI)	verifier_model_list_challenge.sh
Model listing (API)	tests/integration/ helixcode_full_stack_test.go:TestFullStack_APIModels_ReturnsV
Model selection with scoring	tests/component/ model_manager_verifier_component_test.go:TestModelManager_Se
Code generation	verifier_model_select_challenge.sh

Feature	Test File / Challenge
Verifier disable + fallback	verifier_disable_fallback_challenge.sh
API key provisioning	verifier_api_key_provision_challenge.sh
Rate limiting display	verifier_rate_limit_display_challenge.sh
Real-time updates	verifier_realtime_update_challenge.sh
MCP integration	verifier_mcp_lsp_acp_challenge.sh
LSP integration	verifier_mcp_lsp_acp_challenge.sh
ACP integration	verifier_mcp_lsp_acp_challenge.sh

Feature	Test File / Challenge
Embedding integration	verifier_mcp_lsp_acp_challenge.sh
Cross-platform CLI	verifier_cross_platform_cli_challenge.sh
Startup pipeline	verifier_startup_pipeline_challenge.sh
Canned response detection	verifier_canned_detection_challenge.sh
Security redaction	verifier_security_redaction_challenge.sh
Scoring accuracy	verifier_scoring_accuracy_challenge.sh
Cache hit latency	tests/performance/model_list_latency_test.go
Database encryption	tests/security/database_encryption_test.go

Feature	Test File / Challenge
Provider fallback chain	tests/integration/helixcode_fallback_chain_test.go
Health monitoring	tests/component/startup_pipeline_component_test.go
Config hot-reload	tests/integration/helixcode_verifier_config_reload_test.go
Event publishing	tests/component/event_bus_verifier_component_test.go
Alias resolution	internal/verifier/aliases_test.go
Subscription detection	internal/verifier/subscription_detector_test.go
Score suffix format	internal/verifier/scoring_test.go

Feature	Test File / Challenge
Verification result persistence	tests/integration/helixcode_verifier_sqlite_test.go
Rate limit header parsing	internal/verifier/rate_limit_test.go
Concurrent model registry	tests/performance/concurrent_registry_test.go
Memory stability	tests/performance/memory_model_registry_test.go
API schema validation	tests/contract/schema_validation_test.go
Error response format	tests/contract/error_response_contract_test.go
JWT auth on verifier API	tests/security/verifier_api_auth_test.go
Config file permissions	tests/security/config_permissions_test.go

Appendix A: Makefile Target Definitions

Add these targets to helix_code/Makefile:

```
# --- Test Infrastructure ---
.PHONY: test-infra-up test-infra-down test-infra-status
test-infra-up:
    docker compose -f ../docker/docker-compose.test.yml up -d --
        wait

test-infra-down:
    docker compose -f ../docker/docker-compose.test.yml down -v

test-infra-status:
    docker compose -f ../docker/docker-compose.test.yml ps

# --- Unit Tests ---
.PHONY: test-unit test-unit-full test-unit-coverage
test-unit:
    go test -short -v ./internal/verifier/... ./internal/
        llm/... ./cmd/cli/... ./internal/services/...

test-unit-full: test-infra-up
    go test -short -v -race -coverprofile=coverage-unit.out ./
        internal/verifier/... ./internal/llm/... ./cmd/
        cli/... ./internal/services/...
    go tool cover -func=coverage-unit.out | tail -1

test-unit-coverage:
    go test -short -coverprofile=coverage-unit.out ./internal/
        verifier/... ./internal/llm/... ./cmd/cli/... ./
        internal/services/...
    go tool cover -html=coverage-unit.out -o coverage-unit.html

# --- Contract Tests ---
.PHONY: test-contract test-contract-full
test-contract: test-infra-up
    go test -v -run Contract ./tests/contract/...

test-contract-full: test-infra-up
    go test -v -race -run Contract -coverprofile=coverage-
        contract.out ./tests/contract/...

# --- Component Tests ---
.PHONY: test-component test-component-full
test-component: test-infra-up
    go test -v -run Component ./tests/component/...
```

```
test-component-full: test-infra-up
    go test -v -race -run Component -coverprofile=coverage-
        component.out ./tests/component/...

# --- Integration Tests ---
.PHONY: test-integration test-integration-full
test-integration: test-infra-up
    go test -v -run Integration ./tests/integration/...

test-integration-full: test-infra-up
    go test -v -race -run Integration -coverprofile=coverage-
        integration.out ./tests/integration/...

# --- E2E / Challenge Tests ---
.PHONY: test-e2e test-e2e-full
test-e2e:
    cd ../challenges/scripts && bash
        run_all_verifier_challenges.sh

test-e2e-full: test-infra-up build-cli build-server
    cd ../challenges/scripts && bash
        run_all_verifier_challenges.sh

# --- Security Tests ---
.PHONY: test-security test-security-full
test-security: test-infra-up
    go test -v -run Security ./tests/security/...

test-security-full: test-infra-up
    go test -v -race -run Security -coverprofile=coverage-
        security.out ./tests/security/...

# --- Performance / Load Tests ---
.PHONY: test-performance test-load-full
test-performance: test-infra-up
    go test -v -bench=. -benchmem -benchtime=30s ./tests/
        performance/...

test-load-full: test-infra-up
    go test -v -bench=. -benchmem -benchtime=60s
        -cpuprofile=cpu.prof -memprofile=mem.prof ./tests/
        performance/...

# --- Coverage & Quality Gates ---
.PHONY: coverage-full no-mocks-above-unit
coverage-full: test-unit-full test-contract-full test-component-
    full test-integration-full
@echo "--- Combined Coverage Report ---"
```

```
go tool cover -func=coverage-unit.out | tail -1
go tool cover -func=coverage-contract.out | tail -1
go tool cover -func=coverage-component.out | tail -1
go tool cover -func=coverage-integration.out | tail -1
```

no-mocks-above-unit:

```
bash ../scripts/no_mocks_above_unit.sh
```

--- Complete Test Suite ---

.PHONY: test-complete test-full

test-complete: test-unit-full test-contract-full test-component-
full test-integration-full test-e2e-full test-security-
full test-load-full coverage-full no-mocks-above-unit

```
@echo "=====
```

```
@echo "ALL TESTS COMPLETE"
```

```
@echo "=====
```

test-full: test-complete

--- Documentation Tests ---

.PHONY: test-docs

test-docs:

Verify all documented test files exist

```
@test -f internal/verifier/client_test.go || (echo "MISSING:  
client_test.go"; exit 1)
```

```
@test -f tests/contract/verifier_api_contract_test.go ||  
(echo "MISSING: verifier_api_contract_test.go"; exit 1)
```

```
@test -f tests/component/  
verifier_client_cache_component_test.go || (echo  
"MISSING:  
verifier_client_cache_component_test.go"; exit 1)
```

```
@test -f tests/integration/helixcode_verifier_sqlite_test.go  
|| (echo "MISSING: helixcode_verifier_sqlite_test.go";  
exit 1)
```

```
@test -f tests/security/api_key_redaction_test.go || (echo  
"MISSING: api_key_redaction_test.go"; exit 1)
```

```
@test -f tests/performance/model_list_latency_test.go ||  
(echo "MISSING: model_list_latency_test.go"; exit 1)
```

```
@test -f challenges/scripts/verifier_model_list_challenge.sh  
|| (echo "MISSING: verifier_model_list_challenge.sh";  
exit 1)
```

```
@echo "[PASS] All documented test files exist"
```

Appendix B: Constitution Cross-Reference

Constitution ID	Requirement	Test Strategy Implementation
CONST-001	No CI/CD	All tests run via Makefile targets and shell scripts; no GitHub Actions, no Jenkins, no pipeline YAML
CONST-002	100% Test Coverage	<code>scripts/enforce_coverage.sh</code> enforces 100% unit, 95%+ integration; <code>coverage-full</code> target generates reports
CONST-002a	No Mocks Above Unit	<code>scripts/no_mock_above_unit.sh</code> scans and rejects any mock usage outside <code>*_test.go</code> short-mode files
CONST-003	No HTTPS for Git	N/A for testing (SSH-only repo access enforced elsewhere)
CONST-004	No manual container commands	<code>docker-compose.test.yml</code> is declarative; <code>make test-infra-up</code> is the only orchestrator-approved entry point
CONST-005	100% real data for non-unit tests	All contract/component/integration/e2e/security tests use real DBs, real APIs, real keys, real CLI binary
CONST-006	Challenge coverage for every component	12 challenge scripts cover every verifier component; matrix in Section 11 maps components to challenges
CONST-017	Zero-Bluff Testing	This entire document is the implementation; every test has anti-bluff criteria
CONST-020	Provider Fallback Chain Reality	<code>helixcode_fallback_chain_test.go</code> tests with real wrong endpoints, not mock errors
CONST-021	No Mocks Above Unit target	<code>make no-mocks-above-unit</code> runs the scanner
CONST-025	Secret Management	Security tests verify keys are never in logs/errors/output; config permissions enforce 0600

Constitution ID	Requirement	Test Strategy Implementation
CONST-035	End-User Usability Mandate	Every challenge script verifies exact user-visible output; <code>t.Skip()</code> requires SKIP-OK justification

Summary of Deliverables

Category	Count	Files
New Unit Test Files	20	internal/verifier/*_test.go, internal/llm/verifier*_test.go, cmd/cli/verifier_cli_test.go, internal/services/ llmsverifier_score_adapter_test.go
New Contract Test Files	6	tests/contract/*_contract_test.go
New Component Test Files	6	tests/component/ *_component_test.go
New Integration Test Files	8	tests/integration/ helixcode*_test.go
New Challenge Scripts	12	challenges/scripts/ verifier*_challenge.sh
New Security Test Files	8	tests/security/*_test.go
New Performance Test Files	8	tests/performance/*_test.go
Docker Compose	1	docker/docker-compose.test.yml
Test Fixtures	5+	tests/fixtures/*
Setup/Teardown Scripts	3	scripts/setup_test_env.sh, scripts/ teardown_test_env.sh, scripts/ run_tests.sh
Coverage Enforcement	2	scripts/enforce_coverage.sh, scripts/no_mock_above_unit.sh
Makefile Targets	20+	Added to helix_code/Makefile

Total New Files: ~75

Total Estimated Lines: ~12,000+

Constitutional Compliance: 100% (CONST-001 through CONST-035)

[End of Section 6]

LLMsVerifier Integration — Constitution & Documentation Updates

Status: Draft for insertion into HelixCode, HelixAgent, and ALL submodules

Mandate: User explicitly demands: “This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules’s Constitution, CLAUDE.MD and AGENTS.MD as well.”

Version: 1.0.0-Draft

Date: 2026-07-01

Table of Contents

- 1. [Executive Summary](#)
 - 2. [CONSTITUTION.md Amendments](#)
 - 3. [CLAUDE.md Updates](#)
 - 4. [AGENTS.md Updates](#)
 - 5. [Configuration Documentation](#)
 - 6. [User Guide](#)
 - 7. [Integration Guide for Developers](#)
 - 8. [Submodule Constitution Template](#)
-

1. Executive Summary

This document specifies the **exact content** that must be inserted into CONSTITUTION.md, CLAUDE.md, AGENTS.md, and all submodule equivalents to codify LLMsVerifier integration as **project law**. The following gaps in HelixCode have been identified from analysis:

Gap	Current State	Required State
BLUFF-002	cmd/cli/main.go:101-128 hardcodes 3 models	Must fetch from LLMsVerifier

Gap	Current State	Required State
BLUFF-001	Simulated LLM responses in .old file	All generation must route through verified providers
No Dynamic Model Source	model_discovery.go hardcodes external models	Must use LLMsVerifier as single source of truth
No Verifier Config	No LLMsVerifier section in config.go	Full verifier YAML schema required
No Anti-Bluff Test Guarantee	CONST-035 exists but lacks verifier-specific language	Must mandate tests that prove features actually work
Submodule Alignment	Submodules have independent constitutions	Must propagate all rules below

Legal Force: Every rule in Section 2 carries the same binding authority as CONST-001 through CONST-035. Violation is a constitutional breach.

2. CONSTITUTION.md Amendments

2.1 Insert After CONST-035 — New Rules CONST-036 through CONST-040

Insertion Point: Immediately after the closing block of CONST-035 in CONSTITUTION.md (HelixCode) and equivalent location in all submodule CONSTITUTION.md files.

CONST-036: LLMsVerifier Single Source of Truth Mandate

****Rule**:** LLMsVerifier SHALL BE the sole authoritative source for:

1. All model metadata (names, IDs, context windows, capabilities)
2. All provider metadata (endpoints, auth types, supported models)
3. All verification status (verified, partial, failed, pending)
4. All scoring data (overall scores, capability scores, tier rankings)
5. All rate-limit and cooldown state

*****Prohibition*****: NO hardcoded model lists, NO hardcoded provider lists, NO simulated model discovery. Any code path that presents a model or provider listing to a user **MUST** fetch that listing from the LLMsVerifier subsystem or its cached replica.

*****Anti-Bluff Verification*****:

- The challenge script ``challenges/scripts/llmsverifier_hardcode_check.sh`` **MUST** scan all Go source files for hardcoded model arrays.
- Any ``[]string{"gpt-4", "claude-3"}`` or equivalent literal in production code is a constitutional violation.
- The only permitted hardcoded data is the LLMsVerifier service endpoint URL and the list of verification test types.

*****Enforcement*****: ``make test-complete`` **MUST** include a test that asserts ``ModelManager.GetAvailableModels()`` returns at least as many models as the verifier's database contains for configured providers. A test that passes while the CLI shows a hardcoded list is a TEST BLUFF and violates CONST-017.

CONST-037: Model Provider Anti-Bluff Guarantee

*****Rule*****: Every model displayed to an end user **MUST** have been verified by LLMsVerifier within the last ``verification_timeout`` period (default: 24h). Models older than this **MUST** display a "stale" indicator and be deprioritized.

*****Prohibition Against Test Bluffing*****:

- A unit test that mocks the verifier client and asserts ``GetAvailableModels()`` returns 3 models **DOES NOT** satisfy this rule.
- An integration test that starts the verifier server, performs real provider discovery, and confirms the model count matches the actual provider API response **DOES** satisfy this rule.
- The Makefile target ``make test-verifier-integration`` **MUST** exist and **MUST** run without mocks.

*****The "Tests Pass But Features Don't Work" Guarantee***** (User-Demand Anti-Bluff):

NO TEST MAY PASS UNLESS THE FEATURE IT TESTS IS DEMONSTRABLY USABLE BY AN END USER IN THE SAME BUILD.

- If `TestModelList` passes but `helixcode --list-models` shows hardcoded
- If `TestProviderHealth` passes but the health endpoint returns `200 OK`
- If `TestLLMGeneration` passes but `--prompt "hello"` returns a simulated
- Bluff tests MUST be rewritten or deleted. There is no "grandfather" exce

****Evidence Standard****: Every test that claims to verify model/provider fun

1. Call a real API endpoint or a real verifier database
2. Assert on response content that could only come from that real source
3. Include a `t.Parallel()` integration test that runs the CLI binary with

CONST-038: Real-Time Model Status Accuracy

****Rule****: Model status (available, rate-limited, cooldown, offline, deprec

****Polling vs. Push****:

- If WebSocket/SSE push is unavailable, the system MUST poll LLMsVerifier
- The TUI MUST display a "last updated" timestamp with every model listing
- Models in "cooldown" or "rate-limited" state MUST show the estimated rec

****Accuracy Verification****:

- Challenge script `challenges/scripts/model\_status\_accuracy\_challenge.sh`
 1. Artificially rate-limit a provider by exhausting its quota
 2. Wait for the status to propagate to the verifier
 3. Check that `helixcode --list-models` shows the rate-limited status wi
 4. Check that `SelectOptimalModel()` no longer selects the rate-limited

****Prohibition****: Status indicators that are "always green" or that lag >60

CONST-039: All Providers and Models Integration Mandate

****Rule****: HelixCode MUST integrate with ALL providers and models that LLMs

1. The provider being explicitly disabled in configuration (`enabled: fals
2. The API key being absent and the provider requiring one
3. The provider being marked `deprecated` in the verifier database

****Minimum Provider Set**** (SHALL NOT be reduced without constitutional amen

Provider	Auth Type	Required Env Var
OpenAI	API Key	`HELIX_OPENAI_API_KEY`
Anthropic	API Key / OAuth	`HELIX_ANTHROPIC_API_KEY`
Gemini	API Key	`HELIX_GEMINI_API_KEY`
DeepSeek	API Key	`HELIX_DEEPSEEK_API_KEY`
Groq	API Key	`HELIX_GROQ_API_KEY`
Together AI	API Key	`HELIX_TOGETHER_API_KEY`
Mistral	API Key	`HELIX_MISTRAL_API_KEY`
xAI	API Key	`HELIX_XAI_API_KEY`
Cerebras	API Key	`HELIX_CEREBRAS_API_KEY`
Cloudflare Workers AI	API Key + Account ID	`HELIX_CLOUDFLARE_API_KEY`

```
| SiliconFlow | API Key | `HELIX_SILICONFLOW_API_KEY` |
| Replicate | Token | `HELIX_REPLICATE_API_TOKEN` |
| OpenRouter | API Key | `HELIX_OPENROUTER_API_KEY` |
| Ollama | Local | None (auto-detect) |
| Llama.cpp | Local | None (auto-detect) |
```

****Integration Requirement**:** For every provider in the minimum set:

- There MUST be a provider adapter file in `internal/llm/` or `internal/verifier/`
- There MUST be a `\*\_test.go` file with real API tests (skipped only if `HELIX\_SKIP\_API\_TESTS` is set)
- There MUST be a challenge script in `challenges/scripts/`
- The model listing MUST include models from this provider when the provider is in the minimum set

CONST-040: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Integration

****Rule**:** LLMsVerifier integration SHALL extend beyond basic model listing

1. ****MCP (Model Context Protocol)**:** The verifier MUST report which models support MCP
2. ****LSP (Language Server Protocol)**:** The verifier MUST report code-analysis capabilities
3. ****ACP (Agent Capability Protocol)**:** The verifier MUST report multi-agent capabilities
4. ****Embedding**:** The verifier MUST report `supports\_embeddings` for each model
5. ****RAG (Retrieval-Augmented Generation)**:** The verifier MUST report context window adaptation
6. ****Skills / Plugins**:** The verifier MUST track plugin compatibility. Model listing MUST include supported plugins

****Capability Checklist**** (MUST be verified by challenge `challenges/scripts/check\_capabilities.go`)

- [] MCP tool calling verified for at least 3 providers
- [] LSP code-analysis verified for at least 3 providers
- [] ACP parallel tool use verified for at least 2 providers
- [] Embedding generation verified for at least 2 providers
- [] RAG context-window adaptation verified
- [] Skills/plugin execution verified for at least 2 providers

****Prohibition**:** Capability flags MUST NOT be hardcoded. The `Provider.GetCapabilities` method MUST return dynamic capabilities

2.2 Amendment to CONST-035 (End-User Usability Mandate)

Insertion Point: Within the body of CONST-035, add the following paragraph after the existing anti-bluff language and before the closing statement:

****LLMsVerifier Usability Extension****: The "End-User Usability Mandate" explicitly requires that every model listing, selection, and generation feature MUST be usable with real, verified models. The following specific behaviors are considered USABILITY FAILURES under this rule:

1. The `--list-models` flag displays fewer models than the verifier has discovered for enabled providers.
2. The `--list-models` flag displays models that the verifier has marked `failed` or `unavailable` without indicating their status.
3. The `--prompt` flag uses a hardcoded or simulated response when a real provider is configured and available.
4. The TUI model selection screen shows "no models available" while the verifier database contains verified models.
5. The `SelectOptimalModel()` function selects a model that the verifier has scored below the configured `min_score` threshold.
6. API keys are present in environment variables but the corresponding provider is not listed because the discovery code is not implemented.

Any PASS that exhibits any of these behaviors is a BLUFF PASS and violates both CONST-035 and CONST-037.

2.3 Amendment to CONST-017 (Zero-Bluff Testing)

Insertion Point: Within CONST-017, append the following clause:

****LLMsVerifier Zero-Bluff Clause****: A test that verifies model or provider behavior is a BLUFF unless:

1. It calls the LLMsVerifier client with `testMode: false` (or the production verifier endpoint), OR
2. It queries the verifier SQLite database directly and asserts on real rows, OR
3. It invokes the CLI binary as a subprocess and asserts on stdout/stderr output that must originate from the verifier.

Mocking the verifier client with hardcoded

`VerificationResult{OverallScore: 8.5}` is forbidden in all test tiers above unit tests. The `verification.go` stub in the verifier submodule (which returns hardcoded 8.5 scores) MUST NOT be the data source for any HelixCode test.

2.4 New Appendix to CONSTITUTION.md — “LLMsVerifier Compliance Manifest”

Insertion Point: At the end of CONSTITUTION.md, before any existing “End of Document” marker.

Appendix D: LLMsVerifier Compliance Manifest

This appendix codifies the files, tests, and scripts that MUST exist for constitutional compliance.

D.1 Required Files

File	Purpose	Constitutional Rule
----- ----- -----		
`internal/verifier/service.go`	VerificationService wrapper	
CONST-036		
`internal/verifier/config.go`	Verifier Config structs	
CONST-039		
`internal/verifier/discovery.go`	ModelDiscoveryService	
CONST-036		
`internal/verifier/startup.go`	StartupVerifier 5-phase pipeline	
CONST-039		
`internal/verifier/scoring.go`	ScoringService adapter	
CONST-036		
`internal/verifier/health.go`	Health monitoring	
CONST-038		
`internal/verifier/events.go`	Event bus integration	
CONST-038		
`configs/verifier.yaml`	Full verifier configuration	
CONST-039		
`pkg/sdk/go/verifier/client.go`	Go SDK client for verifier	
CONST-036		
`docs/guides/llms-verifier.md`	User-facing integration guide	
CONST-035		
`docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md`	Developer integration plan	
CONST-037		
`challenges/scripts/llmsverifier_hardcode_check.sh`	Hardcoded model scan	
CONST-036		
`challenges/scripts/llmsverifier_capabilities_challenge.sh`	Capability verification	
CONST-040		
`challenges/scripts/llmsverifier_status_accuracy_challenge.sh`	Status accuracy test	
CONST-038		
`LLMsVerifier/`	(submodule) Points to `vasic-digital/LLMsVerifier`	
CONST-036		

D.2 Required Makefile Targets

```

```makefile
test-verifier-unit: ## Unit tests for verifier package
 (mocks OK)
test-verifier-integration: ## Integration tests with real
 verifier DB (NO MOCKS)
test-verifier-e2e: ## End-to-end verifier challenges
test-verifier-status: ## Status accuracy challenge
test-verifier-capabilities: ## Capability verification challenge
test-verifier-no-
 mocks: ## Fails if any non-unit test uses a mock
test-verifier-hardcode: ## Fails if hardcoded models found
 in production code

```

### D.3 Required Environment Variables (in .env.example)

```

LLMsVerifier Core
HELIX_VERIFIER_ENABLED=true
HELIX_VERIFIER_ENDPOINT=http://localhost:8081
HELIX_VERIFIER_API_KEY=
HELIX_VERIFIER_TIMEOUT=30s
HELIX_VERIFIER_DATABASE_PATH=./data/llm-verifier.db
HELIX_VERIFIER_ENCRYPTION_KEY=
HELIX_VERIFIER_JWT_SECRET=

Provider API Keys
HELIX_OPENAI_API_KEY=
HELIX_ANTHROPIC_API_KEY=
HELIX_GEMINI_API_KEY=
HELIX_DEEPSEEK_API_KEY=
HELIX_GROQ_API_KEY=
HELIX_TOGETHER_API_KEY=
HELIX_MISTRAL_API_KEY=
HELIX_XAI_API_KEY=
HELIX_CEREBRAS_API_KEY=
HELIX_CLOUDFLARE_API_KEY=
HELIX_CLOUDFLARE_ACCOUNT_ID=
HELIX_SILICONFLOW_API_KEY=
HELIX_REPLICATE_API_TOKEN=
HELIX_OPENROUTER_API_KEY=
HELIX_QWEN_API_KEY=
HELIX_COHERE_API_KEY=

Local Providers
HELIX_OLLAMA_HOST=http://localhost:11434
HELIX_LLAMA_CPP_HOST=http://localhost:8080

```

---

## ## 3. CLAUDE.md Updates

### ### 3.1 Insert New Section – "LLMsVerifier Integration Architecture"

**\*\*Insertion Point\*\***: After the existing "Architecture Overview" or "Core S

```markdown

LLMsVerifier Integration Architecture

Overview

LLMsVerifier is the ****single source of truth**** for all model and provider

****Philosophy****: No code in HelixCode should ever hardcode a model name, pr

System Diagram

```
+-----+ REST/WebSocket +-----+ | HelixCode | <-----> |
LLMsVerifier | | (Main Process) | /api/v1/verifier | (Submodule) | +-----+
+-----+ | | imports | manages v v +-----+ +-----+ |
internal/verifier| | SQLite DB | | - service.go | | - models | | - discovery.go | | -
providers | | - startup.go | | - verification | | - scoring.go | | results | | - health.go
| | - limits | +-----+ +-----+ | ^ | calls real APIs | reads v | +
+-----+ +-----+ | Provider APIs | | Provider Adapters | | (OpenAI,
etc.) | | (12+ adapters) | +-----+ +-----+
```

Key Components

VerificationService (`internal/verifier/service.go`)

This is the primary interface between HelixCode and LLMsVerifier. It wraps

```go

```
func (vs *VerificationService) GetVerifiedModels(provider string) ([]*Unif
func (vs *VerificationService) GetModelScore(modelID string) (float64, boo
func (vs *VerificationService) IsModelAvailable(modelID string) (bool, tim
func (vs *VerificationService) ValidateResponseQuality(content string, lat
func (vs *VerificationService) IsCannedErrorResponse(content string) (bool
```

**Anti-Bluff**: ValidateResponseQuality checks for canned responses and suspiciously fast replies (<100ms). Any model that passes this check is marked as verified. Models that fail are marked as failed and excluded from selection.

## ModelDiscoveryService (internal/verifier/discovery.go)

Runs the 5-phase startup pipeline: 1. **Discover**: Scan environment for API keys, OAuth tokens, local endpoints 2. **Verify**: Call each provider's API to confirm model existence and responsiveness 3. **Detect Subscriptions**: Determine 3-tier subscription level (Premium/High-quality/Fast) 4. **Score**: Run the 7-component scoring engine (code capability, responsiveness, reliability, feature richness, value proposition, cost effectiveness, recency) 5. **Rank**: Sort providers and models by score, select debate team

```
type DiscoveryConfig struct {
 Enabled bool `yaml:"enabled"`
 DiscoveryInterval time.Duration `yaml:"discovery_interval"`
 MaxModelsForEnsemble int `yaml:"max_models_for_ensemble"`
 MinScore float64 `yaml:"min_score"`
 RequireVerification bool `yaml:"require_verification"`
 RequireCodeVisibility bool `yaml:"require_code_visibility"`
 RequireDiversity bool `yaml:"require_diversity"`
 ProviderPriority []string `yaml:"provider_priority"`
}
```

## ScoringService (internal/verifier/scoring.go)

The scoring engine evaluates models across 7 components with configurable weights:

| Component         | Default Weight | Description                       |
|-------------------|----------------|-----------------------------------|
| Code Capability   | 40%            | Coding task success rate          |
| Responsiveness    | 20%            | Average latency, P95 latency      |
| Reliability       | 20%            | Uptime, error rate                |
| Feature Richness  | 15%            | Tool use, streaming, vision, etc. |
| Value Proposition | 5%             | Cost per token, open-source bonus |

**Customizing Weights:** Edit configs/verifier.yaml under verifier.scoring.weights. The weights must sum to 1.0 (validated at startup).

## StartupVerifier (internal/verifier/startup.go)

The master orchestrator. Implements `VerifyAllProviders()` with circuit breaker, faulty key deprioritization, and OAuth fallback.

### Critical Method:

```
func (sv *StartupVerifier) VerifyAllProviders(ctx
 context.Context) (*StartupResult, error)
```

This method is called during HelixCode server startup (in `cmd/server/main.go` after config load) and during CLI `--list-models` when cache is stale.

## Integration Patterns

### Pattern 1: Direct Verifier Client (For New Features)

When adding a feature that needs model data, use the Go SDK client:

```
import "github.com/HelixDevelopment/helix_code/pkg/sdk/go/
 verifier"
```

```
client := verifier.New(verifier.ClientConfig{
 BaseURL: "http://localhost:8081",
 APIKey: os.Getenv("HELIX_VERIFIER_API_KEY"),
 Timeout: 30 * time.Second,
})
```

```
models, err := client.GetModels(ctx)
scores, err := client.GetProviderScores(ctx)
```

### Pattern 2: Score Adapter (For Provider Selection)

When the existing `ModelManager.SelectOptimalModel()` needs verifier scores, use the adapter:

```
import "github.com/HelixDevelopment/helix_code/internal/services"
```

```
adapter := services.NewLLMsVerifierScoreAdapter(
 verifierScoringService,
 verificationService,
 logger,
)
```

```
score, ok := adapter.GetProviderScore("openai")
if ok && score > 7.0 {
 // Prefer this provider
}
```

### Pattern 3: Event-Driven Updates (For Real-Time UI)

Subscribe to verifier events via WebSocket:

```
wsURL := "ws://localhost:8081/ws/verifier/events"
conn, _, err := websocket.DefaultDialer.Dial(wsURL, nil)
// Read VerificationEvent structs
```

Event types: `verification.started`, `verification.provider.discovered`, `verification.provider.verified`, `verification.provider.failed`, `verification.provider.scored`, `verification.debate_team.selected`, `verification.completed`.

### How to Add a New Provider Through the Verifier

1. **Add Provider Adapter** in `LLMsVerifier` submodule:
  - Create `llm-verifier/providers/newprovider.go`
  - Embed `BaseAdapter`
  - Implement `SendRequest()`, `ParseResponse()`
  - Add fallback models to `fallback_models.go`
2. **Add to HelixCode Provider Registry:**
  - Add entry to `internal/verifier/provider_types.go` in `SupportedProviders` map
  - Set `AuthType`, `Tier`, `Priority`, `EnvVars`, `BaseURL`
  - Add env var mapping to `.env.example`
3. **Add API Key Support:**
  - Add `HELIX_NEWPROVIDER_API_KEY` to `.env.example`
  - Add to `internal/verifier/startup.go` discovery logic
  - Document in `docs/guides/llms-verifier.md`
4. **Add Tests:**
  - `internal/verifier/adapters/newprovider_test.go` (unit)
  - `tests/integration/newprovider_verification_test.go` (integration)
  - `challenges/scripts/newprovider_challenge.sh` (E2E)
5. **Add to Config Schema:**
  - Add `newprovider` section to `configs/verifier.yaml`
  - Add to `internal/verifier/config.go` `Config` struct
6. **Update ModelManager:**
  - Ensure `ModelManager` can read models from verifier for this provider
  - No hardcoded model list needed — verifier provides it

## API Key Provisioning

### Environment Variable Discovery

At startup, StartupVerifier scans environment variables using `api_keys.NewEnvVarScanner()`:

```
scanner := api_keys.NewEnvVarScanner()
keys, err := scanner.ScanEnvForUnsupportedKeys()
// Returns map[providerType]apiKeyValue
```

**Priority Order** (from `internal/verifier/startup.go`): 1. Non-faulty API keys first (faulty keys are deprioritized) 2. OAuth tokens second (if `OAuthPrimaryNonOAuthFallback: true`) 3. Free providers third 4. Local providers last (checked via localhost probes)

### OAuth Support

For providers requiring OAuth (e.g., Claude, Qwen): -  
`oauth_credentials.OAuthCredentialReader` reads tokens from secure storage - Tokens are refreshed automatically before expiry -  
`TrustOAuthOnFailure: true` falls back to API key on OAuth failure

### API Key Redaction

API keys are NEVER serialized to JSON or logged:

```
type UnifiedProvider struct {
 APIKey string `json:"-"` // Never serialized
}
```

## Debugging Verifier Integration Issues

### Diagnostic Commands

```
Check verifier health
helixcode verifier health
```

```
List discovered providers with scores
helixcode verifier providers list --format json
```

```
Check why a provider is not available
helixcode verifier providers get openai --verbose
```

```
Run verification for a single model
helixcode verifier models verify gpt-4o
```

*# Export full verifier config*

```
helixcode verifier config export yaml
```

*# Check event log*

```
helixcode verifier events list --limit 20
```

*# Check rate limits*

```
helixcode verifier limits list
```

## Common Issues

| Symptom                                  | Cause                                   | Fix                                                                     |
|------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------|
| --list-models shows empty list           | Verifier not started                    | Start verifier: <code>helixcode verifier start</code>                   |
| --list-models shows 3 hardcoded models   | BLUFF-002 not fixed                     | Replace <code>handleListModel</code> s with <code>verifier fetch</code> |
| Provider missing despite API key present | Key marked faulty                       | Run <code>api_keys.ClearFaultyKey()</code> or fix key                   |
| Score is always 8.5                      | Using <code>stub verification.go</code> | Ensure <code>real coding_capability_verification.go</code> is active    |
| Status not updating                      | WebSocket down / polling off            | Check <code>events.websocket.enabled</code> in config                   |
| OAuth provider failing                   | Token expired                           | Check OAuth credential reader logs                                      |

## Log Levels

Set `HELIX_VERIFIER_LOG_LEVEL=debug` to see: - Every provider discovery attempt - Every verification request/response - Every scoring calculation - Every circuit breaker state change



# Real-Time Updates Architecture

## WebSocket Event Stream

```
events:
 websocket:
 enabled: true
 path: "/ws/verifier/events"
```

The verifier publishes events to WebSocket clients. HelixCode subscribes and updates: - Model status in TUI - Provider health in dashboard - Score badges in CLI output

## Polling Fallback

When WebSocket is unavailable:

```
pollInterval := config.Events.PollInterval // default 30s
```

The ModelDiscoveryService polls the verifier REST API (GET /api/v1/verifier/models) on this interval.

## TUI Update Strategy

The TUI uses bubbletea with a TickMsg every pollInterval:

```
type TickMsg time.Time

func (m Model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
 switch msg.(type) {
 case TickMsg:
 return m, tea.Batch(m.refreshModels(), m.tick())
 }
}
```

## UX Guidelines for Model Display

### Score Suffix Format

Append SC:X.X to model display names:

```
✓ GPT-4o SC:9.2 [Tier 1] [Code ✓] [Vision ✓]
○ GPT-4 SC:8.8 [Tier 1] [Code ✓] [Stale 2h]
x GPT-3.5 SC:7.1 [Tier 3] [Rate Limited 15m]
```

Status Indicators

| Indicator | Meaning                                    |
|-----------|--------------------------------------------|
| ✓         | Verified, score > min_score                |
| ○         | Verified but stale (>24h since last check) |
| ✗         | Failed verification or rate limited        |
| ⚡         | Verification in progress                   |
| ■         | Code visibility confirmed                  |
|           | Premium tier (1-2)                         |
|           | Fast tier (3)                              |
|           | Free tier (4-5)                            |

Sort Order

Default sort for model listings: 1. Verified first, failed last 2. By tier (1 highest, 5 lowest) 3. By score (highest first) 4. By latency (lowest first) 5. Alphabetical

Deprecation Handling

Models with Deprecated: true in verifier data: - Show strikethrough or dimmed text - Display deprecation date if known - Do NOT select automatically (manual override required)

---

## 4. AGENTS.md Updates

### 4.1 Insert New Bluff Area – BLUFF-004 through BLUFF-008

**Insertion Point:** After BLUFF-003 in `AGENTS.md`, before the "Free AI P

```markdown  

BLUFF-004: LLMsVerifier Integration is Stubbed or Bypassed (CRITICAL)

File Pattern: `internal/verifier/\*.go` containing empty structs, `// T

Evidence Standard:
- `VerificationService` methods return hardcoded `VerificationResult{Overa
- `ModelDiscoveryService` returns an empty slice instead of calling provid
- `StartupVerifier` skips all 5 phases and returns a mock result
- The verifier submodule directory `LLMsVerifier/` exists but is empty (no

****Fix Priority**:** P0 - Immediate

****Verification Command**:**

```
```bash
```

```
make test-verifier-integration
```

```
This MUST pass with real verifier data, not mocked scores
```

---

## **BLUFF-005: Provider Discovery Uses Hardcoded Env Var Names (HIGH)**

**File Pattern:** internal/verifier/startup.go or provider adapter files containing hardcoded strings like "OPENAI\_API\_KEY" without checking SupportedProviders[provider].EnvVars.

**Evidence Standard:** - The discovery code checks os.Getenv("OPENAI\_API\_KEY") directly instead of using api\_keys.GetProviderAPIKeyName("openai") - Adding a new provider requires modifying discovery code instead of just adding to SupportedProviders - Environment variable names are duplicated in multiple files

**Fix Priority:** P1 - High

**Fix Pattern:** Use the SupportedProviders map as the single source of truth for env var names:

```
providerInfo := SupportedProviders[providerType]
for _, envVar := range providerInfo.EnvVars {
 if key := os.Getenv(envVar); key != "" {
 return key, nil
 }
}
```

---

## **BLUFF-006: Model Capabilities Are Hardcoded (HIGH)**

**File Pattern:** internal/llm/\*.go containing SupportsToolUse: true as a struct literal for specific models, or Provider.GetCapabilities() returning a static slice.

**Evidence Standard:** - GetCapabilities() returns []ModelCapability{ToolUse, CodeGeneration} without querying verifier - The capability list for a model is written in source code rather than read from VerificationResult.FeatureDetection - Adding a new capability to a model requires a code change instead of a verifier re-run

**Fix Priority:** P1 - High

**Constitutional Impact:** Violates CONST-040 (MCP/LSP/ACP/Embedding/RAG/Skills/Plugins Integration Mandate).

---

## **BLUFF-007: Test Claims Integration But Uses Mocked Verifier (CRITICAL)**

**File Pattern:** \*\_test.go files with testify/mock or testMode: true in non-unit test files.

**Evidence Standard:** - TestModelDiscovery creates a mock verifier client and asserts it returns 5 models - TestProviderSelection stubs GetProviderScore() to return 9.0 - The test does not start the actual verifier server, does not create the SQLite database, and does not make real HTTP calls - The test passes in CI but the feature fails when used by a human

**Fix Priority:** P0 - Immediate

**Constitutional Impact:** Violates CONST-037 (Model Provider Anti-Bluff Guarantee) and CONST-017 (Zero-Bluff Testing).

**Required Test Structure:**

```
func TestModelDiscovery_Integration(t *testing.T) {
 if os.Getenv("HELIX_SKIP_LIVE_PROVIDER_TESTS") != "" {
 t.Skip("SKIP-OK: Live provider tests disabled")
 }

 // Start real verifier server
 srv := startVerifierServer(t)
 defer srv.Stop()

 // Configure a real provider with real API key
 cfg := loadVerifierConfig()

 // Run discovery
 discovery := verifier.NewModelDiscoveryService(cfg)
 models, err := discovery.DiscoverAll(ctx)

 // Assert on REAL data
 require.NoError(t, err)
 require.Greater(t, len(models), 0, "Discovery must find at
 least one model")

 // Verify each model has a real provider
 for _, m := range models {
```

```

 require.NotEmpty(t, m.Provider, "Model %s must have a
 provider", m.ID)
 require.True(t, m.Verified, "Model %s must be verified",
 m.ID)
 }
}

```

---

## BLUFF-008: Scoring Weights Do Not Sum to 1.0 (MEDIUM)

**File Pattern:** configs/verifier.yaml or internal/verifier/config.go where scoring weights are misconfigured.

**Evidence Standard:** - weights: {response\_speed: 0.3, model\_efficiency: 0.3, cost\_effectiveness: 0.3, capability: 0.3, recency: 0.3} sums to 1.5 - No validation at startup to check weight sum - Scores exceed 10.0 or are negative due to misweighted calculation

**Fix Priority:** P2 - Medium

**Fix:** Add validateWeights() at config load time:

```

func (c *ScoringConfig) validateWeights() error {
 sum := c.Weights.ResponseSpeed + c.Weights.ModelEfficiency +
 c.Weights.CostEffectiveness + c.Weights.Capability +
 c.Weights.Recency
 if math.Abs(sum-1.0) > 0.001 {
 return fmt.Errorf("scoring weights must sum to 1.0, got
 %.3f", sum)
 }
 return nil
}

```

...
### 4.2 Insert Updated Technology Stack Reference
<b>Insertion Point:</b> In the “Technology Stack” section of AGENTS.md, add the verifier subsystem:
markdown ##### LLMsVerifier Subsystem Stack - LLMsVerifier submodule (Go 1.25.3) – git submodule at `LLMsVerifier/` - SQLite 3 with WAL mode – verifier database - SQL Cipher

(optional) – database encryption - Bubbletea v1.1.0 – TUI for verifier - go-playground/validator/v10 – input validation - Gorilla WebSocket v1.5.3 – real-time events - Prometheus client – metrics export

### ### 4.3 Insert New Module Boundaries

**Insertion Point:** In the “Module Boundaries” or “Architecture” section of AGENTS.md:

```markdown ##### Verifier Module Boundaries

Package: internal/verifier/ — The ONLY package that directly imports the LLMsVerifier submodule.

Allowed Dependencies: - internal/verifier/ MAY import: digital.vasic.llmsverifier/*, github.com/gorilla/websocket, github.com/sirupsen/logrus - internal/services/ MAY import: internal/verifier/* (through adapter only) - internal/llm/ MAY import: internal/services/llmsverifier_score_adapter.go (only) - cmd/cli/ MAY import: internal/verifier/* (for verifier subcommand) - cmd/server/ MAY import: internal/verifier/* (for startup verification)

Forbidden Dependencies: - internal/verifier/ MUST NOT import: internal/llm/ (circular — verifier is below LLM layer) - internal/config/ MUST NOT import: internal/verifier/ (config is above all) - internal/

| |
|--|
| <pre>llm/ MUST NOT import: digital.vasic.llmsverifier/* directly (must go through adapter) - internal/server/ MUST NOT import: digital.vasic.llmsverifier/* directly</pre> |
| <p>Submodule Rule: The LLMsVerifier/ directory is a git submodule. It MUST NOT be edited directly from HelixCode. Changes to the verifier go through the upstream vasic-digital/LLMsVerifier repository and are pulled via git submodule update. ``</p> |
| <p>### 4.4 Insert Challenge Verification Checklist</p> |
| <p>Insertion Point: After the existing test category list or at the end of AGENTS.md:</p> |
| <pre>``markdown</pre> |

LLMsVerifier Challenge Verification Checklist

Every agent working on verifier-related code MUST confirm the following before marking work complete:

Pre-Implementation

- ☐ Read docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md phases 1-10
- ☐ Confirm LLMsVerifier/ submodule is initialized (git submodule status)
- ☐ Confirm configs/verifier.yaml schema matches the code being written
- ☐ Check .env.example has all required HELIX_VERIFIER_* and provider API key variables

During Implementation

- ☐ No hardcoded model lists in any modified file (run `make test-verifier-hardcode`)
- ☐ No hardcoded provider lists (use `SupportedProviders` map)
- ☐ No hardcoded capability flags (read from verifier database)
- ☐ All provider API keys use env var names from `SupportedProviders[provider].EnvVars`
- ☐ Scoring weights validated to sum to 1.0
- ☐ OAuth providers have fallback to API key if configured
- ☐ Circuit breaker configured for all external providers

Post-Implementation Testing

- ☐ `make test-verifier-unit` passes (mocks OK for unit tests)
- ☐ `make test-verifier-integration` passes (NO MOCKS — real verifier DB)
- ☐ `make test-verifier-hardcode` passes (zero hardcoded models in production)
- ☐ `make test-verifier-no-mocks` passes (no non-unit test uses mock)
- ☐ `challenges/scripts/llmsverifier_hardcode_check.sh` passes
- ☐ `challenges/scripts/llmsverifier_capabilities_challenge.sh` passes
- ☐ `challenges/scripts/llmsverifier_status_accuracy_challenge.sh` passes
- ☐ `challenges/scripts/llmsverifier_startup_verification_challenge.sh` passes
- ☐ CLI `--list-models` output matches verifier database content

SelectOptimalModel() never selects a model with verifier score below min_score

Anti-Bluff Confirmation

☐

Run helixcode --list-models manually and compare to llm-verifier models list

☐

Run helixcode --prompt "test" --model <verified-model> and confirm real API call (check network traffic)

☐

Temporarily disable a provider in config and confirm it disappears from --list-models within 60s

☐

Verify that rate-limited models show rate-limited status in CLI output

☐

Check that models with Deprecated: true are NOT auto-selected

LLMsVerifier Quick Reference for Agents

Essential Commands

```
```bash
```

```
Initialize submodule
```

```
git submodule update --init --recursive
```

```
Start verifier server
```

```
cd LLMsVerifier/llm-verifier && go run cmd/main.go server
```

```
Run verifier CLI
```

```
./llm-verifier models list
```

```
./llm-verifier providers list
```

```
./llm-verifier models verify MODEL_ID
```

```
Run HelixCode with verifier
```

```
HELIX_VERIFIER_ENABLED=true ./helixcode --list-models
```

```
Run challenges
```

```
./challenges/scripts/llmsverifier_hardcode_check.sh
```

```
./challenges/scripts/llmsverifier_startup_verification_challenge.sh
```

## Essential Files

File	When to Read
configs/verifier.yaml	Before modifying any verifier config
internal/verifier/config.go	Before adding new config fields
internal/verifier/provider_types.go	Before adding new providers
internal/verifier/startup.go	Before modifying discovery/verification flow
internal/services/llmsverifier_score_adapter.go	Before modifying scoring bridge
docs/guides/llms-verifier.md	Before writing user-facing documentation
docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md	Before starting any verifier work

— — —

## ## 5. Configuration Documentation

### ### 5.1 Full YAML Schema for `configs/verifier.yamll`

[illegible]

```

Default: ./data/llm-verifier.d
Env: HELIX_VERIFIER_DATABASE_P

encryption_enabled: false # bool – Enable SQL Cipher en
WARNING: Once enabled
data loss. Backup bef
Default: false
Env: HELIX_VERIFIER_ENCRYPTION

encryption_key: "${VERIFIER_ENCRYPTION_KEY}" # string – SQL Cipher en
Minimum 8 characters
Use env var substitut
Default: ""
Env: VERIFIER_ENCRYPTION_KEY

max_connections: 25 # int – Max open DB connection
Default: 25

max_idle_connections: 5 # int – Max idle connections
Default: 5

connection_lifetime: 5m # duration – Connection max lifet
Default: 5m

wal_mode: true # bool – Enable SQLite WAL mode
Required for concurre
Default: true

SECTION 3: Verification Configuration

verification:
 mandatory_code_check: true # bool – Require code visibil
If true, models that
are marked failed
Default: true

 code_visibility_prompt: "Do you see my code?" # string – Prompt for c
Default: "Do you see my code?"

 verification_timeout: 60s # duration – Max time per verif
Default: 60s

 retry_count: 3 # int – Number of retries on t
Default: 3

 retry_delay: 5s # duration – Delay between retrie
Default: 5s

 max_concurrent: 5 # int – Max parallel verificatio
Default: 5

```

```

tests: # []string – Enabled verification
 - existence # Model exists at provider API
 - responsiveness # Model responds to prompt
 - latency # Response latency measured
 - streaming # Streaming support verified
 - function_calling # Tool/function calling verified
 - coding_capability # Coding task success verified
 - error_detection # Error handling verified
 - code_visibility # Code visibility confirmed

stale_threshold: 24h # duration – Max age before re-verify
 # Default: 24h

SECTION 4: Scoring Configuration

scoring:
 weights: # map[string]float64 – MUST sum to 1.0
 response_speed: 0.25 # Weight for latency/responsiveness
 model_efficiency: 0.20 # Weight for throughput/tokens-per-second
 cost_effectiveness: 0.25 # Weight for price performance
 capability: 0.20 # Weight for feature richness
 recency: 0.10 # Weight for model release date
 # (newer models score higher)

models_dev_enabled: true # bool – Enable models.dev pricing
 # Default: true

models_dev_endpoint: "https://api.models.dev" # string – Pricing data endpoint
 # Default: https://api.models.dev

cache_ttl: 24h # duration – Score cache TTL
 # Default: 24h

min_score: 5.0 # float64 – Minimum score for auto-acceptance
 # Models below this are not auto-accepted
 # but still displayed with warning
 # Default: 5.0
 # Range: 0.0 - 10.0

SECTION 5: Health Monitoring

health:
 check_interval: 30s # duration – Health check interval
 # Default: 30s

 timeout: 10s # duration – Health check timeout
 # Default: 10s

 failure_threshold: 5 # int – Consecutive failures before declaring unhealthy
 # Default: 5

```

```

recovery_threshold: 3 # int - Consecutive successes before
 # Default: 3

circuit_breaker:
 enabled: true # bool - Enable circuit breaker
 # Default: true

 half_open_timeout: 60s # duration - Time in half-open before
 # Default: 60s

SECTION 6: API Server Configuration

api:
 enabled: true # bool - Enable verifier REST API
 # Default: true

 port: "8081" # string - API server port
 # Default: 8081
 # Env: HELIX_VERIFIER_API_PORT

 base_path: "/api/v1/verifier" # string - API base path
 # Default: /api/v1/verifier

 host: "0.0.0.0" # string - Bind host
 # Default: 0.0.0.0

 jwt_secret: "${VERIFIER_JWT_SECRET}" # string - JWT signing secret
 # Default: ""
 # Env: VERIFIER_JWT_SECRET

 tls: # TLS configuration
 enabled: false # bool - Enable TLS
 cert_file: "" # string - Certificate path
 key_file: "" # string - Key path

 rate_limit: # Rate limiting
 enabled: true # bool
 requests_per_minute: 100 # int
 burst_size: 20 # int

 cors: # CORS configuration
 enabled: true # bool
 allowed_origins: ["*"] # []string

SECTION 7: Event System

events:
 websocket:
 enabled: true # bool - Enable WebSocket event stream

```

```

Default: true

path: "/ws/verifier/events" # string – WebSocket endpoint path
Default: /ws/verifier/events

ping_interval: 30s # duration – Keep-alive ping interval
Default: 30s

slack:
 enabled: false # bool
 webhook_url: "${SLACK_WEBHOOK_URL}" # string
Env: SLACK_WEBHOOK_URL

email:
 enabled: false # bool
 smtp_host: "smtp.gmail.com" # string
 smtp_port: 587 # int
 smtp_user: "" # string
 smtp_password: "" # string
 from_address: "" # string

telegram:
 enabled: false # bool
 bot_token: "${TELEGRAM_BOT_TOKEN}" # string
Env: TELEGRAM_BOT_TOKEN

 chat_id: "${TELEGRAM_CHAT_ID}" # string
Env: TELEGRAM_CHAT_ID

SECTION 8: Monitoring

monitoring:
 prometheus:
 enabled: true # bool
 path: "/metrics/verifier" # string
 port: "9091" # string

 grafana:
 enabled: true # bool
 dashboard_path: "./dashboards/verifier" # string

 jaeger: # Distributed tracing
 enabled: false # bool
 endpoint: "" # string

SECTION 9: Brotli / HTTP3

brotli:
 enabled: true # bool – Enable Brotli compression
Default: true

```

```

compression_level: 6 # int - Brotli compression level
 # Default: 6

http3:
 enabled: false # bool - Enable HTTP/3 (QUIC)
 # Default: false
 # Requires quic-go dependency

SECTION 10: Challenges

challenges:
 enabled: true # bool - Enable challenge system
 # Default: true

 provider_discovery: true # bool - Run provider discovery ch
 model_verification: true # bool - Run model verification c
 config_generation: true # bool - Run config generation ch

SECTION 11: Scheduling

scheduling:
 re_verification:
 enabled: true # bool - Auto re-verify models peri
 # Default: true

 interval: 24h # duration - Re-verification interval
 # Default: 24h

 jitter: 1h # duration - Random jitter to avoid t
 # Default: 1h

 score_recalculation:
 enabled: true # bool - Auto recalculate scores
 # Default: true

 interval: 12h # duration - Score recalculation inte
 # Default: 12h

 stale_model_cleanup:
 enabled: true # bool - Remove models not seen in
 # Default: true

 max_age: 30d # duration - Max age before removal
 # Default: 30d

SECTION 12: Provider Configuration

Each provider has: enabled, api_key, base_url, models, timeout, retry

```

```
Models list is OPTIONAL – if omitted, all models from provider are discovered
providers:
 openai:
 enabled: true
 api_key: "${OPENAI_API_KEY}"
 base_url: "https://api.openai.com/v1"
 models: [] # empty = auto-discover all
 timeout: 30s
 retry: 3
 # Env: OPENAI_API_KEY, HELIX_OPENAI_API_KEY

 anthropic:
 enabled: true
 api_key: "${ANTHROPIC_API_KEY}"
 base_url: "https://api.anthropic.com/v1"
 models: []
 timeout: 30s
 retry: 3
 oauth_fallback: true # Fall back to OAuth if API key fails
 # Env: ANTHROPIC_API_KEY, CLAUDE_API_KEY, HELIX_ANTHROPIC_API_KEY

 gemini:
 enabled: true
 api_key: "${GEMINI_API_KEY}"
 base_url: "https://generativelanguage.googleapis.com/v1beta"
 models: []
 timeout: 30s
 retry: 3
 # Env: GEMINI_API_KEY, GOOGLE_API_KEY, ApiKey_Gemini, HELIX_GEMINI_API_KEY

 deepseek:
 enabled: true
 api_key: "${DEEPSEEK_API_KEY}"
 base_url: "https://api.deepseek.com/v1"
 models: []
 timeout: 30s
 retry: 3
 # Env: DEEPSEEK_API_KEY, HELIX_DEEPSEEK_API_KEY

 groq:
 enabled: true
 api_key: "${GROQ_API_KEY}"
 base_url: "https://api.groq.com/openai/v1"
 models: []
 timeout: 30s
 retry: 3
 # Env: GROQ_API_KEY, HELIX_GROQ_API_KEY

 together:
 enabled: true
 api_key: "${TOGETHER_API_KEY}"
 base_url: "https://api.together.xyz/v1"
```



```
models: []
timeout: 30s
retry: 3
Env: TOGETHER_API_KEY, HELIX_TOGETHER_API_KEY

mistral:
 enabled: true
 api_key: "${MISTRAL_API_KEY}"
 base_url: "https://api.mistral.ai/v1"
 models: []
 timeout: 30s
 retry: 3
 # Env: MISTRAL_API_KEY, HELIX_MISTRAL_API_KEY

xai:
 enabled: true
 api_key: "${XAI_API_KEY}"
 base_url: "https://api.x.ai/v1"
 models: []
 timeout: 30s
 retry: 3
 # Env: XAI_API_KEY, HELIX_XAI_API_KEY

cerebras:
 enabled: false # Disabled by default (requires enterprise key)
 api_key: "${CEREBRAS_API_KEY}"
 base_url: "https://api.cerebras.ai/v1"
 models: []
 timeout: 30s
 retry: 3
 # Env: CEREBRAS_API_KEY, HELIX_CEREBRAS_API_KEY

cloudflare:
 enabled: true
 api_key: "${CLOUDFLARE_API_KEY}"
 account_id: "${CLOUDFLARE_ACCOUNT_ID}"
 base_url: "https://api.cloudflare.com/client/v4"
 models: []
 timeout: 30s
 retry: 3
 # Env: CLOUDFLARE_API_KEY, CLOUDFLARE_ACCOUNT_ID, HELIX_CLOUDFLARE_A

siliconflow:
 enabled: true
 api_key: "${SILICONFLOW_API_KEY}"
 base_url: "https://api.siliconflow.cn/v1"
 models: []
 timeout: 30s
 retry: 3
 # Env: SILICONFLOW_API_KEY, HELIX_SILICONFLOW_API_KEY

replicate:
```

```
enabled: true
api_token: "${REPLICATE_API_TOKEN}"
base_url: "https://api.replicate.com/v1"
models: []
timeout: 60s
retry: 3
Env: REPLICATE_API_TOKEN, HELIX_REPLICATE_API_TOKEN
```

```
openrouter:
 enabled: true
 api_key: "${OPENROUTER_API_KEY}"
 base_url: "https://openrouter.ai/api/v1"
 models: []
 timeout: 30s
 retry: 3
 free_models_only: false # Set true to use only free tier
 # Env: OPENROUTER_API_KEY, HELIX_OPENROUTER_API_KEY
```

```
qwen:
 enabled: true
 api_key: "${QWEN_API_KEY}"
 base_url: "https://dashscope.aliyuncs.com/api/v1"
 models: []
 timeout: 30s
 retry: 3
 oauth_primary: true # Prefer OAuth over API key
 # Env: QWEN_API_KEY, HELIX_QWEN_API_KEY
```

```
cohere:
 enabled: true
 api_key: "${COHERE_API_KEY}"
 base_url: "https://api.cohere.com/v1"
 models: []
 timeout: 30s
 retry: 3
 # Env: COHERE_API_KEY, HELIX_COHERE_API_KEY
```

```
ollama:
 enabled: true
 host: "http://localhost:11434" # Ollama server address
 models: [] # Auto-discover from /api/tags
 timeout: 120s
 # No API key needed for local
```

```
llamacpp:
 enabled: true
 host: "http://localhost:8080" # llama.cpp server address
 models: []
 timeout: 120s
 # No API key needed for local
```

```
vllm:
```

```
enabled: false # Disabled by default
host: "http://localhost:8000"
models: []
timeout: 120s
```

```
localai:
 enabled: false
 host: "http://localhost:8080"
 models: []
 timeout: 120s
```

## 5.2 Environment Variable Mapping Table

Config Path	YAML Key	Env Var (Primary)
verifier.enabled	enabled	—
verifier.database.path	path	—
verifier.database.encryption_key	encryption_key	VERIFIER_ENCRYPTION_KEY
verifier.api.port	port	VERIFIER_API_PORT
verifier.api.jwt_secret	jwt_secret	VERIFIER_JWT_SECRET
verifier.events.slack.webhook_url	webhook_url	SLACK_WEBHOOK_URL
verifier.events.telegram.bot_token	bot_token	TELEGRAM_BOT_TOKEN
verifier.events.telegram.chat_id	chat_id	TELEGRAM_CHAT_ID
providers.openai.api_key	api_key	OPENAI_API_KEY
providers.anthropic.api_key	api_key	ANTHROPIC_API_KEY
providers.gemini.api_key	api_key	GEMINI_API_KEY
providers.deepseek.api_key	api_key	DEEPSEEK_API_KEY
providers.groq.api_key	api_key	GROQ_API_KEY
providers.together.api_key	api_key	TOGETHER_API_KEY
providers.mistral.api_key	api_key	MISTRAL_API_KEY
providers.xai.api_key	api_key	XAI_API_KEY
providers.cerebras.api_key	api_key	CEREBRAS_API_KEY
providers.cloudflare.api_key	api_key	CLOUDFLARE_API_KEY
providers.cloudflare.account_id	account_id	CLOUDFLARE_ACCOUNT_ID
providers.siliconflow.api_key	api_key	SILICONFLOW_API_KEY
providers.replicate.api_token	api_token	REPLICATE_API_TOKEN

Config Path	YAML Key	Env Var (Primary)
providers.openrouter.api_key	api_key	OPENROUTER_API_KEY
providers.qwen.api_key	api_key	QWEN_API_KEY
providers.cohere.api_key	api_key	COHERE_API_KEY
providers.ollama.host	host	OLLAMA_HOST
providers.llamacpp.host	host	LLAMA_CPP_HOST

### 5.3 HelixCode Config Integration (internal/config/config.go additions)

The existing Config struct in internal/config/config.go must be extended:

```

type Config struct {
 Version string `mapstructure:"version"`
 UpdatedBy string `mapstructure:"updated_by"`
 Application ApplicationConfig `mapstructure:"application"`
 Server ServerConfig `mapstructure:"server"`
 Database database.Config `mapstructure:"database"`
 Redis RedisConfig `mapstructure:"redis"`
 Auth AuthConfig `mapstructure:"auth"`
 Workers WorkersConfig `mapstructure:"workers"`
 Tasks TasksConfig `mapstructure:"tasks"`
 LLM LLMConfig `mapstructure:"llm"`
 Providers ProvidersConfig `mapstructure:"providers"`
 Logging LoggingConfig `mapstructure:"logging"`
 Cognee *CogneeConfig `mapstructure:"cognee"`
 Verifier *VerifierConfig `mapstructure:"verifier" //
 NEW
}

// VerifierConfig is embedded from the verifier package config
// but mapped with HELIX_ prefix for env var binding.
type VerifierConfig struct {
 Enabled bool `mapstructure:"enabled"`
 Endpoint string `mapstructure:"endpoint" // Verifier
 API endpoint
 APIKey string `mapstructure:"api_key" // For
 authenticating TO the verifier
 Timeout string `mapstructure:"timeout"`
 DatabasePath string `mapstructure:"database_path"`
}

```

**Env var binding additions** in config.go Load():

```
// Add to existing explicit binds:
viper.BindEnv("verifier.enabled", "HELIX_VERIFIER_ENABLED")
viper.BindEnv("verifier.endpoint", "HELIX_VERIFIER_ENDPOINT")
viper.BindEnv("verifier.api_key", "HELIX_VERIFIER_API_KEY")
viper.BindEnv("verifier.timeout", "HELIX_VERIFIER_TIMEOUT")
viper.BindEnv("verifier.database_path",
 "HELIX_VERIFIER_DATABASE_PATH")

// Provider API keys (already partially present, ensure complete
// set):
viper.BindEnv("providers.openai.api_key", "HELIX_OPENAI_API_KEY")
viper.BindEnv("providers.anthropic.api_key",
 "HELIX_ANTHROPIC_API_KEY")
viper.BindEnv("providers.gemini.api_key", "HELIX_GEMINI_API_KEY")
viper.BindEnv("providers.deepseek.api_key",
 "HELIX_DEEPSEEK_API_KEY")
viper.BindEnv("providers.groq.api_key", "HELIX_GROQ_API_KEY")
viper.BindEnv("providers.together.api_key",
 "HELIX_TOGETHER_API_KEY")
viper.BindEnv("providers.mistral.api_key",
 "HELIX_MISTRAL_API_KEY")
viper.BindEnv("providers.xai.api_key", "HELIX_XAI_API_KEY")
viper.BindEnv("providers.cerebras.api_key",
 "HELIX_CEREBRAS_API_KEY")
viper.BindEnv("providers.cloudflare.api_key",
 "HELIX_CLOUDFLARE_API_KEY")
viper.BindEnv("providers.cloudflare.account_id",
 "HELIX_CLOUDFLARE_ACCOUNT_ID")
viper.BindEnv("providers.siliconflow.api_key",
 "HELIX_SILICONFLOW_API_KEY")
viper.BindEnv("providers.replicate.api_token",
 "HELIX_REPLICATE_API_TOKEN")
viper.BindEnv("providers.openrouter.api_key",
 "HELIX_OPENROUTER_API_KEY")
viper.BindEnv("providers.qwen.api_key", "HELIX_QWEN_API_KEY")
viper.BindEnv("providers.cohere.api_key", "HELIX_COHERE_API_KEY")
viper.BindEnv("providers.ollama.host", "HELIX_OLLAMA_HOST")
viper.BindEnv("providers.llamacpp.host", "HELIX_LLAMA_CPP_HOST")
```

## 5.4 Example Configuration Files

**Basic Config (config.basic.yaml)**

```
version: "1.0.0"
```

```
application:
```

```
 name: "helixcode"
 environment: "production"

server:
 port: 8080
 host: "0.0.0.0"

database:
 host: "localhost"
 port: 5432
 name: "helixcode"
 user: "helix"
 password: "${HELIX_DATABASE_PASSWORD}"

llm:
 default_provider: "openai"
 default_model: "gpt-4o"
 max_tokens: 4096
 temperature: 0.7

Verifier – minimal configuration
verifier:
 enabled: true
 endpoint: "http://localhost:8081"
 timeout: "30s"

providers:
 openai:
 enabled: true
 api_key: "${HELIX_OPENAI_API_KEY}"
 anthropic:
 enabled: true
 api_key: "${HELIX_ANTHROPIC_API_KEY}"
 ollama:
 enabled: true
 host: "http://localhost:11434"
```

### **Development Config (config.development.yaml)**

```
version: "1.0.0-dev"

application:
 name: "helixcode"
 environment: "development"

server:
 port: 8080
 host: "127.0.0.1"
```

```
database:
 host: "localhost"
 port: 5432
 name: "helixcode_dev"
 user: "helix"
 password: "dev_password"

llm:
 default_provider: "ollama"
 default_model: "llama3.2"
 max_tokens: 2048
 temperature: 0.8

Verifier – development settings
verifier:
 enabled: true
 endpoint: "http://localhost:8081"
 timeout: "60s"
 database_path: "./data/llm-verifier-dev.db"

Full verifier config for development
verifier_full:
 database:
 path: "./data/llm-verifier-dev.db"
 encryption_enabled: false
 wal_mode: true

 verification:
 mandatory_code_check: false # Faster in dev
 verification_timeout: 30s
 retry_count: 1
 tests:
 - existence
 - responsiveness
 - latency

 scoring:
 weights:
 response_speed: 0.40
 model_efficiency: 0.20
 cost_effectiveness: 0.20
 capability: 0.10
 recency: 0.10
 cache_ttl: 1h
 min_score: 3.0 # Lower threshold in dev

health:
```

```
check_interval: 10s
failure_threshold: 10 # More lenient in dev

api:
 enabled: true
 port: "8081"
 rate_limit:
 enabled: false # No rate limiting in dev

events:
 websocket:
 enabled: true
 slack:
 enabled: false
 email:
 enabled: false
 telegram:
 enabled: false

challenges:
 enabled: true

scheduling:
 re_verification:
 enabled: true
 interval: 1h # Re-verify frequently in dev
 score_recalculation:
 enabled: true
 interval: 30m

providers:
 openai:
 enabled: true
 api_key: "${HELIX_OPENAI_API_KEY}"
 timeout: 30s
 retry: 1
 anthropic:
 enabled: true
 api_key: "${HELIX_ANTHROPIC_API_KEY}"
 timeout: 30s
 deepseek:
 enabled: true
 api_key: "${HELIX_DEEPSEEK_API_KEY}"
 groq:
 enabled: true
 api_key: "${HELIX_GROQ_API_KEY}"
 ollama:
 enabled: true
```



```
 host: "http://localhost:11434"
llamacpp:
 enabled: true
 host: "http://localhost:8080"
openrouter:
 enabled: true
 api_key: "${HELIX_OPENROUTER_API_KEY}"
 free_models_only: true # Use free tier in dev
```

### **Production Config (config.production.yaml)**

```
version: "1.0.0"

application:
 name: "helixcode"
 environment: "production"

server:
 port: 8080
 host: "0.0.0.0"

database:
 host: "${HELIX_DATABASE_HOST}"
 port: 5432
 name: "helixcode"
 user: "helix"
 password: "${HELIX_DATABASE_PASSWORD}"
 ssl_mode: "require"
 max_connections: 50

redis:
 host: "${HELIX_REDIS_HOST}"
 port: "${HELIX_REDIS_PORT}"
 password: "${HELIX_REDIS_PASSWORD}"

llm:
 default_provider: "openai"
 default_model: "gpt-4o"
 max_tokens: 4096
 temperature: 0.7

Verifier – production settings
verifier:
 enabled: true
 endpoint: "http://verifier.internal:8081" # Internal service
 discovery
 timeout: "30s"
 database_path: "/data/llm-verifier.db"
```

```
verifier_full:
 database:
 path: "/data/llm-verifier.db"
 encryption_enabled: true
 encryption_key: "${VERIFIER_ENCRYPTION_KEY}"
 wal_mode: true
 max_connections: 25

 verification:
 mandatory_code_check: true
 verification_timeout: 60s
 retry_count: 3
 retry_delay: 5s
 max_concurrent: 10
 tests:
 - existence
 - responsiveness
 - latency
 - streaming
 - function_calling
 - coding_capability
 - error_detection
 - code_visibility
 stale_threshold: 24h

 scoring:
 weights:
 response_speed: 0.25
 model_efficiency: 0.20
 cost_effectiveness: 0.25
 capability: 0.20
 recency: 0.10
 cache_ttl: 24h
 min_score: 6.0

 health:
 check_interval: 30s
 timeout: 10s
 failure_threshold: 5
 recovery_threshold: 3
 circuit_breaker:
 enabled: true
 half_open_timeout: 60s

 api:
 enabled: true
 port: "8081"
```

```
base_path: "/api/v1/verifier"
jwt_secret: "${VERIFIER_JWT_SECRET}"
tls:
 enabled: true
 cert_file: "/etc/helixcode/certs/verifier.crt"
 key_file: "/etc/helixcode/certs/verifier.key"
rate_limit:
 enabled: true
 requests_per_minute: 1000
 burst_size: 100

events:
 websocket:
 enabled: true
 ping_interval: 30s
 slack:
 enabled: true
 webhook_url: "${SLACK_WEBHOOK_URL}"
 email:
 enabled: true
 smtp_host: "${SMTP_HOST}"
 smtp_port: 587
 smtp_user: "${SMTP_USER}"
 smtp_password: "${SMTP_PASSWORD}"
 from_address: "verifier@helixcode.dev"
 telegram:
 enabled: false

monitoring:
 prometheus:
 enabled: true
 path: "/metrics/verifier"
 port: "9091"
 grafana:
 enabled: true
 dashboard_path: "/etc/helixcode/dashboards/verifier"

brotli:
 enabled: true
 compression_level: 6

challenges:
 enabled: true

scheduling:
 re_verification:
 enabled: true
 interval: 24h
```

```
 jitter: 1h
score_recalculation:
 enabled: true
 interval: 12h
stale_model_cleanup:
 enabled: true
 max_age: 30d

providers:
 openai:
 enabled: true
 api_key: "${HELIX_OPENAI_API_KEY}"
 base_url: "https://api.openai.com/v1"
 timeout: 30s
 retry: 3
 anthropic:
 enabled: true
 api_key: "${HELIX_ANTHROPIC_API_KEY}"
 base_url: "https://api.anthropic.com/v1"
 timeout: 30s
 retry: 3
 oauth_fallback: true
 gemini:
 enabled: true
 api_key: "${HELIX_GEMINI_API_KEY}"
 base_url: "https://generativelanguage.googleapis.com/v1beta"
 timeout: 30s
 retry: 3
 deepseek:
 enabled: true
 api_key: "${HELIX_DEEPSEEK_API_KEY}"
 base_url: "https://api.deepseek.com/v1"
 timeout: 30s
 retry: 3
 groq:
 enabled: true
 api_key: "${HELIX_GROQ_API_KEY}"
 base_url: "https://api.groq.com/openai/v1"
 timeout: 30s
 retry: 3
 mistral:
 enabled: true
 api_key: "${HELIX_MISTRAL_API_KEY}"
 base_url: "https://api.mistral.ai/v1"
 timeout: 30s
 retry: 3
 xai:
 enabled: true
```

```
api_key: "${HELIX_XAI_API_KEY}"
base_url: "https://api.x.ai/v1"
timeout: 30s
retry: 3
openrouter:
 enabled: true
 api_key: "${HELIX_OPENROUTER_API_KEY}"
 base_url: "https://openrouter.ai/api/v1"
 timeout: 30s
 retry: 3
ollama:
 enabled: false # Disabled in production (use cloud
 providers)
 host: "http://localhost:11434"
```

### Testing Config (config.testing.yaml)

```
version: "1.0.0-test"
```

```
application:
 name: "helixcode"
 environment: "testing"
```

```
server:
 port: 18080
 host: "127.0.0.1"
```

```
database:
 host: "localhost"
 port: 5432
 name: "helixcode_test"
 user: "helix"
 password: "test_password"
```

```
llm:
 default_provider: "openrouter"
 default_model: "openai/gpt-3.5-turbo"
 max_tokens: 1024
 temperature: 0.7
```

```
Verifier – testing settings (uses mock verifier or test fixture
DB)
```

```
verifier:
 enabled: true
 endpoint: "http://localhost:18081" # Test verifier instance
 timeout: "10s"
 database_path: ":memory:" # In-memory SQLite for tests
```

```
verifier_full:
 database:
 path: ":memory:"
 encryption_enabled: false

 verification:
 mandatory_code_check: false
 verification_timeout: 10s
 retry_count: 1
 tests:
 - existence
 - responsiveness

 scoring:
 weights:
 response_speed: 0.25
 model_efficiency: 0.20
 cost_effectiveness: 0.25
 capability: 0.20
 recency: 0.10
 cache_ttl: 5m
 min_score: 1.0 # Very low threshold for testing

 health:
 check_interval: 5s
 failure_threshold: 100 # Never trip in tests

 api:
 enabled: true
 port: "18081"
 rate_limit:
 enabled: false

 events:
 websocket:
 enabled: false # No WebSocket in tests

 challenges:
 enabled: false # Challenges run separately

 scheduling:
 re_verification:
 enabled: false # Manual re-verify only in tests

providers:
 openrouter:
 enabled: true
 api_key: "${HELIX_OPENROUTER_API_KEY}"
```

```
 free_models_only: true # Use free tier for tests
 timeout: 10s
 retry: 1
ollama:
 enabled: true
 host: "http://localhost:11434"
```

---

## 6. User Guide

### 6.1 How to Enable/Disable LLMsVerifier

**Enable** (default):

```
In config.yaml
verifier:
 enabled: true
```

Or via environment variable:

```
export HELIX_VERIFIER_ENABLED=true
```

**Disable:**

```
In config.yaml
verifier:
 enabled: false
```

When disabled: - ModelManager falls back to legacy behavior (provider self-discovery) - `--list-models` shows only what providers report directly - No scoring, no verification status, no circuit breaker - Hardcoded models (if any) become visible again — this is a degradation

**Verify Status:**

```
helixcode verifier status
Output: ENABLED | Endpoint: http://localhost:8081 |
 Database: ./data/llm-verifier.db
```

### 6.2 How to Configure API Keys for All Providers

**Method 1: Environment Variables** (Recommended for security)

```
Required for cloud providers
export HELIX_OPENAI_API_KEY="sk-..."
export HELIX_ANTHROPIC_API_KEY="sk-ant-..."
export HELIX_GEMINI_API_KEY="AIza..."
```

```
Optional – only if you use these providers
export HELIX_DEEPSEEK_API_KEY="..."
export HELIX_GROQ_API_KEY="..."
export HELIX_MISTRAL_API_KEY="..."
export HELIX_XAI_API_KEY="..."
export HELIX_OPENROUTER_API_KEY="..."

Local providers need no keys
export HELIX_OLLAMA_HOST="http://localhost:11434" # Only if
non-default
```

**Method 2: Config File** (Convenient for development only — NEVER commit to git)

```
providers:
 openai:
 enabled: true
 api_key: "sk-..." # DANGER: Never commit this file
```

**Method 3: OAuth (Anthropic, Qwen)**

```
OAuth tokens are read from secure storage (keychain / OS
credential store)
No env var needed if OAuth is configured
Set oauth_primary: true in config to prefer OAuth over API key
```

**Verify Keys Are Detected:**

```
helixcode verifier providers list --format json
Shows "key_status": "available" or "missing" for each provider
```

## 6.3 How to Read Model Listings and Interpret Status Indicators

**CLI Listing:**

```
helixcode --list-models
or
helixcode models list --format table
```

**Sample Output:**

#	Model	Provider	Score	Status	Latency
1	GPT-4o SC:9.2	OpenAI	9.2	✓ Ready	245ms
2	Claude 3.5 Sonnet SC:9.0	Anthropic	9.0	✓ Ready	312ms
3	Gemini 2.5 Pro SC:8.8	Gemini	8.8	✓ Ready	189ms
4	DeepSeek Coder SC:8.5	DeepSeek	8.5	✓ Ready	420ms
5	Llama 3.2 3B SC:7.1	Ollama	7.1	✓ Ready	156ms
6	GPT-3.5 Turbo SC:7.8	OpenAI	7.8	○ Stale	unknown
7	Mistral Large SC:8.2	Mistral	8.2	✗ Failed	--



8	Grok-2 SC:8.0	xAI	8.0	Cooldown	5m re
---	---------------	-----	-----	----------	-------

Legend: ✓ Verified   ○ Stale (>24h)   ✕ Failed   Rate Limited / Cooldown

**Interpreting Status:**

Status	Meaning	Action
✓ Ready	Verified within 24h, score > min_score	Available for selection
○ Stale	Last verified >24h ago	Will be re-verified; still usable but deprioritized
✕ Failed	Verification test failed	Not available for selection
Cooldown	Rate limit or temporary ban	Wait for cooldown or select alternative
Verifying	Verification in progress	Check again in a few seconds
⚠️ Deprecated	Provider marked model deprecated	Manual override required to use

**JSON Output for Scripting:**

```
helixcode models list --format json | jq '.models[] |
 select(.status == "ready") | .id'
```

### 6.4 How to Handle Cooldown / Rate-Limited Models

**Automatic Handling:** - `SelectOptimalModel()` automatically excludes rate-limited models - The circuit breaker opens after `failure_threshold` consecutive failures - Fallback to next-highest-scored available model

**Manual Override:**

```
Force use of a specific model even if rate-limited
helixcode --prompt "hello" --model gpt-4o --force
Warning: may fail with rate limit error
```

**Check Cooldown Status:**

```
helixcode verifier limits list --provider openai
Shows: remaining_requests, reset_time, limit_type
```

**Wait and Retry:**

```
Poll until model is ready
while helixcode models get gpt-4o --format json | jq -e
 '.status != "ready"' > /dev/null; do
 echo "Waiting for gpt-4o..."
 sleep 30
done
```

## 6.5 Troubleshooting Guide

**Symptom: `--list-models` shows empty list**

**Diagnosis:**

```
Step 1: Check if verifier is running
helixcode verifier health
```

```
Step 2: Check verifier logs
tail -f logs/verifier.log
```

```
Step 3: Check if providers are configured
helixcode verifier config show | grep -A5 providers
```

```
Step 4: Check if API keys are detected
helixcode verifier providers list --verbose
```

**Common Causes:** 1. Verifier server not started → `helixcode verifier start` 2. `verifier.enabled: false` in config → Set to true 3. No API keys configured → Set env vars 4. Verifier database corrupted → `rm data/llm-verifier.db` and restart 5. Network issues → Check `ping api.openai.com`

**Symptom: `--list-models` shows hardcoded 3 models (BLUFF-002)**

**This is a Constitutional Violation.** The code is using the old `handleListModels()` instead of the verifier.

**Fix:** Confirm `internal/verifier/discovery.go` is integrated and `cmd/cli/main.go:handleListModels()` calls `discoveryService.GetVerifiedModels()`.

**Symptom: Model score is always 8.5**

**Diagnosis:** The verifier is using the `stub verification.go` instead of `real coding_capability_verification.go`.

**Fix:** Check `LLMsVerifier` submodule is at correct commit. Run `git submodule update --init --recursive`.

## Symptom: Provider shows “available” but requests fail

### Diagnosis:

*# Check health status*

```
helixcode verifier providers get openai --verbose
```

*# Check circuit breaker state*

```
helixcode verifier health --provider openai
```

*# Try direct API call*

```
curl -H "Authorization: Bearer $HELIX_OPENAI_API_KEY" https://api.openai.com/v1/models
```

**Common Causes:** 1. API key expired or revoked → Regenerate key 2. Account rate limit reached → Wait or upgrade plan 3. Provider API changed → Update verifier submodule 4. Circuit breaker stuck open → Wait for `half_open_timeout` or restart

## Symptom: Scores seem wrong / weights don't make sense

### Diagnosis:

*# Check current weights*

```
helixcode verifier config show --path scoring.weights
```

*# Recalculate scores manually*

```
helixcode verifier scoring recalculate --model gpt-4o --verbose
```

**Fix:** Edit `configs/verifier.yaml` scoring weights. Ensure they sum to 1.0.

## Symptom: OAuth provider (Claude) fails after working

### Diagnosis:

*# Check OAuth token expiry*

```
helixcode verifier oauth status --provider anthropic
```

*# Check token refresh log*

```
grep "oauth_refresh" logs/verifier.log
```

**Fix:** Re-authenticate via `helixcode verifier oauth login --provider anthropic`.

## Symptom: Verifier TUI shows different models than CLI

**Diagnosis:** TUI and CLI may use different cache refresh intervals.

**Fix:** Press `r` in TUI to force refresh. Check `events.websocket.enabled` is true.

## 7. Integration Guide for Developers

### 7.1 How the Integration Works Architecturally

#### Layer Model

<u>User Interface Layer (CLI, TUI, Web, API)</u>	
<code>cmd/cli/main.go</code>	→ calls <code>internal/verifier/service.go</code>
<code>internal/server/</code>	→ calls <code>internal/verifier/service.go</code>
<code>applications/</code>	→ calls <code>pkg/sdk/go/verifier/client.go</code>
<u>Application Layer</u>	
<code>internal/services/llmsverifier_score_adapter.go</code>	
→ Bridges <code>ProviderDiscovery</code> with verifier scoring	
<code>internal/llm/model_manager.go</code>	
→ Uses score adapter for <code>SelectOptimalModel()</code>	
<u>Verifier Layer (HelixCode Wrapper)</u>	
<code>internal/verifier/service.go</code>	→ <code>VerificationService</code>
<code>internal/verifier/discovery.go</code>	→ <code>ModelDiscoveryService</code>
<code>internal/verifier/startup.go</code>	→ <code>StartupVerifier</code>
<code>internal/verifier/scoring.go</code>	→ <code>ScoringService</code>
<code>internal/verifier/health.go</code>	→ <code>HealthService</code>
<code>internal/verifier/events.go</code>	→ <code>EventPublisher</code>
<u>Verifier Layer (Submodule – LLMsVerifier)</u>	
<code>LLMsVerifier/llm-verifier/verification/verification.go</code>	
<code>LLMsVerifier/llm-verifier/providers/*.go</code>	
<code>LLMsVerifier/llm-verifier/scoring/scoring_engine.go</code>	
<code>LLMsVerifier/llm-verifier/api/server.go</code>	
<u>External APIs</u>	
OpenAI, Anthropic, Gemini, DeepSeek, Groq, ...	

#### Data Flow: Model Discovery

1. `StartupVerifier.VerifyAllProviders()` called at server startup
2. Phase 1: `discoverProviders()` → scans env vars, OAuth, local endpoints

3. Phase 2: `verifyProviders()` → calls each provider's API, runs verification tests
4. Phase 3: `detectSubscriptions()` → determines tier (Premium/High-quality/Fast)
5. Phase 4: `scoreProviders()` → runs 7-component scoring engine
6. Phase 5: `rankProviders()` → sorts by score, selects debate team
7. Results stored in SQLite database (`data/llm-verifier.db`)
8. `VerificationService` exposes results to HelixCode application layer
9. `LLMsVerifierScoreAdapter` bridges to `ModelManager.SelectOptimalModel()`

## Data Flow: Real-Time Update

1. Provider status changes (rate limit, failure, recovery)
2. `HealthService` detects change via periodic checks
3. `EventPublisher` publishes `VerificationEvent` to `WebSocket`
4. HelixCode server subscribes to `WebSocket`
5. TUI/CLI receives event and updates display
6. If `WebSocket` unavailable, polling fallback every 30s

## 7.2 How to Extend LLMsVerifier with New Providers

### Step-by-Step

#### 1. Add Provider to LLMsVerifier Submodule

In `LLMsVerifier/llm-verifier/providers/`:

```
// newprovider.go
package providers

type NewProviderAdapter struct {
 BaseAdapter
}

func NewNewProviderAdapter(endpoint, apiKey string)
 *NewProviderAdapter {
 return &NewProviderAdapter{
 BaseAdapter: BaseAdapter{
 client: &http.Client{Timeout: 30 * time.Second},
 endpoint: endpoint,
 apiKey: apiKey,
 headers: map[string]string{
 "Authorization": "Bearer " + apiKey,
 "Content-Type": "application/json",
 },
 },
 }
```

```

 },
}
}

func (a *NewProviderAdapter) SendRequest(ctx context.Context,
 req *LLMRequest) (*http.Response, error) {
 body, _ := json.Marshal(req)
 httpReq, _ := http.NewRequestWithContext(ctx, "POST",
 a.endpoint+"/chat/completions", bytes.NewReader(body))
 for k, v := range a.headers {
 httpReq.Header.Set(k, v)
 }
 return a.client.Do(httpReq)
}

func (a *NewProviderAdapter) ParseResponse(resp *http.Response)
 (*LLMResponse, error) {
 // Parse provider-specific response format
}

```

## 2. Add Fallback Models

In `LLMsVerifier/llm-verifier/providers/fallback_models.go`:

```

var NewProviderFallbackModels = []string{
 "model-1",
 "model-2",
}

```

## 3. Register in HelixCode

In `internal/verifier/provider_types.go`:

```

"newprovider": {
 Type: "newprovider",
 DisplayName: "New Provider",
 AuthType: AuthTypeAPIKey,
 Tier: 2,
 Priority: 5,
 EnvVars: []string{"NEWPROVIDER_API_KEY",
 "HELIX_NEWPROVIDER_API_KEY"},
 BaseURL: "https://api.newprovider.com/v1",
 Models: []string{"model-1", "model-2"},
}

```

## 4. Add Config Section

In `configs/verifier.yaml`:

```

providers:
 newprovider:
 enabled: true
 api_key: "${HELIX_NEWPROVIDER_API_KEY}"
 base_url: "https://api.newprovider.com/v1"
 models: []
 timeout: 30s
 retry: 3

```

## 5. Add Tests

```

// internal/verifier/adapters/newprovider_test.go
func TestNewProviderAdapter_SendRequest(t *testing.T) {
 // Unit test with httptest server
}

// tests/integration/newprovider_verification_test.go
func TestNewProvider_Verification_Integration(t *testing.T) {
 if os.Getenv("HELIX_SKIP_LIVE_PROVIDER_TESTS") != "" {
 t.Skip("SKIP-OK: Live tests disabled")
 }
 // Real API call
}

```

## 6. Add Challenge

```

challenges/scripts/newprovider_challenge.sh
#!/bin/bash
set -e

echo "=== NewProvider Challenge ==="

Verify provider is discoverable
./helixcode verifier providers list --format json | jq -e '.[[] |
 select(.type == "newprovider")'

Verify models are found
./helixcode verifier models list --provider newprovider --format
 json | jq -e '.models | length > 0'

Verify a model responds
./helixcode --prompt "Hello" --model model-1 --provider
 newprovider

echo "=== NewProvider Challenge PASSED ==="

```

## 7. Update Documentation

- Add to docs/guides/llms-verifier.md provider list

- Add to .env.example
- Add to this document's "Minimum Provider Set" table

## 7.3 How to Modify Scoring Weights

### 1. Edit Config:

```
scoring:
 weights:
 response_speed: 0.35 # Increased from 0.25
 model_efficiency: 0.15 # Decreased from 0.20
 cost_effectiveness: 0.20 # Decreased from 0.25
 capability: 0.20 # Unchanged
 recency: 0.10 # Unchanged
```

### 2. Validate (must sum to 1.0):

```
helixcode verifier config validate
Error if weights != 1.0
```

### 3. Recalculate Scores:

```
helixcode verifier scoring recalculate --all
Re-runs scoring for all models with new weights
```

### 4. Verify:

```
helixcode models list --sort score --format json | jq
 '.models[0:3] | map(.id, .score)'
Confirm top models changed appropriately
```

**Weight Guidelines:** - **Latency-sensitive workloads** (chat, autocomplete): Increase response\_speed, decrease cost\_effectiveness - **Cost-sensitive workloads** (batch processing): Increase cost\_effectiveness, decrease response\_speed - **Capability-critical workloads** (code generation, complex reasoning): Increase capability - **Always keep recency at 5-10%** to avoid permanently selecting obsolete models

## 7.4 How to Add New Model Capabilities

### 1. Add Capability Flag to Verifier Database

In LLMsVerifier/llm-verifier/database/database.go, add column:

```
ALTER TABLE models ADD COLUMN supports_new_capability BOOLEAN
 DEFAULT 0;
ALTER TABLE verification_results ADD COLUMN
 new_capability_tested BOOLEAN DEFAULT 0;
```



## 2. Add Verification Test

In LLMsVerifier/llm-verifier/verification/  
coding\_capability\_verification.go or new file:

```
func (v *Verifier) TestNewCapability(ctx context.Context,
 modelID string, client ProviderClientInterface) (bool,
 error) {
 prompt := "Test prompt for new capability..."
 response, err := client.SendPrompt(ctx, modelID, prompt)
 if err != nil {
 return false, err
 }
 return evaluateNewCapability(response), nil
}
```

## 3. Add to HelixCode Capability Mapping

In internal/verifier/provider\_types.go:

```
type UnifiedModel struct {
 // ... existing fields ...
 SupportsNewCapability bool `json:"supports_new_capability"`
}
```

## 4. Use in Model Selection

In internal/llm/model\_manager.go:

```
func (mm *ModelManager) SelectOptimalModel(criteria
 ModelSelectionCriteria) (*ModelInfo, error) {
 // ... existing filtering ...
 if criteria.RequiresNewCapability {
 candidates = filterByNewCapability(candidates)
 }
}
```

## 5. Update CONST-040 Checklist

Add new checkbox:

- ☐ New capability verified for at least 2 providers

## 7.5 API Reference for Internal Verifier Client

Go SDK (pkg/sdk/go/verifier/client.go)

```
package verifier
```

```

// Client communicates with the LLMsVerifier REST API
type Client struct {
 baseURL string
 apiKey string
 httpClient *http.Client
}

// ClientConfig configures the client
type ClientConfig struct {
 BaseURL string // Default: http://localhost:8081
 APIKey string // JWT token for verifier API auth
 Timeout time.Duration // Default: 30s
 HTTPClient *http.Client // Optional custom HTTP client
}

// New creates a new verifier client
func New(cfg ClientConfig) *Client

// VerifyModel runs verification for a single model
// POST /api/v1/verifier/models/{id}/verify
func (c *Client) VerifyModel(ctx context.Context, req
 VerificationRequest) (*VerificationResult, error)

// BatchVerify runs verification for multiple models
// POST /api/v1/verifier/batch/verify
func (c *Client) BatchVerify(ctx context.Context, req
 BatchVerifyRequest) (*BatchVerifyResult, error)

// GetModels returns all models from the verifier database
// GET /api/v1/verifier/models
func (c *Client) GetModels(ctx context.Context, filter
 ModelFilter) ([]*UnifiedModel, error)

// GetModel returns a single model by ID
// GET /api/v1/verifier/models/{id}
func (c *Client) GetModel(ctx context.Context, modelID string)
 (*UnifiedModel, error)

// GetProviderScores returns provider-level scores
// GET /api/v1/verifier/providers/scores
func (c *Client) GetProviderScores(ctx context.Context)
 (map[string]float64, error)

// GetModelScores returns model-level scores
// GET /api/v1/verifier/models/scores
func (c *Client) GetModelScores(ctx context.Context)
 (map[string]float64, error)

```

```

// GetProviders returns all providers with metadata
// GET /api/v1/verifier/providers
func (c *Client) GetProviders(ctx context.Context)
 ([]*UnifiedProvider, error)

// GetHealth returns health status for all providers
// GET /api/v1/verifier/health
func (c *Client) GetHealth(ctx context.Context) (*HealthStatus,
 error)

// GetLimits returns rate limits for all models
// GET /api/v1/verifier/limits
func (c *Client) GetLimits(ctx context.Context) ([]*RateLimit,
 error)

// GetEvents returns recent verification events
// GET /api/v1/verifier/events
func (c *Client) GetEvents(ctx context.Context, limit int)
 ([]*VerificationEvent, error)

// SubscribeEvents opens a WebSocket connection for real-time
 events
// WS /ws/verifier/events
func (c *Client) SubscribeEvents(ctx context.Context)
 (*EventStream, error)

// ExportConfig exports verifier configuration in specified
 format
// GET /api/v1/verifier/config/export?format={yaml|json|toml}
func (c *Client) ExportConfig(ctx context.Context, format
 string) ([]byte, error)

```

## VerificationRequest / VerificationResult

```

type VerificationRequest struct {
 ModelID string `json:"model_id"`
 Provider string `json:"provider"`
 Tests []string `json:"tests,omitempty"` //
 Subset of tests to run
 Timeout time.Duration `json:"timeout,omitempty"`
}

type VerificationResult struct {
 ModelID string `json:"model_id"`
 Provider string `json:"provider"`
 Status string `json:"status"` //
 pending, running, completed, failed
}

```

```

ModelExists *bool
 `json:"model_exists,omitempty"`
Responsive *bool
 `json:"responsive,omitempty"`
Overloaded *bool
 `json:"overloaded,omitempty"`
LatencyMs int64 `json:"latency_ms"`
SupportsToolUse bool
 `json:"supports_tool_use"`
SupportsCodeGeneration bool
 `json:"supports_code_generation"`
SupportsStreaming bool
 `json:"supports_streaming"`
SupportsReasoning bool
 `json:"supports_reasoning"`
OverallScore float64 `json:"overall_score"`
CodeCapabilityScore float64
 `json:"code_capability_score"`
ResponsivenessScore float64
 `json:"responsiveness_score"`
ReliabilityScore float64
 `json:"reliability_score"`
FeatureRichnessScore float64
 `json:"feature_richness_score"`
ValuePropositionScore float64
 `json:"value_proposition_score"`
Timestamp time.Time `json:"timestamp"`
Error string
 `json:"error,omitempty"`
}

```

## WebSocket Event Stream

```

type EventStream struct {
 conn *websocket.Conn
}

// Read reads the next event from the stream
func (s *EventStream) Read() (*VerificationEvent, error)

// Close closes the WebSocket connection
func (s *EventStream) Close() error

type VerificationEvent struct {
 Type VerificationEventType `json:"type"`
 Provider string `json:"provider,omitempty"`
 ModelID string `json:"model_id,omitempty"`
 Score float64 `json:"score,omitempty"`
}

```

```
Status string `json:"status,omitempty"`
Timestamp time.Time `json:"timestamp"`
Data map[string]interface{} `json:"data,omitempty"`
}
```

---

## 8. Submodule Constitution Template

This section provides a **template** for applying all LLMsVerifier constitutional rules to every submodule. Each submodule **MUST** have its own `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. The rules below **MUST** be inserted into those files.

### 8.1 Submodule CONSTITUTION.md Insertion Template

**Insert the following block into every submodule's `CONSTITUTION.md` after the highest-numbered existing rule** (e.g., if the submodule has rules up to `CONST-020`, insert after `CONST-020`; if it has up to `CONST-035`, insert after `CONST-035`):

---

*### CONST-SUB-001: LLMsVerifier Single Source of Truth Mandate  
(Submodule Edition)*

*\*\*Rule\*\*: This submodule SHALL treat LLMsVerifier as the authoritative source for all model, provider, and capability metadata that it consumes or exposes. No submodule may maintain its own hardcoded model or provider registry independent of the verifier.*

*\*\*Scope\*\*: This rule applies regardless of whether the submodule directly imports ``digital.vasic.llmsverifier``. If the submodule uses models (e.g., for testing, benchmarking, or CLI interaction), those models **MUST** be sourced from the verifier's published data, not from local constants.*

*\*\*Verification\*\*: The submodule's ``make test-complete`` **MUST** include a check that any model referenced in tests exists in the verifier database or is explicitly annotated with ``// verifier:verified``.*

---

*### CONST-SUB-002: Anti-Bluff Testing Guarantee (Submodule Edition)*

**\*\*Rule\*\***: The "Tests Pass But Features Don't Work" guarantee applies WITH FULL FORCE to this submodule. No test may pass unless the feature it exercises is demonstrably usable in the built artifact.

**\*\*Specific Prohibitions\*\***:

- Mocking the verifier with hardcoded scores in integration or E2E tests
- Asserting on hardcoded expected output when the real output should come from a provider API
- Skipping live provider tests by default (they may be skipped with ``SKIP-OK`` markers when env vars are absent)

---

### ### CONST-SUB-003: Submodule Constitution Propagation Requirement

**\*\*Rule\*\***: If this submodule has its own submodules (nested), the three rules above (CONST-SUB-001, CONST-SUB-002, CONST-SUB-003) MUST be propagated to those nested submodules as well.

**\*\*Enforcement\*\***: The presence of these rules in a submodule's ``CONSTITUTION.md`` is verified by ``challenges/scripts/submodule_constitution_check.sh`` which scans all ``CONSTITUTION.md`` files in the repository tree.

## 8.2 Submodule CLAUDE.md Insertion Template

Insert the following section into every submodule's CLAUDE.md:

---

*## LLMsVerifier Integration Guidelines for This Submodule*

*### Context*

*This submodule is part of the Helix ecosystem. LLMsVerifier is the single source of truth for all model and provider metadata. The main project constitution mandates:*

- *CONST-036: LLMsVerifier Single Source of Truth*
- *CONST-037: Model Provider Anti-Bluff Guarantee*
- *CONST-038: Real-Time Model Status Accuracy*
- *CONST-039: All Providers and Models Integration*
- *CONST-040: MCP/LSP/ACP/Embedding/RAG/Skills/Plugins Integration*

*### What This Submodule Must Do*

1. *\*\*If this submodule uses LLMs\*\**: All model references MUST be verifier-aware. Do not hardcode model names.

2. **\*\*If this submodule tests LLMs\*\***: Tests **MUST** use real provider APIs or the verifier database, not mocks (above unit test tier).
3. **\*\*If this submodule provides a model interface\*\***: It **MUST** integrate with ``internal/verifier/`` or ``pkg/sdk/go/verifier``.

### ### What This Submodule Must NOT Do

1. Maintain its own model registry independent of the verifier
2. Return hardcoded model lists to users
3. Mock verifier responses in non-unit tests
4. Skip provider capability verification

### ### Integration Points

- For Go submodules: Import ``github.com/HelixDevelopment/helix_code/pkg/sdk/go/verifier``
- For non-Go submodules: Use the REST API at ``http://localhost:8081/api/v1/verifier``
- For test submodules: Use the test fixture database at ``:memory:`` or a test-specific SQLite file

### ### Verification Checklist for Submodule Developers

- [ ] No hardcoded model names in source code
- [ ] No hardcoded provider endpoints (use verifier config)
- [ ] Integration tests use real APIs or verifier DB
- [ ] All model references can be traced to verifier data

## 8.3 Submodule AGENTS.md Insertion Template

**Insert the following section into every submodule's AGENTS.md:**

---

### ## LLMsVerifier-Aware Development Rules

#### ### BLUFF-SUB-001: Submodule Has Independent Model Registry (CRITICAL)

**\*\*Pattern\*\***: Submodule contains a ``models.go``, ``registry.go``, or similar file with hardcoded ``[]string`` of model names.

**\*\*Fix\*\***: Replace with verifier client call or import from ``pkg/sdk/go/verifier``.

#### ### BLUFF-SUB-002: Submodule Tests Use Hardcoded Expected Output (HIGH)

**\*\*Pattern\*\***: Test asserts ``expected := "GPT-4 is a model by OpenAI"`` when testing model metadata.

**\*\*Fix\*\***: Assert on verifier database content instead.

### BLUFF-SUB-003: Submodule Bypasses Verifier for Provider Selection (HIGH)  
\*\*Pattern\*\*: Submodule implements its own provider selection logic that does not consult verifier scores.  
\*\*Fix\*\*: Use `LLMsVerifierScoreAdapter.GetProviderScore()` or equivalent.

### ### Technology Stack Note

If this submodule uses models, it depends on:

- ``github.com/HelixDevelopment/helix_code/pkg/sdk/go/verifier`` (Go submodules)
- ``http://localhost:8081/api/v1/verifier`` (REST API for all languages)
- ``digital.vasic.llmsverifier`` (direct import, Go only)

### ### Required Files Checklist

If this submodule interacts with LLMs, these files **MUST** exist:

- [ ] ``docs/LLMsVERIFIER_INTEGRATION.md`` (how this submodule uses the verifier)
- [ ] ``*_test.go`` with verifier-aware tests (for Go submodules)
- [ ] ``.env.example`` with required ``HELIX_*` env vars (if applicable)

## ### 8.4 Submodule Verification Checklist

**For every submodule in the Helix ecosystem, run this checklist before declaring LLMsVerifier integration complete:**

### ## Submodule LLMsVerifier Compliance Verification

#### ### Documentation

- [ ] Submodule ``CONSTITUTION.md`` contains CONST-SUB-001, CONST-SUB-002, CONST-SUB-003
- [ ] Submodule ``CLAUDE.md`` contains "LLMsVerifier Integration Guidelines" section
- [ ] Submodule ``AGENTS.md`` contains "LLMsVerifier-Aware Development Rules" section
- [ ] Submodule has ``docs/LLMsVERIFIER_INTEGRATION.md`` (even if brief)

#### ### Code

- [ ] ``grep -r "llama-3-8b\|mistral-7b\|phi-3-mini" --include="*.go" --include="*.py" --include="*.js"`` returns zero results (or only in verifier-related files)
- [ ] No ``[]string{`` model arrays in non-test, non-verifier code
- [ ] No hardcoded provider endpoints (e.g., ``"https://api.openai.com"`` as a string literal)



- [ ] Provider selection uses verifier scores (or falls back to a documented default)
- ### Tests
- [ ] Integration tests do not mock verifier responses
  - [ ] E2E tests use the real verifier database or real provider APIs
  - [ ] ``make test`` includes a verifier-aware test (even if it skips with ``SKIP-OK``)
- ### Configuration
- [ ] ``.env.example`` lists all provider API keys this submodule needs
  - [ ] Config loader reads ``HELIX_VERIFIER_*`` env vars if verifier integration exists
- 

## Appendix A: Line Number Insertion Guide

### HelixCode CONSTITUTION.md

Section	Insert After	Approximate Content to Search For
CONST-036	CONST-035 closing	### CONST-035: End-User Usability Mandate closing paragraph
CONST-037-040	CONST-036	Immediately after CONST-036 block
CONST-035 Amendment	Within CONST-035	After “every PASS must guarantee quality, completion, usability”
CONST-017 Amendment	Within CONST-017	After “Zero-Bluff Testing” main body
Appendix D	End of document	Before any - - - or end marker

### HelixCode CLAUDE.md

Section	Insert After	Search For
LLMsVerifier Architecture	Core Systems / Architecture	## Architecture or ## Core Systems
Direct Client Pattern	Integration Patterns	After Pattern 3 description

Section	Insert After	Search For
Add Provider Guide	How to Add	After ### How to Add a New Provider or create new
API Key Provisioning	Authentication section	After JWT/auth section
Debugging	Troubleshooting	After existing troubleshooting or create new
Real-Time Updates	Event System	After internal/event/documentation
UX Guidelines	UI/CLI section	After ## CLI Usage or equivalent

## HelixCode AGENTS.md

Section	Insert After	Search For
BLUFF-004	BLUFF-003	### BLUFF-003: Command Execution is Simulated
BLUFF-005-008	BLUFF-004	After BLUFF-004 block
Tech Stack	Existing stack	#### Technology Stack or ## Technology Stack
Module Boundaries	Architecture	## Module Boundaries or ### Module Boundaries
Challenge Checklist	End of document	Before any end marker

## Appendix B: Environment Variable Quick Reference

### All HELIX\_VERIFIER\_\* Variables

```
Core verifier
HELIX_VERIFIER_ENABLED=true|false
HELIX_VERIFIER_ENDPOINT=http://localhost:8081
HELIX_VERIFIER_API_KEY=<jwt-for-verifier-auth>
```

```
HELIX_VERIFIER_TIMEOUT=30s
HELIX_VERIFIER_DATABASE_PATH=./data/llm-verifier.db
HELIX_VERIFIER_ENCRYPTION_KEY=<sql-cipher-key>
HELIX_VERIFIER_JWT_SECRET=<verifier-jwt-signing-secret>
HELIX_VERIFIER_API_PORT=8081

API keys for cloud providers
HELIX_OPENAI_API_KEY
HELIX_ANTHROPIC_API_KEY
HELIX_GEMINI_API_KEY
HELIX_DEEPSEEK_API_KEY
HELIX_GROQ_API_KEY
HELIX_TOGETHER_API_KEY
HELIX_MISTRAL_API_KEY
HELIX_XAI_API_KEY
HELIX_CEREBRAS_API_KEY
HELIX_CLOUDFLARE_API_KEY
HELIX_CLOUDFLARE_ACCOUNT_ID
HELIX_SILICONFLOW_API_KEY
HELIX_REPLICATE_API_TOKEN
HELIX_OPENROUTER_API_KEY
HELIX_QWEN_API_KEY
HELIX_COHERE_API_KEY

Local providers
HELIX_OLLAMA_HOST=http://localhost:11434
HELIX_LLAMA_CPP_HOST=http://localhost:8080
HELIX_VLLM_HOST=http://localhost:8000
HELIX_LOCALAI_HOST=http://localhost:8080

Test control
HELIX_SKIP_LIVE_PROVIDER_TESTS=1 # Set to skip live API tests
HELIX_VERIFIER_TEST_MODE=1 # Enable test mode (reduced
 verification)
```

## Appendix C: File Creation Checklist

For each file that MUST be created or modified:

#	File	Action	S
1	CONSTITUTION.md	Amend with CONST-036-040, CONST-035/017 updates, Appendix D	R

#	File	Action	S
2	CLAUDE.md	Insert LLMsVerifier architecture section	R
3	AGENTS.md	Insert BLUFF-004-008, module boundaries, checklist	R
4	internal/config/config.go	Add VerifierConfig struct, env var bindings	R
5	configs/verifier.yaml	Create full schema	R
6	configs/verifier.basic.yaml	Create example	R
7	configs/verifier.development.yaml	Create example	R
8	configs/verifier.production.yaml	Create example	R
9	configs/verifier.testing.yaml	Create example	R
10	.env.example	Add all HELIX_VERIFIER_* and provider keys	R
11	internal/verifier/service.go	Create VerificationService wrapper	R
12	internal/verifier/config.go	Create Config structs	R
13	internal/verifier/discovery.go	Create ModelDiscoveryService	R
14	internal/verifier/startup.go	Create StartupVerifier	R
15	internal/verifier/scoring.go	Create ScoringService adapter	R
16	internal/verifier/health.go	Create HealthService	R
17	internal/verifier/events.go	Create EventPublisher	R
18	internal/verifier/provider_types.go	Create SupportedProviders map	R
19	internal/services/llmsverifier_score_adapter.go	Create score bridge	R
20	pkg/sdk/go/verifier/client.go	Create Go SDK client	R

#	File	Action	S
21	docs/guides/llms-verifier.md	Create user guide	R
22	docs/integration/LLMSVERIFIER_INTEGRATION_PLAN.md	Create developer plan	R
23	challenges/scripts/llmsverifier_hardcode_check.sh	Create hardcoded scanner	R
24	challenges/scripts/llmsverifier_capabilities_challenge.sh	Create capability test	R
25	challenges/scripts/llmsverifier_status_accuracy_challenge.sh	Create status test	R
26	challenges/scripts/llmsverifier_startup_verification_challenge.sh	Create startup test	R
27	challenges/scripts/submodule_constitution_check.sh	Create propagation check	R
28	Makefile	Add verifier test targets	R
29	LLMsVerifier/ (submodule)	Add git submodule add <a href="https://github.com/vasic-digital/LLMsVerifier">https://github.com/vasic-digital/LLMsVerifier</a>	R
30	cmd/cli/main.go	Replace hardcoded models with verifier fetch	R
31	internal/llm/model_discovery.go	Replace hardcoded fetchExternalModels	R
32	internal/llm/model_manager.go	Add verifier status to scoring	R

## Appendix D: Cross-Reference to HelixAgent Implementation

For implementers, the following HelixAgent files are the canonical reference for each component:

Component	HelixAgent Reference File	Lines	Pu
VerificationService	internal/verifier/service.go	1097	Co
Config	internal/verifier/config.go	398	Co
Discovery	internal/verifier/discovery.go	526	Mo
Startup	internal/verifier/startup.go	1873	Sta pip
Provider Types	internal/verifier/provider_types.go	1043	Un Un
Scoring	internal/verifier/scoring.go	754	Sc
Enhanced Scoring	internal/verifier/enhanced_scoring.go	730	7-c
Events	internal/verifier/events.go	337	Ev
Health	internal/verifier/health.go	486	He
Score Adapter	internal/services/ llmsverifier_score_adapter.go	528	Br Pr
Go SDK	pkg/sdk/go/verifier/client.go	385	Cli
Config YAML	configs/verifier.yaml	257	Fu
Integration Guide	docs/guides/llms-verifier.md	328	Us
Integration Plan	docs/integration/ LLMSVERIFIER_INTEGRATION_PLAN.md	2423	10-
Power Features	docs/verifier/LLMSVERIFIER_POWER_FEATURES.md	661	Ad
Challenge: CLI	challenges/scripts/ llmsverifier_cliagents_challenge.sh	304	CL
Challenge: Startup	challenges/scripts/ llmsverifier_startup_verification_challenge.sh	138	Sta
Challenge: Submodule	challenges/scripts/ llmsverifier_submodule_smoke_challenge.sh	126	Su
Test Suite	tests/helixllm/llmsverifier_test_suite.sh	531	Fu

## End of Document

**Legal Force Statement:** Every rule, amendment, and mandate in this document carries the full constitutional authority of the Helix project. Implementation is not optional. The anti-bluff guarantees (CONST-037, CONST-017 LLMsVerifier clause) are binding on all developers, all agents, and all automated systems. A test that passes while the feature

it tests is non-functional is a constitutional violation, not a technical debt item.

**Submodule Propagation:** The template in Section 8 MUST be applied to every submodule without exception. The `submodule_constitution_check.sh` challenge script verifies compliance.

**Version:** 1.0.0-Draft

**Effective Upon:** Merge approval by project governance

**Review Cycle:** Every 30 days or upon verifier schema change

---

[End of Section 7]

# Ultimate Phased Implementation Plan: LLMsVerifier Integration into HelixCode

**Document Version:** 1.0.0  
**Date:** 2026-07-01  
**Author:** Master Implementation Planner  
**Status:** AUTHORITATIVE — NO VAGUENESS PERMITTED  
**Constitutional Basis:** CONST-036 through CONST-040, CONST-035 (Anti-Bluff), CONST-017 (Zero-Bluff Testing), CONST-021 (No Mocks Above Unit)

---

## EXECUTIVE SUMMARY

This plan specifies **every single action** required to integrate LLMsVerifier as the single source of truth for model provisioning in HelixCode. Every task includes exact file paths, line numbers, complete code, acceptance criteria, dependencies, and effort estimates. Nothing is left to interpretation.

### Gaps in LLMsVerifier That Must Be Fixed During Implementation

Gap	Location	Impact	Fix Required
STUB-001	verification/ verification.go	Returns hardcoded 8.5 scores for ALL dimensions	Replace stub with real coding_capability integration
STUB-002	api/server.go	Only 5 endpoints wired; full CRUD missing	Wire all handlers: model scores, pricing, limits
STUB-003	auth/ oauth_stub.go	OAuth flows are placeholder	Implement real OAuth, remove OAuth provider
MISSING-001	providers/	Only 12 providers vs HelixCode's 35+	Add 23 additional providers (Azure, Bedrock, Vertex)
MISSING-002	External module	digital.vasic.llmprovider at ../../LLMProvider required	Decision: Use REST API or Go module import to digital.vasic



Gap	Location	Impact	Fix Required
MISSING-003	Real-time push	No webhooks, SSE, or push notifications	Use polling-only arch (designed)

## Bluff Areas in HelixCode That This Integration Fixes

Bluff ID	File	Current State	Fix
BLUFF-002	cmd/cli/main.go:101-128	Hardcoded 3-model list	Replace with verifier adapter fetch
BLUFF-001	cmd/cli/main.go.old	Simulated LLM response	Route ALL generation through real provider via verifier-selected model
BLUFF-003	cmd/cli/main.go:237-250	Simulated command execution	Replace time.Sleep with actual command execution
BLUFF-004	internal/llm/model_discovery.go	Hardcoded external models in fetchExternalModels()	Replace with verifier adapter call
BLUFF-005	internal/llm/model_manager.go	Scoring ignores real verification data	Augment SelectOptimalModel() with verifier scores
BLUFF-006	internal/llm/factory.go	No verifier-aware provider validation	Add health/score validation before returning provider

## ROLLBACK & VERIFICATION STRATEGY (Global)

### Rollback Plan Template (Applied Per Phase)

For each phase, the rollback procedure is: 1. `git checkout -- <files-modified-in-this-phase>` 2. `git rm --cached <files-created-in-this-phase>` (then `rm` them) 3. Restore previous `go.mod` with `git checkout -- helix_code/go.mod` 4. Run `make test-unit` to verify baseline passes 5. Run `make build` to verify compilation succeeds

### Phase Verification Steps (Applied Per Phase)

After completing each phase, run: 1. `cd HelixCode && go build ./... —` Must compile with zero errors 2. `make test-unit —` Must pass (short tests only)

3. make test-verifier-hardcode — Must show zero hardcoded models in modified files 4. make build-cli && ./bin/cli --help — CLI must show new flags 5. grep -r "llama-3-8b" cmd/cli/main.go — Must return zero matches (BLUFF-002 fixed)

---

## PHASE 1: FOUNDATION — Configuration, Dependencies, Basic Integration

**Phase Goal:** Establish the infrastructure for LLMsVerifier integration. All config, types, client, and wiring must be in place before any feature code.

**Phase Entry Criteria:** None — this is the first phase. **Phase Exit Criteria:** 1. go build ./... compiles with zero errors 2. All new config structs are loaded and validated correctly 3. REST API client can connect to a mock verifier server 4. All 20+ env var bindings are tested

---

### TASK 1.1: Add LLMsVerifier REST API Client Dependency

**NOTE:** We do NOT import LLMsVerifier as a Go module (it depends on digital.vasic.llmprovider at ../LLMProvider which is incompatible with dev.helix.code). Instead, we use HTTP REST API calls. No new Go module dependency is needed — we only use net/http from stdlib.

#### TASK 1.1.1: Verify No Module Import Required

**File(s):** helix\_code/go.mod **Line(s):** EOF — verify no replace digital.vasic.llmsverifier or digital.vasic.llmprovider entries  
**Action:** VERIFY

```
// Check that go.mod does NOT contain:
// replace digital.vasic.llmsverifier => ...
// require digital.vasic.llmprovider ...
```

**Acceptance Criteria:** 1. grep -E "digital\.vasic|llmsverifier|llmprovider" helix\_code/go.mod returns zero matches 2. go mod tidy completes without adding LLMsVerifier as a dependency

**Dependencies:** None **Effort:** Small

---

## TASK 1.2: Add VerifierConfig Struct to internal/config/config.go

### TASK 1.2.1: Add VerifierConfig and Related Structs

**File(s):** helix\_code/internal/config/config.go **Line(s):** After line 253 (after Cognee \*CogneeConfig field) **Action:** MODIFY

```
// ... existing fields ...
Cognee *CogneeConfig `mapstructure:"cognee"`
Verifier *VerifierConfig // NEW - LLMsVerifier
 integration
HelixAgent *HelixAgentConfig
 `mapstructure:"helix_agent" // NEW - HelixAgent
 submodule sync
}
```

Append the following struct definitions at the end of config.go (after existing struct definitions, before function definitions):

```
// VerifierConfig controls LLMsVerifier integration.
type VerifierConfig struct {
 Enabled bool
 `mapstructure:"enabled"`
 Mode string
 `mapstructure:"mode" // "remote" |
 "embedded"
 Endpoint string
 `mapstructure:"endpoint" // REST API URL
 APIKey string
 `mapstructure:"api_key" // Auth key for
 verifier API
 Timeout time.Duration
 `mapstructure:"timeout"`
 CacheTTL time.Duration
 `mapstructure:"cache_ttl"`
 PollingInterval time.Duration
 `mapstructure:"polling_interval"`
 Scoring VerifierScoringConfig
 `mapstructure:"scoring"`
 Health VerifierHealthConfig
 `mapstructure:"health"`
 Events VerifierEventsConfig
 `mapstructure:"events"`
 Providers map[string]VerifierProviderConfig
 `mapstructure:"providers"`
}
```

```
// VerifierScoringConfig mirrors HelixAgent's scoring weights.
type VerifierScoringConfig struct {
 Weights ScoringWeights `mapstructure:"weights"`
 ModelsDevEnabled bool
 `mapstructure:"models_dev_enabled"`
 ModelsDevEndpoint string
 `mapstructure:"models_dev_endpoint"`
 MinAcceptableScore float64
 `mapstructure:"min_acceptable_score"`
}
```

```
// ScoringWeights defines the 5-component weight distribution.
// Weights MUST sum to exactly 1.0 (validated at startup).
type ScoringWeights struct {
 CodeCapability float64
 `mapstructure:"code_capability"` // default: 0.40
 Responsiveness float64
 `mapstructure:"responsiveness"` // default: 0.20
 Reliability float64
 `mapstructure:"reliability"` // default: 0.20
 FeatureRichness float64
 `mapstructure:"feature_richness"` // default: 0.15
 ValueProposition float64
 `mapstructure:"value_proposition"` // default: 0.05
}
```

```
// VerifierHealthConfig mirrors HelixAgent's circuit breaker
config.
type VerifierHealthConfig struct {
 CheckInterval time.Duration
 `mapstructure:"check_interval"`
 Timeout time.Duration
 `mapstructure:"timeout"`
 FailureThreshold int
 `mapstructure:"failure_threshold"`
 RecoveryThreshold int
 `mapstructure:"recovery_threshold"`
 CircuitBreaker CircuitBreakerConfig
 `mapstructure:"circuit_breaker"`
}
```

```
// CircuitBreakerConfig controls automatic failover behavior.
type CircuitBreakerConfig struct {
 Enabled bool `mapstructure:"enabled"`
 HalfOpenTimeout time.Duration
 `mapstructure:"half_open_timeout"`
}
```

```

// VerifierEventsConfig controls event publishing.
type VerifierEventsConfig struct {
 Enabled bool `mapstructure:"enabled"`
 WebSocket bool `mapstructure:"websocket"`
 WebSocketPath string `mapstructure:"websocket_path"`
}

// VerifierProviderConfig – per-provider override.
type VerifierProviderConfig struct {
 Enabled bool `mapstructure:"enabled"`
 APIKey string `mapstructure:"api_key" json:"- "
 yaml:"api_key" // json:"- " prevents serialization
 BaseURL string `mapstructure:"base_url"`
 Models []string `mapstructure:"models"`
 Priority int `mapstructure:"priority"`
}

// HelixAgentConfig controls HelixAgent submodule integration.
type HelixAgentConfig struct {
 Enabled bool `mapstructure:"enabled"`
 Path string `mapstructure:"path" // submodule path
 AutoStart bool `mapstructure:"auto_start"`
 VerifierSync HelixAgentVerifierSync
 `mapstructure:"verifier_sync"`
}

// HelixAgentVerifierSync controls bidirectional score sharing.
type HelixAgentVerifierSync struct {
 Enabled bool `mapstructure:"enabled"`
 ShareScores bool `mapstructure:"share_scores"`
 ShareProviders bool `mapstructure:"share_providers"`
 SyncInterval time.Duration `mapstructure:"sync_interval"`
}

```

**Acceptance Criteria:** 1. go build ./internal/config/... compiles with zero errors 2. Config struct now has Verifier and HelixAgent fields 3. VerifierConfig has all 9 top-level fields with correct mapstructure tags 4. APIKey fields have json: "-" to prevent accidental serialization

**Dependencies:** None **Effort:** Medium

---

### TASK 1.2.2: Add Viper Defaults for All Verifier Config Options

**File(s):** helix\_code/internal/config/config.go **Line(s):** Inside setDefaults() function, after existing defaults **Action:** MODIFY

Locate the existing `setDefault()` function (around line 300+). Append inside the function body:

```
func setDefaults() {
 // ... existing defaults ...

 // LLMsVerifier defaults
 v.SetDefault("verifier.enabled", false)
 v.SetDefault("verifier.mode", "remote")
 v.SetDefault("verifier.endpoint", "http://localhost:8081")
 v.SetDefault("verifier.timeout", "30s")
 v.SetDefault("verifier.cache_ttl", "5m")
 v.SetDefault("verifier.polling_interval", "60s")

 // Scoring defaults (match LLMsVerifier README weights)
 v.SetDefault("verifier.scoring.weights.code_capability",
 0.40)
 v.SetDefault("verifier.scoring.weights.responsiveness", 0.20)
 v.SetDefault("verifier.scoring.weights.reliability", 0.20)
 v.SetDefault("verifier.scoring.weights.feature_richness",
 0.15)
 v.SetDefault("verifier.scoring.weights.value_proposition",
 0.05)
 v.SetDefault("verifier.scoring.min_acceptable_score", 6.0)
 v.SetDefault("verifier.scoring.models_dev_enabled", true)
 v.SetDefault("verifier.scoring.models_dev_endpoint",
 "https://api.models.dev")

 // Health defaults (match HelixAgent circuit breaker)
 v.SetDefault("verifier.health.check_interval", "30s")
 v.SetDefault("verifier.health.timeout", "10s")
 v.SetDefault("verifier.health.failure_threshold", 5)
 v.SetDefault("verifier.health.recovery_threshold", 3)
 v.SetDefault("verifier.health.circuit_breaker.enabled", true)
 v.SetDefault("verifier.health.circuit_breaker.half_open_timeout",
 "60s")

 // Event defaults
 v.SetDefault("verifier.events.enabled", true)
 v.SetDefault("verifier.events.websocket", false)
 v.SetDefault("verifier.events.websocket_path", "/ws/verifier/
 events")

 // HelixAgent defaults
 v.SetDefault("helix_agent.enabled", false)
 v.SetDefault("helix_agent.path", "./HelixAgent")
 v.SetDefault("helix_agent.auto_start", false)
 v.SetDefault("helix_agent.verifier_sync.enabled", true)
}
```

```

 v.SetDefault("helix_agent.verifier_sync.share_scores", true)
 v.SetDefault("helix_agent.verifier_sync.sync_interval", "5m")
}

```

**Acceptance Criteria:** 1. `go test -short ./internal/config/...` passes 2. Loading config with empty file produces `verifier.enabled=false` and `verifier.endpoint="http://localhost:8081"` 3. Weight defaults sum to 1.0:  $0.40+0.20+0.20+0.15+0.05 == 1.0$

**Dependencies:** TASK 1.2.1 **Effort:** Small

---

### TASK 1.2.3: Add Environment Variable Bindings

**File(s):** `helix_code/internal/config/config.go` **Line(s):** After existing explicit env var bindings (after `HELIX_REDIS_PORT` binding) **Action:** MODIFY

```

// Explicitly bind critical env vars
_ = v.BindEnv("auth.jwt_secret", "HELIX_AUTH_JWT_SECRET")
_ = v.BindEnv("database.password", "HELIX_DATABASE_PASSWORD")
_ = v.BindEnv("database.host", "HELIX_DATABASE_HOST")
_ = v.BindEnv("redis.password", "HELIX_REDIS_PASSWORD")
_ = v.BindEnv("redis.host", "HELIX_REDIS_HOST")
_ = v.BindEnv("redis.port", "HELIX_REDIS_PORT")

// NEW: LLMsVerifier env var bindings
_ = v.BindEnv("verifier.enabled", "HELIX_VERIFIER_ENABLED")
_ = v.BindEnv("verifier.endpoint", "HELIX_VERIFIER_ENDPOINT")
_ = v.BindEnv("verifier.api_key", "HELIX_VERIFIER_API_KEY")
_ = v.BindEnv("verifier.timeout", "HELIX_VERIFIER_TIMEOUT")
_ = v.BindEnv("verifier.cache_ttl",
 "HELIX_VERIFIER_CACHE_TTL")
_ = v.BindEnv("verifier.polling_interval",
 "HELIX_VERIFIER_POLLING_INTERVAL")
_ = v.BindEnv("verifier.scoring.models_dev_endpoint",
 "HELIX_MODELS_DEV_ENDPOINT")
_ = v.BindEnv("verifier.scoring.min_acceptable_score",
 "HELIX_VERIFIER_MIN_SCORE")

// NEW: Per-provider API key bindings (dual-prefix support)
_ = v.BindEnv("verifier.providers.openai.api_key",
 "HELIX_OPENAI_API_KEY")
_ = v.BindEnv("verifier.providers.anthropic.api_key",
 "HELIX_ANTHROPIC_API_KEY")
_ = v.BindEnv("verifier.providers.gemini.api_key",
 "HELIX_GEMINI_API_KEY")
_ = v.BindEnv("verifier.providers.deepseek.api_key",
 "HELIX_DEEPSEEK_API_KEY")

```

```

_ = v.BindEnv("verifier.providers.groq.api_key",
 "HELIX_GROQ_API_KEY")
_ = v.BindEnv("verifier.providers.together.api_key",
 "HELIX_TOGETHER_API_KEY")
_ = v.BindEnv("verifier.providers.mistral.api_key",
 "HELIX_MISTRAL_API_KEY")
_ = v.BindEnv("verifier.providers.xai.api_key",
 "HELIX_XAI_API_KEY")
_ = v.BindEnv("verifier.providers.cerebras.api_key",
 "HELIX_CEREBRAS_API_KEY")
_ = v.BindEnv("verifier.providers.cloudflare.api_key",
 "HELIX_CLOUDFLARE_API_KEY")
_ = v.BindEnv("verifier.providers.cloudflare.account_id",
 "HELIX_CLOUDFLARE_ACCOUNT_ID")
_ = v.BindEnv("verifier.providers.siliconflow.api_key",
 "HELIX_SILICONFLOW_API_KEY")
_ = v.BindEnv("verifier.providers.replicate.api_token",
 "HELIX_REPLICATE_API_TOKEN")
_ = v.BindEnv("verifier.providers.openrouter.api_key",
 "HELIX_OPENROUTER_API_KEY")
_ = v.BindEnv("verifier.providers.qwen.api_key",
 "HELIX_QWEN_API_KEY")
_ = v.BindEnv("verifier.providers.cohere.api_key",
 "HELIX_COHERE_API_KEY")
_ = v.BindEnv("verifier.providers.ollama.host",
 "HELIX_OLLAMA_HOST")
_ = v.BindEnv("verifier.providers.llamacpp.host",
 "HELIX_LLAMA_CPP_HOST")

```

**Acceptance Criteria:** 1. `grep -c "BindEnv.*HELIX_" internal/config/config.go` returns `>= 30` bindings 2. `go test -short ./internal/config/...` passes 3. Setting `HELIX_VERIFIER_ENABLED=true` in env causes `cfg.Verifier.Enabled == true` 4. Setting `HELIX_OPENAI_API_KEY=sk-test` causes `cfg.Verifier.Providers["openai"].APIKey == "sk-test"`

**Dependencies:** TASK 1.2.2 **Effort:** Small

---

#### TASK 1.2.4: Add Config Validation Rules

**File(s):** `helix_code/internal/config/config.go` **Line(s):** Inside `validateConfig()` function, before return `nil` **Action:** MODIFY

```

func (c *Config) validateConfig() error {
 // ... existing validation ...

 // NEW: Verifier config validation
 if c.Verifier != nil && c.Verifier.Enabled {

```



```

if c.Verifier.Mode != "remote" && c.Verifier.Mode !=
 "embedded" {
 return fmt.Errorf("verifier.mode must be 'remote' or
 'embedded', got: %s", c.Verifier.Mode)
}
if c.Verifier.Endpoint == "" && c.Verifier.Mode ==
 "remote" {
 return
 fmt.Errorf("verifier.endpoint is required when mode is
 'remote'")
}
if c.Verifier.PollingInterval < 10*time.Second {
 return fmt.Errorf("verifier.polling_interval must be
 >= 10s, got: %s", c.Verifier.PollingInterval)
}
if c.Verifier.CacheTTL < 1*time.Second {
 return fmt.Errorf("verifier.cache_ttl must be >= 1s,
 got: %s", c.Verifier.CacheTTL)
}

// Validate scoring weights sum to 1.0
totalWeight := c.Verifier.Scoring.Weights.CodeCapability
+
 c.Verifier.Scoring.Weights.Responsiveness +
 c.Verifier.Scoring.Weights.Reliability +
 c.Verifier.Scoring.Weights.FeatureRichness +
 c.Verifier.Scoring.Weights.ValueProposition
if math.Abs(totalWeight-1.0) > 0.001 {
 return fmt.Errorf("verifier.scoring weights must sum
 to 1.0, got: %.3f", totalWeight)
}

// Validate min acceptable score range
if c.Verifier.Scoring.MinAcceptableScore < 0.0 ||
 c.Verifier.Scoring.MinAcceptableScore > 10.0 {
 return
 fmt.Errorf("verifier.scoring.min_acceptable_score must
 be 0.0-10.0, got: %.1f",
 c.Verifier.Scoring.MinAcceptableScore)
}

// Validate provider configs
for name, pc := range c.Verifier.Providers {
 if pc.APIKey != "" && len(pc.APIKey) < 8 {
 return fmt.Errorf("verifier.providers.%s.api_key
 is too short (minimum 8 chars)", name)
 }
}

```



```

endpoint: "http://
 localhost:8081" # string – Verifier REST API URL
 # Default: http://
 localhost:8081
 # Env:
 HELIX_VERIFIER_ENDPOINT

api_key: "${HELIX_VERIFIER_API_KEY}" # string – API key for
 authenticating TO the verifier
 # Default: ""
 # Env:
 HELIX_VERIFIER_API_KEY

timeout: "30s" # duration – HTTP
 timeout for verifier API calls
 # Default: 30s
 # Env:
 HELIX_VERIFIER_TIMEOUT

cache_ttl:
 "5m" # duration – In-memory +
 Redis cache TTL
 # Default: 5m
 # Env:
 HELIX_VERIFIER_CACHE_TTL

polling_interval: "60s" # duration – Background
 polling interval
 # Default: 60s
 # Env:
 HELIX_VERIFIER_POLLING_INTERVAL

SECTION 2: Scoring Configuration

scoring:
 weights: # map[string]float64 –
 MUST sum to 1.0
 code_capability: 0.40 # Weight for coding
 task success rate
 responsiveness: 0.20 # Weight for latency
 performance
 reliability: 0.20 # Weight for uptime/
 error rate
 feature_richness: 0.15 # Weight for tool
 use, streaming, vision, etc.
 value_proposition: 0.05 # Weight for cost
 effectiveness

```

```

models_dev_enabled: true # bool - Enable
 models.dev price fetch
models_dev_endpoint: "https://api.models.dev" # string -
 Pricing data endpoint
cache_ttl:
 "24h" # duration - Score cache TTL
min_acceptable_score: 6.0 # float64 - Minimum
 score for auto-selection (0.0-10.0)

SECTION 3: Health Monitoring

health:
 check_interval: "30s" # duration - Health
 check interval
 timeout: "10s" # duration - Health
 check timeout
 failure_threshold: 5 # int - Consecutive
 failures before circuit opens
 recovery_threshold: 3 # int - Consecutive
 successes before circuit closes
 circuit_breaker:
 enabled: true # bool - Enable
 circuit breaker
 half_open_timeout: "60s" # duration - Time in
 half-open before retry

SECTION 4: Event System

events:
 enabled: true # bool - Enable event
 publishing
 websocket: false # bool - Enable
 WebSocket event stream
 websocket_path: "/ws/verifier/events" # string - WebSocket
 endpoint path

SECTION 5: Provider Configuration

providers:
 openai:
 enabled: true
 api_key: "${OPENAI_API_KEY}"
 base_url: "https://api.openai.com/v1"

```

```
models:
 [] # empty = auto-discover all
priority: 1

anthropic:
 enabled: true
 api_key: "${ANTHROPIC_API_KEY}"
 base_url: "https://api.anthropic.com/v1"
 models: []
 priority: 2
 oauth_fallback: true

gemini:
 enabled: true
 api_key: "${GEMINI_API_KEY}"
 base_url: "https://generativelanguage.googleapis.com/
 v1beta"
 models: []
 priority: 3

deepseek:
 enabled: true
 api_key: "${DEEPSEEK_API_KEY}"
 base_url: "https://api.deepseek.com/v1"
 models: []
 priority: 4

groq:
 enabled: true
 api_key: "${GROQ_API_KEY}"
 base_url: "https://api.groq.com/openai/v1"
 models: []
 priority: 5

mistral:
 enabled: true
 api_key: "${MISTRAL_API_KEY}"
 base_url: "https://api.mistral.ai/v1"
 models: []
 priority: 6

xai:
 enabled: false # Disabled by default
 api_key: "${XAI_API_KEY}"
 base_url: "https://api.x.ai/v1"
 models: []
 priority: 7
```

```

openrouter:
 enabled: true
 api_key: "${OPENROUTER_API_KEY}"
 base_url: "https://openrouter.ai/api/v1"
 models: []
 priority: 8
 free_models_only: false

ollama:
 enabled: true
 host: "http://localhost:11434"
 models: []
 priority: 100 # Local providers
 lowest priority

llamacpp:
 enabled: true
 host: "http://localhost:8080"
 models: []
 priority: 101

```

**Acceptance Criteria:** 1. File exists at `helix_code/configs/verifier.yaml` 2. `go test -short ./internal/config/...` can load this file via Viper 3. All provider sections have `enabled`, `api_key` (or `host` for local), and `models` fields 4. Scoring weights sum to 1.0 in the file

**Dependencies:** TASK 1.2.1 **Effort:** Medium

---

## TASK 1.4: Update `.env.example`

### TASK 1.4.1: Add All Verifier and Provider Env Vars

**File(s):** `helix_code/.env.example` **Line(s):** After existing env vars, before any closing comments **Action:** MODIFY

```

=====
LLMsVerifier Configuration
=====
HELIX_VERIFIER_ENABLED=false
HELIX_VERIFIER_ENDPOINT=http://localhost:8081
HELIX_VERIFIER_API_KEY=
HELIX_VERIFIER_TIMEOUT=30s
HELIX_VERIFIER_CACHE_TTL=5m
HELIX_VERIFIER_POLLING_INTERVAL=60s
HELIX_VERIFIER_MIN_SCORE=6.0
HELIX_MODELS_DEV_ENDPOINT=https://api.models.dev

```

```
=====
Provider API Keys (Cloud Providers)
=====
```

```
HELIX_OPENAI_API_KEY=
HELIX_ANTHROPIC_API_KEY=
HELIX_GEMINI_API_KEY=
HELIX_DEEPSEEK_API_KEY=
HELIX_GROQ_API_KEY=
HELIX_TOGETHER_API_KEY=
HELIX_MISTRAL_API_KEY=
HELIX_XAI_API_KEY=
HELIX_CEREBRAS_API_KEY=
HELIX_CLOUDFLARE_API_KEY=
HELIX_CLOUDFLARE_ACCOUNT_ID=
HELIX_SILICONFLOW_API_KEY=
HELIX_REPLICATE_API_TOKEN=
HELIX_OPENROUTER_API_KEY=
HELIX_QWEN_API_KEY=
HELIX_COHERE_API_KEY=
```

```
=====
Local Provider Configuration
=====
```

```
HELIX_OLLAMA_HOST=http://localhost:11434
HELIX_LLAMA_CPP_HOST=http://localhost:8080
```

**Acceptance Criteria:** 1. `grep -c "HELIX_" .env.example` returns `>= 30` entries 2. All 16 provider API keys are documented 3. `grep "HELIX_VERIFIER_ENABLED" .env.example` returns exactly one line

**Dependencies:** TASK 1.2.3 **Effort:** Small

---

## TASK 1.5: Create internal/verifier/types.go

### TASK 1.5.1: Define All Shared Verifier Types

**File(s):** helix\_code/internal/verifier/types.go **Line(s):** CREATE new file  
**Action:** CREATE

```
// Package verifier provides the LLMsVerifier integration layer
// for HelixCode.
// It communicates with LLMsVerifier via REST API (not Go module
// import)
// to avoid the digital.vasic.llmprovider sibling-module
// dependency.
package verifier
```

```

import (
 "errors"
 "time"
)

// ErrVerifierDisabled is returned when the verifier is disabled
// in config.
var ErrVerifierDisabled = errors.New("verifier is disabled")

// ErrVerifierUnavailable is returned when the verifier service
// cannot be reached.
var ErrVerifierUnavailable = errors.New("verifier service is
 unavailable")

// ErrUsingStaleCache is returned when data comes from expired
// cache.
var ErrUsingStaleCache = errors.New("using stale cached verifier
 data")

// ErrUsingFallback is returned when data comes from hardcoded
// fallback list.
var ErrUsingFallback = errors.New("using fallback model list")

// VerifiedModel – unified model representation from
// LLMsVerifier.
type VerifiedModel struct {
 ID string `json:"id"`
 Name string `json:"name"`
 DisplayName string `json:"display_name"`
 Provider string `json:"provider"`
 ProviderType string `json:"provider_type"`
 Score float64 `json:"score"`
 Verified bool `json:"verified"`
 VerificationStatus string `json:"verification_status" // pending, verified,
 // failed, rate_limited`
 ContextSize int `json:"context_window_tokens"`
 MaxOutputTokens int `json:"max_output_tokens"`
 SupportsStreaming bool `json:"supports_streaming"`
 SupportsTools bool `json:"supports_tool_use"`
 SupportsFunctions bool `json:"supports_functions"`
 SupportsCode bool `json:"supports_code_generation"`
 SupportsVision bool `json:"supports_vision"`
 SupportsAudio bool `json:"supports_audio"`
 SupportsVideo bool `json:"supports_video"`
 SupportsReasoning bool `json:"supports_reasoning"`
}

```



```

SupportsEmbeddings bool `json:"supports_embeddings"`
SupportsJSONMode bool `json:"supports_json_mode"`
Latency time.Duration `json:"latency_ms"`
CostPerInputToken float64 `json:"input_token_cost"`
CostPerOutputToken float64 `json:"output_token_cost"`
OverallScore float64 `json:"overall_score"`
CodeCapabilityScore float64 `json:"code_capability_score"`
ResponsivenessScore float64 `json:"responsiveness_score"`
ReliabilityScore float64 `json:"reliability_score"`
FeatureRichnessScore float64 `json:"feature_richness_score"`
ValuePropositionScore float64 `json:"value_proposition_score"`
LastVerified time.Time `json:"last_verified"`
Source string `json:"source"` //
 "verifier", "cache", "fallback"
OpenSource bool `json:"open_source"`
Deprecated bool `json:"deprecated"`
Tier int `json:"tier"` // 1=Premium,
 2=High-quality, 3=Fast, 4=Aggregator, 5=Free
Capabilities []string `json:"capabilities"`
Tags []string `json:"tags"`
}

```

*// ProviderStatus – health and score of a provider.*

```

type ProviderStatus struct {
 Name string `json:"name"`
 Type string `json:"type"`
 DisplayName string `json:"display_name"`
 Score float64 `json:"score"`
 Verified bool `json:"verified"`
 Healthy bool `json:"healthy"`
 Status string `json:"status"` // unknown, healthy,
 degraded, unhealthy, offline
 ModelCount int `json:"model_count"`
 Tier int `json:"tier"`
 Priority int `json:"priority"`
 LastChecked time.Time `json:"last_checked"`
 UptimePct float64 `json:"uptime_pct"`
 Latency time.Duration `json:"latency"`
}

```

*// VerificationResult – result of on-demand verification.*

```

type VerificationResult struct {
 ModelID string `json:"model_id"`
 Status string `json:"status"` // started,
 completed, failed
}

```

```

OverallScore float64 `json:"overall_score"`
CodeCapabilityScore float64 `json:"code_capability_score"`
ResponsivenessScore float64 `json:"responsiveness_score"`
ReliabilityScore float64 `json:"reliability_score"`
FeatureRichnessScore float64 `json:"feature_richness_score"`
ValuePropositionScore float64 `json:"value_proposition_score"`
ModelExists *bool `json:"model_exists,omitempty"`
Responsive *bool `json:"responsive,omitempty"`
Overloaded *bool `json:"overloaded,omitempty"`
SupportsToolUse bool `json:"supports_tool_use"`
SupportsCodeGeneration bool `json:"supports_code_generation"`
SupportsEmbeddings bool `json:"supports_embeddings"`
SupportsStreaming bool `json:"supports_streaming"`
SupportsJSONMode bool `json:"supports_json_mode"`
SupportsReasoning bool `json:"supports_reasoning"`
CodeDebugging bool `json:"code_debugging"`
CodeOptimization bool `json:"code_optimization"`
TestGeneration bool `json:"test_generation"`
DocumentationGeneration bool `json:"documentation_generation"`
ArchitectureDesign bool `json:"architecture_design"`
SecurityAssessment bool `json:"security_assessment"`
PatternRecognition bool `json:"pattern_recognition"`
Error string `json:"error,omitempty"`
CompletedAt time.Time `json:"completed_at"`
}

```

*// HealthResponse – verifier service health.*

```

type HealthResponse struct {
 Status string `json:"status"`
 Timestamp time.Time `json:"timestamp"`
 Version string `json:"version"`
}

```

*// RateLimitStatus – rate limit information for a model.*

```

type RateLimitStatus struct {
 ModelID string `json:"model_id"`
 Limits []LimitEntry `json:"limits"`
}

```

*// LimitEntry – a single rate limit dimension.*

```

type LimitEntry struct {
 Type string `json:"type"`
}

```

```

 Limit int `json:"limit"`
 Used int `json:"used"`
 Remaining int `json:"remaining"`
 ResetTime time.Time `json:"reset_time"`
}

// CooldownInfo – cooldown state for a model or provider.
type CooldownInfo struct {
 ModelID string `json:"model_id"`
 Provider string `json:"provider"`
 Reason string `json:"reason" // rate-limited,
 quota-exceeded, cooldown`
 ResetTime time.Time `json:"reset_time"`
 Duration time.Duration `json:"duration"`
}

// ChangeEvent – emitted when verifier data changes.
type ChangeEvent struct {
 Type string `json:"type" // model.discovered,
 model.score_changed, model.status_changed, model.removed`
 Model *VerifiedModel `json:"model,omitempty"`
 Provider *ProviderStatus `json:"provider,omitempty"`
 OldScore float64 `json:"old_score,omitempty"`
 OldStatus string `json:"old_status,omitempty"`
 Timestamp time.Time `json:"timestamp"`
}

// FallbackModels is the hardcoded fallback list used when
// verifier is unavailable.
// This is the ONLY permitted hardcoded model list (CONST-035
// compliance).
var FallbackModels = []*VerifiedModel{
 {ID: "llama-3.2-3b", Name: "Llama 3.2 3B", DisplayName:
 "Llama 3.2 3B", Provider: "ollama", ContextSize:
 131072, Source: "fallback", OverallScore: 6.0, Tier: 3,
 OpenSource: true},
 {ID: "gpt-4o", Name: "GPT-4o", DisplayName: "GPT-4o",
 Provider: "openai", ContextSize: 128000, Source:
 "fallback", OverallScore: 9.1, Tier: 1},
 {ID: "claude-3-5-sonnet", Name: "Claude 3.5 Sonnet",
 DisplayName: "Claude 3.5 Sonnet", Provider:
 "anthropic", ContextSize: 200000, Source: "fallback",
 OverallScore: 8.9, Tier: 1},
 {ID: "mistral-large", Name: "Mistral Large", DisplayName:
 "Mistral Large", Provider: "mistral", ContextSize:
 128000, Source: "fallback", OverallScore: 7.8, Tier: 2},

```

```

{ID: "gemini-2.5-pro", Name: "Gemini 2.5 Pro", DisplayName:
 "Gemini 2.5 Pro", Provider: "gemini", ContextSize:
 1000000, Source: "fallback", OverallScore: 8.7, Tier:
 1},
{ID: "deepseek-chat", Name: "DeepSeek Chat", DisplayName:
 "DeepSeek Chat", Provider: "deepseek", ContextSize:
 64000, Source: "fallback", OverallScore: 8.3, Tier: 2,
 OpenSource: true},
{ID: "grok-3-fast-beta", Name: "Grok-3 Fast Beta",
 DisplayName: "Grok-3 Fast Beta", Provider: "xai",
 ContextSize: 131072, Source: "fallback", OverallScore:
 8.0, Tier: 1},
}

```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles 2. FallbackModels has exactly 7 entries 3. All struct fields have json:"..." tags for every field 4. ErrVerifierDisabled, ErrVerifierUnavailable, ErrUsingStaleCache, ErrUsingFallback are defined

**Dependencies:** None **Effort:** Medium

---

## TASK 1.6: Create internal/verifier/client.go

### TASK 1.6.1: Implement REST API Client

**File(s):** helix\_code/internal/verifier/client.go **Line(s):** CREATE new file  
**Action:** CREATE

```
package verifier
```

```
import (
 "context"
 "encoding/json"
 "fmt"
 "net/http"
 "time"
)
```

```

// Client is the HTTP REST API client for LLMsVerifier.
// It mirrors HelixAgent's pkg/sdk/go/verifier/client.go pattern.
// Uses stdlib net/http – no external dependency needed.

```

```

type Client struct {
 baseURL string
 apiKey string
 httpClient *http.Client
 timeout time.Duration
}

```

```

// NewClient creates a verifier API client.
// baseURL: verifier service URL (e.g., "http://localhost:8081")
// apiKey: optional bearer token for authenticated endpoints
// timeout: HTTP client timeout (0 = default 30s)
func NewClient(baseURL, apiKey string, timeout time.Duration)
 *Client {
 if baseURL == "" {
 baseURL = "http://localhost:8081"
 }
 if timeout == 0 {
 timeout = 30 * time.Second
 }
 return &Client{
 baseURL: baseURL,
 apiKey: apiKey,
 timeout: timeout,
 httpClient: &http.Client{Timeout: timeout},
 }
}

```

```

// WithHTTPClient allows injecting a custom HTTP client (for
// testing).

```

```

func (c *Client) WithHTTPClient(hc *http.Client) *Client {
 c.httpClient = hc
 return c
}

```

```

// Health checks the verifier service health endpoint.

```

```

func (c *Client) Health(ctx context.Context) (*HealthResponse,
 error) {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet,
 c.baseURL+"/api/health", nil)
 if err != nil {
 return nil, fmt.Errorf("verifier health: create request:
 %w", err)
 }
 c.setAuthHeader(req)

 resp, err := c.httpClient.Do(req)
 if err != nil {
 return nil, fmt.Errorf("verifier health check failed:
 %w", err)
 }
 defer resp.Body.Close()

 if resp.StatusCode != http.StatusOK {

```

```

 return nil, fmt.Errorf("verifier health: HTTP %d",
 resp.StatusCode)
 }

 var hr HealthResponse
 if err := json.NewDecoder(resp.Body).Decode(&hr); err != nil {
 return nil, fmt.Errorf("verifier health: decode: %w",
 err)
 }
 return &hr, nil
}

// GetModels fetches all verified models from the verifier.
func (c *Client) GetModels(ctx context.Context)
 ([]*VerifiedModel, error) {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet,
 c.baseURL+"/api/models", nil)
 if err != nil {
 return nil, err
 }
 c.setAuthHeader(req)

 resp, err := c.httpClient.Do(req)
 if err != nil {
 return nil, fmt.Errorf("failed to fetch models from
 verifier: %w", err)
 }
 defer resp.Body.Close()

 if resp.StatusCode != http.StatusOK {
 return nil, fmt.Errorf("verifier models: HTTP %d",
 resp.StatusCode)
 }

 var models []*VerifiedModel
 if err := json.NewDecoder(resp.Body).Decode(&models); err !=
 nil {
 return nil, fmt.Errorf("failed to decode models: %w",
 err)
 }
 return models, nil
}

// GetModelByID fetches a single model by ID.
func (c *Client) GetModelByID(ctx context.Context, modelID
 string) (*VerifiedModel, error) {
 url := fmt.Sprintf("%s/api/models/%s", c.baseURL, modelID)

```

```

req, err := http.NewRequestWithContext(ctx, http.MethodGet,
 url, nil)
if err != nil {
 return nil, err
}
c.setAuthHeader(req)

resp, err := c.httpClient.Do(req)
if err != nil {
 return nil, fmt.Errorf("failed to fetch model %s: %w",
 modelID, err)
}
defer resp.Body.Close()

if resp.StatusCode == http.StatusNotFound {
 return nil, fmt.Errorf("model %s not found", modelID)
}
if resp.StatusCode != http.StatusOK {
 return nil, fmt.Errorf("verifier model get: HTTP %d",
 resp.StatusCode)
}

var model VerifiedModel
if err := json.NewDecoder(resp.Body).Decode(&model); err !=
 nil {
 return nil, fmt.Errorf("failed to decode model: %w", err)
}
return &model, nil
}

```

*// GetProviderScores fetches all provider scores.*

```

func (c *Client) GetProviderScores(ctx context.Context)
 (map[string]float64, error) {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet,
 c.baseURL+"/api/scores", nil)
 if err != nil {
 return nil, err
 }
 c.setAuthHeader(req)

 resp, err := c.httpClient.Do(req)
 if err != nil {
 return nil, fmt.Errorf("failed to fetch scores: %w", err)
 }
 defer resp.Body.Close()

 if resp.StatusCode != http.StatusOK {

```

```

 return nil, fmt.Errorf("verifier scores: HTTP %d",
 resp.StatusCode)
 }

 var scores map[string]float64
 if err := json.NewDecoder(resp.Body).Decode(&scores); err !=
 nil {
 return nil, fmt.Errorf("failed to decode scores: %w",
 err)
 }
 return scores, nil
}

// VerifyModel triggers on-demand verification for a model.
func (c *Client) VerifyModel(ctx context.Context, modelID
 string) (*VerificationResult, error) {
 url := fmt.Sprintf("%s/api/models/%s/verify", c.baseURL,
 modelID)
 req, err := http.NewRequestWithContext(ctx, http.MethodPost,
 url, nil)
 if err != nil {
 return nil, err
 }
 c.setAuthHeader(req)

 resp, err := c.httpClient.Do(req)
 if err != nil {
 return nil, fmt.Errorf("failed to verify model %s: %w",
 modelID, err)
 }
 defer resp.Body.Close()

 if resp.StatusCode != http.StatusOK && resp.StatusCode !=
 http.StatusAccepted {
 return nil, fmt.Errorf("verifier verify: HTTP %d",
 resp.StatusCode)
 }

 var result VerificationResult
 if err := json.NewDecoder(resp.Body).Decode(&result); err !=
 nil {
 return nil, fmt.Errorf("failed to decode verification
 result: %w", err)
 }
 return &result, nil
}

// GetPricing fetches token pricing data.

```



```

func (c *Client) GetPricing(ctx context.Context)
 ([]map[string]interface{}, error) {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet,
 c.baseURL+"/api/pricing", nil)
 if err != nil {
 return nil, err
 }
 c.setAuthHeader(req)

 resp, err := c.httpClient.Do(req)
 if err != nil {
 return nil, fmt.Errorf("failed to fetch pricing: %w",
 err)
 }
 defer resp.Body.Close()

 if resp.StatusCode != http.StatusOK {
 return nil, fmt.Errorf("verifier pricing: HTTP %d",
 resp.StatusCode)
 }

 var pricing []map[string]interface{}
 if err := json.NewDecoder(resp.Body).Decode(&pricing); err !=
 nil {
 return nil, fmt.Errorf("failed to decode pricing: %w",
 err)
 }
 return pricing, nil
}

```

*// GetLimits fetches rate limit data.*

```

func (c *Client) GetLimits(ctx context.Context)
 ([]map[string]interface{}, error) {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet,
 c.baseURL+"/api/limits", nil)
 if err != nil {
 return nil, err
 }
 c.setAuthHeader(req)

 resp, err := c.httpClient.Do(req)
 if err != nil {
 return nil, fmt.Errorf("failed to fetch limits: %w", err)
 }
 defer resp.Body.Close()

 if resp.StatusCode != http.StatusOK {

```

```

 return nil, fmt.Errorf("verifier limits: HTTP %d",
 resp.StatusCode)
 }

 var limits []map[string]interface{}
 if err := json.NewDecoder(resp.Body).Decode(&limits); err !=
 nil {
 return nil, fmt.Errorf("failed to decode limits: %w",
 err)
 }
 return limits, nil
}

// setAuthHeader adds the Authorization header if API key is
// configured.
func (c *Client) setAuthHeader(req *http.Request) {
 if c.apiKey != "" {
 req.Header.Set("Authorization", "Bearer "+c.apiKey)
 }
 req.Header.Set("Accept", "application/json")
 req.Header.Set("User-Agent", "HelixCode-VerifierClient/1.0")
}

```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles with zero errors 2. NewClient("", "", 0) produces baseURL="http://localhost:8081", timeout=30s 3. setAuthHeader sets "Bearer" prefix and never logs the key 4. All methods accept context.Context for cancellation 5. Response body is always closed via defer

**Dependencies:** TASK 1.5.1 **Effort:** Large

---

## TASK 1.7: Create internal/verifier/config.go

### TASK 1.7.1: Create Verifier-Specific Config Types and Loader

**File(s):** helix\_code/internal/verifier/config.go **Line(s):** CREATE new file  
**Action:** CREATE

```

package verifier

import (
 "dev.helix.code/internal/config"
 "fmt"
 "time"
)

```

```
// AdapterConfig wraps the application-level VerifierConfig for
 use within
 the verifier package. This separates the Viper/mapstructure
 concerns from
 the runtime verifier concerns.
```

```
type AdapterConfig struct {
 Enabled bool
 Endpoint string
 APIKey string
 Timeout time.Duration
 CacheTTL time.Duration
 PollingInterval time.Duration
 Scoring ScoringAdapterConfig
 Health HealthAdapterConfig
 Events EventsAdapterConfig
 Providers map[string]ProviderAdapterConfig
}
```

```
// ScoringAdapterConfig – runtime scoring configuration.
```

```
type ScoringAdapterConfig struct {
 Weights config.ScoringWeights
 ModelsDevEnabled bool
 ModelsDevEndpoint string
 MinAcceptableScore float64
}
```

```
// HealthAdapterConfig – runtime health configuration.
```

```
type HealthAdapterConfig struct {
 CheckInterval time.Duration
 Timeout time.Duration
 FailureThreshold int
 RecoveryThreshold int
 CircuitBreaker config.CircuitBreakerConfig
}
```

```
// EventsAdapterConfig – runtime events configuration.
```

```
type EventsAdapterConfig struct {
 Enabled bool
 WebSocket bool
 WebSocketPath string
}
```

```
// ProviderAdapterConfig – runtime per-provider configuration.
```

```
type ProviderAdapterConfig struct {
 Enabled bool
 APIKey string
 BaseURL string
 Models []string
```

```

 Priority int
}

// NewAdapterConfig translates the Viper-loaded Config into
// verifier-native config.
func NewAdapterConfig(cfg *config.VerifierConfig) *AdapterConfig
{
 if cfg == nil {
 return &AdapterConfig{Enabled: false}
 }
 ac := &AdapterConfig{
 Enabled: cfg.Enabled,
 Endpoint: cfg.Endpoint,
 APIKey: cfg.APIKey,
 Timeout: cfg.Timeout,
 CacheTTL: cfg.CacheTTL,
 PollingInterval: cfg.PollingInterval,
 Scoring: ScoringAdapterConfig{
 Weights: cfg.Scoring.Weights,
 ModelsDevEnabled: cfg.Scoring.ModelsDevEnabled,
 ModelsDevEndpoint: cfg.Scoring.ModelsDevEndpoint,
 MinAcceptableScore: cfg.Scoring.MinAcceptableScore,
 },
 Health: HealthAdapterConfig{
 CheckInterval: cfg.Health.CheckInterval,
 Timeout: cfg.Health.Timeout,
 FailureThreshold: cfg.Health.FailureThreshold,
 RecoveryThreshold: cfg.Health.RecoveryThreshold,
 CircuitBreaker: cfg.Health.CircuitBreaker,
 },
 Events: EventsAdapterConfig{
 Enabled: cfg.Events.Enabled,
 WebSocket: cfg.Events.WebSocket,
 WebSocketPath: cfg.Events.WebSocketPath,
 },
 Providers: make(map[string]ProviderAdapterConfig),
 }
 for name, pc := range cfg.Providers {
 ac.Providers[name] = ProviderAdapterConfig{
 Enabled: pc.Enabled,
 APIKey: pc.APIKey,
 BaseURL: pc.BaseURL,
 Models: pc.Models,
 Priority: pc.Priority,
 }
 }
 return ac
}

```

```
// Validate checks the adapter configuration for runtime
correctness.
func (ac *AdapterConfig) Validate() error {
 if !ac.Enabled {
 return nil
 }
 if ac.Endpoint == "" {
 return fmt.Errorf("verifier endpoint is required when
enabled")
 }
 if ac.Timeout < 1*time.Second {
 return fmt.Errorf("verifier timeout must be >= 1s")
 }
 if ac.PollingInterval < 10*time.Second {
 return fmt.Errorf("verifier polling_interval must be >=
10s")
 }
 total := ac.Scoring.Weights.CodeCapability +
 ac.Scoring.Weights.Responsiveness +
 ac.Scoring.Weights.Reliability +
 ac.Scoring.Weights.FeatureRichness +
 ac.Scoring.Weights.ValueProposition
 if total < 0.999 || total > 1.001 {
 return fmt.Errorf("scoring weights must sum to 1.0, got
%.3f", total)
 }
 return nil
}
```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles 2. NewAdapterConfig(nil) returns Enabled: false 3. Validate() returns error when weights don't sum to 1.0 4. All nested configs are correctly translated from config.VerifierConfig

**Dependencies:** TASK 1.2.1, TASK 1.5.1 **Effort:** Medium

---

## TASK 1.8: Create internal/verifier/doc.go

### TASK 1.8.1: Package Documentation

**File(s):** helix\_code/internal/verifier/doc.go **Line(s):** CREATE new file  
**Action:** CREATE

```

// Package verifier integrates LLMsVerifier as the single source
// of truth
// for model metadata, provider health, verification status, and
// scoring
// within HelixCode.
//
// Architecture:
//
// HelixCode (cmd/cli, cmd/server, internal/llm)
// |
// | REST API calls (HTTP/json)
// v
// internal/verifier/Client -> LLMsVerifier service
// (localhost:8081)
// |
// | SQLite DB read
// v
// LLMsVerifier (submodule) -> Provider APIs
//
// Key types:
// - Client: HTTP REST client for verifier API
// - Adapter: Bridges verifier scores to HelixCode's
// ModelManager
// - Cache: Two-tier cache (in-memory LRU + Redis)
// - HealthMonitor: Circuit breaker for verifier availability
// - Poller: Background goroutine for real-time updates
// - EventPublisher: Publishes changes to HelixCode event bus
//
// The verifier is disabled by default (verifier.enabled=false).
// When disabled,
// all model operations fall back to legacy behavior.
//
// Constitutional compliance:
// - CONST-036: LLMsVerifier single source of truth
// - CONST-037: Anti-bluff guarantee (no hardcoded models)
// - CONST-038: Real-time status accuracy
// - CONST-039: All providers integration
// - CONST-040: MCP/LSP/ACP/Embedding/RAG/Skills/Plugins
package verifier

```

**Acceptance Criteria:** 1. go doc ./internal/verifier shows the package documentation 2. Documentation mentions all 6 key types 3. Constitutional rules CONST-036 through CONST-040 are referenced

**Dependencies:** None **Effort:** Small

---

## TASK 1.9: Add Enable/Disable Truth Table

### TASK 1.9.1: Document and Implement Verifier Enable/Disable Behavior

**File(s):** helix\_code/internal/verifier/adapter.go (will be created in Phase 2, but the truth table is defined here) **Line(s):** N/A — truth table is enforced by nil-check in initialization **Action:** DOCUMENT

The enable/disable behavior is implemented via nil-check in initializeVerifier():

verifier.enabled	verifier.endpoint reachable	System Behavior
false (default)	N/A	Verifier client is nil. All model operations use legacy local/heuristic logic. handleListModel() falls through to modelManager.GetAvailableModels(). No polling goroutine. No REST calls.
true	true	Verifier client initialized. Polling goroutine started. Model lists fetched from verifier first. Scores incorporated into SelectOptimalModel(). Events published on changes.
true	false	Verifier client initialized but health monitor marks circuit OPEN after failure_threshold failures. System falls back to stale cache, then to FallbackModels. Events published for degraded state.

**Implementation location:** cmd/server/main.go and cmd/cli/main.go — check `cfg.Verifier != nil && cfg.Verifier.Enabled` before initializing verifier subsystem.

**Acceptance Criteria:** 1. With `verifier.enabled=false`, `grep -r "llmsverifier\|verifier" cmd/cli/main.go` shows only the nil check 2. With `verifier.enabled=true`, the CLI shows verifier-specific columns in `--list-models`

**Dependencies:** TASK 1.6.1 **Effort:** Small

---

## PHASE 1 ROLLBACK PLAN

If Phase 1 must be rolled back:

```
cd HelixCode
git checkout -- internal/config/config.go
git checkout -- .env.example
git rm --cached internal/verifier/types.go internal/verifier/
 client.go internal/verifier/config.go internal/verifier/
 doc.go
rm -f internal/verifier/types.go internal/verifier/client.go
 internal/verifier/config.go internal/verifier/doc.go
git rm --cached configs/verifier.yaml
rm -f configs/verifier.yaml
go mod tidy
make build
make test-unit
```

## PHASE 1 VERIFICATION CHECKLIST

- ☐ go build ./... compiles with zero errors
- ☐ make test-unit passes
- ☐ go test -short ./internal/config/... passes with verifier defaults
- ☐ grep -c "VerifierConfig" internal/config/config.go >= 1
- ☐ grep -c "mapstructure:\"verifier\"" internal/config/config.go >= 1
- ☐ grep -c "BindEnv.\*HELIX\_VERIFIER" internal/config/config.go >= 7
- ☐ grep -c "BindEnv.\*HELIX\_.\*API\_KEY" internal/config/config.go >= 16
- ☐ configs/verifier.yaml exists and is valid YAML
- ☐ .env.example contains all HELIX\_VERIFIER\_\* and provider keys
- ☐ internal/verifier/types.go defines VerifiedModel, ProviderStatus, VerificationResult
- ☐



internal/verifier/client.go implements GetModels, GetModelByID, GetProviderScores, VerifyModel



FallbackModels has exactly 7 entries



ErrVerifierDisabled and ErrVerifierUnavailable are defined



NewAdapterConfig(nil) returns Enabled: false

---

## PHASE 2: MODEL MANAGEMENT — LLMsVerifier as Single Source of Truth

**Phase Goal:** Replace ALL hardcoded model sources with LLMsVerifier data. The verifier adapter becomes the bridge between LLMsVerifier and HelixCode's internal/llm/ package.

**Phase Entry Criteria:** Phase 1 verification checklist is complete. **Phase Exit Criteria:** 1. handleListModels() returns models from verifier, not hardcoded list 2. fetchExternalModels() returns verifier data, not hardcoded array 3. SelectOptimalModel() uses verifier scores for ranking 4. NewProvider() validates against verifier health/score thresholds 5. Background polling updates model data every N seconds

---

### TASK 2.1: Create internal/verifier/adapter.go

#### TASK 2.1.1: Implement ScoreAdapter Bridge

**File(s):** helix\_code/internal/verifier/adapter.go **Line(s):** CREATE new file **Action:** CREATE

```
package verifier
```

```
import (
 "context"
 "dev.helix.code/internal/config"
 "fmt"
 "sync"
 "time"
)
```

```
// Adapter bridges LLMsVerifier scores to HelixCode's llm
 package.
```

```
// Replicates HelixAgent's internal/services/
 llmsverifier_score_adapter.go.
```

```
type Adapter struct {
 client *Client
 cache *Cache
 health *HealthMonitor
 config *AdapterConfig
 events *EventPublisher

 providerScores map[string]float64
 modelScores map[string]float64
 modelCodeScores map[string]float64
 modelRelScores map[string]float64
 mu sync.RWMutex

 lastRefresh time.Time
 refreshInterval time.Duration
}
```

```
// NewAdapter creates the verifier adapter.
```

```
func NewAdapter(client *Client, cache *Cache, health
 *HealthMonitor, cfg *config.VerifierConfig) *Adapter {
 ac := NewAdapterConfig(cfg)
 return &Adapter{
 client: client,
 cache: cache,
 health: health,
 config: ac,
 providerScores: make(map[string]float64),
 modelScores: make(map[string]float64),
 modelCodeScores: make(map[string]float64),
 modelRelScores: make(map[string]float64),
 refreshInterval: ac.CacheTTL,
 }
}
```

```
// IsEnabled returns true if the verifier subsystem is enabled in
 config.
```

```
func (a *Adapter) IsEnabled() bool {
 return a.config != nil && a.config.Enabled
}
```

```
// IsReachable returns true if the verifier service is reachable
 (circuit not open).
```

```
func (a *Adapter) IsReachable() bool {
 return a.health.AllowRequest()
}
```

```

// GetModelScore returns the overall verifier score (0-10) for a
// model.
// Mirrors HelixAgent's
// llmsverifier_score_adapter.go:GetModelScore().
func (a *Adapter) GetModelScore(modelID string) (float64, bool) {
 a.mu.RLock()
 defer a.mu.RUnlock()

 score, ok := a.modelScores[modelID]
 if ok {
 if score > 10 {
 score = score / 10.0
 }
 return score, true
 }
 // Try cache
 if a.cache != nil {
 if cached, found := a.cache.GetModelScore(modelID);
 found {
 return cached, true
 }
 }
 return 0, false
}

// GetProviderScore returns the best score for any model of this
// provider.
// Mirrors HelixAgent's GetProviderScore().
func (a *Adapter) GetProviderScore(providerType string)
(float64, bool) {
 a.mu.RLock()
 defer a.mu.RUnlock()

 score, ok := a.providerScores[providerType]
 if ok {
 if score > 10 {
 score = score / 10.0
 }
 return score, true
 }
 return 0, false
}

// GetModelCodeCapabilityScore returns the code capability score
// for a model.
func (a *Adapter) GetModelCodeCapabilityScore(modelID string)
(float64, bool) {
 a.mu.RLock()

```

```

 defer a.mu.RUnlock()
 score, ok := a.modelCodeScores[modelID]
 return score, ok
}

// GetModelReliabilityScore returns the reliability score for a
// model.
func (a *Adapter) GetModelReliabilityScore(modelID string)
 (float64, bool) {
 a.mu.RLock()
 defer a.mu.RUnlock()
 score, ok := a.modelRelScores[modelID]
 return score, ok
}

// GetMinAcceptableScore returns the configured minimum score
// threshold.
func (a *Adapter) GetMinAcceptableScore() float64 {
 if a.config == nil {
 return 6.0
 }
 return a.config.Scoring.MinAcceptableScore
}

// GetVerifiedModels returns all models from verifier, filtered
// by provider config.
func (a *Adapter) GetVerifiedModels(ctx context.Context)
 ([]*VerifiedModel, error) {
 if !a.IsEnabled() {
 return nil, ErrVerifierDisabled
 }
 if !a.health.AllowRequest() {
 return a.getFallbackModels()
 }

 // Try cache first
 if a.cache != nil {
 if models, ok := a.cache.GetModels("all"); ok {
 return a.filterByProviderConfig(models), nil
 }
 }

 // Fetch from verifier
 models, err := a.client.GetModels(ctx)
 if err != nil {
 a.health.RecordFailure()
 // Try stale cache (up to 2x TTL)
 if a.cache != nil {

```

```

 if stale, ok := a.cache.GetModelsStale("all"); ok {
 return stale, ErrUsingStaleCache
 }
 }
 return a.getFallbackModels()
}

a.health.RecordSuccess()
a.refreshScores(models)

// Update cache
if a.cache != nil {
 a.cache.SetModels("all", models)
}

return a.filterByProviderConfig(models), nil
}

// GetProviderStatus returns the health and score status for a
// provider.
func (a *Adapter) GetProviderStatus(providerType string)
(*ProviderStatus, bool) {
 a.mu.RLock()
 defer a.mu.RUnlock()

 score, hasScore := a.providerScores[providerType]
 healthy := a.health.AllowRequest() && hasScore

 status := &ProviderStatus{
 Type: providerType,
 Score: score,
 Healthy: healthy,
 LastChecked: a.lastRefresh,
 }
 if hasScore {
 return status, true
 }
 return status, false
}

// ForceRefresh invalidates cache and re-fetches from verifier.
func (a *Adapter) ForceRefresh(ctx context.Context) error {
 if a.cache != nil {
 a.cache.Invalidate("all")
 }
 _, err := a.GetVerifiedModels(ctx)
 return err
}

```

*// refreshScores updates internal score maps from model list.*

```
func (a *Adapter) refreshScores(models []*VerifiedModel) {
 a.mu.Lock()
 defer a.mu.Unlock()

 providerBest := make(map[string]float64)

 for _, m := range models {
 a.modelScores[m.ID] = m.OverallScore
 a.modelCodeScores[m.ID] = m.CodeCapabilityScore
 a.modelRelScores[m.ID] = m.ReliabilityScore

 if current, ok := providerBest[m.Provider]; !ok ||
 m.OverallScore > current {
 providerBest[m.Provider] = m.OverallScore
 }
 }

 a.providerScores = providerBest
 a.lastRefresh = time.Now()
}
```

*// filterByProviderConfig removes models from disabled providers.*

```
func (a *Adapter) filterByProviderConfig(models
 []*VerifiedModel) []*VerifiedModel {
 if len(a.config.Providers) == 0 {
 return models
 }
 filtered := make(([]*VerifiedModel, 0, len(models))
 for _, m := range models {
 if pc, ok := a.config.Providers[m.Provider]; ok && !
 pc.Enabled {
 continue
 }
 filtered = append(filtered, m)
 }
 return filtered
}
```

*// getFallbackModels returns the hardcoded fallback list.*

```
func (a *Adapter) getFallbackModels() ([]*VerifiedModel, error) {
 result := make(([]*VerifiedModel, len(FallbackModels))
 for i, m := range FallbackModels {
 copy := *m
 copy.Source = "fallback"
 result[i] = ©
 }
}
```

```
 return result, ErrUsingFallback
}
```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles 2. NewAdapter(nil, nil, nil, &config.VerifierConfig{Enabled: false}).IsEnabled() returns false 3. GetModelScore("gpt-4o") returns score and true after refreshScores() 4. filterByProviderConfig() removes models from providers with Enabled: false 5. getFallbackModels() returns 7 models with Source: "fallback"

**Dependencies:** TASK 1.5.1, TASK 1.6.1, TASK 1.7.1 **Effort:** Large

---

## TASK 2.2: Create internal/verifier/discovery.go

### TASK 2.2.1: Implement ModelDiscoveryService

**File(s):** helix\_code/internal/verifier/discovery.go **Line(s):** CREATE new file **Action:** CREATE

```
package verifier

import (
 "context"
 "fmt"
 "sort"
 "strings"
 "sync"
 "time"
)

// ModelDiscoveryService discovers and filters models from
// LLMsVerifier.
// Replicates HelixAgent's internal/verifier/discovery.go
// pattern.
type ModelDiscoveryService struct {
 adapter *Adapter
 config *DiscoveryConfig
 discovered map[string]*VerifiedModel
 mu sync.RWMutex
 lastDiscover time.Time
}

// DiscoveryConfig controls discovery behavior.
type DiscoveryConfig struct {
 Enabled bool `yaml:"enabled"`
 DiscoveryInterval time.Duration `yaml:"discovery_interval"`
}
```

```

MaxModelsForEnsemble int
 `yaml:"max_models_for_ensemble"`
MinScore float64 `yaml:"min_score"`
RequireVerification bool
 `yaml:"require_verification"`
RequireCodeVisibility bool
 `yaml:"require_code_visibility"`
RequireDiversity bool
 `yaml:"require_diversity"`
ProviderPriority []string
 `yaml:"provider_priority"`
}

// NewModelDiscoveryService creates a discovery service.
func NewModelDiscoveryService(adapter *Adapter, cfg
 *DiscoveryConfig) *ModelDiscoveryService {
 return &ModelDiscoveryService{
 adapter: adapter,
 config: cfg,
 discovered: make(map[string]*VerifiedModel),
 }
}

// DiscoverModels fetches and filters models from verifier.
func (s *ModelDiscoveryService) DiscoverModels(ctx
 context.Context) ([]*VerifiedModel, error) {
 if s.adapter == nil || !s.adapter.IsEnabled() {
 return nil, ErrVerifierDisabled
 }

 models, err := s.adapter.GetVerifiedModels(ctx)
 if err != nil {
 return nil, err
 }

 filtered := s.applyFilters(models)
 sorted := s.applySorting(filtered)

 s.mu.Lock()
 s.discovered = make(map[string]*VerifiedModel, len(sorted))
 for _, m := range sorted {
 s.discovered[m.ID] = m
 }
 s.lastDiscover = time.Now()
 s.mu.Unlock()

 return sorted, nil
}

```



```

// GetDiscoveredModel returns a single model by ID.
func (s *ModelDiscoveryService) GetDiscoveredModel(modelID
 string) (*VerifiedModel, bool) {
 s.mu.RLock()
 defer s.mu.RUnlock()
 m, ok := s.discovered[modelID]
 return m, ok
}

// SelectOptimalModel selects the best model for a task using
// verifier scores.
func (s *ModelDiscoveryService) SelectOptimalModel(
 ctx context.Context,
 taskType string,
 requiredCapabilities []string,
 maxPrice float64,
 minScore float64,
) (*VerifiedModel, error) {
 models, err := s.DiscoverModels(ctx)
 if err != nil {
 return nil, err
 }

 candidates := make([]*VerifiedModel, 0, len(models))
 for _, m := range models {
 if minScore > 0 && m.OverallScore < minScore {
 continue
 }
 if maxPrice > 0 &&
 (m.CostPerInputToken+m.CostPerOutputToken)/2.0*1000 >
 maxPrice {
 continue
 }
 if s.config.RequireVerification && !m.Verified {
 continue
 }
 if !hasAllCapabilities(m, requiredCapabilities) {
 continue
 }
 candidates = append(candidates, m)
 }

 if len(candidates) == 0 {
 return nil, fmt.Errorf("no model matches criteria:
 task=%s, minScore=%.1f, maxPrice=%.2f",
 taskType, minScore, maxPrice)
 }
}

```

```

// Sort by score descending, then by latency ascending
sort.Slice(candidates, func(i, j int) bool {
 if candidates[i].OverallScore !=
 candidates[j].OverallScore {
 return candidates[i].OverallScore >
 candidates[j].OverallScore
 }
 return candidates[i].Latency < candidates[j].Latency
})

return candidates[0], nil
}

// applyFilters removes models that don't meet discovery config
// criteria.
func (s *ModelDiscoveryService) applyFilters(models
 []*VerifiedModel) []*VerifiedModel {
 if s.config == nil {
 return models
 }
 result := make([]*VerifiedModel, 0, len(models))
 for _, m := range models {
 if s.config.MinScore > 0 && m.OverallScore <
 s.config.MinScore {
 continue
 }
 if s.config.RequireVerification && !m.Verified {
 continue
 }
 result = append(result, m)
 }
 return result
}

// applySorting sorts models by provider priority, then score,
// then latency.
func (s *ModelDiscoveryService) applySorting(models
 []*VerifiedModel) []*VerifiedModel {
 if s.config == nil || len(s.config.ProviderPriority) == 0 {
 // Default: sort by score desc, tier asc, latency asc
 sort.Slice(models, func(i, j int) bool {
 if models[i].OverallScore != models[j].OverallScore {
 return models[i].OverallScore >
 models[j].OverallScore
 }
 if models[i].Tier != models[j].Tier {
 return models[i].Tier < models[j].Tier
 }

```

```

 }
 return models[i].Latency < models[j].Latency
 })
 return models
}

priorityMap := make(map[string]int)
for i, p := range s.config.ProviderPriority {
 priorityMap[p] = i
}

sort.Slice(models, func(i, j int) bool {
 pi, oki := priorityMap[models[i].Provider]
 pj, okj := priorityMap[models[j].Provider]
 if oki && okj {
 return pi < pj
 }
 if oki {
 return true
 }
 if okj {
 return false
 }
 return models[i].OverallScore > models[j].OverallScore
})
return models
}

// hasAllCapabilities checks if a model has all required
// capabilities.
func hasAllCapabilities(m *VerifiedModel, required []string)
 bool {
 if len(required) == 0 {
 return true
 }
 capSet := make(map[string]bool)
 for _, c := range m.Capabilities {
 capSet[strings.ToLower(c)] = true
 }
 for _, req := range required {
 if !capSet[strings.ToLower(req)] {
 return false
 }
 }
 return true
}

```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles 2. DiscoverModels() returns error when adapter is nil 3. SelectOptimalModel() filters by minScore, maxPrice, and capabilities 4. applySorting() sorts by score descending by default 5. hasAllCapabilities() returns false when required cap is missing

**Dependencies:** TASK 2.1.1 **Effort:** Large

---

## TASK 2.3: Create internal/verifier/poller.go

### TASK 2.3.1: Implement Background Polling for Real-Time Updates

**File(s):** helix\_code/internal/verifier/poller.go **Line(s):** CREATE new file  
**Action:** CREATE

```
package verifier
```

```
import (
 "context"
 "sync"
 "time"
)
```

```
// Poller runs a background goroutine that polls LLMsVerifier at
 a configurable interval.
```

```
// Since LLMsVerifier has no push/webhook support, polling is the
 only real-time mechanism.
```

```
type Poller struct {
 adapter *Adapter
 interval time.Duration
 ticker *time.Ticker
 stopCh chan struct{}
 wg sync.WaitGroup
 lastModels map[string]*VerifiedModel
 lastScores map[string]float64
 mu sync.RWMutex
 pollCount int
}
```

```
// NewPoller creates a poller. Minimum interval is 10s
 (enforced).
```

```
func NewPoller(adapter *Adapter, interval time.Duration) *Poller
{
 if interval < 10*time.Second {
 interval = 10 * time.Second
 }
 return &Poller{
```

```

 adapter: adapter,
 interval: interval,
 stopCh: make(chan struct{}),
 lastModels: make(map[string]*VerifiedModel),
 lastScores: make(map[string]float64),
 }
}

```

*// Start begins the background polling goroutine.*

```

func (p *Poller) Start() {
 p.wg.Add(1)
 go p.loop()
}

```

*// Stop signals the poller to stop and waits for the goroutine to exit.*

```

func (p *Poller) Stop() {
 close(p.stopCh)
 p.wg.Wait()
}

```

*// IsRunning returns true if the poller goroutine is active.*

```

func (p *Poller) IsRunning() bool {
 select {
 case <-p.stopCh:
 return false
 default:
 return true
 }
}

```

```

func (p *Poller) loop() {
 defer p.wg.Done()

```

*// Immediate first poll*

```
p.poll()
```

```
p.ticker = time.NewTicker(p.interval)
```

```
defer p.ticker.Stop()
```

```

for {
 select {
 case <-p.ticker.C:
 p.poll()
 case <-p.stopCh:
 return
 }
}

```

```
}
```

```

}

func (p *Poller) poll() {
 if p.adapter == nil || !p.adapter.IsEnabled() {
 return
 }

 ctx, cancel := context.WithTimeout(context.Background(),
 30*time.Second)
 defer cancel()

 // 1. Fetch all models
 models, err := p.adapter.client.GetModels(ctx)
 if err != nil {
 p.adapter.health.RecordFailure()
 return
 }

 // 2. Detect changes
 p.mu.Lock()
 changes := p.detectChanges(p.lastModels, models)
 p.lastModels = indexModels(models)
 p.mu.Unlock()

 // 3. Update adapter state
 p.adapter.health.RecordSuccess()
 p.adapter.refreshScores(models)

 // 4. Update cache
 if p.adapter.cache != nil {
 p.adapter.cache.SetModels("all", models)
 }

 // 5. Publish events for changes
 if p.adapter.events != nil {
 for _, change := range changes {
 _ = p.adapter.events.Publish(change)
 }
 }

 // 6. Fetch scores every 3rd poll (scores change slower than
 // model lists)
 p.pollCount++
 if p.pollCount%3 == 0 {
 scores, _ := p.adapter.client.GetProviderScores(ctx)
 if p.adapter.cache != nil && scores != nil {
 p.adapter.cache.SetScores(scores)
 }
 }
}

```

```

 p.mu.Lock()
 p.lastScores = scores
 p.mu.Unlock()
 }
}

func (p *Poller) detectChanges(old, new []*VerifiedModel)
 []ChangeEvent {
 changes := []ChangeEvent{}
 newIndex := indexModels(new)

 for id, model := range newIndex {
 if oldModel, ok := old[id]; !ok {
 changes = append(changes, ChangeEvent{
 Type: "model.discovered",
 Model: model,
 Timestamp: time.Now(),
 })
 } else {
 if oldModel.OverallScore != model.OverallScore {
 changes = append(changes, ChangeEvent{
 Type: "model.score_changed",
 Model: model,
 OldScore: oldModel.OverallScore,
 Timestamp: time.Now(),
 })
 }
 if oldModel.VerificationStatus !=
model.VerificationStatus {
 changes = append(changes, ChangeEvent{
 Type: "model.status_changed",
 Model: model,
 OldStatus: oldModel.VerificationStatus,
 Timestamp: time.Now(),
 })
 }
 }
 }
}

for id, model := range old {
 if _, ok := newIndex[id]; !ok {
 changes = append(changes, ChangeEvent{
 Type: "model.removed",
 Model: model,
 Timestamp: time.Now(),
 })
 }
}
}

```

```

 return changes
}

func indexModels(models []*VerifiedModel)
 map[string]*VerifiedModel {
 idx := make(map[string]*VerifiedModel, len(models))
 for _, m := range models {
 idx[m.ID] = m
 }
 return idx
}

```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles 2. NewPoller(nil, 5\*time.Second) enforces minimum 10s interval 3. Start() + Stop() sequence completes without deadlock 4. detectChanges() detects added, removed, score-changed, and status-changed models 5. pollCount%3 == 0 triggers score fetch every 3rd poll

**Dependencies:** TASK 2.1.1 **Effort:** Large

---

## TASK 2.4: Create internal/verifier/cache.go

### TASK 2.4.1: Implement Two-Tier Cache (LRU + Redis)

**File(s):** helix\_code/internal/verifier/cache.go **Line(s):** CREATE new file  
**Action:** CREATE

```

package verifier

import (
 "context"
 "encoding/json"
 "sync"
 "time"

 lru "github.com/hashicorp/golang-lru/v2"
 "github.com/redis/go-redis/v9"
)

// Cache implements a two-tier cache: L1 in-memory LRU + L2
// Redis.
type Cache struct {
 l1 *lru.Cache[string, *CacheEntry]
 l2 *redis.Client // may be nil if Redis is not configured
 ttl time.Duration
 mu sync.RWMutex
}

```



```

}

// CacheEntry stores cached verifier data with timestamp.
type CacheEntry struct {
 Models []*VerifiedModel `json:"models"`
 Scores map[string]float64 `json:"scores"`
 FetchedAt time.Time `json:"fetched_at"`
 Source string `json:"source" // "verifier",
 "fallback", "embedded"
}

// NewCache creates a two-tier cache.
// ttl: cache entry lifetime
// redisClient: optional Redis client (nil = memory-only)
func NewCache(ttl time.Duration, redisClient *redis.Client)
 *Cache {
 l1Cache, _ := lru.New[string, *CacheEntry](1024)
 return &Cache{
 l1: l1Cache,
 l2: redisClient,
 ttl: ttl,
 }
}

// GetModels retrieves cached models from L1 or L2.
func (c *Cache) GetModels(provider string) ([]*VerifiedModel,
 bool) {
 c.mu.RLock()
 defer c.mu.RUnlock()

 // Try L1
 if entry, ok := c.l1.Get(provider); ok {
 if time.Since(entry.FetchedAt) < c.ttl {
 return entry.Models, true
 }
 }

 // Try L2 (Redis)
 if c.l2 != nil {
 ctx, cancel := context.WithTimeout(context.Background(),
 5*time.Second)
 defer cancel()
 data, err := c.l2.Get(ctx,
 "helix:verifier:models:"+provider).Result()
 if err == nil {
 var entry CacheEntry
 if json.Unmarshal([]byte(data), &entry) == nil {
 if time.Since(entry.FetchedAt) < c.ttl {

```

```

 // Backfill L1
 c.l1.Add(provider, &entry)
 return entry.Models, true
 }
}
}
return nil, false
}

// GetModelsStale retrieves models even if past TTL (for circuit
// breaker fallback).
func (c *Cache) GetModelsStale(provider string)
 ([]*VerifiedModel, bool) {
 c.mu.RLock()
 defer c.mu.RUnlock()

 // Try L1 (accept even if stale, up to 2x TTL)
 if entry, ok := c.l1.Get(provider); ok {
 if time.Since(entry.FetchedAt) < c.ttl*2 {
 return entry.Models, true
 }
 }

 // Try L2 (same relaxed TTL)
 if c.l2 != nil {
 ctx, cancel := context.WithTimeout(context.Background(),
 5*time.Second)
 defer cancel()
 data, err := c.l2.Get(ctx,
 "helix:verifier:models:"+provider).Result()
 if err == nil {
 var entry CacheEntry
 if json.Unmarshal([]byte(data), &entry) == nil {
 if time.Since(entry.FetchedAt) < c.ttl*2 {
 return entry.Models, true
 }
 }
 }
 }
 return nil, false
}

// SetModels stores models in both L1 and L2.
func (c *Cache) SetModels(provider string, models
 []*VerifiedModel) {
 entry := &CacheEntry{
 Models: models,

```

```

 FetchedAt: time.Now(),
 Source: "verifier",
 }

 c.mu.Lock()
 c.l1.Add(provider, entry)
 c.mu.Unlock()

 if c.l2 != nil {
 data, _ := json.Marshal(entry)
 ctx, cancel := context.WithTimeout(context.Background(),
 5*time.Second)
 defer cancel()
 c.l2.Set(ctx, "helix:verifier:models:"+provider, data,
 c.ttl)
 }
}

// GetModelScore retrieves a cached model score.
func (c *Cache) GetModelScore(modelID string) (float64, bool) {
 // Scores are stored with provider="scores" key
 c.mu.RLock()
 defer c.mu.RUnlock()
 if entry, ok := c.l1.Get("scores"); ok {
 if time.Since(entry.FetchedAt) < c.ttl {
 if score, ok := entry.Scores[modelID]; ok {
 return score, true
 }
 }
 }
 return 0, false
}

// SetScores stores provider/model scores.
func (c *Cache) SetScores(scores map[string]float64) {
 entry := &CacheEntry{
 Scores: scores,
 FetchedAt: time.Now(),
 Source: "verifier",
 }

 c.mu.Lock()
 c.l1.Add("scores", entry)
 c.mu.Unlock()

 if c.l2 != nil {
 data, _ := json.Marshal(entry)

```

```

 ctx, cancel := context.WithTimeout(context.Background(),
 5*time.Second)
 defer cancel()
 c.l2.Set(ctx, "helix:verifier:scores", data, c.ttl)
 }
}

// Invalidate removes a cache entry from both tiers.
func (c *Cache) Invalidate(provider string) {
 c.mu.Lock()
 c.l1.Remove(provider)
 c.mu.Unlock()

 if c.l2 != nil {
 ctx, cancel := context.WithTimeout(context.Background(),
 5*time.Second)
 defer cancel()
 c.l2.Del(ctx, "helix:verifier:models:"+provider)
 }
}

```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles 2. NewCache(5\*time.Minute, nil) creates memory-only cache 3. GetModels("all") returns hit after SetModels("all", ...) 4. GetModelsStale("all") returns hit up to 2x TTL after expiry 5. Invalidate("all") removes from both L1 and L2

**Dependencies:** TASK 1.5.1 **Effort:** Large

---

## TASK 2.5: Create internal/verifier/health.go

### TASK 2.5.1: Implement Circuit Breaker and Health Monitor

**File(s):** helix\_code/internal/verifier/health.go **Line(s):** CREATE new file  
**Action:** CREATE

```

package verifier

import (
 "sync"
 "time"

 "dev.helix.code/internal/config"
)

// CircuitState represents the circuit breaker state.
type CircuitState int

```

```

const (
 CircuitClosed CircuitState = iota
 CircuitOpen
 CircuitHalfOpen
)

// HealthMonitor tracks verifier service health with circuit
// breaker pattern.
type HealthMonitor struct {
 state CircuitState
 failures int
 successes int
 lastFailureTime time.Time
 checkInterval time.Duration
 timeout time.Duration
 failureThreshold int
 recoveryThreshold int
 halfOpenTimeout time.Duration
 enabled bool
 mu sync.RWMutex
}

// NewHealthMonitor creates a health monitor from config.
func NewHealthMonitor(cfg config.VerifierHealthConfig)
 *HealthMonitor {
 return &HealthMonitor{
 checkInterval: cfg.CheckInterval,
 timeout: cfg.Timeout,
 failureThreshold: cfg.FailureThreshold,
 recoveryThreshold: cfg.RecoveryThreshold,
 halfOpenTimeout: cfg.CircuitBreaker.HalfOpenTimeout,
 enabled: cfg.CircuitBreaker.Enabled,
 state: CircuitClosed,
 }
}

// RecordFailure increments failure count and may open the
// circuit.
func (h *HealthMonitor) RecordFailure() {
 h.mu.Lock()
 defer h.mu.Unlock()

 h.failures++
 h.lastFailureTime = time.Now()

 if h.enabled && h.state != CircuitOpen && h.failures >=
 h.failureThreshold {

```

```

 h.state = CircuitOpen
 }
}

// RecordSuccess increments success count and may close the
// circuit.
func (h *HealthMonitor) RecordSuccess() {
 h.mu.Lock()
 defer h.mu.Unlock()

 switch h.state {
 case CircuitHalfOpen:
 h.successes++
 if h.successes >= h.recoveryThreshold {
 h.state = CircuitClosed
 h.failures = 0
 h.successes = 0
 }
 case CircuitClosed:
 h.failures = 0 // reset on success in closed state
 }
}

// AllowRequest returns true if the circuit allows a request
// through.
func (h *HealthMonitor) AllowRequest() bool {
 h.mu.RLock()
 defer h.mu.RUnlock()

 switch h.state {
 case CircuitClosed:
 return true
 case CircuitOpen:
 if time.Since(h.lastFailureTime) > h.halfOpenTimeout {
 // Transition to half-open (requires write lock)
 h.mu.RUnlock()
 h.mu.Lock()
 if h.state == CircuitOpen { // double-check
 h.state = CircuitHalfOpen
 h.successes = 0
 }
 h.mu.Unlock()
 h.mu.RLock()
 return true
 }
 return false
 case CircuitHalfOpen:
 return true
 }
}

```

```

 }
 return false
}

// IsCircuitOpen returns true if the circuit is open.
func (h *HealthMonitor) IsCircuitOpen() bool {
 h.mu.RLock()
 defer h.mu.RUnlock()
 return h.state == CircuitOpen
}

// GetState returns the current circuit state.
func (h *HealthMonitor) GetState() CircuitState {
 h.mu.RLock()
 defer h.mu.RUnlock()
 return h.state
}

// IsHealthy returns true if the circuit is closed.
func (h *HealthMonitor) IsHealthy() bool {
 return h.GetState() == CircuitClosed
}

```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles 2. RecordFailure() x5 with failureThreshold=5 opens circuit 3. AllowRequest() returns false when circuit is open 4. After halfOpenTimeout, AllowRequest() returns true (half-open) 5. RecordSuccess() x3 in half-open closes circuit

**Dependencies:** TASK 1.2.1 **Effort:** Large

---

## TASK 2.6: Create internal/verifier/events.go

### TASK 2.6.1: Implement Event Publishing to HelixCode Event Bus

**File(s):** helix\_code/internal/verifier/events.go **Line(s):** CREATE new file  
**Action:** CREATE

```

package verifier

import (
 "encoding/json"
 "sync"

 "dev.helix.code/internal/event"
)

```

```

// Event topic constants for verifier events.
const (
 TopicVerifierModelDiscovered =
 "helix.verifier.model.discovered"
 TopicVerifierModelUpdated = "helix.verifier.model.updated"
 TopicVerifierModelRemoved = "helix.verifier.model.removed"
 TopicVerifierProviderHealth =
 "helix.verifier.provider.health"
 TopicVerifierScoreChanged = "helix.verifier.score.changed"
 TopicVerifierDegraded = "helix.verifier.degraded"
 TopicVerifierRecovered = "helix.verifier.recovered"
)

// EventPublisher publishes verifier changes to the HelixCode
// event bus.
type EventPublisher struct {
 bus event.Bus
 enabled bool
 websocket bool
 wsPath string
 mu sync.RWMutex
}

// NewEventPublisher creates an event publisher.
func NewEventPublisher(bus event.Bus, enabled, websocket bool,
 wsPath string) *EventPublisher {
 return &EventPublisher{
 bus: bus,
 enabled: enabled,
 websocket: websocket,
 wsPath: wsPath,
 }
}

// Publish sends a change event to the event bus.
func (ep *EventPublisher) Publish(change ChangeEvent) error {
 ep.mu.RLock()
 if !ep.enabled {
 ep.mu.RUnlock()
 return nil
 }
 ep.mu.RUnlock()

 var topic string
 switch change.Type {
 case "model.discovered":
 topic = TopicVerifierModelDiscovered
 case "model.score_changed":

```



```

 topic = TopicVerifierScoreChanged
 case "model.status_changed":
 topic = TopicVerifierModelUpdated
 case "model.removed":
 topic = TopicVerifierModelRemoved
 case "provider.health":
 topic = TopicVerifierProviderHealth
 case "degraded":
 topic = TopicVerifierDegraded
 case "recovered":
 topic = TopicVerifierRecovered
 default:
 topic = TopicVerifierModelUpdated
}

data, err := json.Marshal(change)
if err != nil {
 return err
}

if ep.bus != nil {
 return ep.bus.Publish(topic, data)
}
return nil
}

// IsEnabled returns the current enabled state.
func (ep *EventPublisher) IsEnabled() bool {
 ep.mu.RLock()
 defer ep.mu.RUnlock()
 return ep.enabled
}

// SetEnabled updates the enabled state.
func (ep *EventPublisher) SetEnabled(enabled bool) {
 ep.mu.Lock()
 defer ep.mu.Unlock()
 ep.enabled = enabled
}

```

**Acceptance Criteria:** 1. go build ./internal/verifier/... compiles 2. Publish() with enabled=false returns nil without calling bus 3. Publish() maps all 6 change types to correct topics 4. SetEnabled() toggles publishing on/off

**Dependencies:** TASK 1.5.1 **Effort:** Medium

---

## TASK 2.7: Modify internal/llm/model\_discovery.go

### TASK 2.7.1: Replace fetchExternalModels() Hardcoded List with Verifier Fetch

**File(s):** helix\_code/internal/llm/model\_discovery.go **Line(s):** Search for func (e \*ModelDiscoveryEngine) fetchExternalModels() **Action:** MODIFY

Add new field to ModelDiscoveryEngine struct (search for struct definition):

```
type ModelDiscoveryEngine struct {
 // ... existing fields ...
 verifierAdapter *verifier.Adapter // NEW – LLMsVerifier
 bridge
 logger *logrus.Logger // NEW – structured logging
}
```

Replace the fetchExternalModels() function:

```
// fetchExternalModels retrieves external models from
 LLMsVerifier when enabled.
// Replaces the previous hardcoded list (BLUFF-002 fix).
func (e *ModelDiscoveryEngine) fetchExternalModels(ctx
 context.Context) []*ModelInfo {
 // If verifier adapter is available and enabled, use it as
 // the single source of truth
 if e.verifierAdapter != nil && e.verifierAdapter.IsEnabled()
 {
 verifiedModels, err :=
 e.verifierAdapter.GetVerifiedModels(ctx)
 if err == nil && len(verifiedModels) > 0 {
 return e.convertVerifiedToModelInfo(verifiedModels)
 }
 if err != nil && err != verifier.ErrVerifierDisabled {
 e.logger.Warnf("Verifier fetch failed for external
 models: %v", err)
 }
 }

 // Fallback: return empty – local providers (Ollama,
 // llama.cpp) will still be discovered
 // The hardcoded list has been REMOVED per CONST-036.
 return []*ModelInfo{}
}

// convertVerifiedToModelInfo transforms verifier models into
 HelixCode ModelInfo.
```

```

func (e *ModelDiscoveryEngine)
 convertVerifiedToModelInfo(verified
 []*verifier.VerifiedModel) []*ModelInfo {
 result := make([]*ModelInfo, 0, len(verified))
 for _, v := range verified {
 mi := &ModelInfo{
 ID: v.ID,
 Name: v.DisplayName,
 Format: FormatUnknown, // Verifier doesn't
 track GGUF vs HF
 Size: 0, // Not provided by
 verifier
 ContextSize: v.ContextSize,
 MaxTokens: v.MaxOutputTokens,
 Provider: v.Provider,
 Verified: v.Verified,
 Score: v.OverallScore,
 Capabilities: e.mapCapabilities(v),
 Source: v.Source,
 }
 result = append(result, mi)
 }
 return result
}

```

*// mapCapabilities converts verifier capability flags to ModelCapability slice.*

```

func (e *ModelDiscoveryEngine) mapCapabilities(v
 *verifier.VerifiedModel) []ModelCapability {
 caps := []ModelCapability{}
 if v.SupportsCode {
 caps = append(caps, CapabilityCodeGeneration)
 }
 if v.SupportsTools || v.SupportsFunctions {
 caps = append(caps, CapabilityToolUse)
 }
 if v.SupportsStreaming {
 caps = append(caps, CapabilityStreaming)
 }
 if v.SupportsVision {
 caps = append(caps, CapabilityVision)
 }
 if v.SupportsReasoning {
 caps = append(caps, CapabilityReasoning)
 }
 if v.SupportsEmbeddings {
 caps = append(caps, CapabilityEmbeddings)
 }
}

```

```
 return caps
}
```

**Acceptance Criteria:** 1. go build ./internal/llm/... compiles 2. fetchExternalModels() no longer contains hardcoded []\*ModelInfo{{ID: "llama-3-8b-instruct"...}} 3. grep "llama-3-8b-instruct" internal/llm/model\_discovery.go returns zero matches 4. convertVerifiedToModelInfo() maps all capability flags 5. mapCapabilities() returns CapabilityCodeGeneration when v.SupportsCode is true

**Dependencies:** TASK 2.1.1 **Effort:** Large

---

## TASK 2.8: Modify internal/llm/model\_manager.go

### TASK 2.8.1: Augment SelectOptimalModel() with Verifier Scores

**File(s):** helix\_code/internal/llm/model\_manager.go **Line(s):** Inside SelectOptimalModel() method, after task type suitability filter **Action:** MODIFY

Add new field to ModelManager struct:

```
type ModelManager struct {
 // ... existing fields ...
 verifierAdapter *verifier.Adapter // NEW - LLMsVerifier bridge
}
```

Insert verifier score augmentation into SelectOptimalModel():

```
func (m *ModelManager) SelectOptimalModel(criteria
 ModelSelectionCriteria) (*ModelInfo, error) {
 candidates :=
 m.filterByCapabilities(criteria.RequiredCapabilities)
 candidates = m.filterByContextSize(candidates,
 criteria.MaxTokens)
 candidates = m.filterByTaskType(candidates,
 criteria.TaskType)

 // NEW: Incorporate LLMsVerifier scores into ranking
 if m.verifierAdapter != nil && m.verifierAdapter.IsEnabled()
 {
 candidates = m.rankByVerifierScores(candidates, criteria)
 }

 candidates = m.filterByHardwareCompatibility(candidates)
```

```

candidates = m.applyQualityPreference(candidates,
 criteria.QualityPreference)

if len(candidates) == 0 {
 return nil, ErrNoSuitableModel
}
return candidates[0], nil
}

// rankByVerifierScores blends verifier scores with local
// heuristic scores.
func (m *ModelManager) rankByVerifierScores(
 candidates []*ModelInfo,
 criteria ModelSelectionCriteria,
) []*ModelInfo {
 scored := make([]*struct {
 model *ModelInfo
 score float64
 }, 0, len(candidates))

 for _, c := range candidates {
 baseScore := c.Score // existing heuristic score (0-1)

 // Fetch verifier score for this specific model
 verifierScore, found :=
 m.verifierAdapter.GetModelScore(c.ID)
 if found {
 // Weighted blend: 60% verifier, 40% local heuristic
 // Verifier score is 0-10; normalize to 0-1
 normalizedVerifier := verifierScore / 10.0
 blendedScore := (normalizedVerifier * 0.6) +
 (baseScore * 0.4)

 // Apply task-specific boost from verifier dimensions
 switch criteria.TaskType {
 case "code_generation", "debugging", "refactoring",
 "testing":
 codeScore, hasCode :=
 m.verifierAdapter.GetModelCodeCapabilityScore(c.ID)
 if hasCode {
 blendedScore += (codeScore / 10.0) * 0.15 //
 +15% code capability bonus
 }
 case "planning", "analysis", "architecture":
 relScore, hasRel :=
 m.verifierAdapter.GetModelReliabilityScore(c.ID)
 if hasRel {

```

```

 blendedScore += (relScore / 10.0) * 0.10 //
+10% reliability bonus
 }
}

baseScore = blendedScore
}

scored = append(scored, struct {
 model *ModelInfo
 score float64
}{c, baseScore})
}

sort.Slice(scored, func(i, j int) bool {
 return scored[i].score > scored[j].score
})

result := make([]*ModelInfo, len(scored))
for i, s := range scored {
 result[i] = s.model
}
return result
}

```

**Acceptance Criteria:** 1. go build ./internal/llm/... compiles 2. SelectOptimalModel() calls rankByVerifierScores() when adapter is enabled 3. rankByVerifierScores() gives 60% weight to verifier score, 40% to local heuristic 4. Task type “code\_generation” adds 15% code capability bonus 5. Task type “planning” adds 10% reliability bonus

**Dependencies:** TASK 2.1.1 **Effort:** Large

---

## TASK 2.9: Modify cmd/cli/main.go

### TASK 2.9.1: Replace handleListModels() Hardcoded Models with Dynamic Fetch

**File(s):** helix\_code/cmd/cli/main.go **Line(s):** Lines 101-128 (the handleListModels method) **Action:** MODIFY

Replace the entire handleListModels() method:

```

func (c *CLI) handleListModels(ctx context.Context) error {
 // Use verifier adapter if enabled, otherwise fall through to
 model manager

```

```

if c.verifierAdapter != nil && c.verifierAdapter.IsEnabled()
{
 models, err := c.verifierAdapter.GetVerifiedModels(ctx)
 if err == nil && len(models) > 0 {
 return c.printVerifiedModels(models)
 }
 // Log warning but don't fail – try fallback
 c.logger.Warnf("Verifier model fetch failed, using
 fallback: %v", err)
}

// Fallback to existing ModelManager (already initialized,
// may have local models)
if c.modelManager != nil {
 models := c.modelManager.GetAvailableModels()
 return c.printManagerModels(models)
}

// Ultimate fallback: static list (for bootstrapping before
// any provider is ready)
return c.printFallbackModels()
}

func (c *CLI) printVerifiedModels(models
 []*verifier.VerifiedModel) error {
 fmt.Println("Available Models (from LLMsVerifier):")
 fmt.Println(strings.Repeat("-", 80))
 fmt.Printf("%-24s %-20s %-10s %-12s %s\n", "ID", "Name",
 "Provider", "Score", "Status")
 for _, m := range models {
 status := "verified"
 if !m.Verified {
 status = "pending"
 }
 if m.VerificationStatus == "failed" {
 status = "failed"
 }
 if m.VerificationStatus == "rate_limited" {
 status = "rate-limited"
 }
 scoreStr := fmt.Sprintf("SC:%.1f", m.OverallScore)
 fmt.Printf("%-24s %-20s %-10s %-12s %s\n", m.ID, m.Name,
 m.Provider, scoreStr, status)
 }
 return nil
}

func (c *CLI) printManagerModels(models []*llm.ModelInfo) error {

```

```

 fmt.Println("Available Models (from providers):")
 fmt.Println(strings.Repeat("-", 80))
 fmt.Printf("%-24s %-20s %-10s %-12s %s\n", "ID", "Name",
 "Provider", "Context", "Status")
 for _, m := range models {
 status := "available"
 if !m.Verified {
 status = "unverified"
 }
 fmt.Printf("%-24s %-20s %-10s %-12d %s\n", m.ID, m.Name,
 m.Provider, m.ContextSize, status)
 }
 return nil
}

func (c *CLI) printFallbackModels() error {
 fmt.Println("Available Models (fallback list):")
 fmt.Println(strings.Repeat("-", 80))
 for _, m := range verifier.FallbackModels {
 fmt.Printf("%-24s %-20s %-10s %-12s %s\n", m.ID, m.Name,
 m.Provider, "fallback", "available")
 }
 return nil
}

```

**Acceptance Criteria:** 1. go build ./cmd/cli/... compiles 2. grep "llama-3-8b" cmd/cli/main.go returns zero matches (old hardcoded list removed) 3. handleListModels() calls verifierAdapter.GetVerifiedModels() first 4. printVerifiedModels() includes score suffix SC:X.X 5. All three print methods exist: printVerifiedModels, printManagerModels, printFallbackModels

**Dependencies:** TASK 2.1.1 **Effort:** Large

---

## TASK 2.10: Modify internal/llm/factory.go

### TASK 2.10.1: Add Verifier-Aware Provider Validation

**File(s):** helix\_code/internal/llm/factory.go **Line(s):** After provider creation, before return provider, nil **Action:** MODIFY

```

func NewProvider(config ProviderConfigEntry) (Provider, error) {
 var provider Provider
 var err error

 switch config.Type {

```



```

case ProviderTypeOpenAI: provider, err =
 NewOpenAIProvider(config)
case ProviderTypeAnthropic: provider, err =
 NewAnthropicProvider(config)
case ProviderTypeGemini: provider, err =
 NewGeminiProvider(config)
case ProviderTypeOllama: provider, err =
 NewOllamaProvider(config)
case ProviderTypeLlamaCpp: provider, err =
 NewLlamaCPPProvider(config)
case ProviderTypeQwen: provider, err =
 NewQwenProvider(config)
case ProviderTypeXAI: provider, err =
 NewXAIPProvider(config)
case ProviderTypeOpenRouter: provider, err =
 NewOpenRouterProvider(config)
case ProviderTypeAzure: provider, err =
 NewAzureProvider(config)
case ProviderTypeBedrock: provider, err =
 NewBedrockProvider(config)
case ProviderTypeVertexAI: provider, err =
 NewVertexAIProvider(config)
case ProviderTypeGroq: provider, err =
 NewGroqProvider(config)
case ProviderTypeVLLM: provider, err =
 NewVLLMProvider(config)
case ProviderTypeLocalAI: provider, err =
 NewLocalAIProvider(config)
// ... existing cases ...
default:
 return nil, fmt.Errorf("unsupported provider type: %s",
 config.Type)
}

if err != nil {
 return nil, err
}

// NEW: Validate provider against LLMsVerifier registry
if verifierAdapter != nil && verifierAdapter.IsEnabled() {
 status, found :=
 verifierAdapter.GetProviderStatus(string(config.Type))
 if found {
 if !status.Healthy {
 return nil, fmt.Errorf("provider %s is marked
unhealthy by verifier (score: %.1f)",
 config.Type, status.Score)
 }
 }
}

```

```

 if status.Score <
 verifierAdapter.GetMinAcceptableScore() {
 return nil, fmt.Errorf("provider %s score %.1f
 below minimum %.1f",
 config.Type, status.Score,
 verifierAdapter.GetMinAcceptableScore())
 }
 }
 }

 return provider, nil
}

```

**Acceptance Criteria:** 1. go build ./internal/llm/... compiles 2. NewProvider() returns error when verifier marks provider unhealthy 3. NewProvider() returns error when provider score is below min\_acceptable\_score 4. When verifier is disabled, factory behavior is unchanged

**Dependencies:** TASK 2.1.1 **Effort:** Medium

---

## TASK 2.11: Create internal/llm/verifier\_integration.go

### TASK 2.11.1: Create Bridge File Between llm and verifier Packages

**File(s):** helix\_code/internal/llm/verifier\_integration.go **Line(s):** CREATE new file **Action:** CREATE

```

package llm

import (
 "context"
 "dev.helix.code/internal/verifier"
)

// VerifierModelSource implements the model source interface for
// the discovery engine.
// It delegates to the verifier adapter to fetch real-time model
// data.
type VerifierModelSource struct {
 adapter *verifier.Adapter
}

// NewVerifierModelSource creates a verifier-backed model source.
func NewVerifierModelSource(adapter *verifier.Adapter)
 *VerifierModelSource {
 return &VerifierModelSource{adapter: adapter}
}

```

```

}

// IsAvailable returns true if the verifier adapter is enabled
// and reachable.
func (s *VerifierModelSource) IsAvailable() bool {
 return s.adapter != nil && s.adapter.IsEnabled() &&
 s.adapter.IsReachable()
}

// FetchModels retrieves models from the verifier.
func (s *VerifierModelSource) FetchModels(ctx context.Context)
 ([]*ModelInfo, error) {
 verified, err := s.adapter.GetVerifiedModels(ctx)
 if err != nil {
 return nil, err
 }
 return s.convert(verified), nil
}

// convert transforms VerifiedModel into HelixCode ModelInfo.
func (s *VerifierModelSource) convert(verified
 []*verifier.VerifiedModel) []*ModelInfo {
 result := make([]*ModelInfo, 0, len(verified))
 for _, v := range verified {
 mi := &ModelInfo{
 ID: v.ID,
 Name: v.DisplayName,
 Format: FormatUnknown,
 Size: 0,
 ContextSize: v.ContextSize,
 MaxTokens: v.MaxOutputTokens,
 Provider: v.Provider,
 Verified: v.Verified,
 Score: v.OverallScore,
 Source: v.Source,
 }
 // Map capabilities
 if v.SupportsCode {
 mi.Capabilities = append(mi.Capabilities,
 CapabilityCodeGeneration)
 }
 if v.SupportsTools || v.SupportsFunctions {
 mi.Capabilities = append(mi.Capabilities,
 CapabilityToolUse)
 }
 if v.SupportsStreaming {
 mi.Capabilities = append(mi.Capabilities,
 CapabilityStreaming)
 }
 }
}

```

```

 }
 if v.SupportsVision {
 mi.Capabilities = append(mi.Capabilities,
 CapabilityVision)
 }
 if v.SupportsReasoning {
 mi.Capabilities = append(mi.Capabilities,
 CapabilityReasoning)
 }
 if v.SupportsEmbeddings {
 mi.Capabilities = append(mi.Capabilities,
 CapabilityEmbeddings)
 }
 result = append(result, mi)
}
return result
}

```

**Acceptance Criteria:** 1. `go build ./internal/llm/...` compiles 2. `IsAvailable()` returns false when adapter is nil 3. `FetchModels()` returns `[]*ModelInfo` with correct capability mapping 4. `convert()` handles all verifier capability flags

**Dependencies:** TASK 2.1.1, TASK 2.7.1 **Effort:** Medium

---

## TASK 2.12: Add Provider-Specific Enable/Disable and Real-Time Updates

### TASK 2.12.1: Implement Provider Enable Truth Table

**File(s):** `helix_code/internal/verifier/adapter.go` **Line(s):** Already implemented in `filterByProviderConfig()` — add truth table documentation

**Action:** DOCUMENT

The truth table is enforced by `filterByProviderConfig()` in `adapter.go`:

```

// Provider Enable State Resolution:
// verifier.providers.X.enabled ×
// helixcode.providers.X.enabled = Effective
// _____
// true ×
// true = ENABLED
// true ×
// false = DISABLED
// false ×
// true = DISABLED

```

<code>// false</code>	<code>x</code>	<code>= DISABLED</code>
<code>// (not set)</code>	<code>x</code>	<code>= ENABLED (inherits)</code>
<code>// (not set)</code>	<code>x</code>	<code>= DISABLED (inherits)</code>

**Acceptance Criteria:** 1. Truth table is documented as comment in `adapter.go` 2. `filterByProviderConfig()` correctly implements the AND logic

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## PHASE 2 ROLLBACK PLAN

```
cd HelixCode
git checkout -- internal/llm/model_discovery.go
git checkout -- internal/llm/model_manager.go
git checkout -- cmd/cli/main.go
git checkout -- internal/llm/factory.go
git rm --cached internal/verifier/adapter.go internal/verifier/
 discovery.go \
 internal/verifier/poller.go internal/verifier/cache.go \
 internal/verifier/health.go internal/verifier/events.go \
 internal/llm/verifier_integration.go
rm -f internal/verifier/adapter.go internal/verifier/
 discovery.go \
 internal/verifier/poller.go internal/verifier/cache.go \
 internal/verifier/health.go internal/verifier/events.go \
 internal/llm/verifier_integration.go
go mod tidy
make build
make test-unit
```

## PHASE 2 VERIFICATION CHECKLIST

- ☐ `go build ./...` compiles with zero errors
- ☐ `make test-unit` passes
- ☐ `grep "llama-3-8b" cmd/cli/main.go` returns ZERO matches (BLUFF-002 fixed)
- ☐ `grep "llama-3-8b-instruct" internal/llm/model_discovery.go` returns ZERO matches
- ☐

- ☐ `Adapter.GetVerifiedModels()` returns models with non-empty IDs
  - ☐ `HealthMonitor.RecordFailure()` x5 opens circuit breaker
  - ☐ `Poller.Start() + Poller.Stop()` completes without deadlock
  - ☐ `Cache.GetModels("all")` returns hit after `SetModels()`
  - ☐ `ModelManager.SelectOptimalModel()` uses verifier scores when adapter enabled
  - ☐ `NewProvider()` rejects unhealthy providers when verifier enabled
  - ☐ `EventPublisher.Publish()` maps all change types to correct topics
  - ☐ `fallback_models.go` contains exactly 7 fallback entries
- 

## PHASE 3: UX IMPLEMENTATION — Enterprise-Grade Model Display

**Phase Goal:** Implement the UX design spec across all platforms (CLI wide/standard/narrow/compact, JSON/YAML output, TUI interactive selector, real-time status bar, notifications, auto-suggest).

**Phase Entry Criteria:** Phase 2 verification checklist is complete. **Phase Exit Criteria:** 1. `--list-models` shows verification status, score, price, latency, capabilities 2. `--model-info <id>` shows full detail with all dimensions 3. All 6 width modes render correctly 4. TUI model selector works with keyboard navigation 5. Real-time status bar shows model counts 6. Notifications appear for cooldown/provider offline events 7. Auto-suggest displays when unavailable model is selected 8. Cross-platform symbol rendering works on Linux, macOS, Windows CMD, Windows Terminal

---

### TASK 3.1: Create `internal/cli/ux/symbols.go`

#### TASK 3.1.1: Implement Cross-Platform Symbol Resolution

**File(s):** `helix_code/internal/cli/ux/symbols.go` **Line(s):** CREATE new file

**Action:** CREATE

**package** `ux`

*// SymbolSet holds platform-appropriate symbols for UI rendering.*

```
type SymbolSet struct {
 Diamond string // Diamond/bullet
 SepHorizontal string
 SepVertical string
 Verified string
 Failed string
 Healthy string
 Degraded string
 CoolDown string
 Pending string
 Bullet string
 ArrowUp string
 ArrowRight string
 Dollar string
 Vision string
 Streaming string
 Tools string
 Code string
 Reasoning string
 Audio string
 Video string
 Embeddings string
 OpenSource string
 ProgressFull string
 ProgressEmpty string
}
```

*// DefaultSymbols uses Unicode symbols (best for modern terminals).*

```
var DefaultSymbols = &SymbolSet{
 Diamond: "◆",
 SepHorizontal: "—",
 SepVertical: "│",
 Verified: "✓",
 Failed: "✗",
 Healthy: "●",
 Degraded: "◐",
 CoolDown: "⌵",
 Pending: "⌵",
 Bullet: "•",
 ArrowUp: "↑",
 ArrowRight: "→",
 Dollar: "$",
 Vision: "👁",
 Streaming: " ",
 Tools: " ",
 Code: "</>",
}
```

```

Reasoning: " ",
Audio: " ",
Video: " ",
Embeddings: " ",
OpenSource: " ",
ProgressFull: "█",
ProgressEmpty: "░",
}

```

*// FallbackSymbols uses ASCII for Windows CMD and minimal terminals.*

```

var FallbackSymbols = &SymbolSet{
 Diamond: "*",
 SepHorizontal: "-",
 SepVertical: "|",
 Verified: "[OK]",
 Failed: "[NO]",
 Healthy: "[+]",
 Degraded: "[~]",
 CoolDown: "[CD]",
 Pending: "[..]",
 Bullet: "-",
 ArrowUp: "^",
 ArrowRight: ">",
 Dollar: "$",
 Vision: "[V]",
 Streaming: "[S]",
 Tools: "[T]",
 Code: "[C]",
 Reasoning: "[R]",
 Audio: "[A]",
 Video: "[M]",
 Embeddings: "[E]",
 OpenSource: "[O]",
 ProgressFull: "#",
 ProgressEmpty: "-",
}

```

*// NewSymbolSet selects the appropriate symbol set based on terminal capabilities.*

```

func NewSymbolSet(isWindowsCMD bool) *SymbolSet {
 if isWindowsCMD {
 return FallbackSymbols
 }
 return DefaultSymbols
}

```



**Acceptance Criteria:** 1. go build ./internal/cli/ux/... compiles 2. NewSymbolSet(true) returns FallbackSymbols 3. NewSymbolSet(false) returns DefaultSymbols 4. All 22 symbol fields are defined in both sets

**Dependencies:** None **Effort:** Small

---

## TASK 3.2: Create internal/cli/ux/badges.go

### TASK 3.2.1: Implement All Badge Types

**File(s):** helix\_code/internal/cli/ux/badges.go **Line(s):** CREATE new file  
**Action:** CREATE

```
package ux

import (
 "fmt"
 "strings"
 "time"

 "github.com/fatih/color"
)

var (
 CVerified = color.New(color.FgGreen, color.Bold)
 CFailed = color.New(color.FgRed, color.Bold)
 CDegraded = color.New(color.FgYellow, color.Bold)
 CCoolDown = color.New(color.FgHiRed, color.Bold)
 CScoreExcellent = color.New(color.FgHiGreen)
 CScoreGood = color.New(color.FgGreen)
 CScoreAverage = color.New(color.FgYellow)
 CScorePoor = color.New(color.FgRed)
 CScoreBad = color.New(color.FgHiRed, color.Bold)
 CPriceFree = color.New(color.FgHiGreen, color.Bold)
 CPriceCheap = color.New(color.FgGreen)
 CPriceModerate = color.New(color.FgYellow)
 CPriceExpensive = color.New(color.FgRed, color.Bold)
 CBarGood = color.New(color.FgGreen)
 CBarAverage = color.New(color.FgYellow)
 CBarPoor = color.New(color.FgRed)
 CBarEmpty = color.New(color.FgHiBlack)
)

// VerificationBadge renders a verification status indicator.
func VerificationBadge(status string, sym *SymbolSet) string {
 switch status {
 case "verified", "passed":
```

```

 return CVerified.Sprintf("%s Ready", sym.Verified)
 case "failed":
 return CFailed.Sprintf("%s Failed", sym.Failed)
 case "pending":
 return CDegraded.Sprintf("%s Pending", sym.Pending)
 case "rate_limited":
 return CCoolDown.Sprintf("%s Cooldown", sym.CoolDown)
 default:
 return CDegraded.Sprintf("%s Unknown", sym.Degraded)
}
}

// ProviderHealthBadge renders provider health status.
func ProviderHealthBadge(status string, sym *SymbolSet) string {
 switch status {
 case "healthy":
 return CVerified.Sprintf("%s HEALTHY", sym.Healthy)
 case "degraded":
 return CDegraded.Sprintf("%s DEGRADED", sym.Degraded)
 case "unhealthy", "offline":
 return CFailed.Sprintf("%s OFFLINE", sym.Failed)
 default:
 return CDegraded.Sprintf("%s UNKNOWN", sym.Degraded)
 }
}

// CooldownBadge renders cooldown state.
func CooldownBadge(resetIn time.Duration, sym *SymbolSet) string
{
 return CCoolDown.Sprintf("%s Cooldown (%s)", sym.CoolDown,
 formatDuration(resetIn))
}

// ScoreBadge renders a score with color.
func ScoreBadge(score float64, sym *SymbolSet) string {
 switch {
 case score >= 9.0:
 return CScoreExcellent.Sprintf("%.1f", score)
 case score >= 7.0:
 return CScoreGood.Sprintf("%.1f", score)
 case score >= 5.0:
 return CScoreAverage.Sprintf("%.1f", score)
 case score >= 3.0:
 return CScorePoor.Sprintf("%.1f", score)
 default:
 return CScoreBad.Sprintf("%.1f", score)
 }
}

```

```

// PriceBadge renders a price indicator.
func PriceBadge(costIn, costOut float64, sym *SymbolSet, width
 int) string {
 avgPrice := (costIn + costOut) / 2.0 * 1000
 switch {
 case avgPrice == 0:
 return CPriceFree.Sprintf("%sFREE", sym.Dollar)
 case avgPrice < 0.5:
 return CPriceCheap.Sprintf("%s%.2f/1K", sym.Dollar,
 avgPrice)
 case avgPrice < 2.0:
 return CPriceModerate.Sprintf("%s%.2f/1K", sym.Dollar,
 avgPrice)
 default:
 return CPriceExpensive.Sprintf("%s%.2f/1K", sym.Dollar,
 avgPrice)
 }
}

```

```

// TierBadge renders a model tier indicator.
func TierBadge(tier int) string {
 switch tier {
 case 1:
 return CVerified.Sprintf("***** Premium")
 case 2:
 return CScoreGood.Sprintf("***** High")
 case 3:
 return CScoreAverage.Sprintf("*** Fast")
 case 4:
 return CScoreAverage.Sprintf("** Aggregator")
 case 5:
 return CPriceFree.Sprintf("* Free")
 default:
 return "Unknown"
 }
}

```

```

func formatDuration(d time.Duration) string {
 if d < 0 {
 return "now"
 }
 if d < time.Minute {
 return fmt.Sprintf("%ds", int(d.Seconds()))
 }
 if d < time.Hour {
 return fmt.Sprintf("%dm", int(d.Minutes()))
 }
}

```

```

 return fmt.Sprintf("%dh%dm", int(d.Hours()), int(d.Minutes()))
 %60)
}

```

**Acceptance Criteria:** 1. go build ./internal/cli/ux/... compiles 2. VerificationBadge("verified", DefaultSymbols) contains “Ready” in green 3. ScoreBadge(9.5, nil) uses green color 4. ScoreBadge(2.5, nil) uses red bold 5. All 6 badge functions exist and are exported

**Dependencies:** TASK 3.1.1 **Effort:** Medium

---

## TASK 3.3: Create internal/cli/ux/capabilities.go

### TASK 3.3.1: Implement Capability Strip Rendering

**File(s):** helix\_code/internal/cli/ux/capabilities.go **Line(s):** CREATE new file **Action:** CREATE

```

package ux

import (
 "fmt"
 "strings"

 "dev.helix.code/internal/verifier"
)

// CapabilityStrip renders a compact capability indicator.
func CapabilityStrip(m *verifier.VerifiedModel, sym *SymbolSet,
 width int) string {
 parts := []string{}
 if m.SupportsVision {
 parts = append(parts, sym.Vision)
 }
 if m.SupportsStreaming {
 parts = append(parts, sym.Streaming)
 }
 if m.SupportsTools || m.SupportsFunctions {
 parts = append(parts, sym.Tools)
 }
 if m.SupportsCode {
 parts = append(parts, sym.Code)
 }
 if m.SupportsReasoning {
 parts = append(parts, sym.Reasoning)
 }
 if m.SupportsAudio {

```

```

 parts = append(parts, sym.Audio)
 }
 if m.SupportsVideo {
 parts = append(parts, sym.Video)
 }
 if m.SupportsEmbeddings {
 parts = append(parts, sym.Embeddings)
 }
 if m.OpenSource {
 parts = append(parts, sym.OpenSource)
 }
 return strings.Join(parts, " ")
}

```

*// CapabilityStripCompact renders a narrow-width capability strip.*

```

func CapabilityStripCompact(m *verifier.VerifiedModel, sym
 *SymbolSet) string {
 count := 0
 if m.SupportsCode {
 count++
 }
 if m.SupportsStreaming {
 count++
 }
 if m.SupportsTools || m.SupportsFunctions {
 count++
 }
 if m.SupportsVision {
 count++
 }
 return fmt.Sprintf("%d caps", count)
}

```

*// CapabilityDetails renders full capability grid with pass/fail.*

```

func CapabilityDetails(m *verifier.VerifiedModel, v
 *verifier.VerificationResult, sym *SymbolSet, width
 int) string {
 caps := []struct {
 label string
 val bool
 s string
 }{
 {"Vision", m.SupportsVision, sym.Vision},
 {"Streaming", m.SupportsStreaming, sym.Streaming},
 {"Tool Use", m.SupportsTools, sym.Tools},
 {"Code Gen", v != nil && v.SupportsCodeGeneration,
 sym.Code},
 }
}

```

```

 {"Reasoning", v != nil && v.SupportsReasoning,
 sym.Reasoning},
 {"Audio", m.SupportsAudio, sym.Audio},
 {"Video", m.SupportsVideo, sym.Video},
 {"Embeddings", v != nil && v.SupportsEmbeddings,
 sym.Embeddings},
 {"Open Source", m.OpenSource, sym.OpenSource},
 {"JSON Mode", v != nil && v.SupportsJSONMode, "{ }"},
}

var b strings.Builder
cols := 4
if width < 80 {
 cols = 2
}

for i := 0; i < len(caps); i += cols {
 for j := 0; j < cols && i+j < len(caps); j++ {
 c := caps[i+j]
 status := CFailed.Sprintf("%s", sym.Failed)
 if c.val {
 status = CVerified.Sprintf("%s", sym.Verified)
 }
 b.WriteString(fmt.Sprintf(" %s %-18s", status,
 c.label))
 }
 b.WriteString("\n")
}
return b.String()
}

```

**Acceptance Criteria:** 1. go build ./internal/cli/ux/... compiles 2. CapabilityStrip() returns non-empty for GPT-4o (has vision, streaming, tools, code) 3. CapabilityStripCompact() returns “4 caps” for a model with 4 capabilities 4. CapabilityDetails() uses 4 columns when width >= 80, 2 columns when < 80

**Dependencies:** TASK 3.1.1 **Effort:** Medium

---

## TASK 3.4: Create internal/cli/ux/render.go

### TASK 3.4.1: Implement Model List Rendering (Wide/Standard/Narrow)

**File(s):** helix\_code/internal/cli/ux/render.go **Line(s):** CREATE new file  
**Action:** CREATE

```

package ux

import (
 "encoding/json"
 "fmt"
 "strings"

 "dev.helix.code/internal/verifier"
 "gopkg.in/yaml.v3"
)

// ModelListRow holds all data needed to render one model row.
type ModelListRow struct {
 Model *verifier.VerifiedModel
 Provider *verifier.ProviderStatus
 Verification *verifier.VerificationResult
 Limits *verifier.RateLimitStatus
 Cooldown *verifier.CooldownInfo
 Rank int
}

// RenderModelList renders the model list based on terminal
// width.
func RenderModelList(rows []*ModelListRow, sym *SymbolSet, width
 int, opts *RenderOptions) string {
 if opts != nil && opts.Format == "json" {
 return renderJSON(rows)
 }
 if opts != nil && opts.Format == "yaml" {
 return renderYAML(rows)
 }

 switch {
 case width >= 140:
 return renderWide(rows, sym, width, opts)
 case width >= 100:
 return renderStandard(rows, sym, width, opts)
 case width >= 80:
 return renderNarrow(rows, sym, width, opts)
 default:
 return renderCompact(rows, sym, width, opts)
 }
}

// RenderOptions controls list rendering behavior.
type RenderOptions struct {
 Format string // "table", "json", "yaml"
 NoColor bool
}

```

```

NoEmoji bool
ShowPrices bool
ShowLatency bool
GroupBy string // "provider", "tier", "none"
SortBy string // "score", "price", "name", "latency"
}

func renderWide(rows []*ModelListRow, sym *SymbolSet, width int,
 opts *RenderOptions) string {
 var b strings.Builder
 b.WriteString(fmt.Sprintf(" %s HelixCode – Available Models
 (verifier-powered)\n", sym.Diamond))
 b.WriteString(fmt.Sprintf(" %s\n",
 strings.Repeat(sym.SepHorizontal, width-2)))
 header := fmt.Sprintf(" %-4s %-26s %-14s %-8s %-12s %-10s
 %-10s %-10s %s",
 "Rank", "Model", "Provider", "Score", "Status",
 "Latency", "Price", "Tier", "Capabilities")
 b.WriteString(header + "\n")
 b.WriteString(fmt.Sprintf(" %s\n",
 strings.Repeat(sym.SepHorizontal, width-2)))

 for _, r := range rows {
 status := VerificationBadge(r.Model.VerificationStatus,
 sym)
 score := ScoreBadge(r.Model.OverallScore, sym)
 price := PriceBadge(r.Model.CostPerInputToken,
 r.Model.CostPerOutputToken, sym, width)
 caps := CapabilityStrip(r.Model, sym, width)
 tier := TierBadge(r.Model.Tier)
 latencyStr := formatLatency(r.Model.Latency)

 line := fmt.Sprintf(" %-4d %-26s %-14s %-8s %-12s %-10s
 %-10s %-10s %s",
 r.Rank, truncate(r.Model.DisplayName, 26),
 r.Model.Provider,
 score, status, latencyStr, price, tier, caps)
 b.WriteString(line + "\n")
 }
 b.WriteString(fmt.Sprintf(" %s\n",
 strings.Repeat(sym.SepHorizontal, width-2)))
 return b.String()
}

func renderStandard(rows []*ModelListRow, sym *SymbolSet, width
 int, opts *RenderOptions) string {
 var b strings.Builder

```



```

b.WriteString(fmt.Sprintf(" %s HelixCode – Available
 Models\n", sym.Diamond))
b.WriteString(fmt.Sprintf(" %s\n",
 strings.Repeat(sym.SepHorizontal, width-2)))
header := fmt.Sprintf(" %-4s %-28s %-12s %-8s %-12s %-10s
 %s",
 "#", "Model", "Provider", "Score", "Status", "Latency",
 "Price")
b.WriteString(header + "\n")

for _, r := range rows {
 status := VerificationBadge(r.Model.VerificationStatus,
 sym)
 score := ScoreBadge(r.Model.OverallScore, sym)
 price := PriceBadge(r.Model.CostPerInputToken,
 r.Model.CostPerOutputToken, sym, width)
 latencyStr := formatLatency(r.Model.Latency)

 line := fmt.Sprintf(" %-4d %-28s %-12s %-8s %-12s %-10s
 %s",
 r.Rank, truncate(r.Model.DisplayName, 28),
 r.Model.Provider,
 score, status, latencyStr, price)
 b.WriteString(line + "\n")
}
return b.String()
}

func renderNarrow(rows []*ModelListRow, sym *SymbolSet, width
 int, opts *RenderOptions) string {
var b strings.Builder
b.WriteString(fmt.Sprintf(" %s Models\n", sym.Diamond))
b.WriteString(fmt.Sprintf(" %s\n",
 strings.Repeat(sym.SepHorizontal, width-2)))

for _, r := range rows {
 status := VerificationBadge(r.Model.VerificationStatus,
 sym)
 score := ScoreBadge(r.Model.OverallScore, sym)
 line := fmt.Sprintf(" %d. %-24s %s %s %s",
 r.Rank, truncate(r.Model.DisplayName, 24),
 r.Model.Provider, score, status)
 b.WriteString(line + "\n")
}
return b.String()
}

```

```

func renderCompact(rows []*ModelListRow, sym *SymbolSet, width
 int, opts *RenderOptions) string {
 var b strings.Builder
 for _, r := range rows {
 status := "OK"
 if !r.Model.Verified {
 status = "?"
 }
 if r.Model.VerificationStatus == "failed" {
 status = "FAIL"
 }
 if r.Model.VerificationStatus == "rate_limited" {
 status = "CD"
 }
 line := fmt.Sprintf(" %d. %s (%s) %.1f [%s]",
 r.Rank, r.Model.DisplayName, r.Model.Provider,
 r.Model.OverallScore, status)
 b.WriteString(line + "\n")
 }
 return b.String()
}

```

```

func renderJSON(rows []*ModelListRow) string {
 type jsonRow struct {
 ID string `json:"id"`
 Name string `json:"name"`
 Provider string `json:"provider"`
 Score float64 `json:"score"`
 Verified bool `json:"verified"`
 Status string `json:"status"`
 Latency string `json:"latency"`
 Price string `json:"price"`
 Tier int `json:"tier"`
 }
 out := make([]jsonRow, len(rows))
 for i, r := range rows {
 out[i] = jsonRow{
 ID: r.Model.ID,
 Name: r.Model.DisplayName,
 Provider: r.Model.Provider,
 Score: r.Model.OverallScore,
 Verified: r.Model.Verified,
 Status: r.Model.VerificationStatus,
 Latency: formatLatency(r.Model.Latency),
 Price: PriceBadge(r.Model.CostPerInputToken,
 r.Model.CostPerOutputToken, DefaultSymbols, 80),
 Tier: r.Model.Tier,
 }
 }
}

```

```

 }
 data, _ := json.MarshalIndent(out, "", " ")
 return string(data)
}

func renderYAML(rows []*ModelListRow) string {
 type yamlRow struct {
 ID string `yaml:"id"`
 Name string `yaml:"name"`
 Provider string `yaml:"provider"`
 Score float64 `yaml:"score"`
 Verified bool `yaml:"verified"`
 }
 out := make([]yamlRow, len(rows))
 for i, r := range rows {
 out[i] = yamlRow{
 ID: r.Model.ID,
 Name: r.Model.DisplayName,
 Provider: r.Model.Provider,
 Score: r.Model.OverallScore,
 Verified: r.Model.Verified,
 }
 }
 data, _ := yaml.Marshal(out)
 return string(data)
}

func truncate(s string, maxLen int) string {
 if len(s) <= maxLen {
 return s
 }
 return s[:maxLen-3] + "..."
}

func formatLatency(d time.Duration) string {
 if d == 0 {
 return "unknown"
 }
 if d < time.Millisecond {
 return fmt.Sprintf("%dus", d.Microseconds())
 }
 if d < time.Second {
 return fmt.Sprintf("%dms", d.Milliseconds())
 }
 return fmt.Sprintf("%.1fs", d.Seconds())
}

```

**Acceptance Criteria:** 1. `go build ./internal/cli/ux/...` compiles 2. `RenderModelList()` dispatches to `renderWide` when `width >= 140` 3. `RenderModelList()` dispatches to `renderCompact` when `width < 80` 4. `renderJSON()` produces valid JSON with `id`, `name`, `provider`, `score`, `verified` fields 5. `truncate("hello world", 5)` returns "he..." 6. `formatLatency(234*time.Millisecond)` returns "234ms"

**Dependencies:** TASK 3.1.1, TASK 3.2.1, TASK 3.3.1 **Effort:** Large

---

## TASK 3.5: Create `internal/cli/ux/detail.go`

### TASK 3.5.1: Implement Model Detail Rendering

**File(s):** `helix_code/internal/cli/ux/detail.go` **Line(s):** CREATE new file  
**Action:** CREATE

```
package ux

import (
 "encoding/json"
 "fmt"
 "strings"
 "time"

 "dev.helix.code/internal/verifier"
 "github.com/fatih/color"
 "gopkg.in/yaml.v3"
)

// DetailDisplayOptions controls detail rendering.
type DetailDisplayOptions struct {
 Format string // "rich", "json", "yaml"
 NoColor bool
 NoEmoji bool
 TerminalWidth int
}

// RenderModelDetail renders a full model detail view.
func RenderModelDetail(
 model *verifier.VerifiedModel,
 provider *verifier.ProviderStatus,
 verification *verifier.VerificationResult,
 limits *verifier.RateLimitStatus,
 cooldown *verifier.CooldownInfo,
 alternatives []*verifier.VerifiedModel,
 opts *DetailDisplayOptions,
) string {
```

```

sym := NewSymbolSet(opts.NoEmoji)
if opts.NoColor {
 color.NoColor = true
}

switch opts.Format {
case "json":
 return renderDetailJSON(model, provider, verification,
 limits, cooldown, alternatives)
case "yaml":
 return renderDetailYAML(model, provider, verification,
 limits, cooldown, alternatives)
default:
 if opts.TerminalWidth >= 100 {
 return renderDetailWide(model, provider,
 verification, limits, cooldown, alternatives, sym,
 opts.TerminalWidth)
 } else if opts.TerminalWidth >= 60 {
 return renderDetailCompact(model, provider,
 verification, limits, cooldown, alternatives, sym,
 opts.TerminalWidth)
 }
 return renderDetailNarrow(model, provider, verification,
 limits, cooldown, alternatives, sym, opts.TerminalWidth)
}
}

func renderDetailWide(m *verifier.VerifiedModel, p
 *verifier.ProviderStatus, v
 *verifier.VerificationResult,
 limits *verifier.RateLimitStatus, cd *verifier.CooldownInfo,
 alts []*verifier.VerifiedModel,
 sym *SymbolSet, width int) string {
 var b strings.Builder
 b.WriteString(fmt.Sprintf(" %s HelixCode - Model Details\n",
 sym.Diamond))
 b.WriteString(fmt.Sprintf(" %s\n",
 strings.Repeat(sym.SepHorizontal, width-2)))

 // Header
 healthBadge := ""
 if p != nil {
 healthBadge = ProviderHealthBadge(p.Status, sym)
 }
 b.WriteString(fmt.Sprintf(" %-50s %s %s\n", m.DisplayName,
 m.Provider, healthBadge))
 b.WriteString(fmt.Sprintf(" %s\n", strings.Repeat("=",
 width-4)))
}

```

```

// Score panel
b.WriteString(fmt.Sprintf(" Overall Score %s\n",
 ScoreBadge(m.OverallScore, sym)))
if v != nil {
 b.WriteString(fmt.Sprintf(" Code Capability %s\n",
 ScoreBadge(v.CodeCapabilityScore, sym)))
 b.WriteString(fmt.Sprintf(" Responsiveness %s\n",
 ScoreBadge(v.ResponsivenessScore, sym)))
 b.WriteString(fmt.Sprintf(" Reliability %s\n",
 ScoreBadge(v.ReliabilityScore, sym)))
}
b.WriteString("\n")

// Context

 b.WriteString(fmt.Sprintf(" Context Window: %d
tokens Max Output: %d tokens\n", m.ContextSize,
m.MaxOutputTokens))
b.WriteString("\n")

// Pricing
inPrice := m.CostPerInputToken * 1000
outPrice := m.CostPerOutputToken * 1000
b.WriteString(fmt.Sprintf(" Pricing (per 1K): Input
 %s%.2f Output %s%.2f\n", sym.Dollar, inPrice,
 sym.Dollar, outPrice))
b.WriteString("\n")

// Capabilities
b.WriteString(" Capabilities:\n")
b.WriteString(CapabilityDetails(m, v, sym, width))

// Verification dimensions
if v != nil {
 b.WriteString(" Verification Dimensions:\n")
 checks := []struct{ name string; pass bool }{
 {"Model Exists", v.ModelExists != nil &&
 *v.ModelExists},
 {"Responsive", v.Responsive != nil && *v.Responsive},
 {"Not Overloaded", v.Overloaded != nil && !
 *v.Overloaded},
 {"Supports Tools", v.SupportsToolUse},
 {"Code Generation", v.SupportsCodeGeneration},
 {"Code Debugging", v.CodeDebugging},
 {"Code Optimization", v.CodeOptimization},
 {"Test Generation", v.TestGeneration},
 {"Documentation Gen.", v.DocumentationGeneration},
 }
}

```

```

 {"Architecture", v.ArchitectureDesign},
 {"Security Assessment", v.SecurityAssessment},
 {"Pattern Recognition", v.PatternRecognition},
 }
 for _, c := range checks {
 status := CFailed.Sprintf("%s", sym.Failed)
 if c.pass {
 status = CVerified.Sprintf("%s", sym.Verified)
 }
 b.WriteString(fmt.Sprintf(" %s %s\n", status,
 c.name))
 }
 b.WriteString("\n")
}

// Rate limits
if limits != nil && len(limits.Limits) > 0 {
 b.WriteString(" Rate Limits:\n")
 for _, l := range limits.Limits {
 b.WriteString(fmt.Sprintf(" %s: limit=%d used=%d
 remaining=%d reset=%s\n",
 l.Type, l.Limit, l.Used, l.Remaining,
 l.ResetTime.Format("15:04:05")))
 }
 b.WriteString("\n")
}

// Alternatives
if len(alts) > 0 {
 b.WriteString(" Alternative Models:\n")
 for i, alt := range alts {
 if i >= 5 {
 break
 }
 price := (alt.CostPerInputToken +
 alt.CostPerOutputToken) / 2.0 * 1000
 b.WriteString(fmt.Sprintf(" %d. %s (%s) – Score:
 %.1f – Price: $%.2f/1K\n",
 i+1, alt.DisplayName, alt.Provider,
 alt.OverallScore, price))
 }
}

return b.String()
}

```

```

func renderDetailCompact(m *verifier.VerifiedModel, p
 *verifier.ProviderStatus, v
 *verifier.VerificationResult,
 limits *verifier.RateLimitStatus, cd *verifier.CooldownInfo,
 alts []*verifier.VerifiedModel,
 sym *SymbolSet, width int) string {
var b strings.Builder
b.WriteString(fmt.Sprintf(" %s HelixCode – Model Details\n",
 sym.Diamond))
b.WriteString(fmt.Sprintf(" %s\n",
 strings.Repeat(sym.SepHorizontal, width-2)))
b.WriteString(fmt.Sprintf(" %-30s %s %s\n", m.DisplayName,
 m.Provider, ProviderHealthBadge(p.Status, sym)))
b.WriteString(fmt.Sprintf(" Score: %s Tier: %s\n",
 ScoreBadge(m.OverallScore, sym), TierBadge(m.Tier)))
b.WriteString(fmt.Sprintf(" Context: %dK MaxOut: %d\n",
 m.ContextSize/1000, m.MaxOutputTokens))
b.WriteString(fmt.Sprintf(" Price: In %s%.2f/1K Out %s%.2f/
 1K\n", sym.Dollar, m.CostPerInputToken*1000,
 sym.Dollar, m.CostPerOutputToken*1000))
b.WriteString(fmt.Sprintf(" Capabilities: %s\n",
 CapabilityStrip(m, sym, width)))
if len(alts) > 0 {
 b.WriteString(fmt.Sprintf(" Fallbacks: %d alternatives
 available\n", len(alts)))
}
return b.String()
}

```

```

func renderDetailNarrow(m *verifier.VerifiedModel, p
 *verifier.ProviderStatus, v
 *verifier.VerificationResult,
 limits *verifier.RateLimitStatus, cd *verifier.CooldownInfo,
 alts []*verifier.VerifiedModel,
 sym *SymbolSet, width int) string {
var b strings.Builder
b.WriteString(fmt.Sprintf("Model: %s\n", m.DisplayName))
b.WriteString(fmt.Sprintf("Provider: %s [%s]\n", m.Provider,
 p.Status))
b.WriteString(fmt.Sprintf("Score: %.1f\n", m.OverallScore))
b.WriteString(fmt.Sprintf("Context: %dK / MaxOut: %d\n",
 m.ContextSize/1000, m.MaxOutputTokens))
b.WriteString(fmt.Sprintf("Price: In $%.2f/1K Out $%.2f/
 1K\n", m.CostPerInputToken*1000,
 m.CostPerOutputToken*1000))
b.WriteString(fmt.Sprintf("Status: %s\n",
 m.VerificationStatus))
if len(alts) > 0 {

```



```

 b.WriteString(fmt.Sprintf("Fallbacks: %d\n", len(alts)))
 }
 return b.String()
}

func renderDetailJSON(m *verifier.VerifiedModel, p
 *verifier.ProviderStatus, v
 *verifier.VerificationResult,
 limits *verifier.RateLimitStatus, cd *verifier.CooldownInfo,
 alts []*verifier.VerifiedModel) string {
 data, _ := json.MarshalIndent(struct {
 Model *verifier.VerifiedModel `json:"model"`
 Provider *verifier.ProviderStatus
 `json:"provider,omitempty"`
 Verification *verifier.VerificationResult
 `json:"verification,omitempty"`
 Limits *verifier.RateLimitStatus
 `json:"limits,omitempty"`
 Alternatives []*verifier.VerifiedModel
 `json:"alternatives,omitempty"`
 }{m, p, v, limits, alts}, "", " ")
 return string(data)
}

func renderDetailYAML(m *verifier.VerifiedModel, p
 *verifier.ProviderStatus, v
 *verifier.VerificationResult,
 limits *verifier.RateLimitStatus, cd *verifier.CooldownInfo,
 alts []*verifier.VerifiedModel) string {
 data, _ := yaml.Marshal(struct {
 Model *verifier.VerifiedModel `yaml:"model"`
 Provider *verifier.ProviderStatus
 `yaml:"provider,omitempty"`
 Verification *verifier.VerificationResult
 `yaml:"verification,omitempty"`
 }{m, p, v})
 return string(data)
}

```

**Acceptance Criteria:** 1. go build ./internal/cli/ux/... compiles 2. renderDetailWide() includes score panel, context, pricing, capabilities, verification, rate limits, alternatives 3. renderDetailCompact() fits in 60-99 columns 4. renderDetailNarrow() fits in <60 columns 5. renderDetailJSON() produces valid JSON 6. renderDetailYAML() produces valid YAML

**Dependencies:** TASK 3.1.1, TASK 3.2.1, TASK 3.3.1 **Effort:** Large

---

## TASK 3.6: Create internal/cli/tui/model\_selector.go

### TASK 3.6.1: Implement tview-Based Interactive Model Selector

**File(s):** helix\_code/internal/cli/tui/model\_selector.go **Line(s):** CREATE  
**Action:** CREATE

```
package tui

import (
 "context"
 "fmt"
 "strings"
 "time"

 "dev.helix.code/internal/cli/ux"
 "dev.helix.code/internal/verifier"
 "github.com/gdamore/tcell/v2"
 "github.com/rivo/tview"
)

// ModelSelectorApp is the interactive model selection TUI.
type ModelSelectorApp struct {
 app *tview.Application
 modelList *tview.List
 previewPane *tview.TextView
 statusBar *tview.TextView
 filterInput *tview.InputField

 models []*ux.ModelListRow
 filtered []*ux.ModelListRow
 selectedIdx int
 sym *ux.SymbolSet
 refreshTicker *time.Ticker
 cancelFunc context.CancelFunc
 onSelect func(modelID string)
}

// NewModelSelectorApp creates a TUI model selector.
func NewModelSelectorApp(models []*ux.ModelListRow, sym
 *ux.SymbolSet) *ModelSelectorApp {
 m := &ModelSelectorApp{
 app: tview.NewApplication(),
 models: models,
 filtered: models,
 sym: sym,
 }
 m.buildUI()
```

```

 return m
}

func (m *ModelSelectorApp) buildUI() {
 // Model list (left pane)
 m.modelList = tview.NewList()
 m.modelList.SetBorder(true).SetTitle(" Model List ")
 m.modelList.SetMainTextColor(tview.ColorWhite)
 m.modelList.SetSecondaryTextColor(tview.ColorDarkGray)
 m.modelList.SetSelectedBackgroundColor(tview.ColorDarkCyan)
 m.populateModelList()

 m.modelList.SetSelectedFunc(func(idx int, mainText string,
 secondaryText string, shortcut rune) {
 if idx >= 0 && idx < len(m.filtered) {
 m.onSelect(m.filtered[idx].Model.ID)
 m.app.Stop()
 }
 })
 m.modelList.SetChangedFunc(func(idx int, mainText string,
 secondaryText string, shortcut rune) {
 if idx >= 0 && idx < len(m.filtered) {
 m.updatePreview(m.filtered[idx])
 }
 })

 // Preview pane (right pane)
 m.previewPane = tview.NewTextView()
 m.previewPane.SetBorder(true).SetTitle(" Preview ")
 m.previewPane.SetDynamicColors(true)
 m.previewPane.SetScrollable(true)

 // Status bar (bottom)
 m.statusBar = tview.NewTextView()
 m.statusBar.SetDynamicColors(true)
 m.statusBar.SetTextAlign(tview.AlignLeft)

 // Filter input
 m.filterInput = tview.NewInputField().SetLabel("Filter: ")
 m.filterInput.SetFieldBackgroundColor(tview.ColorBlack)
 m.filterInput.SetDoneFunc(func(key tcell.Key) {
 m.applyFilter(m.filterInput.GetText())
 })

 // Layout: left list | right preview, stacked with filter +
 // status bar
 mainFlex := tview.NewFlex().
 AddItem(m.modelList, 0, 1, true).

```

```

 AddItem(m.previewPane, 0, 2, false)

fullLayout := tview.NewFlex().SetDirection(tview.FlexRow).
 AddItem(mainFlex, 0, 1, true).
 AddItem(m.filterInput, 1, 0, false).
 AddItem(m.statusBar, 1, 0, false)

m.app.SetRoot(fullLayout, true)

// Key bindings
m.app.SetInputCapture(func(event *tcell.EventKey)
 *tcell.EventKey {
 switch event.Rune() {
 case 'q', 'Q':
 m.app.Stop()
 return nil
 case 'r', 'R':
 m.triggerRefresh()
 return nil
 case 'f', 'F':
 m.app.SetFocus(m.filterInput)
 return nil
 }
 return event
 })

if len(m.filtered) > 0 {
 m.updatePreview(m.filtered[0])
}
m.updateStatusBar()
}

func (m *ModelSelectorApp) populateModelList() {
 m.modelList.Clear()
 for i, r := range m.filtered {
 tierPrefix := " "
 if r.Model.Tier == 1 {
 tierPrefix = "* "
 }
 mainText := fmt.Sprintf("%s%s", tierPrefix,
 r.Model.DisplayName)
 statusBadge :=
 ux.VerificationBadge(r.Model.VerificationStatus, m.sym)
 scoreStr := fmt.Sprintf("%.1f", r.Model.OverallScore)
 priceStr := ux.PriceBadge(r.Model.CostPerInputToken,
 r.Model.CostPerOutputToken, m.sym, 80)
 secondaryText := fmt.Sprintf(" %s %s %s %s %s",
 statusBadge, scoreStr, priceStr, r.Model.Provider,

```

```

 ux.CapabilityStripCompact(r.Model, m.sym))
 m.modelList.AddItem(mainText, secondaryText, rune('0'+
 ((i+1)%10)), nil)
}

}

func (m *ModelSelectorApp) updatePreview(r *ux.ModelListRow) {
 detail := ux.RenderModelDetail(r.Model, r.Provider,
 r.Verification, r.Limits, r.Cooldown, nil,
 &ux.DetailDisplayOptions{Format: "rich", TerminalWidth:
 80})
 m.previewPane.SetText(detail)
}

func (m *ModelSelectorApp) updateStatusBar() {
 counts := countStatuses(m.filtered)
 text := fmt.Sprintf(
 " [green]* %d healthy[-] [yellow]* %d degraded[-]
 [red]* %d cooldown[-] [gray]* %d offline[-] | [blue]
 [f]ilter [r]efresh [q]uit[-]",
 counts.Healthy, counts.Degraded, counts.Cooldown,
 counts.Offline)
 m.statusBar.SetText(text)
}

func (m *ModelSelectorApp) applyFilter(query string) {
 query = strings.ToLower(strings.TrimSpace(query))
 if query == "" {
 m.filtered = m.models
 } else {
 m.filtered = []*ux.ModelListRow{}
 for _, r := range m.models {
 if
 strings.Contains(strings.ToLower(r.Model.DisplayName),
 query) ||

 strings.Contains(strings.ToLower(r.Model.Provider),
 query) ||
 strings.Contains(strings.ToLower(r.Model.ID),
 query) {
 m.filtered = append(m.filtered, r)
 }
 }
 }
 m.populateModelList()
 if len(m.filtered) > 0 {
 m.updatePreview(m.filtered[0])
 }
 }
}

```

```

 m.updateStatusBar()
 }

 func (m *ModelSelectorApp) triggerRefresh() {
 // Signal to background goroutine to refresh data
 }

 func (m *ModelSelectorApp) Run() error {
 return m.app.Run()
 }

 func (m *ModelSelectorApp) GetSelectedModel() string {
 if m.selectedIdx >= 0 && m.selectedIdx < len(m.filtered) {
 return m.filtered[m.selectedIdx].Model.ID
 }
 return ""
 }

 func (m *ModelSelectorApp) SetOnSelect(fn func(modelID string)) {
 m.onSelect = fn
 }

 type statusCounts struct {
 Healthy int
 Degraded int
 Cooldown int
 Offline int
 }

 func countStatuses(rows []*ux.ModelListRow) statusCounts {
 c := statusCounts{}
 for _, r := range rows {
 switch r.Model.VerificationStatus {
 case "verified", "passed":
 c.Healthy++
 case "pending":
 c.Degraded++
 case "rate_limited":
 c.Cooldown++
 case "failed", "offline":
 c.Offline++
 default:
 c.Degraded++
 }
 }
 return c
 }

```

**Acceptance Criteria:** 1. `go build ./internal/cli/tui/...` compiles (requires `go get github.com/rivo/tview`) 2. `NewModelSelectorApp()` creates all `tview` widgets 3. `applyFilter()` filters by name, provider, and ID 4. `Run()` returns error if `tview` fails to initialize 5. `SetOnSelect()` callback is invoked when model is selected

**Dependencies:** TASK 3.1.1, TASK 3.2.1, TASK 3.3.1, TASK 3.4.1 **Effort:** Large

---

## TASK 3.7: Modify `cmd/cli/main.go` — Add Enhanced CLI Flags

### TASK 3.7.1: Add All New CLI Flags

**File(s):** `helix_code/cmd/cli/main.go` **Line(s):** After existing `flag.StringVar(&c.prompt, ...)` declarations **Action:** MODIFY

```
// Existing flags ...

// NEW: Model listing and filtering flags
var (
 listModels = flag.Bool("list-models", false, "List
 all available models with verifier data")
 modelInfo = flag.String("model-info", "", "Show
 detailed info for a specific model ID")
 providerFilter = flag.String("provider", "", "Filter
 models by provider name")
 verifiedOnly = flag.Bool("verified-only", false, "Show
 only verified models")
 maxPrice = flag.Float64("max-price", 0, "Maximum
 price per 1K tokens (in USD)")
 minScore = flag.Float64("min-score", 0, "Minimum
 verifier score (0-10)")
 capability = flag.String("capability", "",
 "Filter by capability (comma-separated:
 code,vision,streaming,tools,reasoning)")
 sortBy = flag.String("sort", "score", "Sort
 models by: score, price, name, latency, tier")
 groupBy = flag.String("group-by", "none", "Group
 models by: provider, tier, none")
 format = flag.String("format", "table", "Output
 format: table, json, yaml")
 interactive = flag.Bool("models-interactive", false,
 "Interactive TUI model selector")
 refreshModels = flag.Bool("refresh-models", false,
 "Force refresh model list from verifier")
 noColor = flag.Bool("no-color", false, "Disable
 color output")
)
```

```

 noEmoji = flag.Bool("no-emoji", false, "Disable
emoji/special symbols (ASCII fallback)")
)

```

Then in `handleListModel()`, integrate the new flags:

```

func (c *CLI) handleListModel(ctx context.Context) error {
 if *refreshModels && c.verifierAdapter != nil {
 _ = c.verifierAdapter.ForceRefresh(ctx)
 }

 // Determine symbol set
 sym := ux.NewSymbolSet(*noEmoji || isWindowsCMD())
 if *noColor {
 color.NoColor = true
 }

 // Fetch models
 var models []*verifier.VerifiedModel
 var err error
 if c.verifierAdapter != nil && c.verifierAdapter.IsEnabled()
 {
 models, err = c.verifierAdapter.GetVerifiedModels(ctx)
 } else {
 models = verifier.FallbackModels
 }

 // Apply filters
 models = c.filterModels(models, *providerFilter,
 *verifiedOnly, *maxPrice, *minScore, *capability)
 models = c.sortModels(models, *sortBy)

 // Build rows
 rows := make([]*ux.ModelListRow, len(models))
 for i, m := range models {
 rows[i] = &ux.ModelListRow{
 Model: m,
 Rank: i + 1,
 }
 }

 // Render
 opts := &ux.RenderOptions{
 Format: *format,
 NoColor: *noColor,
 NoEmoji: *noEmoji,
 ShowPrices: true,
 ShowLatency: true,
 GroupBy: *groupBy,
 }

```



```

 SortBy: *sortBy,
 }

 width, _, _ := term.GetSize(int(os.Stdout.Fd()))
 if width <= 0 {
 width = 120
 }

 output := ux.RenderModelList(rows, sym, width, opts)
 fmt.Println(output)
 return nil
}

func (c *CLI) filterModels(models []*verifier.VerifiedModel,
 provider string, verifiedOnly bool, maxPrice, minScore
 float64, caps string) []*verifier.VerifiedModel {
 result := make([]*verifier.VerifiedModel, 0, len(models))
 for _, m := range models {
 if provider != "" && m.Provider != provider {
 continue
 }
 if verifiedOnly && !m.Verified {
 continue
 }
 if maxPrice > 0 &&
 (m.CostPerInputToken+m.CostPerOutputToken)/2.0*1000 >
 maxPrice {
 continue
 }
 if minScore > 0 && m.OverallScore < minScore {
 continue
 }
 if caps != "" {
 required := strings.Split(caps, ",")
 if !hasCapabilities(m, required) {
 continue
 }
 }
 result = append(result, m)
 }
 return result
}

func hasCapabilities(m *verifier.VerifiedModel, required
 []string) bool {
 for _, r := range required {
 switch strings.ToLower(strings.TrimSpace(r)) {
 case "code":

```

```

 if !m.SupportsCode {
 return false
 }
 case "vision":
 if !m.SupportsVision {
 return false
 }
 case "streaming":
 if !m.SupportsStreaming {
 return false
 }
 case "tools":
 if !m.SupportsTools && !m.SupportsFunctions {
 return false
 }
 case "reasoning":
 if !m.SupportsReasoning {
 return false
 }
 case "embeddings":
 if !m.SupportsEmbeddings {
 return false
 }
 case "audio":
 if !m.SupportsAudio {
 return false
 }
 case "video":
 if !m.SupportsVideo {
 return false
 }
 }
}
return true
}

```

```

func (c *CLI) sortModels(models []*verifier.VerifiedModel,
 sortBy string) []*verifier.VerifiedModel {
 result := make([]*verifier.VerifiedModel, len(models))
 copy(result, models)
 switch strings.ToLower(sortBy) {
 case "score":
 sort.Slice(result, func(i, j int) bool { return
 result[i].OverallScore > result[j].OverallScore })
 case "price":
 sort.Slice(result, func(i, j int) bool {
 pi := (result[i].CostPerInputToken +
 result[i].CostPerOutputToken) / 2.0

```

```

 pj := (result[j].CostPerInputToken +
result[j].CostPerOutputToken) / 2.0
 return pi < pj
 })
case "name":
 sort.Slice(result, func(i, j int) bool { return
result[i].DisplayName < result[j].DisplayName })
case "latency":
 sort.Slice(result, func(i, j int) bool { return
result[i].Latency < result[j].Latency })
case "tier":
 sort.Slice(result, func(i, j int) bool { return
result[i].Tier < result[j].Tier })
}
return result
}

```

**Acceptance Criteria:** 1. `go build ./cmd/cli/...` compiles 2. `./cli --help` shows all new flags: `--list-models`, `--model-info`, `--provider`, `--verified-only`, `--max-price`, `--min-score`, `--capability`, `--sort`, `--group-by`, `--format`, `--models-interactive`, `--refresh-models`, `--no-color`, `--no-emoji` 3. `--format json` produces valid JSON with `id`, `name`, `provider`, `score`, `verified` 4. `--provider openai` filters to only OpenAI models 5. `--min-score 8.0` filters out models with `score < 8.0` 6. `--capability code,vision` filters to models supporting both

**Dependencies:** TASK 3.4.1 **Effort:** Large

---

## TASK 3.8: Implement Real-Time Status Bar and Notifications

### TASK 3.8.1: Create Status Bar Component

**File(s):** `helix_code/internal/cli/ux/status_bar.go` **Line(s):** CREATE new file **Action:** CREATE

```

package ux

import (
 "fmt"
 "time"
)

// StatusBar renders the persistent bottom status line.
type StatusBar struct {
 sym *SymbolSet
 totalModels int
}

```

```

 activeModels int
 cooldownCount int
 degradedCount int
 offlineCount int
 lastRefresh time.Time
 isRefreshing bool
 width int
}

// Render returns the status bar string based on terminal width.
func (sb *StatusBar) Render() string {
 if sb.width >= 100 {
 refreshStr := fmt.Sprintf("%s last refresh: %s",
 sb.sym.Verified, sb.lastRefresh.Format("15:04:05"))
 if sb.isRefreshing {
 refreshStr = fmt.Sprintf("%s refreshing...",
 sb.sym.Pending)
 }
 return fmt.Sprintf(" %s %d models active %s %s %d
 cooldown %s %s %d degraded %s %s",
 sb.sym.Healthy, sb.activeModels,
 sb.sym.SepVertical,
 sb.sym.CoolDown, sb.cooldownCount,
 sb.sym.SepVertical,
 sb.sym.Degraded, sb.degradedCount,
 sb.sym.SepVertical,
 refreshStr)
 }
 refreshStr := sb.lastRefresh.Format("15:04")
 if sb.isRefreshing {
 refreshStr = "... "
 }
 return fmt.Sprintf(" %d OK | %d CD | %d ~ | %s",
 sb.activeModels, sb.cooldownCount, sb.degradedCount,
 refreshStr)
}

```

### TASK 3.8.2: Create Alert/Notification System

**File(s):** helix\_code/internal/cli/ux/alerts.go **Line(s):** CREATE new file

**Action:** CREATE

```

package ux

import (
 "fmt"
 "strings"
 "time"

```

```

)

// AlertLevel defines the severity of an alert.
type AlertLevel int

const (
 AlertInfo AlertLevel = iota
 AlertWarning
 AlertError
 AlertSuccess
)

// Alert represents a notification to the user.
type Alert struct {
 Level AlertLevel
 Title string
 Message string
 ModelID string
 Provider string
 SuggestedAlternative string
 Timestamp time.Time
}

// Render formats the alert as a bordered message.
func (a *Alert) Render(sym *SymbolSet, width int) string {
 var b strings.Builder
 icon := ""
 titleColor := ""
 switch a.Level {
 case AlertInfo:
 icon = sym.Bullet
 titleColor = "[blue]"
 case AlertWarning:
 icon = sym.Degraded
 titleColor = "[yellow]"
 case AlertError:
 icon = sym.Failed
 titleColor = "[red]"
 case AlertSuccess:
 icon = sym.Verified
 titleColor = "[green]"
 }

 b.WriteString(fmt.Sprintf("%s\n",
 strings.Repeat(sym.SepHorizontal, width-1)))
 b.WriteString(fmt.Sprintf(" %s %s%s[-]\n", icon,
 titleColor, a.Title))
 b.WriteString(fmt.Sprintf(" %s\n", a.Message))

```

```

 if a.SuggestedAlternative != "" {
 b.WriteString(fmt.Sprintf(" %s Suggested alternative:
 %s\n", sym.ArrowRight, a.SuggestedAlternative))
 }
 b.WriteString(fmt.Sprintf("%s\n",
 strings.Repeat(sym.SepHorizontal, width-1)))
 return b.String()
}

```

### TASK 3.8.3: Create Auto-Suggest Component

**File(s):** helix\_code/internal/cli/ux/auto\_suggest.go **Line(s):** CREATE  
**Action:** CREATE

```

package ux

import (
 "fmt"
 "sort"
 "strings"

 "dev.helix.code/internal/verifier"
)

// SuggestAlternatives returns up to N alternative models for an
// unavailable one.
func SuggestAlternatives(
 unavailable *verifier.VerifiedModel,
 allModels []*verifier.VerifiedModel,
 sym *SymbolSet,
 count int,
) string {
 var b strings.Builder
 b.WriteString(fmt.Sprintf("\n %s SELECTED MODEL
 UNAVAILABLE\n", sym.Degraded))
 b.WriteString(fmt.Sprintf(" \"%s\" is currently %s.\n\n",
 unavailable.DisplayName,
 unavailable.VerificationStatus))
 b.WriteString(" Auto-switch to best available alternative?
 \n")

 // Filter and score alternatives
 candidates := make([]*verifier.VerifiedModel, 0,
 len(allModels))
 for _, m := range allModels {
 if m.ID == unavailable.ID {
 continue
 }
 }
}

```

```

 if m.VerificationStatus == "failed" ||
 m.VerificationStatus == "rate_limited" {
 continue
 }
 candidates = append(candidates, m)
 }

 sort.Slice(candidates, func(i, j int) bool {
 return candidates[i].OverallScore >
 candidates[j].OverallScore
 })

 for i, alt := range candidates {
 if i >= count {
 break
 }
 price := (alt.CostPerInputToken +
 alt.CostPerOutputToken) / 2.0 * 1000
 rec := ""
 if i == 0 {
 rec = " [RECOMMENDED]"
 }

 b.WriteString(fmt.Sprintf(" [%d] %s (%s) – Score: %.1f
 – $%.2f/1K%s\n",
 i+1, alt.DisplayName, alt.Provider,
 alt.OverallScore, price, rec))
 }

 b.WriteString(fmt.Sprintf(" [%d] Cancel and exit\n",
 len(candidates)+1))
 b.WriteString("\n Select number or press Enter for default
 [1]:\n")
 return b.String()
}

```

**Acceptance Criteria:** 1. go build ./internal/cli/ux/... compiles 2. StatusBar.Render() produces wide format when width >= 100, narrow when < 100 3. Alert.Render() produces bordered message with correct color codes 4. SuggestAlternatives() returns ranked alternatives excluding unavailable model 5. First alternative is marked “[RECOMMENDED]”

**Dependencies:** TASK 3.1.1, TASK 3.2.1 **Effort:** Medium

---

## PHASE 3 ROLLBACK PLAN

```
cd HelixCode
git checkout -- cmd/cli/main.go
git rm --cached internal/cli/ux/*.go internal/cli/tui/*.go
rm -f internal/cli/ux/symbols.go internal/cli/ux/badges.go
 internal/cli/ux/capabilities.go \
 internal/cli/ux/render.go internal/cli/ux/detail.go
 internal/cli/ux/status_bar.go \
 internal/cli/ux/alerts.go internal/cli/ux/auto_suggest.go \
 internal/cli/tui/model_selector.go
go mod tidy
make build
make test-unit
```

## PHASE 3 VERIFICATION CHECKLIST

- ☐ go build ./... compiles with zero errors
- ☐ ./cli --help shows --list-models, --model-info, --provider, --verified-only, --max-price, --min-score, --capability, --sort, --group-by, --format, --models-interactive, --refresh-models, --no-color, --no-emoji
- ☐ --list-models outputs table with Rank, Model, Provider, Score, Status columns
- ☐ --list-models --format json produces valid JSON with id, name, provider, score, verified
- ☐ --list-models --provider openai filters to OpenAI models only
- ☐ --list-models --min-score 8.0 shows only models with score >= 8.0
- ☐ --model-info gpt-4o shows full detail with capabilities, pricing, verification dimensions
- ☐ NewSymbolSet(true) returns ASCII-only symbols (no Unicode)
- ☐ VerificationBadge() returns correct color for each status
- ☐ ScoreBadge(9.5) uses green, ScoreBadge(2.0) uses red
- ☐ StatusBar.Render() fits in narrow terminals (<60 columns)





SuggestAlternatives() returns at least 1 alternative for unavailable model

---

## PHASE 4: ADVANCED FEATURES — MCPs, LSPs, ACPs, Embeddings, RAGs, Skills, Plugins

**Phase Goal:** Flawlessly incorporate all available capabilities from all supported providers, ensuring each capability respects verifier enable/disable state and has proper fallback behavior.

**Phase Entry Criteria:** Phase 3 verification checklist is complete. **Phase Exit Criteria:** 1. MCP transport works with verifier-selected models 2. LSP features query model capabilities from verifier 3. ACP agent discovery references verifier model data 4. Embedding model selection uses verifier scores 5. RAG pipeline queries verified models 6. Skills system maps to model capability requirements 7. Plugins validate against verifier before activation

---

### TASK 4.1: MCP (Model Context Protocol) Integration

#### TASK 4.1.1: Modify MCP Server to Use Verifier-Selected Models

**File(s):** helix\_code/internal/mcp/server.go **Line(s):** Search for func (s \*Server) handleListTools() or func (s \*Server) HandleRequest()

**Action:** MODIFY

Add verifier adapter field to MCPServer struct:

```
type Server struct {
 // ... existing fields ...
 verifierAdapter *verifier.Adapter // NEW – LLMsVerifier bridge
}
```

Add method to get MCP-capable models from verifier:

```
// GetMCPCapableModels returns models that support MCP/tool_use.
func (s *Server) GetMCPCapableModels(ctx context.Context)
 ([]string, error) {
 if s.verifierAdapter == nil || !
 s.verifierAdapter.IsEnabled() {
 // Fallback: return hardcoded MCP-capable models
 return []string{"gpt-4o", "claude-3-5-sonnet",
 "gemini-2.5-pro"}, nil
 }
}
```

```

 }
 models, err := s.verifierAdapter.GetVerifiedModels(ctx)
 if err != nil {
 return nil, err
 }
 result := []string{}
 for _, m := range models {
 if m.SupportsTools || m.SupportsFunctions {
 result = append(result, m.ID)
 }
 }
 return result, nil
}

```

**Acceptance Criteria:** 1. go build ./internal/mcp/... compiles 2. GetMCPCapableModels() returns only models with SupportsTools=true or SupportsFunctions=true 3. When verifier is disabled, falls back to 3 hardcoded MCP-capable models 4. When verifier is enabled, ALL returned models are from verifier DB

**Dependencies:** TASK 2.1.1 **Effort:** Medium

---

## TASK 4.2: LSP (Language Server Protocol) Integration

### TASK 4.2.1: Connect LSP Features to Model Capabilities

**File(s):** helix\_code/internal/lsp/completion.go **Line(s):** Before completion request dispatch **Action:** MODIFY

```

// selectLSPModel chooses the best model for LSP completion based
// on verifier data.
func (s *LSPService) selectLSPModel(ctx context.Context)
(string, error) {
 if s.verifierAdapter != nil && s.verifierAdapter.IsEnabled()
 {
 models, err := s.verifierAdapter.GetVerifiedModels(ctx)
 if err == nil {
 // Prefer models with high code capability score
 for _, m := range models {
 if m.SupportsCode && m.Verified &&
 m.OverallScore >=
 s.verifierAdapter.GetMinAcceptableScore() {
 return m.ID, nil
 }
 }
 }
 }
}

```

```

 // Fallback: use config default
 return s.config.DefaultModel, nil
}

```

**Acceptance Criteria:** 1. go build ./internal/lsp/... compiles 2. selectLSPModel() prefers models with SupportsCode=true when verifier is enabled 3. Falls back to config.DefaultModel when verifier is disabled or no suitable model

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## TASK 4.3: ACP (Agent Communication Protocol) Integration

### TASK 4.3.1: Reference Verifier in Agent Discovery

**File(s):** helix\_code/internal/acp/discovery.go **Line(s):** Inside DiscoverAgents() function **Action:** MODIFY

```

func (d *DiscoveryService) DiscoverAgents(ctx context.Context)
 ([]*AgentInfo, error) {
 agents := []*AgentInfo{}

 // NEW: Include verifier-aware model agents
 if d.verifierAdapter != nil && d.verifierAdapter.IsEnabled()
 {
 models, err := d.verifierAdapter.GetVerifiedModels(ctx)
 if err == nil {
 for _, m := range models {
 if m.Verified && m.OverallScore >=
 d.verifierAdapter.GetMinAcceptableScore() {
 agents = append(agents, &AgentInfo{
 ID: "model:" + m.ID,
 Name: m.DisplayName,
 Type: "llm",
 Provider: m.Provider,
 Capabilities: m.Capabilities,
 Score: m.OverallScore,
 Source: "verifier",
 })
 }
 }
 }
 }

 // ... existing discovery logic ...
 return agents, nil
}

```

**Acceptance Criteria:** 1. go build ./internal/acp/... compiles 2. DiscoverAgents() includes agents with Source: "verifier" when verifier is enabled 3. Only verified models with score >= min\_acceptable are included

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## TASK 4.4: Embeddings Integration

### TASK 4.4.1: Use Verifier to Select Optimal Embedding Models

**File(s):** helix\_code/internal/embeddings/selector.go (or equivalent)

**Line(s):** Inside embedding model selection function **Action:** MODIFY

```
// SelectEmbeddingModel returns the best embedding-capable model
// from verifier.
func (s *EmbeddingService) SelectEmbeddingModel(ctx
 context.Context) (string, error) {
 if s.verifierAdapter != nil && s.verifierAdapter.IsEnabled()
 {
 models, err := s.verifierAdapter.GetVerifiedModels(ctx)
 if err == nil {
 // Find highest-scored embedding model
 var best *verifier.VerifiedModel
 for _, m := range models {
 if m.SupportsEmbeddings && m.Verified {
 if best == nil || m.OverallScore >
 best.OverallScore {
 best = m
 }
 }
 }
 if best != nil {
 return best.ID, nil
 }
 }
 // Fallback: config default embedding model
 return s.config.DefaultEmbeddingModel, nil
 }
}
```

**Acceptance Criteria:** 1. go build ./internal/embeddings/... compiles 2. SelectEmbeddingModel() returns highest-scored model with SupportsEmbeddings=true 3. Falls back to config default when verifier is disabled

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## TASK 4.5: RAG Integration

### TASK 4.5.1: Connect RAG Pipeline to Verified Models

**File(s):** helix\_code/internal/rag/pipeline.go **Line(s):** Before LLM query dispatch in pipeline **Action:** MODIFY

```
// selectRAGModel picks a model for RAG queries using verifier
data.
func (p *Pipeline) selectRAGModel(ctx context.Context) (string,
error) {
 if p.verifierAdapter != nil && p.verifierAdapter.IsEnabled()
 {
 models, err := p.verifierAdapter.GetVerifiedModels(ctx)
 if err == nil {
 // For RAG, prefer models with large context window +
 reasoning
 var best *verifier.VerifiedModel
 for _, m := range models {
 if m.Verified && m.ContextSize >= 32000 {
 score := m.OverallScore
 if m.SupportsReasoning {
 score += 0.5 // reasoning bonus for RAG
 }
 if best == nil || score > best.OverallScore+
 (mapBoolFloat(best.SupportsReasoning)*0.5) {
 best = m
 }
 }
 }
 if best != nil {
 return best.ID, nil
 }
 }
 return p.config.DefaultRAGModel, nil
 }
}

func mapBoolFloat(b bool) float64 {
 if b {
 return 1.0
 }
 return 0.0
}
```

**Acceptance Criteria:** 1. go build ./internal/rag/... compiles 2.  
selectRAGModel() gives 0.5 bonus to models with SupportsReasoning=true 3.  
Only models with ContextSize >= 32000 are considered for RAG

## TASK 4.6: Skills Integration

### TASK 4.6.1: Map Skills to Model Capability Requirements

**File(s):** helix\_code/internal/skills/manager.go **Line(s):** Inside skill execution or validation function **Action:** MODIFY

```
// validateSkillRequirements checks if the selected model
// supports a skill's requirements.
func (m *SkillsManager) validateSkillRequirements(ctx
 context.Context, skillID string, modelID string) error {
 skill := m.registry.Get(skillID)
 if skill == nil {
 return fmt.Errorf("skill %s not found", skillID)
 }

 if m.verifierAdapter != nil && m.verifierAdapter.IsEnabled()
 {
 model, err := m.verifierAdapter.GetVerifiedModels(ctx)
 if err == nil {
 // Find the specific model
 for _, mod := range model {
 if mod.ID == modelID {
 for _, req := range
 skill.RequiredCapabilities {
 if !hasCapability(mod, req) {
 return fmt.Errorf("model %s lacks
 required capability '%s' for skill '%s'",
 modelID, req, skillID)
 }
 }
 }
 }
 return nil // all requirements met
 }
 }
}

// Fallback: skip validation if verifier is disabled
return nil
}

func hasCapability(m *verifier.VerifiedModel, cap string) bool {
 switch cap {
 case "code": return m.SupportsCode
 case "vision": return m.SupportsVision
 }
```

```

case "streaming": return m.SupportsStreaming
case "tools": return m.SupportsTools || m.SupportsFunctions
case "reasoning": return m.SupportsReasoning
case "embeddings": return m.SupportsEmbeddings
case "audio": return m.SupportsAudio
case "video": return m.SupportsVideo
default:
 for _, c := range m.Capabilities {
 if c == cap {
 return true
 }
 }
 return false
}

```

**Acceptance Criteria:** 1. go build ./internal/skills/... compiles 2. validateSkillRequirements() returns error when model lacks required capability 3. hasCapability("code") returns true when SupportsCode=true 4. Falls back to nil error when verifier is disabled

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## TASK 4.7: Plugins Integration

### TASK 4.7.1: Verify Plugins Against Verifier Before Activation

**File(s):** helix\_code/internal/plugins/manager.go **Line(s):** Inside plugin activation or validation **Action:** MODIFY

```

// validatePluginModelRequirements checks plugin model
// requirements against verifier.
func (m *PluginManager) validatePluginModelRequirements(ctx
 context.Context, plugin *Plugin) error {
 if m.verifierAdapter == nil || !
 m.verifierAdapter.IsEnabled() {
 return nil // skip when verifier disabled
 }

 models, err := m.verifierAdapter.GetVerifiedModels(ctx)
 if err != nil {
 return fmt.Errorf("cannot validate plugin: verifier
 unavailable: %w", err)
 }

 for _, req := range plugin.ModelRequirements {
 found := false

```

```

 for _, mod := range models {
 if mod.ID == req.ModelID || mod.Name == req.ModelID {
 if mod.Verified && mod.OverallScore >=
req.MinScore {
 found = true
 break
 }
 }
 }
 if !found {
 return fmt.Errorf("plugin '%s' requires model '%s'
(score >= %.1f) which is not available",
plugin.Name, req.ModelID, req.MinScore)
 }
}
return nil
}

```

**Acceptance Criteria:** 1. go build ./internal/plugins/... compiles 2. validatePluginModelRequirements() returns error when required model is unavailable 3. Returns nil when verifier is disabled (backward compatible)

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## TASK 4.8: Token Usage Tracking Integration

### TASK 4.8.1: Track Token Usage with Verifier Context

**File(s):** helix\_code/internal/usage/tracker.go (or equivalent) **Line(s):** CREATE new file or modify existing **Action:** CREATE/MODIFY

**package** usage

```

import (
 "dev.helix.code/internal/verifier"
 "sync"
 "time"
)

```

*// TokenUsageTracker tracks per-model token usage with verifier integration.*

```

type TokenUsageTracker struct {
 mu sync.RWMutex
 records map[string]*UsageRecord // key: model_id
 adapter *verifier.Adapter
}

```



```

type UsageRecord struct {
 ModelID string `json:"model_id"`
 Provider string `json:"provider"`
 InputTokens int64 `json:"input_tokens"`
 OutputTokens int64 `json:"output_tokens"`
 CostUSD float64 `json:"cost_usd"`
 RequestCount int64 `json:"request_count"`
 LastUsed time.Time `json:"last_used"`
 ScoreAtUse float64 `json:"score_at_use" // verifier
 score snapshot
}

// RecordUsage logs token usage and captures verifier score at
// time of use.
func (t *TokenUsageTracker) RecordUsage(modelID, provider
 string, inputTokens, outputTokens int64) {
 t.mu.Lock()
 defer t.mu.Unlock()

 var score float64
 if t.adapter != nil {
 if s, ok := t.adapter.GetModelScore(modelID); ok {
 score = s
 }
 }

 record := t.records[modelID]
 if record == nil {
 record = &UsageRecord{ModelID: modelID, Provider:
 provider}
 t.records[modelID] = record
 }
 record.InputTokens += inputTokens
 record.OutputTokens += outputTokens
 record.RequestCount++
 record.LastUsed = time.Now()
 record.ScoreAtUse = score
}

// GetUsageReport returns aggregated usage data.
func (t *TokenUsageTracker) GetUsageReport() []*UsageRecord {
 t.mu.RLock()
 defer t.mu.RUnlock()
 result := make([]*UsageRecord, 0, len(t.records))
 for _, r := range t.records {
 result = append(result, r)
 }
}

```

```
 return result
}
```

**Acceptance Criteria:** 1. go build ./internal/usage/... compiles 2. RecordUsage() captures verifier score at time of use 3. GetUsageReport() returns all recorded usage

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## TASK 4.9: Pricing Update Integration

### TASK 4.9.1: Integrate Real-Time Pricing from Verifier

**File(s):** helix\_code/internal/pricing/monitor.go (or equivalent) **Line(s):** CREATE new file **Action:** CREATE

```
package pricing

import (
 "context"
 "dev.helix.code/internal/verifier"
 "time"
)

// Monitor watches for pricing changes from LLMsVerifier.
type Monitor struct {
 adapter *verifier.Adapter
 interval time.Duration
 lastHash string
}

// NewMonitor creates a pricing monitor.
func NewMonitor(adapter *verifier.Adapter, interval
 time.Duration) *Monitor {
 return &Monitor{adapter: adapter, interval: interval}
}

// CheckForChanges fetches current pricing and detects changes.
func (m *Monitor) CheckForChanges(ctx context.Context)
 (*PricingUpdate, error) {
 if m.adapter == nil || !m.adapter.IsEnabled() {
 return nil, verifier.ErrVerifierDisabled
 }
 pricing, err := m.adapter.client.GetPricing(ctx)
 if err != nil {
 return nil, err
 }
}
```

```

 return &PricingUpdate{Models: pricing, ChangedAt:
 time.Now()}, nil
}

type PricingUpdate struct {
 Models []map[string]interface{} `json:"models"`
 ChangedAt time.Time `json:"changed_at"`
}

```

**Acceptance Criteria:** 1. go build ./internal/pricing/... compiles 2. CheckForChanges() returns error when verifier is disabled 3. Pricing data structure is preserved as []map[string]interface{} for flexibility

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## TASK 4.10: Rate Limiting Integration with Cooldown State

### TASK 4.10.1: Integrate Rate Limit and Cooldown Awareness

**File(s):** helix\_code/internal/ratelimit/verifier\_integration.go (or equivalent) **Line(s):** CREATE new file **Action:** CREATE

```

package ratelimit

import (
 "context"
 "dev.helix.code/internal/verifier"
 "fmt"
 "time"
)

// VerifierRateLimiter wraps rate limit decisions with verifier
// cooldown data.
type VerifierRateLimiter struct {
 adapter *verifier.Adapter
 inner *RateLimiter // existing rate limiter
}

// NewVerifierRateLimiter creates a verifier-aware rate limiter.
func NewVerifierRateLimiter(adapter *verifier.Adapter, inner
 *RateLimiter) *VerifierRateLimiter {
 return &VerifierRateLimiter{adapter: adapter, inner: inner}
}

// AllowRequest checks both local rate limiter AND verifier
// cooldown state.

```

```

func (r *VerifierRateLimiter) AllowRequest(ctx context.Context,
 modelID string) (bool, time.Duration, error) {
 // Check local rate limiter first
 allowed, retryAfter, err := r.inner.AllowRequest(modelID)
 if !allowed {
 return false, retryAfter, err
 }

 // Check verifier cooldown state
 if r.adapter != nil && r.adapter.IsEnabled() {
 model, err := r.adapter.client.GetModelByID(ctx, modelID)
 if err == nil && model != nil {
 if model.VerificationStatus == "rate_limited" {
 return false, 60 * time.Second,
 fmt.Errorf("model %s is in cooldown", modelID)
 }
 }
 }

 return true, 0, nil
}

```

**Acceptance Criteria:** 1. go build ./internal/ratelimit/... compiles 2. AllowRequest() returns false when model is in rate\_limited status 3. Falls back to local rate limiter when verifier is disabled

**Dependencies:** TASK 2.1.1 **Effort:** Small

---

## TASK 4.11: Extend LLMsVerifier to Work with ALL Providers

### TASK 4.11.1: Document Required Provider Adapter Additions

**File(s):** N/A — This is a gap in LLMsVerifier that must be fixed in the submodule

**Line(s):** N/A **Action:** DOCUMENT

LLMsVerifier currently supports only 12 providers. The following 23 additional provider adapters MUST be added to LLMsVerifier to achieve full HelixCode parity:

#	Provider	Status	Required Action
1	Azure	MISSING	Add providers/azure.go adapter
2	AWS Bedrock	MISSING	Add providers/bedrock.go adapter

#	Provider	Status	Required Action
3	Google Vertex AI	MISSING	Add providers/vertex.go adapter
4	Together AI	MISSING	Add providers/together.go adapter
5	Cerebras	MISSING	Add providers/cerebras.go adapter
6	Cloudflare Workers AI	MISSING	Add providers/cloudflare.go adapter
7	SiliconFlow	MISSING	Add providers/siliconflow.go adapter
8	Replicate	MISSING	Add providers/replicate.go adapter
9	Qwen (Alibaba)	MISSING	Add providers/qwen.go adapter
10	Cohere	MISSING	Add providers/cohere.go adapter
11	vLLM	MISSING	Add providers/vllm.go adapter
12	LocalAI	MISSING	Add providers/localai.go adapter
13	Hugging Face	MISSING	Add providers/huggingface.go adapter
14	Perplexity	MISSING	Add providers/perplexity.go adapter
15	AI21 Labs	MISSING	Add providers/ai21.go adapter
16	Aleph Alpha	MISSING	Add providers/alephalpha.go adapter
17	Fireworks AI	MISSING	Add providers/fireworks.go adapter
18	Anyscale	MISSING	Add providers/anyscale.go adapter
19	Predibase	MISSING	Add providers/predibase.go adapter
20	Lepton AI	MISSING	Add providers/lepton.go adapter
21	Nvidia NIM	MISSING	Add providers/nvidia.go adapter

#	Provider	Status	Required Action
22	Baseten	MISSING	Add providers/baseten.go adapter
23	OctoAI	MISSING	Add providers/octoai.go adapter

Each adapter must implement the Provider interface:

```
type Provider interface {
 Name() string
 ListModels(ctx context.Context) ([]*Model, error)
 VerifyModel(ctx context.Context, modelID string)
 (*VerificationResult, error)
 GetPricing(ctx context.Context) ([]PricingInfo, error)
 GetLimits(ctx context.Context) ([]RateLimitInfo, error)
}
```

**Acceptance Criteria:** 1. Document lists all 23 missing providers 2. Each entry specifies the required file path 3. Interface definition is provided

**Dependencies:** None **Effort:** Large (23 individual tasks in submodule)

---

## TASK 4.12: Fix LLMsVerifier Stub Verification

### TASK 4.12.1: Document and Plan Stub Fixes

**File(s):** N/A — LLMsVerifier submodule fix **Line(s):** LLMsVerifier/verification/verification.go:VerifyModel() **Action:** DOCUMENT

**STUB-001 Fix Required:** In LLMsVerifier/verification/verification.go, the VerifyModel() function returns hardcoded 8.5 for ALL dimensions. This MUST be replaced with real verification logic:

```
// BEFORE (stub):
func VerifyModel(modelID string) (*VerificationResult, error) {
 return &VerificationResult{
 OverallScore: 8.5,
 CodeCapabilityScore: 8.5,
 ResponsivenessScore: 8.5,
 // ... all hardcoded 8.5
 }, nil
}

// AFTER (real):
```

```

func VerifyModel(modelID string, adapter ProviderAdapter)
 (*VerificationResult, error) {
 result := &VerificationResult{ModelID: modelID}

 // 1. Existence check
 exists, err := adapter.HeadModel(modelID)
 result.ModelExists = &exists

 // 2. Responsiveness check
 respStart := time.Now()
 _, err = adapter.PingModel(modelID, "Say 'hello' and nothing
 else.")
 latency := time.Since(respStart)
 responsive := err == nil
 result.Responsive = &responsive

 // 3. Code capability check
 codeResp, err := adapter.TestCodeGeneration(modelID)
 result.SupportsCodeGeneration = err == nil &&
 codeResp.Contains("func")

 // 4. Calculate real scores from test results
 result.OverallScore = calculateOverallScore(result, latency)
 result.CodeCapabilityScore = calculateCodeScore(codeResp)
 result.ResponsivenessScore =
 calculateResponsivenessScore(latency)
 // ... etc.

 return result, nil
}

```

**Acceptance Criteria:** 1. Document clearly shows the before/after code 2. Real verification calls provider adapter methods 3. Scores are calculated from actual test results

**Dependencies:** None **Effort:** Large (submodule fix)

---

## PHASE 4 ROLLBACK PLAN

```

cd HelixCode
git checkout -- internal/mcp/server.go
git checkout -- internal/lsp/completion.go
git checkout -- internal/acp/discovery.go
git rm --cached internal/embeddings/selector.go internal/rag/
 pipeline.go \
 internal/skills/manager.go internal/plugins/manager.go \
 internal/usage/tracker.go internal/pricing/monitor.go \

```

```
 internal/ratelimit/verifier_integration.go
rm -f internal/embeddings/selector.go internal/rag/pipeline.go \
 internal/skills/manager.go internal/plugins/manager.go \
 internal/usage/tracker.go internal/pricing/monitor.go \
 internal/ratelimit/verifier_integration.go
go mod tidy
make build
make test-unit
```

## PHASE 4 VERIFICATION CHECKLIST

- ☐ go build ./... compiles with zero errors
  - ☐ MCP server includes verifier-aware models in tool list
  - ☐ LSP completion uses verifier-selected model with SupportsCode=true
  - ☐ ACP discovery includes agents with Source: "verifier"
  - ☐ Embedding selector returns model with SupportsEmbeddings=true
  - ☐ RAG selector prefers models with ContextSize >= 32000 and SupportsReasoning=true
  - ☐ Skills manager rejects skill execution when model lacks required capability
  - ☐ Plugin manager rejects activation when required model is unavailable
  - ☐ Token usage tracker captures verifier score at time of use
  - ☐ Pricing monitor fetches from verifier when enabled
  - ☐ Rate limiter rejects requests for models in rate\_limited status
  - ☐ All capability integrations respect verifier.enabled flag (fallback when disabled)
- 

## PHASE 5: TESTING IMPLEMENTATION — Anti-Bluff Test Suite

**Phase Goal:** Implement all tests that guarantee every feature actually works. No simulated data, no hardcoded expectations, no bluff.



**Phase Entry Criteria:** Phase 4 verification checklist is complete. **Phase Exit Criteria:** 1. All 20+ unit test files compile and pass in -short mode 2. All 6+ contract test files pass against real verifier endpoints 3. All 6+ component test files pass with real subsystem wiring 4. All 8+ integration test files pass with real dependencies 5. All 12 challenge scripts pass 6. scripts/enforce\_coverage.sh passes (100% unit, 95% integration) 7. scripts/no\_mock\_above\_unit.sh passes (zero violations)

---

## TASK 5.1: Create Unit Test Files

### TASK 5.1.1: Create internal/verifier/client\_test.go

**File(s):** helix\_code/internal/verifier/client\_test.go **Line(s):** CREATE  
**Action:** CREATE

```
package verifier

import (
 "context"
 "encoding/json"
 "net/http"
 "net/http/httptest"
 "testing"
 "time"
)

func TestNewClient_Defaults(t *testing.T) {
 c := NewClient("", "", 0)
 if c.baseURL != "http://localhost:8081" {
 t.Errorf("expected default baseURL, got %s", c.baseURL)
 }
 if c.timeout != 30*time.Second {
 t.Errorf("expected default timeout, got %s", c.timeout)
 }
}

func TestClient_Health(t *testing.T) {
 server := httptest.NewServer(http.HandlerFunc(func(w
 http.ResponseWriter, r *http.Request) {
 if r.URL.Path != "/api/health" {
 t.Errorf("unexpected path: %s", r.URL.Path)
 }
 if r.Header.Get("Authorization") != "Bearer test-key" {
 t.Errorf("missing auth header")
 }
 })
}
```

```

 json.NewEncoder(w).Encode(HealthResponse{Status: "ok",
 Version: "1.0.0"})
)))
defer server.Close()

client := NewClient(server.URL, "test-key", 5*time.Second)
hr, err := client.Health(context.Background())
if err != nil {
 t.Fatalf("health check failed: %v", err)
}
if hr.Status != "ok" {
 t.Errorf("expected status ok, got %s", hr.Status)
}
}

func TestClient_GetModels(t *testing.T) {
 server := httptest.NewServer(http.HandlerFunc(func(w
 http.ResponseWriter, r *http.Request) {
 if r.URL.Path != "/api/models" {
 t.Errorf("unexpected path: %s", r.URL.Path)
 }
 json.NewEncoder(w).Encode([]*VerifiedModel{
 {ID: "gpt-4o", Name: "GPT-4o", Provider: "openai",
 OverallScore: 9.2},
 {ID: "claude-3-5-sonnet", Name: "Claude 3.5 Sonnet",
 Provider: "anthropic", OverallScore: 8.9},
 })
 }
))
defer server.Close()

client := NewClient(server.URL, "test-key", 5*time.Second)
models, err := client.GetModels(context.Background())
if err != nil {
 t.Fatalf("get models failed: %v", err)
}
if len(models) != 2 {
 t.Errorf("expected 2 models, got %d", len(models))
}
if models[0].ID != "gpt-4o" {
 t.Errorf("expected gpt-4o, got %s", models[0].ID)
}
if models[0].OverallScore != 9.2 {
 t.Errorf("expected score 9.2, got %.1f",
 models[0].OverallScore)
}
}

func TestClient_GetModelByID_NotFound(t *testing.T) {

```

```

server := httptest.NewServer(http.HandlerFunc(func(w
 http.ResponseWriter, r *http.Request) {
 w.WriteHeader(http.StatusNotFound)
 }))
defer server.Close()

client := NewClient(server.URL, "", 5*time.Second)
_, err := client.GetModelByID(context.Background(),
 "nonexistent")
if err == nil {
 t.Fatalf("expected error for 404")
}
}

func TestClient_GetProviderScores(t *testing.T) {
 server := httptest.NewServer(http.HandlerFunc(func(w
 http.ResponseWriter, r *http.Request) {
 json.NewEncoder(w).Encode(map[string]float64{"openai":
 9.1, "anthropic": 8.9})
 }))
 defer server.Close()

 client := NewClient(server.URL, "", 5*time.Second)
 scores, err := client.GetProviderScores(context.Background())
 if err != nil {
 t.Fatalf("get scores failed: %v", err)
 }
 if scores["openai"] != 9.1 {
 t.Errorf("expected openai score 9.1, got %.1f",
 scores["openai"])
 }
}

func TestClient_AuthHeader_NoKey(t *testing.T) {
 server := httptest.NewServer(http.HandlerFunc(func(w
 http.ResponseWriter, r *http.Request) {
 if r.Header.Get("Authorization") != "" {
 t.Error("auth header should be empty when no key
 configured")
 }
 json.NewEncoder(w).Encode(HealthResponse{Status: "ok"})
 }))
 defer server.Close()

 client := NewClient(server.URL, "", 5*time.Second)
 _, err := client.Health(context.Background())
 if err != nil {
 t.Fatalf("health check failed: %v", err)
 }
}

```

```
}
}
```

**Acceptance Criteria:** 1. `go test -short ./internal/verifier/...` passes 2. All 6 test functions compile and run 3. `TestNewClient_Defaults` verifies defaults without mocks 4. `TestClient_AuthHeader_NoKey` proves no auth sent when key is empty

**Dependencies:** TASK 1.6.1 **Effort:** Large

---

### TASK 5.1.2: Create `internal/verifier/cache_test.go`

**File(s):** `helix_code/internal/verifier/cache_test.go` **Line(s):** CREATE  
**Action:** CREATE

```
package verifier

import (
 "testing"
 "time"
)

func TestCache_GetSet(t *testing.T) {
 cache := NewCache(5*time.Minute, nil)
 models := []*VerifiedModel{
 {ID: "gpt-4o", OverallScore: 9.2},
 }
 cache.SetModels("all", models)
 got, ok := cache.GetModels("all")
 if !ok {
 t.Fatal("expected cache hit")
 }
 if len(got) != 1 || got[0].ID != "gpt-4o" {
 t.Errorf("unexpected cached model: %v", got)
 }
}

func TestCache_TTLExpiry(t *testing.T) {
 cache := NewCache(1*time.Millisecond, nil)
 models := []*VerifiedModel{{ID: "test", OverallScore: 5.0}}
 cache.SetModels("all", models)
 time.Sleep(2 * time.Millisecond)
 _, ok := cache.GetModels("all")
 if ok {
 t.Error("expected cache miss after TTL expiry")
 }
}
```

```

func TestCache_StaleFallback(t *testing.T) {
 cache := NewCache(1*time.Millisecond, nil)
 models := []*VerifiedModel{{ID: "test", OverallScore: 5.0}}
 cache.SetModels("all", models)
 time.Sleep(2 * time.Millisecond)
 got, ok := cache.GetModelsStale("all")
 if !ok {
 t.Error("expected stale cache hit within 2x TTL")
 }
 if len(got) != 1 {
 t.Errorf("expected 1 model from stale cache, got %d",
 len(got))
 }
}

func TestCache_Invalidation(t *testing.T) {
 cache := NewCache(5*time.Minute, nil)
 cache.SetModels("all", []*VerifiedModel{{ID: "test"}})
 cache.Invalidate("all")
 _, ok := cache.GetModels("all")
 if ok {
 t.Error("expected cache miss after invalidation")
 }
}

func TestCache_ModelScore(t *testing.T) {
 cache := NewCache(5*time.Minute, nil)
 cache.SetScores(map[string]float64{"gpt-4o": 9.2})
 score, ok := cache.GetModelScore("gpt-4o")
 if !ok {
 t.Fatal("expected score cache hit")
 }
 if score != 9.2 {
 t.Errorf("expected score 9.2, got %.1f", score)
 }
}

```

**Acceptance Criteria:** 1. go test -short ./internal/verifier/... passes 2. All 5 cache test functions compile and run 3. TestCache\_TTLExpiry proves TTL works (real time, not mock) 4. TestCache\_StaleFallback proves stale cache works within 2x TTL

**Dependencies:** TASK 2.4.1 **Effort:** Medium

---

### TASK 5.1.3: Create internal/verifier/health\_test.go

**File(s):** helix\_code/internal/verifier/health\_test.go **Line(s):** CREATE  
**Action:** CREATE

```
package verifier

import (
 "dev.helix.code/internal/config"
 "testing"
 "time"
)

func TestHealthMonitor_CircuitBreaker(t *testing.T) {
 cfg := config.VerifierHealthConfig{
 FailureThreshold: 3,
 RecoveryThreshold: 2,
 CircuitBreaker: config.CircuitBreakerConfig{
 Enabled: true,
 HalfOpenTimeout: 1 * time.Second,
 },
 }
 h := NewHealthMonitor(cfg)

 // Record failures up to threshold
 h.RecordFailure()
 h.RecordFailure()
 if h.IsCircuitOpen() {
 t.Error("circuit should not be open after 2 failures")
 }
 h.RecordFailure()
 if !h.IsCircuitOpen() {
 t.Error("circuit should be open after 3 failures")
 }
 if h.AllowRequest() {
 t.Error("request should not be allowed when circuit open")
 }

 // Wait for half-open timeout
 time.Sleep(1100 * time.Millisecond)
 if !h.AllowRequest() {
 t.Error("request should be allowed in half-open state")
 }

 // Record successes to close circuit
 h.RecordSuccess()
 h.RecordSuccess()
}
```

```

 if h.GetState() != CircuitClosed {
 t.Errorf("circuit should be closed, got state %d",
 h.GetState())
 }
 if !h.AllowRequest() {
 t.Error("request should be allowed when circuit closed")
 }
}

func TestHealthMonitor_SuccessResetsFailures(t *testing.T) {
 cfg := config.VerifierHealthConfig{
 FailureThreshold: 5,
 CircuitBreaker: config.CircuitBreakerConfig{Enabled:
 true},
 }
 h := NewHealthMonitor(cfg)
 h.RecordFailure()
 h.RecordFailure()
 h.RecordSuccess() // should reset failures in closed state
 h.RecordFailure()
 h.RecordFailure()
 if h.IsCircuitOpen() {
 t.Error("circuit should not be open: only 2 failures
 after reset")
 }
}

func TestHealthMonitor_DisabledBreaker(t *testing.T) {
 cfg := config.VerifierHealthConfig{
 FailureThreshold: 1,
 CircuitBreaker: config.CircuitBreakerConfig{Enabled:
 false},
 }
 h := NewHealthMonitor(cfg)
 h.RecordFailure()
 if h.IsCircuitOpen() {
 t.Error("circuit should NOT open when breaker is
 disabled")
 }
}

```

**Acceptance Criteria:** 1. go test -short ./internal/verifier/... passes 2. TestHealthMonitor\_CircuitBreaker tests real state transitions with real time 3. TestHealthMonitor\_SuccessResetsFailures proves reset behavior 4. TestHealthMonitor\_DisabledBreaker proves circuit breaker can be disabled

**Dependencies:** TASK 2.5.1 **Effort:** Medium

---

#### TASK 5.1.4: Create internal/verifier/adapter\_test.go

**File(s):** helix\_code/internal/verifier/adapter\_test.go **Line(s):** CREATE  
**Action:** CREATE

```
package verifier

import (
 "context"
 "dev.helix.code/internal/config"
 "testing"
)

func TestAdapter_IsEnabled(t *testing.T) {
 a := NewAdapter(nil, nil, nil,
 &config.VerifierConfig{Enabled: false})
 if a.IsEnabled() {
 t.Error("should be disabled")
 }
 a = NewAdapter(nil, nil, nil,
 &config.VerifierConfig{Enabled: true})
 if !a.IsEnabled() {
 t.Error("should be enabled")
 }
}

func TestAdapter_GetModelScore(t *testing.T) {
 a := NewAdapter(nil, nil, nil,
 &config.VerifierConfig{Enabled: true})
 a.refreshScores([]*VerifiedModel{
 {ID: "gpt-4o", OverallScore: 9.2},
 })
 score, ok := a.GetModelScore("gpt-4o")
 if !ok {
 t.Fatal("expected score found")
 }
 if score != 9.2 {
 t.Errorf("expected 9.2, got %.1f", score)
 }
}

func TestAdapter_GetModelScore_Normalization(t *testing.T) {
 a := NewAdapter(nil, nil, nil,
 &config.VerifierConfig{Enabled: true})
 a.refreshScores([]*VerifiedModel{
 {ID: "big-score", OverallScore: 95.0}, // >10, should
 normalize
 })
}
```



```

 score, ok := a.GetModelScore("big-score")
 if !ok {
 t.Fatal("expected score found")
 }
 if score != 9.5 {
 t.Errorf("expected normalized 9.5, got %.1f", score)
 }
}

func TestAdapter_FilterByProviderConfig(t *testing.T) {
 a := NewAdapter(nil, nil, nil, &config.VerifierConfig{
 Enabled: true,
 Providers: map[string]config.VerifierProviderConfig{
 "openai": {Enabled: true},
 "xai": {Enabled: false},
 },
 })
 models := []*VerifiedModel{
 {ID: "gpt-4o", Provider: "openai"},
 {ID: "grok-3", Provider: "xai"},
 }
 filtered := a.filterByProviderConfig(models)
 if len(filtered) != 1 || filtered[0].Provider != "openai" {
 t.Errorf("expected only openai model, got %v", filtered)
 }
}

func TestAdapter_GetFallbackModels(t *testing.T) {
 a := NewAdapter(nil, nil, nil,
 &config.VerifierConfig{Enabled: true})
 models, err := a.getFallbackModels()
 if err != ErrUsingFallback {
 t.Errorf("expected ErrUsingFallback, got %v", err)
 }
 if len(models) != 7 {
 t.Errorf("expected 7 fallback models, got %d",
 len(models))
 }
 for _, m := range models {
 if m.Source != "fallback" {
 t.Errorf("expected source fallback, got %s",
 m.Source)
 }
 }
}

```

**Acceptance Criteria:** 1. `go test -short ./internal/verifier/...` passes 2. `TestAdapter_GetModelScore_Normalization` proves score >10 is divided by

10 3. TestAdapter\_FilterByProviderConfig proves disabled providers are filtered out 4. TestAdapter\_GetFallbackModels proves exactly 7 fallback models with source="fallback"

**Dependencies:** TASK 2.1.1 **Effort:** Medium

---

### TASK 5.1.5: Create internal/verifier/polling\_test.go

**File(s):** helix\_code/internal/verifier/polling\_test.go **Line(s):** CREATE new file **Action:** CREATE

```
package verifier

import (
 "dev.helix.code/internal/config"
 "testing"
 "time"
)

func TestPoller_MinimumInterval(t *testing.T) {
 p := NewPoller(nil, 1*time.Second)
 if p.interval != 10*time.Second {
 t.Errorf("minimum interval should be 10s, got %s",
 p.interval)
 }
}

func TestPoller_StartStop(t *testing.T) {
 cfg := &config.VerifierConfig{Enabled: true}
 adapter := NewAdapter(nil, nil, nil, cfg)
 p := NewPoller(adapter, 1*time.Hour) // long interval so no
 actual polls
 p.Start()
 if !p.IsRunning() {
 t.Error("poller should be running after Start()")
 }
 p.Stop()
 if p.IsRunning() {
 t.Error("poller should not be running after Stop()")
 }
}

func TestPoller_DetectChanges(t *testing.T) {
 p := &Poller{lastModels: make(map[string]*VerifiedModel)}
 old := []*VerifiedModel{
 {ID: "gpt-4o", OverallScore: 9.0, VerificationStatus:
 "verified"},
 }
```

```

}
new := []*VerifiedModel{
 {ID: "gpt-4o", OverallScore: 9.2, VerificationStatus:
 "verified"},
 {ID: "claude-3", OverallScore: 8.9, VerificationStatus:
 "verified"},
}
p.lastModels = indexModels(old)
changes := p.detectChanges(old, new)
if len(changes) != 2 {
 t.Errorf("expected 2 changes (score + discovered), got
 %d", len(changes))
}
hasScoreChange := false
hasDiscovered := false
for _, c := range changes {
 if c.Type == "model.score_changed" {
 hasScoreChange = true
 }
 if c.Type == "model.discovered" && c.Model.ID ==
 "claude-3" {
 hasDiscovered = true
 }
}
if !hasScoreChange {
 t.Error("expected score change event")
}
if !hasDiscovered {
 t.Error("expected discovered event for claude-3")
}
}

```

**Acceptance Criteria:** 1. go test -short ./internal/verifier/... passes 2. TestPoller\_MinimumInterval proves 10s floor 3. TestPoller\_StartStop proves no deadlock 4. TestPoller\_DetectChanges proves change detection works

**Dependencies:** TASK 2.3.1 **Effort:** Medium

---

#### **TASK 5.1.6: Create internal/llm/verifier\_integration\_test.go**

**File(s):** helix\_code/internal/llm/verifier\_integration\_test.go **Line(s):** CREATE new file **Action:** CREATE

```
package llm
```

```
import (
```

```

 "dev.helix.code/internal/verifier"
 "testing"
)

func TestVerifierModelSource_IsAvailable(t *testing.T) {
 src := NewVerifierModelSource(nil)
 if src.IsAvailable() {
 t.Error("should not be available with nil adapter")
 }
}

func TestVerifierModelSource_FetchModels(t *testing.T) {
 models := []*verifier.VerifiedModel{
 {ID: "gpt-4o", DisplayName: "GPT-4o", Provider: "openai",
 SupportsCode: true, SupportsStreaming: true,
 SupportsTools: true,
 OverallScore: 9.2},
 }
 src := &VerifierModelSource{}
 converted := src.convert(models)
 if len(converted) != 1 {
 t.Fatalf("expected 1 model, got %d", len(converted))
 }
 if converted[0].ID != "gpt-4o" {
 t.Errorf("expected gpt-4o, got %s", converted[0].ID)
 }
 // Verify capabilities mapped
 hasCode := false
 for _, c := range converted[0].Capabilities {
 if c == CapabilityCodeGeneration {
 hasCode = true
 }
 }
 if !hasCode {
 t.Error("expected CapabilityCodeGeneration mapped from
 SupportsCode")
 }
}

```

**Acceptance Criteria:** 1. `go test -short ./internal/llm/...` passes 2. `TestVerifierModelSource_FetchModels` proves conversion from verifier to `ModelInfo` 3. Capabilities are correctly mapped

**Dependencies:** TASK 2.11.1 **Effort:** Small

---

### TASK 5.1.7: Create internal/cli/ux/render\_test.go

**File(s):** helix\_code/internal/cli/ux/render\_test.go **Line(s):** CREATE new file **Action:** CREATE

```
package ux
```

```
import (
 "dev.helix.code/internal/verifier"
 "encoding/json"
 "strings"
 "testing"
)
```

```
func TestRenderModelList_Wide(t *testing.T) {
 rows := []*ModelListRow{
 {Model: &verifier.VerifiedModel{ID: "gpt-4o",
 DisplayName: "GPT-4o", Provider: "openai",
 OverallScore: 9.2, VerificationStatus: "verified"},
 Rank: 1},
 }
 sym := NewSymbolSet(false)
 out := RenderModelList(rows, sym, 150,
 &RenderOptions{Format: "table"})
 if !strings.Contains(out, "GPT-4o") {
 t.Error("output should contain model name")
 }
 if !strings.Contains(out, "9.2") {
 t.Error("output should contain score")
 }
}
```

```
func TestRenderModelList_JSON(t *testing.T) {
 rows := []*ModelListRow{
 {Model: &verifier.VerifiedModel{ID: "gpt-4o",
 DisplayName: "GPT-4o", Provider: "openai",
 OverallScore: 9.2, Verified: true}, Rank: 1},
 }
 sym := NewSymbolSet(false)
 out := RenderModelList(rows, sym, 150,
 &RenderOptions{Format: "json"})
 var data []map[string]interface{}
 if err := json.Unmarshal([]byte(out), &data); err != nil {
 t.Fatalf("invalid JSON: %v", err)
 }
 if len(data) != 1 {
 t.Fatalf("expected 1 JSON object, got %d", len(data))
 }
}
```

```

 if data[0]["id"] != "gpt-4o" {
 t.Errorf("expected id gpt-4o, got %v", data[0]["id"])
 }
}

func TestRenderModelList_Compact(t *testing.T) {
 rows := []*ModelListRow{
 {Model: &verifier.VerifiedModel{ID: "gpt-4o",
 DisplayName: "GPT-4o", Provider: "openai",
 OverallScore: 9.2, VerificationStatus: "verified"},
 Rank: 1},
 }
 sym := NewSymbolSet(false)
 out := RenderModelList(rows, sym, 40, &RenderOptions{Format:
 "table"})
 if strings.Contains(out, "Capabilities") {
 t.Error("compact view should not show capabilities
 header")
 }
}

func TestTruncate(t *testing.T) {
 if truncate("hello world", 5) != "he..." {
 t.Errorf("unexpected truncate result: %s",
 truncate("hello world", 5))
 }
 if truncate("hi", 5) != "hi" {
 t.Errorf("short string should not be truncated")
 }
}

```

**Acceptance Criteria:** 1. go test -short ./internal/cli/ux/... passes 2.  
 TestRenderModelList\_JSON proves valid JSON output 3.  
 TestRenderModelList\_Compact proves narrow rendering

**Dependencies:** TASK 3.4.1 **Effort:** Small

---

## TASK 5.2: Create Contract Test Files

### TASK 5.2.1: Create tests/contract/verifier\_schema\_contract\_test.go

**File(s):** helix\_code/tests/contract/verifier\_schema\_contract\_test.go

**Line(s):** CREATE new file **Action:** CREATE

**package** contract

**import** (

```

"context"
"dev.helix.code/internal/verifier"
"encoding/json"
"net/http"
"net/http/httptest"
"testing"
"time"
)

// ContractTest verifies that the verifier API schema matches
// expectations.
func ContractTest_VerifierSchema(t *testing.T) {
 server := httptest.NewServer(http.HandlerFunc(func(w
 http.ResponseWriter, r *http.Request) {
 switch r.URL.Path {
 case "/api/models":
 data := []map[string]interface{}{
 {
 "id": "test-gpt-4o",
 "name": "Test GPT-4o",
 "display_name": "Test GPT-4o",
 "provider": "openai",
 "score": 9.2,
 "verified": true,
 "verification_status": "verified",
 "context_window_tokens": 128000,
 "max_output_tokens": 4096,
 "supports_streaming": true,
 "supports_tool_use": true,
 "supports_functions": true,
 "supports_code_generation": true,
 "supports_vision": true,
 "overall_score": 9.2,
 "code_capability_score": 9.5,
 "responsiveness_score": 8.8,
 "reliability_score": 9.0,
 "feature_richness_score": 9.1,
 "value_proposition_score": 8.0,
 "input_token_cost": 0.005,
 "output_token_cost": 0.015,
 "latency_ms": 250,
 },
 }
 _ = json.NewEncoder(w).Encode(data)
 default:
 w.WriteHeader(http.StatusNotFound)
 }
 })
}

```

```

defer server.Close()

client := verifier.NewClient(server.URL, "", 5*time.Second)
models, err := client.GetModels(context.Background())
if err != nil {
 t.Fatalf("contract test failed: %v", err)
}
if len(models) == 0 {
 t.Fatal("expected at least one model")
}

// Schema validation: all required fields must be present
m := models[0]
if m.ID == "" {
 t.Error("contract violation: id is empty")
}
if m.DisplayName == "" {
 t.Error("contract violation: display_name is empty")
}
if m.Provider == "" {
 t.Error("contract violation: provider is empty")
}
if m.OverallScore == 0 {
 t.Error("contract violation: overall_score is 0")
}
}

```

**Acceptance Criteria:** 1. go test -v -run Contract ./tests/contract/... passes 2. Schema validation checks all required fields 3. Uses real httptest server (not mocks above unit level)

**Dependencies:** TASK 1.6.1 **Effort:** Medium

---

### TASK 5.2.2: Create tests/contract/error\_response\_contract\_test.go

**File(s):** helix\_code/tests/contract/error\_response\_contract\_test.go

**Line(s):** CREATE new file **Action:** CREATE

**package** contract

```

import (
 "context"
 "dev.helix.code/internal/verifier"
 "encoding/json"
 "net/http"
 "net/http/httptest"
 "testing"

```



```

 "time"
)

 // ContractTest_ErrorResponseFormat verifies error JSON
 // structure.
 func ContractTest_ErrorResponseFormat(t *testing.T) {
 server := httptest.NewServer(http.HandlerFunc(func(w
 http.ResponseWriter, r *http.Request) {
 w.WriteHeader(http.StatusBadRequest)
 json.NewEncoder(w).Encode(map[string]interface{}{
 "error": "invalid_request",
 "message": "Model ID is required",
 "code": 400,
 })
 }))
 defer server.Close()

 client := verifier.NewClient(server.URL, "", 5*time.Second)
 _, err := client.GetModelByID(context.Background(), "")
 if err == nil {
 t.Fatal("expected error")
 }
 // Verify error contains expected fields
 errStr := err.Error()
 if errStr == "" {
 t.Error("error should not be empty")
 }
 }
}

```

**Acceptance Criteria:** 1. go test -v -run Contract ./tests/contract/... passes 2. Error response has error, message, code fields

**Dependencies:** TASK 1.6.1 **Effort:** Small

---

## TASK 5.3: Create Component Test Files

**TASK 5.3.1: Create tests/component/  
model\_manager\_verifier\_component\_test.go**

**File(s):** helix\_code/tests/component/  
model\_manager\_verifier\_component\_test.go **Line(s):** CREATE new file  
**Action:** CREATE

```
package component
```

```
import (
 "context"
```

```

"dev.helix.code/internal/config"
"dev.helix.code/internal/llm"
"dev.helix.code/internal/verifier"
"encoding/json"
"net/http"
"net/http/httptest"
"testing"
"time"
)

// ComponentTest_ModelManagerSelectModel_UsesVerifierScores
// proves that
// SelectOptimalModel incorporates verifier scores.
func ComponentTest_ModelManagerSelectModel_UsesVerifierScores(t
 *testing.T) {
 server := httptest.NewServer(http.HandlerFunc(func(w
 http.ResponseWriter, r *http.Request) {
 if r.URL.Path == "/api/models" {
 data := []map[string]interface{}{
 {
 "id": "high-score-model", "display_name":
 "High Score", "provider": "openai",
 "overall_score": 9.5,
 "supports_code_generation": true,
 "verification_status": "verified",
 "context_window_tokens": 128000,
 },
 {
 "id": "low-score-model", "display_name":
 "Low Score", "provider": "openai",
 "overall_score": 4.0,
 "supports_code_generation": true,
 "verification_status": "verified",
 "context_window_tokens": 128000,
 },
 }
 _ = json.NewEncoder(w).Encode(data)
 }
 })
 defer server.Close()

 client := verifier.NewClient(server.URL, "", 5*time.Second)
 cfg := &config.VerifierConfig{Enabled: true, Endpoint:
 server.URL, CacheTTL: 1 * time.Hour}
 cache := verifier.NewCache(1*time.Hour, nil)
 health :=
 verifier.NewHealthMonitor(config.VerifierHealthConfig{

```

```

 CircuitBreaker: config.CircuitBreakerConfig{Enabled:
 true},
 })
 adapter := verifier.NewAdapter(client, cache, health, cfg)

 mm := llm.NewModelManager(/* ... setup ... */)
 mm.SetVerifierAdapter(adapter)

 criteria := llm.ModelSelectionCriteria{
 TaskType: "code_generation",
 RequiredCapabilities:
 []llm.ModelCapability{llm.CapabilityCodeGeneration},
 }
 selected, err := mm.SelectOptimalModel(criteria)
 if err != nil {
 t.Fatalf("selection failed: %v", err)
 }
 if selected.ID != "high-score-model" {
 t.Errorf("expected high-score-model (9.5), got %s",
 selected.ID)
 }
}

```

**Acceptance Criteria:** 1. go test -v -run Component ./tests/component/... passes 2. Test proves higher-scored model is selected over lower-scored 3. Uses real httptest server (no mocks)

**Dependencies:** TASK 2.8.1 **Effort:** Large

---

## TASK 5.4: Create Integration Test Files

### TASK 5.4.1: Create tests/integration/helixcode\_full\_stack\_test.go

**File(s):** helix\_code/tests/integration/helixcode\_full\_stack\_test.go

**Line(s):** CREATE new file **Action:** CREATE

**package** integration

```

import (
 "context"
 "dev.helix.code/internal/verifier"
 "os"
 "testing"
 "time"
)

```

```

// IntegrationTest_FullStack_APIModels_ReturnsVerifierData proves
// the full stack
// from config -> verifier adapter -> model manager -> API
// returns verifier data.
func IntegrationTest_FullStack_APIModels_ReturnsVerifierData(t
 *testing.T) {
 if testing.Short() {
 t.Skip("skipping integration test in short mode")
 }

 ctx, cancel := context.WithTimeout(context.Background(),
 30*time.Second)
 defer cancel()

 endpoint := os.Getenv("HELIX_VERIFIER_ENDPOINT")
 if endpoint == "" {
 endpoint = "http://localhost:8081"
 }

 client := verifier.NewClient(endpoint, "", 30*time.Second)
 models, err := client.GetModels(ctx)
 if err != nil {
 t.Fatalf("full stack integration failed: %v", err)
 }
 if len(models) == 0 {
 t.Fatal("no models returned from verifier – integration
 failure")
 }

 // ANTI-BLUFF: Verify the response is NOT a hardcoded list
 if len(models) <= 3 {
 t.Logf("WARNING: only %d models returned – may be
 fallback list", len(models))
 }

 for _, m := range models {
 if m.ID == "" {
 t.Error("model has empty ID")
 }
 if m.Provider == "" {
 t.Error("model has empty provider")
 }
 }
}

```

**Acceptance Criteria:** 1. go test -v -run Integration ./tests/integration/... passes (when run with real verifier) 2. Test connects to real

verifier endpoint 3. Returns error if no models are returned 4. Warns if only <= 3 models returned (hardcoded list detection)

**Dependencies:** TASK 1.6.1 **Effort:** Large

---

## TASK 5.5: Create Challenge Scripts

### TASK 5.5.1: Create All 12 Challenge Shell Scripts

**Challenge 1:** challenges/scripts/verifier\_model\_list\_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
OUTPUT_FILE="/tmp/verifier_model_list_output.txt"

echo "[CHALLENGE] verifier_model_list_challenge: START"
"${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true
if grep -q "llama-3-8b.*mistral-7b.*phi-3-mini" "$
 {OUTPUT_FILE}"; then
 echo "[FAIL] Output contains only hardcoded 3-model list
 (BLUFF-002)"
 exit 1
fi
if ! grep -q "Score" "${OUTPUT_FILE}"; then
 echo "[FAIL] Missing Score column"
 exit 1
fi
echo "[CHALLENGE] verifier_model_list_challenge: PASS"
```

**Challenge 2:** challenges/scripts/verifier\_model\_select\_challenge.sh

```
#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
OUTPUT_FILE="/tmp/verifier_select_output.txt"

echo "[CHALLENGE] verifier_model_select_challenge: START"
"${CLI_BIN}" --prompt "Write a Go function named AntiBluff" --
 model gpt-4o > "${OUTPUT_FILE}" 2>&1 || true
if grep -q "Generated response for:" "${OUTPUT_FILE}"; then
 echo "[FAIL] Simulated response detected (BLUFF-001)"
 exit 1
fi
```

```

fi
if grep -q "func AntiBluff" "${OUTPUT_FILE}"; then
 echo "[CHALLENGE] verifier_model_select_challenge: PASS"
else
 echo "[FAIL] No real code generated"
 exit 1
fi

```

**Challenge 3:** challenges/scripts/  
verifier\_disable\_fallback\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
OUTPUT_FILE="/tmp/verifier_disable_output.txt"

echo "[CHALLENGE] verifier_disable_fallback_challenge: START"
HELIX_VERIFIER_ENABLED=false "${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true
MODEL_COUNT=$(grep -c "openai\|anthropic\|ollama\|llama" "${OUTPUT_FILE}" || true)
if [["${MODEL_COUNT}" -lt 1]]; then
 echo "[FAIL] No models returned when verifier disabled"
 exit 1
fi
echo "[CHALLENGE] verifier_disable_fallback_challenge: PASS"

```

**Challenge 4:** challenges/scripts/  
verifier\_api\_key\_provision\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CONFIG_FILE="${PROJECT_ROOT}/helix_code/configs/config.yaml"

echo "[CHALLENGE] verifier_api_key_provision_challenge: START"
if grep -rP 'api_key:\s*sk-[a-zA-Z0-9]+' "${CONFIG_FILE}" 2>/dev/null; then
 echo "[FAIL] Literal API key found in config (security violation)"
 exit 1
fi
ENV_FILE="${PROJECT_ROOT}/helix_code/.env.example"
KEY_COUNT=$(grep -c "HELIX_.*API_KEY\|HELIX_.*API_TOKEN" "${ENV_FILE}" || true)
if [["${KEY_COUNT}" -lt 10]]; then

```

```

 echo "[FAIL] Only ${KEY_COUNT} API keys documented
 in .env.example (expected >= 10)"
 exit 1
fi
echo "[CHALLENGE] verifier_api_key_provision_challenge: PASS"

```

**Challenge 5:** challenges/scripts/  
verifier\_rate\_limit\_display\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

echo "[CHALLENGE] verifier_rate_limit_display_challenge: START"
curl -sf "http://localhost:8081/api/models" > /tmp/
 verifier_rate_models.json 2>&1 || true
if ! grep -q "rate_limited\|cooldown" /tmp/
 verifier_rate_models.json 2>/dev/null; then
 echo "[WARN] No rate-limited models in verifier DB – test
 inconclusive"
fi
echo "[CHALLENGE] verifier_rate_limit_display_challenge: PASS"

```

**Challenge 6:** challenges/scripts/  
verifier\_realtime\_update\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

echo "[CHALLENGE] verifier_realtime_update_challenge: START"
MAX_WAIT=300
for i in $(seq 1 ${MAX_WAIT}); do
 curl -sf "http://localhost:8081/api/models" > "/tmp/
 verifier_poll_${i}.json" 2>&1 || true
 sleep 1
done
if [[! -f "/tmp/verifier_poll_1.json"]]; then
 echo "[FAIL] No verifier data received"
 exit 1
fi
echo "[CHALLENGE] verifier_realtime_update_challenge: PASS"

```

**Challenge 7:** challenges/scripts/verifier\_mcp\_lsp\_acp\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail

```

```

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
SERVER_URL="http://localhost:8080"

echo "[CHALLENGE] verifier_mcp_lsp_acp_challenge: START"
curl -sf "${SERVER_URL}/api/v1/mcp/models" > /tmp/
 verifier_mcp_output.json 2>&1 || true
if ! grep -q "gpt-4o\|claude" /tmp/verifier_mcp_output.json 2>/
 dev/null; then
 echo "[FAIL] MCP endpoint does not return verifier models"
 exit 1
fi
curl -sf "${SERVER_URL}/api/v1/lsp/completion" \
 -H "Content-Type: application/json" \
 -d '{"model": "gpt-4o", "prompt": "func AntiBluff"}' > /tmp/
 verifier_lsp_output.json 2>&1 || true
if ! grep -q "AntiBluff" /tmp/verifier_lsp_output.json 2>/dev/
 null; then
 echo "[FAIL] LSP endpoint does not return completions with
 verifier model"
 exit 1
fi
curl -sf "${SERVER_URL}/api/v1/acp/agents/discover" > /tmp/
 verifier_acp_output.json 2>&1 || true
if ! grep -q "verifier\|model:" /tmp/verifier_acp_output.json 2>/
 dev/null; then
 echo "[FAIL] ACP endpoint does not reference verifier"
 exit 1
fi
curl -sf "${SERVER_URL}/api/v1/embeddings" \
 -H "Content-Type: application/json" \
 -d '{"model": "text-embedding-3-small", "input": "anti-bluff
 test"}' > /tmp/verifier_embed_output.json 2>&1 || true
if ! grep -q "embedding" /tmp/verifier_embed_output.json 2>/dev/
 null; then
 echo "[FAIL] Embedding endpoint does not return embeddings"
 exit 1
fi
echo "[CHALLENGE] verifier_mcp_lsp_acp_challenge: PASS"

```

**Challenge 8:** challenges/scripts/  
 verifier\_cross\_platform\_cli\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"

```



```

echo "[CHALLENGE] verifier_cross_platform_cli_challenge: START"
PLATFORM=$(uname -s)
ARCH=$(uname -m)
OUTPUT_FILE="/tmp/verifier_cross_platform_${PLATFORM}_${ARCH}.txt"
JSON_FILE="/tmp/verifier_cross_platform_${PLATFORM}_${ARCH}.json"

"${CLI_BIN}" --list-models > "${OUTPUT_FILE}" 2>&1 || true
"${CLI_BIN}" --list-models --format json > "${JSON_FILE}" 2>&1
|| true

if [["${PLATFORM}" != "MINGW*" && "${PLATFORM}" !=
 "CYGWIN*"]]; then
 if grep -q $'\r' "${OUTPUT_FILE}"; then
 echo "[FAIL] Table output contains CRLF on non-Windows"
 exit 1
 fi
fi
if ! python3 -m json.tool "${JSON_FILE}" > /dev/null 2>&1; then
 echo "[FAIL] JSON output is invalid on ${PLATFORM}"
 exit 1
fi
for KEY in "id" "name" "provider" "score" "verified"; do
 if ! python3 -c "import json; d=json.load(open('${JSON_FILE}')); print('${KEY}' in d[0])" | grep -q
 "True"; then
 echo "[FAIL] JSON missing required key '${KEY}'"
 exit 1
 fi
done
MODEL_COUNT=$(python3 -c "import json; d=json.load(open('${JSON_FILE}')); print(len(d))")
if [["${MODEL_COUNT}" -lt 1]]; then
 echo "[FAIL] No models in JSON output"
 exit 1
fi
echo "[CHALLENGE] verifier_cross_platform_cli_challenge: PASS (${PLATFORM} ${ARCH}, ${MODEL_COUNT} models)"

```

**Challenge 9:** challenges/scripts/  
verifier\_startup\_pipeline\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
SERVER_BIN="${PROJECT_ROOT}/helix_code/bin/server"

```

```

LOG_FILE="/tmp/verifier_startup_pipeline.log"

echo "[CHALLENGE] verifier_startup_pipeline_challenge: START"
timeout 60 "${SERVER_BIN}" > "${LOG_FILE}" 2>&1 &
SERVER_PID=$!
sleep 10

if ! grep -qi "discover" "${LOG_FILE}"; then
 echo "[FAIL] Phase 1 (Discover) not logged"
 kill "${SERVER_PID}" 2>/dev/null || true; exit 1
fi
if ! grep -qi "verif" "${LOG_FILE}"; then
 echo "[FAIL] Phase 2 (Verify) not logged"
 kill "${SERVER_PID}" 2>/dev/null || true; exit 1
fi
if ! grep -qi "score" "${LOG_FILE}"; then
 echo "[FAIL] Phase 3 (Score) not logged"
 kill "${SERVER_PID}" 2>/dev/null || true; exit 1
fi
if ! grep -qi "debate\|team\|selected" "${LOG_FILE}"; then
 echo "[FAIL] Phase 5 (Debate Team) not logged"
 kill "${SERVER_PID}" 2>/dev/null || true; exit 1
fi
if ! curl -sf "http://localhost:8080/health" > /dev/null 2>&1;
then
 echo "[FAIL] Server health check failed"
 kill "${SERVER_PID}" 2>/dev/null || true; exit 1
fi
kill "${SERVER_PID}" 2>/dev/null || true
echo "[CHALLENGE] verifier_startup_pipeline_challenge: PASS"

```

**Challenge 10:** challenges/scripts/  
verifier\_canned\_detection\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

echo "[CHALLENGE] verifier_canned_detection_challenge: START"
API_OUTPUT="/tmp/verifier_canned_api.json"
curl -sf "http://localhost:8081/api/models/test-canned-model" >
 "${API_OUTPUT}" 2>&1 || true

STATUS=$(python3 -c "
import json, sys
try:
 d=json.load(open('${API_OUTPUT}'))

```

```

 print(d.get('verification_status', 'UNKNOWN'))
except:
 print('UNKNOWN')
")

if [["${STATUS}" != "failed" && "${STATUS}" != "unverified"]];
then
 echo "[FAIL] Canned-response model has status '${STATUS}'
 instead of failed"
 exit 1
fi

SCORE=$(python3 -c "
import json, sys
try:
 d=json.load(open('${API_OUTPUT}'))
 print(d.get('overall_score', 999))
except:
 print(999)")

if (($(echo "${SCORE} > 3.0" | bc -l))); then
 echo "[FAIL] Canned-response model has score ${SCORE} (>
 3.0)"
 exit 1
fi
echo "[CHALLENGE] verifier_canned_detection_challenge: PASS"

```

**Challenge 11:** challenges/scripts/  
verifier\_security\_redaction\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
CLI_BIN="${PROJECT_ROOT}/helix_code/bin/cli"
CLI_LOG="/tmp/verifier_security_cli.log"

echo "[CHALLENGE] verifier_security_redaction_challenge: START"
FAKE_KEY="sk-antibluff-test-key-9876543210abcdef"
export HELIX_OPENAI_API_KEY="${FAKE_KEY}"

"${CLI_BIN}" --list-models > "${CLI_LOG}" 2>&1 || true

if grep -q "${FAKE_KEY}" "${CLI_LOG}"; then
 echo "[FAIL] API key found in CLI output"
 exit 1
fi
if grep -r "${FAKE_KEY}" "${PROJECT_ROOT}/helix_code/" > /dev/
 null 2>&1; then

```

```

 echo "[FAIL] API key found in HelixCode directory"
 exit 1
fi

KEY_FRAGMENT="${FAKE_KEY:0:8}"
if grep -q "${KEY_FRAGMENT}" "${CLI_LOG}"; then
 echo "[FAIL] API key fragment found in CLI output"
 exit 1
fi
echo "[CHALLENGE] verifier_security_redaction_challenge: PASS"

```

**Challenge 12:** challenges/scripts/  
verifier\_scoring\_accuracy\_challenge.sh

```

#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"

echo "[CHALLENGE] verifier_scoring_accuracy_challenge: START"
HIGH_OUTPUT="/tmp/verifier_score_high.json"
LOW_OUTPUT="/tmp/verifier_score_low.json"

curl -sf "http://localhost:8081/api/models/test-high-score" > "${HIGH_OUTPUT}" 2>&1 || true
curl -sf "http://localhost:8081/api/models/test-low-score" > "${LOW_OUTPUT}" 2>&1 || true

HIGH_SCORE=$(python3 -c "import json; d=json.load(open('${HIGH_OUTPUT}')); print(d.get('overall_score', 0))")
LOW_SCORE=$(python3 -c "import json; d=json.load(open('${LOW_OUTPUT}')); print(d.get('overall_score', 10))")

if (($(echo "${HIGH_SCORE} < 9.0" | bc -l))); then
 echo "[FAIL] High-score model has score ${HIGH_SCORE} (< 9.0)"
 exit 1
fi
if (($(echo "${LOW_SCORE} > 5.0" | bc -l))); then
 echo "[FAIL] Low-score model has score ${LOW_SCORE} (> 5.0)"
 exit 1
fi
if (($(echo "${HIGH_SCORE} == ${LOW_SCORE}" | bc -l))); then
 echo "[FAIL] Scores are identical – hardcoded score detected"
 exit 1
fi
if (($(echo "${HIGH_SCORE} == 8.5" | bc -l))) || (($(echo "${LOW_SCORE} == 8.5" | bc -l))); then
 echo "[FAIL] Score is exactly 8.5 – likely stub value"

```

```

 exit 1
 fi
 echo "[CHALLENGE] verifier_scoring_accuracy_challenge: PASS"

```

**Acceptance Criteria:** 1. All 12 scripts are executable (chmod +x) 2. Each script has set -euo pipefail 3. Each script outputs [CHALLENGE] name: START and [CHALLENGE] name: PASS 4. Each script includes at least one anti-bluff assertion

**Dependencies:** All previous phases **Effort:** Large

---

## TASK 5.6: Create Coverage and Quality Scripts

### TASK 5.6.1: Create scripts/enforce\_coverage.sh

**File(s):** helix\_code/scripts/enforce\_coverage.sh **Line(s):** CREATE new file  
**Action:** CREATE

```

#!/usr/bin/env bash
set -euo pipefail

UNIT_THRESHOLD=100
INTEGRATION_THRESHOLD=95
CONTRACT_THRESHOLD=100

UNIT_COVER=$(go test -short ./internal/verifier/... ./internal/
 llm/... ./cmd/cli/... ./internal/services/... -cover 2>/
 dev/null | grep -oP '\d+\.\d+%' | tail -1 | tr -d '%')
if (($(echo "${UNIT_COVER} < ${UNIT_THRESHOLD}" | bc -l)));
then
 echo "[FAIL] Unit coverage ${UNIT_COVER}% < ${UNIT_THRESHOLD}
 %"
 exit 1
fi

INTEGRATION_COVER=$(go test -run Integration ./tests/
 integration/... -cover 2>/dev/null | grep -oP '\d+\.\d+
 %' | tail -1 | tr -d '%')
if (($(echo "${INTEGRATION_COVER} < ${INTEGRATION_THRESHOLD}" |
 bc -l))); then
 echo "[FAIL] Integration coverage ${INTEGRATION_COVER}% < $
 {INTEGRATION_THRESHOLD}%"
 exit 1
fi

echo "[PASS] Coverage check passed: Unit=${UNIT_COVER}%,
 Integration=${INTEGRATION_COVER}%"

```

### TASK 5.6.2: Create scripts/no\_mock\_above\_unit.sh

**File(s):** helix\_code/scripts/no\_mock\_above\_unit.sh **Line(s):** CREATE new file **Action:** CREATE

```
#!/usr/bin/env bash
set -euo pipefail

VIOLATIONS=0

for FILE in $(find tests/ internal/ -name "*_test.go" | grep -v
 "^internal/verifier/*_test.go" | grep -v "^internal/
 llm/*_test.go" | grep -v "^cmd/cli/*_test.go"); do
 if grep -qE "(mock|Mock|gomock|mockery|httpptest)" "${FILE}";
 then
 echo "[VIOLATION] Mock usage found in non-unit test: $
 {FILE}"
 VIOLATIONS=$((VIOLATIONS + 1))
 fi
done

if [["${VIOLATIONS}" -gt 0]]; then
 echo "[FAIL] ${VIOLATIONS} mock violations found above unit
 test level"
 exit 1
fi

echo "[PASS] No mocks above unit tests"
```

### TASK 5.6.3: Create docker-compose.test.yml

**File(s):** helix\_code/docker-compose.test.yml **Line(s):** CREATE new file **Action:** CREATE

```
version: "3.9"

services:
 postgres-test:
 image: postgres:15-alpine
 environment:
 POSTGRES_USER: helix
 POSTGRES_PASSWORD: ${HELIX_DATABASE_PASSWORD:?set
 HELIX_DATABASE_PASSWORD in .env (run ./setup.sh, or
 cp .env.example .env)}
 POSTGRES_DB: helixcode_test
 ports:
 - "5433:5432"
 healthcheck:
```

```
test: ["CMD-SHELL", "pg_isready -U helix"]
interval: 2s
timeout: 5s
retries: 10
tmpfs:
 - /var/lib/postgresql/data
```

```
redis-test:
 image: redis:7-alpine
 ports:
 - "6380:6379"
 healthcheck:
 test: ["CMD", "redis-cli", "ping"]
 interval: 2s
 timeout: 5s
 retries: 10
```

```
verifier-test:
 build:
 context: ../LLMsVerifier
 dockerfile: Dockerfile
 environment:
 - VERIFIER_DATABASE_PATH=/data/verifier-test.db
 - VERIFIER_API_PORT=8081
 ports:
 - "8081:8081"
 volumes:
 - verifier-test-data:/data
 healthcheck:
 test: ["CMD-SHELL", "curl -sf http://localhost:8081/api/health || exit 1"]
 interval: 5s
 timeout: 5s
 retries: 10
```

```
volumes:
 verifier-test-data:
```

**Acceptance Criteria:** 1. scripts/enforce\_coverage.sh is executable 2. scripts/no\_mock\_above\_unit.sh is executable 3. docker-compose.test.yml defines postgres, redis, and verifier services 4. All services have health checks

**Dependencies:** All previous tasks **Effort:** Medium

---

## TASK 5.7: Modify Makefile

### TASK 5.7.1: Add All Test Targets

**File(s):** helix\_code/Makefile **Line(s):** Before existing .PHONY line or at end of file **Action:** MODIFY

```
--- Test Infrastructure ---
.PHONY: test-infra-up test-infra-down test-infra-status
test-infra-up:
 docker compose -f docker-compose.test.yml up -d --wait

test-infra-down:
 docker compose -f docker-compose.test.yml down -v

test-infra-status:
 docker compose -f docker-compose.test.yml ps

--- Unit Tests ---
.PHONY: test-unit test-unit-full test-unit-coverage
test-unit:
 go test -short -v ./internal/verifier/... ./internal/
 llm/... ./cmd/cli/... ./internal/services/...

test-unit-full: test-infra-up
 go test -short -v -race -coverprofile=coverage-unit.out ./
 internal/verifier/... ./internal/llm/... ./cmd/
 cli/... ./internal/services/...
 go tool cover -func=coverage-unit.out | tail -1

test-unit-coverage:
 go test -short -coverprofile=coverage-unit.out ./internal/
 verifier/... ./internal/llm/... ./cmd/cli/... ./
 internal/services/...
 go tool cover -html=coverage-unit.out -o coverage-unit.html

--- Contract Tests ---
.PHONY: test-contract test-contract-full
test-contract: test-infra-up
 go test -v -run Contract ./tests/contract/...

test-contract-full: test-infra-up
 go test -v -race -run Contract -coverprofile=coverage-
 contract.out ./tests/contract/...

--- Component Tests ---
.PHONY: test-component test-component-full
test-component: test-infra-up
```



```

go test -v -run Component ./tests/component/...

test-component-full: test-infra-up
go test -v -race -run Component -coverprofile=coverage-
component.out ./tests/component/...

--- Integration Tests ---
.PHONY: test-integration test-integration-full
test-integration: test-infra-up
go test -v -run Integration ./tests/integration/...

test-integration-full: test-infra-up
go test -v -race -run Integration -coverprofile=coverage-
integration.out ./tests/integration/...

--- E2E / Challenge Tests ---
.PHONY: test-e2e test-e2e-full
test-e2e:

 cd challenges/scripts && for s in
 verifier_*_challenge.sh; do bash "$$s"; done

test-e2e-full: test-infra-up build-cli build-server

 cd challenges/scripts && for s in
 verifier_*_challenge.sh; do bash "$$s"; done

--- Security Tests ---
.PHONY: test-security test-security-full
test-security: test-infra-up
go test -v -run Security ./tests/security/...

test-security-full: test-infra-up
go test -v -race -run Security -coverprofile=coverage-
security.out ./tests/security/...

--- Performance Tests ---
.PHONY: test-performance test-load-full
test-performance: test-infra-up
go test -v -bench=. -benchmem -benchtime=30s ./tests/
performance/...

test-load-full: test-infra-up
go test -v -bench=. -benchmem -benchtime=60s
-cpuprofile=cpu.prof -memprofile=mem.prof ./tests/
performance/...

--- Coverage & Quality Gates ---

```

```
.PHONY: coverage-full no-mocks-above-unit
coverage-full: test-unit-full test-contract-full test-component-
full test-integration-full
@echo "--- Combined Coverage Report ---"
go tool cover -func=coverage-unit.out | tail -1
go tool cover -func=coverage-contract.out | tail -1
go tool cover -func=coverage-component.out | tail -1
go tool cover -func=coverage-integration.out | tail -1

no-mocks-above-unit:
 bash scripts/no_mock_ above_unit.sh
```

```
--- Anti-Bluff: Hardcoded Model Detection ---
.PHONY: test-no-hardcoded-models
test-no-hardcoded-models:
@echo "Checking for hardcoded model lists (BLUFF-002)..."
@if grep -r "llama-3-8b.*mistral-7b.*phi-3-mini" cmd/
internal/ 2>/dev/null; then \
 echo "[FAIL] Hardcoded 3-model list detected"; exit 1; \
fi
@echo "[PASS] No hardcoded model lists detected"
```

**Acceptance Criteria:** 1. make test-unit runs unit tests with -short flag 2. make test-contract runs contract tests 3. make test-component runs component tests 4. make test-integration runs integration tests 5. make test-e2e runs all challenge scripts 6. make test-no-hardcoded-models detects BLUFF-002

**Dependencies:** All previous tasks **Effort:** Medium

---

## PHASE 5 ROLLBACK PLAN

```
cd HelixCode
git rm --cached tests/unit/verifier/*.go tests/contract/*.go
tests/component/*.go \
tests/integration/*.go tests/security/*.go tests/performance/
*.go \
tests/fixtures/* challenges/scripts/*.sh \
scripts/enforce_coverage.sh scripts/no_mock_ above_unit.sh \
docker-compose.test.yml
rm -rf tests/ challenges/scripts/verifier*.sh scripts/
enforce_coverage.sh \
scripts/no_mock_ above_unit.sh docker-compose.test.yml
git checkout -- Makefile
go mod tidy
make build
make test-unit
```

## PHASE 5 VERIFICATION CHECKLIST

- ☐ `go test -short ./internal/verifier/... passes` (client, cache, health, adapter, polling tests)
  - ☐ `go test -short ./internal/llm/... passes` (verifier integration test)
  - ☐ `go test -short ./internal/cli/ux/... passes` (render test)
  - ☐ `go test -v -run Contract ./tests/contract/... passes`
  - ☐ `go test -v -run Component ./tests/component/... passes`
  - ☐ `go test -v -run Integration ./tests/integration/... passes` (with test infra)
  - ☐ All 12 challenge scripts exist and are executable
  - ☐ `scripts/enforce_coverage.sh` exists and is executable
  - ☐ `scripts/no_mock_above_unit.sh` exists and is executable
  - ☐ `docker-compose.test.yml` defines postgres-test, redis-test, verifier-test
  - ☐ Makefile has targets: test-unit, test-contract, test-component, test-integration, test-e2e, test-security, test-performance, coverage-full, no-mocks-above-unit, test-no-hardcoded-models
  - ☐ `make test-no-hardcoded-models` passes (zero hardcoded lists)
- 

## PHASE 6: DOCUMENTATION & CONSTITUTION — Complete Documentation Update

**Phase Goal:** All documentation, guides, manuals, and constitutional updates are complete and consistent.

**Phase Entry Criteria:** Phase 5 verification checklist is complete. **Phase Exit Criteria:** 1. CONSTITUTION.md updated with CONST-036 through CONST-040 2. CLAUDE.md has verifier architecture section 3. AGENTS.md has new bluff areas BLUFF-004 through BLUFF-008 4. All 5 docs/verifier/ guides exist and are

complete 5. README.md references verifier features 6. All submodules have constitution template applied

---

## TASK 6.1: Update CONSTITUTION.md

### TASK 6.1.1: Add CONST-036 through CONST-040

**File(s):** helix\_code/CONSTITUTION.md **Line(s):** After CONST-035 (before ## MANIFESTO section) **Action:** MODIFY

#### ### CONST-036: LLMsVerifier Single Source of Truth Mandate

**\*\*The Verifier Is Truth.\*\*** LLMsVerifier is the sole, authoritative source of all model metadata within HelixCode. No model ID, provider status, capability flag, or pricing data may be hardcoded in HelixCode source files. All model data MUST be fetched at runtime from LLMsVerifier via its REST API. When the verifier is unavailable, the system MUST use a sanctioned fallback list (``FallbackModels`` in ``internal/verifier/types.go``) and log ``ErrUsingFallback``.

#### ### CONST-037: Anti-Bluff Guarantee for Model Provisioning

**\*\*Verified Models Only.\*\*** Every model presented to users or selected for tasks MUST have a verification status from LLMsVerifier. Models with status ``unverified``, ``failed``, or ``rate_limited`` MUST be clearly labeled and MUST NOT be auto-selected unless explicitly permitted by user configuration. Scores MUST NOT be simulated, stubbed, or hardcoded.

#### ### CONST-038: Real-Time Model Status Accuracy Mandate

**\*\*Fresh Data or Transparent Staleness.\*\*** Model status data displayed to users MUST be no more than ``verifier.cache_ttl`` old. If stale data is used due to verifier unavailability, the display MUST indicate staleness with a ``~`` prefix or ``[STALE]`` label. Background polling MUST update model data at ``verifier.polling_interval`` intervals.

#### ### CONST-039: All Providers and Models Integration Mandate

**\*\*No Provider Left Behind.\*\*** All providers supported by HelixCode MUST be supported by LLMsVerifier. A provider adapter MUST exist in ``LLMsVerifier/providers/`` for every provider type listed in ``internal/llm/factory.go``. This is a structural completeness requirement, not an optional enhancement.

### CONST-040: Advanced Capabilities Integration Mandate

**\*\*Capabilities Must Be Verified.\*\*** MCP, LSP, ACP, Embeddings, RAG, Skills, and Plugins **MUST** query LLMsVerifier for model capability information before execution. A capability **MUST NOT** be assumed present based on model name or provider type alone. The verifier **MUST** return explicit capability flags for every model.

**Acceptance Criteria:** 1. grep "CONST-036" CONSTITUTION.md returns exactly one match 2. grep "CONST-037" CONSTITUTION.md returns exactly one match 3. grep "CONST-038" CONSTITUTION.md returns exactly one match 4. grep "CONST-039" CONSTITUTION.md returns exactly one match 5. grep "CONST-040" CONSTITUTION.md returns exactly one match

**Dependencies:** None **Effort:** Medium

---

## TASK 6.2: Update CLAUDE.md

### TASK 6.2.1: Add Verifier Architecture Section

**File(s):** helix\_code/CLAUDE.md **Line(s):** After existing architecture section (or at end of document) **Action:** MODIFY

#### ## Verifier Architecture

HelixCode integrates LLMsVerifier as its single source of truth for model metadata via a REST API client (not Go module import, to avoid sibling-module dependency conflicts).

#### ### Architecture Diagram (ASCII)

HelixCode (cmd/cli, cmd/server, internal/llm, internal/cli/ux) | | REST API (HTTP/json) — baseUrl from verifier.endpoint v internal/verifier/Client -> LLMsVerifier service (localhost:8081) | | | polling (N seconds) | SQLite read v v internal/verifier/Cache LLMsVerifier (submodule) (L1 LRU + L2 Redis) providers/ | (12 adapters today, 23 needed) | | v v internal/verifier/Adapter Provider APIs (real HTTP calls) (score bridge) | v internal/llm/ModelManager (SelectOptimalModel with verifier scores)

#### ### Key Components

Component	File	Responsibility
Client	`internal/verifier/client.go`	HTTP REST client for verifier A
Adapter	`internal/verifier/adapter.go`	Bridges verifier scores to Mo
Cache	`internal/verifier/cache.go`	Two-tier cache (L1 LRU + L2 Redis

```
| HealthMonitor | `internal/verifier/health.go` | Circuit breaker for veri
| Poller | `internal/verifier/poller.go` | Background goroutine for real-t
| EventPublisher | `internal/verifier/events.go` | Publishes changes to He
| DiscoveryService | `internal/verifier/discovery.go` | Model discovery wi
```

### ### Data Flow

1. **\*\*Startup\*\***: `NewAdapter()` + `NewPoller()` -> immediate first poll
2. **\*\*Polling\*\***: Every `verifier.polling\_interval` -> `client.GetModels()`
3. **\*\*Change Detection\*\***: `poller.detectChanges()` compares old vs new mode
4. **\*\*Cache Update\*\***: `cache.SetModels()` / `cache.SetScores()`
5. **\*\*Event Publish\*\***: `eventPublisher.Publish(change)` -> HelixCode event
6. **\*\*CLI Display\*\***: `ux.RenderModelList()` with badges, scores, status ind

**Acceptance Criteria:** 1. grep "Verifier Architecture" CLAUDE.md returns one match 2. ASCII diagram includes all 7 components 3. Data flow has exactly 6 numbered steps

**Dependencies:** None **Effort:** Medium

---

## TASK 6.3: Update AGENTS.md

### TASK 6.3.1: Add New Bluff Areas BLUFF-004 through BLUFF-008

**File(s):** helix\_code/AGENTS.md **Line(s):** After existing BLUFF-003 (or at end of Known Bluff Areas section) **Action:** MODIFY

#### ### BLUFF-004: Hardcoded Model Discovery

**\*\*Location\*\***: ``internal/llm/`  
    model_discovery.go:fetchExternalModels()``

**\*\*Claim\*\***: "External models are dynamically discovered."

**\*\*Reality\*\***: The function contains a hardcoded  
    ``[]*ModelInfo{{ID: "llama-3-8b-instruct"...}}`` array.

**\*\*Fix\*\***: Replace with ``verifierAdapter.GetVerifiedModels()``  
    call.

**\*\*Status\*\***: Fixed by Phase 2, TASK 2.7.1

#### ### BLUFF-005: Ignored Verification Data in Model Selection

**\*\*Location\*\***: ``internal/llm/`  
    model_manager.go:SelectOptimalModel()``

**\*\*Claim\*\***: "The best model is selected based on scores."

**\*\*Reality\*\***: The function uses local heuristic scores only,  
    ignoring LLMsVerifier scores even when available.

**\*\*Fix\*\***: Augment ``SelectOptimalModel()`` with  
    ``rankByVerifierScores()`` (60% verifier, 40%  
    heuristic).

**\*\*Status\*\***: Fixed by Phase 2, TASK 2.8.1

### ### BLUFF-006: No Provider Health Validation on Factory Creation

**\*\*Location\*\*:** ``internal/llm/factory.go:NewProvider()``  
**\*\*Claim\*\*:** "All providers are validated before use."  
**\*\*Reality\*\*:** The factory returns a provider without checking verifier health or score thresholds.  
**\*\*Fix\*\*:** Add verifier health/score validation after provider creation.  
**\*\*Status\*\*:** Fixed by Phase 2, TASK 2.10.1

### ### BLUFF-007: Simulated LLM Responses (Legacy)

**\*\*Location\*\*:** ``cmd/cli/main.go:handleGeneration()`` (legacy path)  
**\*\*Claim\*\*:** "LLM responses are generated by the selected model."  
**\*\*Reality\*\*:** Some paths return ``fmt.Sprintf("Generated response for: %s", modelID)`` as a simulated response.  
**\*\*Fix\*\*:** Route ALL generation through real provider with verifier-selected model.  
**\*\*Status\*\*:** Fixed by Phase 2, TASK 2.9.1

### ### BLUFF-008: Simulated Command Execution

**\*\*Location\*\*:** ``cmd/cli/main.go:handleExecuteCommand()``  
**\*\*Claim\*\*:** "Commands are executed by the agent."  
**\*\*Reality\*\*:** The function contains ``time.Sleep(1 * time.Second)`` instead of actual command execution.  
**\*\*Fix\*\*:** Replace ``time.Sleep`` with ``exec.Command()`` or appropriate executor.  
**\*\*Status\*\*:** Requires separate fix outside verifier scope

**Acceptance Criteria:** 1. `grep "BLUFF-004" AGENTS.md` returns one match 2. `grep "BLUFF-005" AGENTS.md` returns one match 3. `grep "BLUFF-006" AGENTS.md` returns one match 4. `grep "BLUFF-007" AGENTS.md` returns one match 5. `grep "BLUFF-008" AGENTS.md` returns one match 6. Each bluff entry has Location, Claim, Reality, Fix, and Status fields

**Dependencies:** None **Effort:** Medium

---

## TASK 6.4: Create docs/verifier/INTEGRATION\_GUIDE.md

### TASK 6.4.1: Write Comprehensive Integration Guide

**File(s):** helix\_code/docs/verifier/INTEGRATION\_GUIDE.md **Line(s):** CREATE new file **Action:** CREATE

### # LLMsVerifier Integration Guide

#### ## Overview

This guide explains how HelixCode integrates with LLMsVerifier as its single source of truth for model metadata, provider health, verification status, and scoring.

## ## Prerequisites

- LLMsVerifier submodule checked out at `../LLMsVerifier` or running service at configurable endpoint
- Go 1.21+ for HelixCode
- Redis 7+ (optional, for distributed cache)
- PostgreSQL 15+ (for HelixCode persistence)

## ## Architecture

See `CLAUDE.md` -> Verifier Architecture for the complete diagram.

### ### REST API (Not Go Module Import)

HelixCode does NOT import LLMsVerifier as a Go module. Instead, it uses an HTTP REST client:

```
```go
client := verifier.NewClient("http://localhost:8081", "api-key",
    30*time.Second)
models, err := client.GetModels(ctx)
```

This avoids the `digital.vasic.llmprovider` sibling-module dependency issue.

Configuration

Minimal Configuration (config.yaml)

```
verifier:
  enabled: true
  endpoint: "http://localhost:8081"
  api_key: "${HELIX_VERIFIER_API_KEY}"
```

Full Configuration

See `configs/verifier.yaml` for the complete schema with all 16 provider configs, scoring weights, health thresholds, and event settings.

Environment Variables

Variable	Required	Default	Description
HELIX_VERIFIER_ENABLED	No	false	Master enable switch
HELIX_VERIFIER_ENDPOINT	If enabled	localhost:8081	Verifier service URL
HELIX_VERIFIER_API_KEY	No	""	Authentication key
HELIX_VERIFIER_TIMEOUT	No	30s	HTTP timeout
HELIX_VERIFIER_CACHE_TTL	No	5m	Cache entry lifetime
HELIX_VERIFIER_POLLING_INTERVAL	No	60s	Background poll interval
HELIX_OPENAI_API_KEY	If using OpenAI	""	Provider API key
...	...	...	(see .env.example for all 20+ keys)

Enable/Disable Behavior

verifier.enabled	Endpoint Reachable	Behavior
false	N/A	Legacy mode. No verifier calls.
true	yes	Verifier-powered. Real-time polling.
true	no	Circuit breaker opens. Fallback to cache, then FallbackModels.

Integration Points

1. Model Discovery (internal/llm/model_discovery.go)

Replaced hardcoded external model list with `verifierAdapter.GetVerifiedModels()`.

2. Model Selection (`internal/llm/model_manager.go`)

Augmented `SelectOptimalModel()` with `rankByVerifierScores()` using 60% verifier / 40% heuristic weighting.

3. CLI Display (`cmd/cli/main.go`)

Replaced `handleListModel()` with dynamic fetch + `ux.RenderModelList()`.

4. Provider Factory (`internal/llm/factory.go`)

Added verifier health/score validation before returning provider instance.

5. Advanced Features (MCP, LSP, ACP, Embeddings, RAG, Skills, Plugins)

Each queries verifier for model capabilities before execution.

Troubleshooting

“verifier service is unavailable”

- Check that LLMsVerifier is running: `curl http://localhost:8081/api/health`
- Check `HELIX_VERIFIER_ENDPOINT` env var
- Check firewall / network connectivity

“no model matches criteria”

- Try widening filters: `helixcode --list-models --min-score 0 --max-price 0`
- Check that verifier has models: `curl http://localhost:8081/api/models`

“circuit breaker open”

- Verifier has failed too many times. Wait for `half_open_timeout` (default 60s) or restart.

Rollback

See Phase Rollback Plans in `phased_implementation_plan.md`.

****Acceptance Criteria**:**

1. File exists at `helix\_code/docs/verifier/INTEGRATION\_GUIDE.md`
2. Contains configuration section with all env vars
3. Contains troubleshooting section with 3+ entries
4. References `CLAUDE.md` and `configs/verifier.yaml`

****Dependencies**:** None

****Effort**:** Large

TASK 6.5: Create docs/verifier/USER_GUIDE.md

TASK 6.5.1: Write User Guide

****File(s)**:** `helix\_code/docs/verifier/USER\_GUIDE.md`

****Line(s)**:** CREATE new file

****Action**:** CREATE

```markdown

# User Guide: Model Selection with LLMsVerifier

## Listing Models

### Basic List

```bash

helixcode --list-models

Shows all available models with verification status, score, provider, and price.

Filtered List

```
helixcode --list-models --provider openai --min-score 8.0 --  
          verified-only
```

JSON Output

```
helixcode --list-models --format json
```

Interactive TUI Selector

```
helixcode --list-models --models-interactive
```

Opens an interactive terminal UI with keyboard navigation.

Model Information

```
helixcode --model-info gpt-4o
```

Shows full details: capabilities, pricing, verification dimensions, rate limits, alternatives.

Using Verifier for Generation

```
helixcode --prompt "Write a Go function" --model gpt-4o
```

HelixCode will verify that gpt-4o is available and healthy before sending the request.

Filtering by Capability

```
helixcode --list-models --capability code,vision,streaming
```

Shows only models supporting code generation, vision, and streaming.

Sorting Models

```
helixcode --list-models --sort price           # Cheapest first
helixcode --list-models --sort latency        # Fastest first
helixcode --list-models --sort score          # Highest score
first (default)
```

Real-Time Updates

The CLI automatically polls the verifier for updates. Use `--refresh-models` to force a refresh.

****Acceptance Criteria**:**

1. File exists at ``helix_code/docs/verifier/USER_GUIDE.md``
2. Contains examples for all CLI flags
3. Contains at least 5 usage examples

****Dependencies**:** TASK 3.7.1

****Effort**:** Medium

TASK 6.6: Create docs/verifier/API_REFERENCE.md

TASK 6.6.1: Write API Reference

****File(s)**:** ``helix_code/docs/verifier/API_REFERENCE.md``

****Line(s)**:** CREATE new file

****Action**:** CREATE

``markdown

API Reference: LLMsVerifier REST API

Endpoints

GET /api/health

Returns service health status.

****Response**:**

```
```json
{
 "status": "ok",
 "timestamp": "2026-07-01T12:00:00Z",
 "version": "1.0.0"
}
```

#### **GET /api/models**

Returns all verified models.

**Response:** Array of VerifiedModel objects. See `internal/verifier/types.go`.

#### **GET /api/models/{id}**

Returns a single model by ID.

**Response:** VerifiedModel object.

#### **GET /api/scores**

Returns provider scores.

**Response:**

```
{
 "openai": 9.1,
 "anthropic": 8.9,
 "gemini": 8.7
}
```

#### **POST /api/models/{id}/verify**

Triggers on-demand verification for a model.

**Response:** VerificationResult object.

#### **GET /api/pricing**

Returns token pricing data.

## GET /api/limits

Returns rate limit data.

## Data Types

### VerifiedModel

See `internal/verifier/types.go:VerifiedModel` for the complete struct definition with 30+ fields.

### VerificationResult

See `internal/verifier/types.go:VerificationResult` for the complete struct with all dimension flags.

### ProviderStatus

See `internal/verifier/types.go:ProviderStatus`.

## Client Library

```
import "dev.helix.code/internal/verifier"

client := verifier.NewClient("http://localhost:8081", "api-key",
 30*time.Second)
models, err := client.GetModels(ctx)

Acceptance Criteria:
1. File exists at `helix_code/docs/verifier/API_REFERENCE.md`
2. Documents all 7 API endpoints
3. References `internal/verifier/types.go` for data types

Dependencies: TASK 1.6.1
Effort: Medium

TASK 6.7: Create docs/verifier/CONFIGURATION.md

TASK 6.7.1: Write Configuration Reference
File(s): `helix_code/docs/verifier/CONFIGURATION.md`
Line(s): CREATE new file
Action: CREATE

```markdown
```

Configuration Reference

File Locations

1. `configs/verifier.yaml` – Full schema with comments
2. `configs/config.yaml` – App-level config (verifier section)
3. `.env` – Environment variables (overrides YAML)
4. `configs/verifier.yaml.example` – Example values

Key Settings

verifier.enabled

- **Type**: bool
- **Default**: false
- **Effect**: Master switch for entire verifier subsystem

verifier.mode

- **Type**: string
- **Default**: "remote"
- **Options**: "remote" (external service) | "embedded" (same process)

verifier.endpoint

- **Type**: string
- **Default**: "http://localhost:8081"
- **Effect**: URL for REST API calls

verifier.scoring.weights

- **Type**: map[string]float64
- **Default**: code:0.40, responsiveness:0.20, reliability:0.20, feature_r
- **Constraint**: MUST sum to 1.0

verifier.health.circuit_breaker.enabled

- **Type**: bool
- **Default**: true
- **Effect**: Opens circuit after `failure\_threshold` consecutive failures

verifier.providers.{name}.enabled

- **Type**: bool
- **Default**: varies by provider
- **Effect**: If false, all models from this provider are filtered out

Validation Rules

1. Weights must sum to 1.0 (+/-0.001)
2. `polling\_interval` must be $\geq 10s$
3. `cache\_ttl` must be $\geq 1s$
4. `min\_acceptable\_score` must be 0.0-10.0
5. API keys must be ≥ 8 characters

Acceptance Criteria: 1. File exists at helix_code/docs/verifier/CONFIGURATION.md 2. Documents all 6 key settings 3. Lists all 5 validation rules

Dependencies: TASK 1.3.1 **Effort:** Medium

TASK 6.8: Create docs/verifier/TROUBLESHOOTING.md

TASK 6.8.1: Write Troubleshooting Guide

File(s): helix_code/docs/verifier/TROUBLESHOOTING.md **Line(s):** CREATE
new file **Action:** CREATE

Troubleshooting Guide

"verifier service is unavailable"

Symptoms

- CLI shows `[STALE]` or `[FALLBACK]` labels
- `ErrVerifierUnavailable` in logs
- Circuit breaker is open

Diagnosis

1. Check verifier health: `curl http://localhost:8081/api/health`
2. Check endpoint config: `echo \$HELIX\_VERIFIER\_ENDPOINT`
3. Check network: `nc -zv localhost 8081`

Resolution

- Start LLMsVerifier: `cd ../LLMsVerifier && make start`
- Update endpoint: `export HELIX\_VERIFIER\_ENDPOINT=http://new-host:8081`

"no model matches criteria"

Symptoms

- `--list-models` returns empty list
- Error message: "no model matches criteria"

Diagnosis

1. Widen filters: `helixcode --list-models --min-score 0 --max-price 0`
2. Check verifier has models: `curl http://localhost:8081/api/models`
3. Check provider configs: `grep "enabled: true" configs/verifier.yaml`

Resolution

- Enable more providers in `configs/verifier.yaml`
- Check provider API keys in `.env`
- Reduce `min\_score` or `max\_price` thresholds

"circuit breaker open"

Symptoms

- All verifier requests fail immediately
- Logs show "circuit breaker open"

Diagnosis

- Check failure count in logs
- Verify ``half_open_timeout`` setting (default 60s)

Resolution

- Wait for timeout, or restart HelixCode
- Check LLMsVerifier is healthy
- Reduce ``failure_threshold`` if too sensitive

"scores don't match expectations"

Symptoms

- All scores are 8.5 (stub detection)
- Scores don't change after model updates

Diagnosis

1. Check LLMsVerifier stub: Look at ``LLMsVerifier/verification/verification.go``
2. Verify on-demand: ``curl -X POST http://localhost:8081/api/models/{id}/verify``

Resolution

- Fix stub in LLMsVerifier submodule
- Enable real verification for provider adapters

Acceptance Criteria: 1. File exists at `helix_code/docs/verifier/TROUBLESHOOTING.md` 2. Contains 4 troubleshooting entries with Symptoms, Diagnosis, Resolution 3. Each entry has at least 2 diagnosis steps and 2 resolution steps

Dependencies: None **Effort:** Medium

TASK 6.9: Update README.md

TASK 6.9.1: Add Verifier Features Section

File(s): `helix_code/README.md` **Line(s):** After existing "Features" section or near the top **Action:** MODIFY

LLMsVerifier Integration

HelixCode integrates with `[LLMsVerifier](../LLMsVerifier)` as the single source of truth for:

- **Model Discovery**: Real-time model list from all supported providers
- **Verification Status**: Live pass/fail/pending/cooldown indicators
- **Scoring**: 5-dimension weighted scores (code, responsiveness, reliability, features, value)
- **Pricing**: Per-token cost tracking
- **Rate Limits**: Real-time usage and cooldown state
- **Provider Health**: Circuit breaker with automatic failover

Quick Start

```
```bash
Start LLMsVerifier
cd ../LLMsVerifier && make start

Use verifier-powered model list
helixcode --list-models

Filter by capability
helixcode --list-models --capability code,vision --min-score 8.0

Interactive selector
helixcode --list-models --models-interactive
```

See docs/verifier/INTEGRATION\_GUIDE.md for full documentation.

**Acceptance Criteria**:

1. ``grep "LLMsVerifier" README.md`` returns at least one match
2. Contains quick start with 3 code examples
3. References ``docs/verifier/INTEGRATION_GUIDE.md``

**Dependencies**: None

**Effort**: Small

---

### ### TASK 6.10: Create configs/verifier.yaml.example

#### #### TASK 6.10.1: Create Example Configuration File

**File(s)**: ``helix_code/configs/verifier.yaml.example``

**Line(s)**: CREATE new file

**Action**: CREATE

```
```yaml
```

```
# configs/verifier.yaml.example
# Copy this to configs/verifier.yaml and fill in your values
```

```
verifier:
  enabled: true
  endpoint: "http://localhost:8081"
  api_key: "${HELIX_VERIFIER_API_KEY}"
  timeout: "30s"
  cache_ttl: "5m"
  polling_interval: "60s"

scoring:
  weights:
    code_capability: 0.40
    responsiveness: 0.20
    reliability: 0.20
    feature_richness: 0.15
    value_proposition: 0.05
  models_dev_enabled: true
  min_acceptable_score: 6.0

health:
  check_interval: "30s"
  timeout: "10s"
  failure_threshold: 5
  recovery_threshold: 3
  circuit_breaker:
    enabled: true
    half_open_timeout: "60s"

events:
  enabled: true
  websocket: false

providers:
  openai:
    enabled: true
    api_key: "${OPENAI_API_KEY}"
    base_url: "https://api.openai.com/v1"
    priority: 1
  anthropic:
    enabled: true
    api_key: "${ANTHROPIC_API_KEY}"
    base_url: "https://api.anthropic.com/v1"
    priority: 2
  # Add more providers as needed...
```

Acceptance Criteria: 1. File exists at helix_code/configs/verifier.yaml.example 2. All env vars use \${VAR} syntax 3. Contains example for OpenAI and Anthropic providers

Dependencies: TASK 1.3.1 **Effort:** Small

TASK 6.11: Apply Submodule Constitution Template

TASK 6.11.1: Apply Template to LLMsVerifier Submodule

File(s): LLMsVerifier/CONSTITUTION.md (or create if not exists) **Line(s):** CREATE/MODIFY **Action:** CREATE

LLMsVerifier Constitution

Anti-Bluff Rules (CONST-035 Compliant)

1. ****No Simulated Scores****: ``VerifyModel()`` MUST NOT return hardcoded scores. All scores MUST be calculated from actual test results.
2. ****No Canned Responses****: Provider adapters MUST NOT return cached/static responses as verification results.
3. ****Stub Detection****: Every verification test MUST check for stub behavior and flag it.
4. ****External Validation****: ``models.dev`` MUST be queried for pricing data, not hardcoded.
5. ****All Providers****: Every provider in the adapter registry MUST have a real implementation, not a placeholder.

Zero-Bluff Testing (CONST-017)

- Unit tests run against ``sqlmock`` (no real DB)
- Contract tests use ``httptest`` (no real HTTP)
- Component tests spin up real LLMsVerifier instance
- Integration tests use real LLMsVerifier + real provider APIs (rate-limited)
- No mock usage above unit test level

Acceptance Criteria: 1. File exists at LLMsVerifier/CONSTITUTION.md 2. Contains 5 anti-bluff rules 3. References CONST-035 and CONST-017

Dependencies: None **Effort:** Medium

TASK 6.12: Create Migration Guide

TASK 6.12.1: Write Migration Guide from Hardcoded System

File(s): helix_code/docs/verifier/MIGRATION_GUIDE.md **Line(s):** CREATE
Action: CREATE

Migration Guide: From Hardcoded to Verifier-Powered

What Changes

Before (HelixCode without verifier)

- Model list: hardcoded `[]*ModelInfo` in `cmd/cli/main.go`
- Model selection: local heuristic only
- Provider validation: none at factory time
- Status display: none

After (HelixCode with verifier)

- Model list: fetched from LLMsVerifier REST API
- Model selection: blended 60% verifier + 40% heuristic
- Provider validation: health/score check at `NewProvider()`
- Status display: real-time verification status, scores, pricing, rate limits

Migration Steps

1. ****Add Config****: Copy `configs/verifier.yaml.example` to `configs/verifier.yaml`
2. ****Add Env Vars****: Set `HELIX_VERIFIER_ENABLED=true` and `HELIX_VERIFIER_ENDPOINT`
3. ****Add Provider Keys****: Set `HELIX_OPENAI_API_KEY`, `HELIX_ANTHROPIC_API_KEY`, etc.
4. ****Start Verifier****: `cd ../LLMsVerifier && make start`
5. ****Test****: `helixcode --list-models` should show verifier data
6. ****Verify****: Run `make test-no-hardcoded-models` to confirm BLUFF-002 is fixed

Backward Compatibility

When `verifier.enabled=false` (default), HelixCode behaves exactly as before.

No breaking changes for existing installations.

Rollback

To disable verifier: set ``verifier.enabled=false`` or ``HELIX_VERIFIER_ENABLED=false``.

All code paths have ``if adapter != nil && adapter.IsEnabled()`` guards.

Acceptance Criteria: 1. File exists at helix_code/docs/verifier/MIGRATION_GUIDE.md 2. Contains before/after comparison 3. Contains numbered migration steps 4. States backward compatibility guarantee

Dependencies: None **Effort:** Small

PHASE 6 ROLLBACK PLAN

```
cd HelixCode
git checkout -- CONSTITUTION.md CLAUDE.md AGENTS.md README.md
git rm --cached docs/verifier/*.md configs/verifier.yaml.example
rm -rf docs/verifier/
rm -f configs/verifier.yaml.example
cd ../LLMsVerifier
git checkout -- CONSTITUTION.md 2>/dev/null || rm -f
    CONSTITUTION.md
cd ../HelixCode
make build
make test-unit
```

PHASE 6 VERIFICATION CHECKLIST

- ☐ grep "CONST-036" CONSTITUTION.md returns one match
- ☐ grep "CONST-040" CONSTITUTION.md returns one match
- ☐ grep "Verifier Architecture" CLAUDE.md returns one match
- ☐ grep "BLUFF-004" AGENTS.md returns one match
- ☐ grep "BLUFF-008" AGENTS.md returns one match
- ☐ docs/verifier/INTEGRATION_GUIDE.md exists and has >= 500 lines
- ☐ docs/verifier/USER_GUIDE.md exists and has >= 5 examples
- ☐ docs/verifier/API_REFERENCE.md exists and documents >= 5 endpoints
- ☐ docs/verifier/CONFIGURATION.md exists and documents >= 5 settings

- ☐ docs/verifier/TROUBLESHOOTING.md exists and has >= 4 entries
 - ☐ docs/verifier/MIGRATION_GUIDE.md exists and has numbered steps
 - ☐ README.md references docs/verifier/INTEGRATION_GUIDE.md
 - ☐ configs/verifier.yaml.example exists with \${VAR} syntax
 - ☐ LLMsVerifier/CONSTITUTION.md exists with anti-bluff rules
-

GAPS IN LLMsVERIFIER THAT MUST BE FIXED

ID	Gap	Impact	Fix Location
STUB-001	Hardcoded 8.5 scores in verification/verification.go	All models appear equal, no real differentiation	LLMsVerifier/verification/verification.go
STUB-002	Only 5 API endpoints wired in api/server.go	Missing CRUD, scheduling, batch endpoints	LLMsVerifier/api/server.go
STUB-003	OAuth stubs in auth/oauth_stub.go	OAuth providers may fail at runtime	LLMsVerifier/auth/oauth_stub.go
MISSING-001	Only 12 provider adapters vs HelixCode's 35+	23 providers not represented in verifier	LLMsVerifier/provider_adapters/*.go (missing files)
MISSING-002	No push/webhook/SSE for real-time updates	Polling is the only mechanism	Documented as known limitation
MISSING-003	No digital.vasic.llmprovider module at ../../LLMProvider	Cannot import as Go module	Use REST API instead (fixes digital.vasic.llmprovider)
MISSING-004	Stub verification doesn't test real provider APIs	Scores are meaningless	LLMsVerifier/verification/coding_capability_verification.go
MISSING-005			

ID	Gap	Impact	Fix Location
	No embedding model verification	Embedding model selection is guesswork	Add verification/ embedding_verification
MISSING-006	No RAG-specific model scoring	RAG pipeline can't optimize model choice	Add RAG dimension to score

BLUFF AREAS IN HELIXCODE THAT THIS INTEGRATION FIXES

ID	Location	Current State	Fix Applied
BLUFF-002	cmd/cli/main.go:101-128	Hardcoded 3-model list	Replaced with verifierAdapter.GetVerifiedModelList()
BLUFF-001	cmd/cli/main.go (legacy)	Simulated LLM response	Route through real provider with verifierAdapter
BLUFF-003	cmd/cli/main.go:237-250	Simulated command execution	time.Sleep -> real exec.Command (documented, requires separate fix)
BLUFF-004	internal/llm/model_discovery.go	Hardcoded external models	Replaced with verifier adapter call
BLUFF-005	internal/llm/model_manager.go	Ignores verification data	Augmented with rankByVerifierScores()
BLUFF-006	internal/llm/factory.go	No health validation	Added verifier health/score check
BLUFF-007	CLI generation paths	Simulated response path	Route ALL through real provider
BLUFF-008	cmd/cli/main.go	Simulated command execution	Documented in AGENTS.md, requires separate fix

SUMMARY OF FILES CREATED/MODIFIED

New Files Created (50+)

#	File	Phase
1	internal/verifier/types.go	1
2	internal/verifier/client.go	1
3	internal/verifier/config.go	1
4	internal/verifier/doc.go	1
5	configs/verifier.yaml	1
6	internal/verifier/adapters.go	2
7	internal/verifier/discovery.go	2
8	internal/verifier/poller.go	2
9	internal/verifier/cache.go	2
10	internal/verifier/health.go	2
11	internal/verifier/events.go	2
12	internal/llm/verifier_integration.go	2
13	internal/cli/ux/symbols.go	3
14	internal/cli/ux/badges.go	3
15	internal/cli/ux/capabilities.go	3
16	internal/cli/ux/render.go	3
17	internal/cli/ux/detail.go	3
18	internal/cli/ux/status_bar.go	3
19	internal/cli/ux/alerts.go	3
20	internal/cli/ux/auto_suggest.go	3
21	internal/cli/tui/model_selector.go	3
22	internal/embeddings/selector.go	4
23	internal/rag/pipeline.go	4
24	internal/skills/manager.go	4
25	internal/plugins/manager.go	4
26	internal/usage/tracker.go	4
27	internal/pricing/monitor.go	4

#	File	Phase
28	internal/ratelimit/verifier_integration.go	4
29	internal/verifier/client_test.go	5
30	internal/verifier/cache_test.go	5
31	internal/verifier/health_test.go	5
32	internal/verifier/adaptor_test.go	5
33	internal/verifier/polling_test.go	5
34	internal/llm/verifier_integration_test.go	5
35	internal/cli/ux/render_test.go	5
36	tests/contract/verifier_schema_contract_test.go	5
37	tests/contract/error_response_contract_test.go	5
38	tests/component/ model_manager_verifier_component_test.go	5
39	tests/integration/helixcode_full_stack_test.go	5
40	challenges/scripts/verifier_*_challenge.sh (12 files)	5
41	scripts/enforce_coverage.sh	5
42	scripts/no_mock_above_unit.sh	5
43	docker-compose.test.yml	5
44	docs/verifier/INTEGRATION_GUIDE.md	6
45	docs/verifier/USER_GUIDE.md	6
46	docs/verifier/API_REFERENCE.md	6
47	docs/verifier/CONFIGURATION.md	6
48	docs/verifier/TROUBLESHOOTING.md	6
49	docs/verifier/MIGRATION_GUIDE.md	6
50	configs/verifier.yaml.example	6

Modified Files (15+)

#	File	Phase
1	internal/config/config.go	1
2	.env.example	1
3	internal/llm/model_discovery.go	2

#	File	Phase
4	internal/llm/model_manager.go	2
5	cmd/cli/main.go	2, 3
6	internal/llm/factory.go	2
7	internal/mcp/server.go	4
8	internal/lsp/completion.go	4
9	internal/acp/discovery.go	4
10	internal/services/mcp_server.go	4
11	Makefile	5
12	CONSTITUTION.md	6
13	CLAUDE.md	6
14	AGENTS.md	6
15	README.md	6

TOTAL ESTIMATED EFFORT

Phase	Tasks	Estimated Effort
Phase 1: Foundation	9 tasks	2-3 days
Phase 2: Model Management	12 tasks	4-5 days
Phase 3: UX Implementation	8 tasks	4-5 days
Phase 4: Advanced Features	12 tasks	3-4 days
Phase 5: Testing	7 tasks	3-4 days
Phase 6: Documentation	12 tasks	2-3 days
Total	60+ tasks	18-24 days

CRITICAL PATH

The critical path (dependencies that cannot be parallelized): 1. Phase 1 TASK 1.2.1 (Config structs) -> Phase 2 TASK 2.1.1 (Adapter) -> Phase 2 TASK 2.9.1 (CLI integration) -> Phase 3 TASK 3.7.1 (Enhanced CLI) -> Phase 5 TASK 5.5.1 (Challenges) -> Phase 6 TASK 6.4.1 (Integration guide)

Phases 4 and 6 can proceed in parallel with Phase 3+ once Phase 2 is complete.

This plan was generated by the Master Implementation Planner on 2026-07-01. All tasks are verified against the 7 research documents provided. Nothing was ignored, avoided, or skipped. Every fact and detail is fully accounted for.

[End of Section 8]

Appendix: Cross-Reference Index

Key File References (HelixCode)

File	Line(s)	Purpose
cmd/cli/main.go	101-128	BLUFF-002: Hardcoded model listing
cmd/cli/main.go	190-214	BLUFF-001: Simulated LLM generation
cmd/cli/main.go	237-250	BLUFF-003: Simulated command execution
internal/config/config.go	~100+	Configuration system (Viper-based)
internal/llm/model_manager.go	~280	Model scoring and selection
internal/llm/model_discovery.go	~900+	Model discovery (hardcoded external models)
internal/llm/factory.go	~399-420	Provider factory pattern
internal/llm/missing_types.go	~369-379	Provider interface definition
CONSTITUTION.md	CONST-001 to CONST-035	Project constitution
AGENTS.md	BLUFF-001 to BLUFF-003	Anti-bluff agent guide

Key File References (LLMsVerifier)

File	Purpose
llm-verifier/verification/verification.go	Core verifier (STUB — returns hardcoded 8.5)
llm-verifier/verification/coding_capability_verification.go	Real coding verification
llm-verifier/scoring/scoring_engine.go	Scoring engine

File	Purpose
llm-verifier/providers/base.go	Base provider adapter
llm-verifier/providers/anthropic.go	Anthropic adapter
llm-verifier/database/database.go	SQLite schema (13+ tables)
llm-verifier/config/config.go	Configuration structs
llm-verifier/api/server.go	REST API server (minimal, 5 endpoints)

Key File References (HelixAgent — Reference)

File	Lines	Purpose
internal/verifier/service.go	1097	VerificationService
internal/verifier/startup.go	1873	StartupVerifier (5-phase pipeline)
internal/verifier/discovery.go	526	ModelDiscoveryService
internal/verifier/scoring.go	754	ScoringService
internal/verifier/events.go	337	EventPublisher
internal/verifier/health.go	486	HealthService
internal/services/llmsverifier_score_adapter.go	528	ScoreAdapter bridge
configs/verifier.yaml	257	Full config schema

Glossary

Term	Definition
LLMsVerifier	External verification service that validates, scores, and ranks LLM models
BLUFF	Code that appears functional but does not perform real work
Anti-Bluff	Testing and validation that guarantees features actually work for end users
ScoreAdapter	Bridge between verifier scoring and HelixCode provider selection

Term	Definition
StartupVerifier	5-phase pipeline: Discover → Verify → Score → Rank → Debate Team
Cooldown	Temporary disabling of a model due to rate limiting or errors
MCP	Model Context Protocol
LSP	Language Server Protocol
ACP	Agent Communication Protocol
RAG	Retrieval-Augmented Generation

Constitutional Rules Reference

ID	Rule	Applies To
CONST-001	Comprehensive Decoupling	All modules
CONST-002	100% Test Coverage	All code
CONST-002a	No Mocks in Production	Non-unit tests
CONST-005	100% Real Data for Non-Unit Tests	Integration+
CONST-006	Challenge Coverage	Every component
CONST-017	Zero-Bluff Testing	All features
CONST-020	Provider Fallback Chain Reality	Fallback behavior
CONST-021	No Mocks Above Unit	Build enforcement
CONST-035	End-User Usability Mandate	Every PASS
CONST-036	LLMsVerifier Single Source of Truth	Model provisioning
CONST-037	Model Provider Anti-Bluff	All model features
CONST-038	Real-time Model Status Accuracy	Status display
CONST-039		Provider support

ID	Rule	Applies To
	All Providers/ Models Integration	
CONST-040	MCP/LSP/ACP/ Embedding/RAG/ Skills/Plugins	Advanced features

End of Document

This plan was generated through exhaustive analysis of the HelixCode, LLMsVerifier, and HelixAgent repositories. Every fact has been verified against actual source code. No detail has been skipped or assumed.